

 > ChatGPT 5.2 Thinking

wieso ist venv nicht verfügbar für meinen partner, wenn er von github die repository pullt

Weil eine venv normalerweise nicht mit ins Git-Repo gehört (und oft sogar bewusst per .gitignore ausgeschlossen wird). Eine virtuelle Umgebung ist **maschinen- und pfadabhängig** – wenn dein Partner das Repo pullt, hat er zwar den Code, aber **nicht deine lokale Python-Umgebung**.

Die häufigsten Gründe:

- venv/ ist in .gitignore → wird nicht gepusht, also kann er sie nicht pullen.
- Selbst wenn ihr venv/ pushen würdet: sie enthält **absolute Pfade** (z. B. zu python.exe) und ist oft **nicht portable** zwischen PCs, OS, Python-Versionen.
- Dein Partner hat ggf. **Python nicht installiert**, oder eine andere Version, oder VS Code nutzt einen anderen Interpreter.

So macht man's richtig (Best Practice)

1. Abhängigkeiten ins Repo, nicht die venv

- Lege eine requirements.txt an (oder pyproject.toml bei Poetry).
 - Wenn du pip nutzt:

```
bash
```

 Code kopieren

```
pip freeze > requirements.txt
```

2. Partner erstellt lokal seine eigene venv und installiert die Pakete Windows (PowerShell):

```
bash
```

 Code kopieren

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

macOS/Linux:

```
bash
```

 Code kopieren

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

3. In VS Code den richtigen Interpreter wählen

- `Ctrl+Shift+P` → **Python: Select Interpreter** → `.venv` auswählen

Mini-Checkliste, wenn es trotzdem "nicht verfügbar" ist

- Ist `venv/` oder `.venv/` in `.gitignore` ? (sehr wahrscheinlich: ja)
- Nutzt dein Partner denselben Projektordner und hat dort wirklich `.venv` erstellt?
- `python --version` (bzw. `python3 --version`) stimmt?
- In VS Code unten rechts: zeigt er den `.venv` Interpreter an?

Wenn du magst, kopier hier kurz den Inhalt deiner `.gitignore` (oder zumindest die Zeilen mit `venv / .venv`) und sag, ob ihr Windows oder macOS/Linux nutzt – dann sag ich dir exakt, was bei euch gerade schief läuft.



auf mac den richtigen interpreter auswählen tastenkombi

Auf dem Mac in **VS Code**:

- **Command Palette öffnen:** `⌘ + ⇧ + P`
 - dann tippen: **Python: Select Interpreter** → Enter → deinen `.venv` auswählen

Alternative, wenn du direkt per Shortcut zur Interpreter-Auswahl willst:

- **Quick Open:** `⌘ + P`
 - dann eingeben: `>Python: Select Interpreter` → Enter

Falls die Python-Extension die Statusleiste nutzt:

- Unten rechts auf die **Python-Version/Interpreter** klicken → `.venv` auswählen.



```

<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>

```

```

<h1>Putzplan</h1>
<h2>WG-Name: {{ wg_name }}</h2>
<h2>

```

Hier kannst du den Putzplan einsehen und neue Einträge hinzufügen.

```

</h2>

```

```

{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
}%alert-success{% else %}%alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}

```

```

<input type="checkbox" id="modalToggle" style="display:
none;">

```

```

<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
  + Neuen Eintrag erstellen
</label>

```

```

<div style="position: fixed; top: 0; left: 0; width: 100%;

```

```
height: 100%; background-color: rgba(0, 0, 0, 0.5); display: none; z-index: 1;" id="modalBackdrop"> </div>
```

```
<div style="position: fixed; top: 50%; left: 50%; transform: translate(-50%, -50%); background-color: white; padding: 30px; border-radius: 8px; width: 400px; z-index: 2; display: none;" id="modalContent">
```

```
  <label for="modalToggle" style="float: right; cursor: pointer; font-size: 24px; font-weight: bold;"> × </label>
```

```
  <h2>Neuen Eintrag erstellen</h2>
```

```
  <h3>Bestimme, wer für was zuständig ist</h3>
```

```
  <form method="POST">
```

```
    {{ form.hidden_tag() }}
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="woche">Kalenderwoche:</label>
```

```
      {{ form.woche(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="von_datum">Von:</label>
```

```
      {{ form.von_datum(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="bis_datum">Bis:</label>
```

```
      {{ form.bis_datum(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="aufgabe">Aufgabe:</label>
```

```
      {{ form.aufgabe(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="zustaendig">Zuständig:</label>
```

```
      <input list="users" name="zustaendig" id="zustaendig" style="display: block; margin-top: 5px; padding: 8px; width: 100%;" placeholder="z.B. Max"
```



```

required>
    <datalist id="users">
        {% for user in all_users %}
            <option value="{{ user.username }}"> </option>
        {% endfor %}
    </datalist>
    {% if form.zustaendig.errors %}<span style="color:
red;">{{ form.zustaendig.errors[0] }}</span>{% endif %}
</div>

    {{ form.submit() }}
</form>
</div>

<style>
    #modalToggle:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggle:checked ~ #modalContent {
        display: block !important;
    }
</style>

<table>
    <thead>
        <tr>
            <th>Woche</th>
            <th>Zeitraum</th>
            <th>Aufgabe</th>
            <th>Zuständig</th>
        </tr>
    </thead>
    <tbody>
        {% for eintrag in putzplan %}
            <tr>
                <td>{{ eintrag.woche }}</td>
                <td>{{ eintrag.von_datum.strftime('%d.%m.%Y') }} -
                {{ eintrag.bis_datum.strftime('%d.%m.%Y') }}</td>
                <td>{{ eintrag.aufgabe }}</td>
                <td>{{ eintrag.zustaendig }}</td>
                <td>

```

```

        {% for task in eintrag.tasks %}
        <div>
            <input type="checkbox" {% if task.status
== 'completed' %}checked{% endif %}>
            {{ task.notes }}
        </div>
        {% endfor %}
    </td>
</tr>
{% endfor %}
</tbody>
</table>

<style>
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    th, td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }
    th {
        background-color: #f2f2f2;
    }

```

</style> wie mache ich es, dass er nach dem erstellen im modal auf der seite die entsprechde erstellte aufgabe anzeigt?

Nachgedacht für 25s >

Damit **nach dem Speichern** der neue Eintrag **direkt in der Tabelle auftaucht**, musst du im Flask-Route-Handler beim `POST` :

1. den Eintrag in der DB speichern
2. danach **redirecten** (Post/Redirect/Get) zurück auf dieselbe Seite
3. bei `GET` die Einträge neu aus der DB laden und rendern

Dann wird die Seite neu geladen und die Tabelle enthält automatisch den neuen Datensatz.

Beispiel Route (Flask)

python

 Code kopieren

```
from flask import render_template, request, redirect, url_for, flash
# from yourapp import db
# from yourapp.models import PutzplanEintrag, User
# from yourapp.forms import PutzplanForm

@app.route("/putzplan", methods=["GET", "POST"])
def putzplan():
    form = PutzplanForm()

    if form.validate_on_submit():
        # WICHTIG: du benutzt im Template ein eigenes <input name="zustaendig">
        # daher kommt der Wert nicht automatisch aus form.zustaendig.data (außer
        zustandig = request.form.get("zustandig")

        neuer_eintrag = PutzplanEintrag(
            woche=form.woche.data,
            von_datum=form.von_datum.data,
            bis_datum=form.bis_datum.data,
            aufgabe=form.aufgabe.data,
            zustandig=zustandig
        )

        db.session.add(neuer_eintrag)
        db.session.commit()

        flash("Eintrag wurde erstellt , "success")
        return redirect(url_for("putzplan")) # <- sorgt dafür, dass die Seite ne

# GET (oder wenn Validation fehlschlägt): Daten laden
putzplan_liste = PutzplanEintrag.query.order_by(PutzplanEintrag.von_datum).des
all_users = User.query.all()
wg_name = "..." # so wie du es aktuell befüllst

return render_template(
    "putzplan.html",
    form=form,
    putzplan=putzplan_liste,
```

```

    all_users=all_users,
    wg_name=wg_name
)

```

Modal nach Fehler offen lassen (optional, aber sehr hilfreich)

Wenn Validierung fehlschlägt, ist es nervig, wenn das Modal zugeht. Dann:

html

 Code kopieren

```

<input type="checkbox" id="modalToggle" style="display: none;"
      {% if form.errors %}checked{% endif %}>

```

So bleibt das Modal bei Fehlern offen (und bei Erfolg geht's durch den Redirect automatisch zu).

Kleiner HTML-Bug bei deiner Tabelle (Spaltenanzahl)

Du hast im Header 4 Spalten, aber im Body 5 `<td>` (wegen `tasks`). Entweder:

- füge ein `<th>Tasks</th>` hinzu **oder**
- entferne die `tasks`-Spalte.

Beispiel fix:

html

 Code kopieren

```

<thead>
  <tr>
    <th>Woche</th>
    <th>Zeitraum</th>
    <th>Aufgabe</th>
    <th>Zuständig</th>
    <th>Tasks</th>
  </tr>
</thead>

```

Wenn du mir kurz deine `putzplan()`-Route + dein WTForms `PutzplanForm` zeigst, kann ich dir das exakt auf deinen Code anpassen (inkl. sauberem Handling für `zustaendig`).



```

@app.route("/putzplan/", methods=['GET', 'POST'])
def putzplan():
    form = PutzplanForm()
    all_users = User.query.all()
    for user in all_users:
        print(f"Benutzer: {user.username}")
    if form.validate_on_submit():
        new_eintrag = CleaningTemplate(
            wg_id=1, # Get from session/logged-in user
            name=form.aufgabe.data,
            description=f"Week {form.woche.data}:
{form.von_datum.data} to {form.bis_datum.data}",
            frequency="weekly"
        )
        db.session.add(new_eintrag)
        db.session.commit()

        flash('Eintrag erfolgreich erstellt!', 'success')
        return redirect(url_for("putzplan"))
    elif form.is_submitted():
        # Form submitted but validation failed
        if form.zustaendig.errors:
            flash('WG-Mitglied existiert nicht', 'error')
        putzplan_eintraege = CleaningTemplate.query.all()
        return render_template("putzplan.html", form=form,
        putzplan=putzplan_eintraege, all_users=all_users) das ist
        meine jetzige route

```

Deine Route macht das **fast richtig** – du machst schon `commit + redirect`, also *sollte* der neue Eintrag nach dem Redirect in der Tabelle erscheinen.

Wenn es bei dir **nicht** angezeigt wird, liegt's in deinem Fall sehr wahrscheinlich an diesen 3 Punkten:

1. Du speicherst nicht die Felder, die du im Template anzeigst.

Im Template greifst du auf `eintrag.woche`, `eintrag.von_datum`, `eintrag.bis_datum`, `eintrag.aufgabe`, `eintrag.zustaendig` zu.

In der DB speicherst du aber nur:

- name (entspricht eher aufgabe)
- description (ein String)
- frequency

→ Das passt nicht zu deinem Template.

2. Du speicherst **zustaendig** **gar nicht**.

Du prüfst `form.zustaendig.errors`, aber im Template benutzt du ein eigenes `<input name="zustaendig">` (nicht `{{ form.zustaendig }}`), und im Model speicherst du es nirgends.

3. Du hast `wg_id=1` **hart drin** (nicht fatal fürs Anzeigen, aber für "richtige" WG-Auswahl später wichtig).

Lösung A (empfohlen): Template an dein Model `CleaningTemplate` anpassen

Wenn `CleaningTemplate` wirklich nur `name`, `description`, `frequency` hat, dann zeig genau das an.

putzplan.html (Tabelle)

html

 Code kopieren

```
<thead>
  <tr>
    <th>Name</th>
    <th>Beschreibung</th>
    <th>Frequenz</th>
  </tr>
</thead>
<tbody>
  {% for eintrag in putzplan %}
    <tr>
      <td>{{ eintrag.name }}</td>
      <td>{{ eintrag.description }}</td>
      <td>{{ eintrag.frequency }}</td>
    </tr>
  {% endfor %}
</tbody>
```

Dann wird dein neuer Eintrag nach dem Redirect garantiert sichtbar.

Lösung B: Du willst genau deine aktuellen Spalten behalten (Woche/Zeitraum/Aufgabe/Zuständig)

Dann musst du entweder:

B1) dein Model erweitern (sauber)

`CleaningTemplate` braucht Felder wie `woche`, `von_datum`, `bis_datum`, `zustaendig` (oder `zustaendig_user_id`).

B2) oder beim Rendern aus `description` wieder "zurückparsen" (nicht empfehlenswert)

Sofort-Fix in deiner Route (damit wenigstens "Zuständig" mitkommt)

Wenn du `zustaendig` als Feld im Model hast (z.B. `assigned_to` oder `responsible`), dann hol es so:

python

 Code kopieren

```
zustaendig = request.form.get("zustaendig")
```

und speicher es:

python

 Code kopieren

```
new_eintrag = CleaningTemplate(
    wg_id=1,
    name=form.aufgabe.data,
    description=f"Week {form.woche.data}: {form.von_datum.data} to {form.bis_datum.data}
    frequency="weekly",
    zustaaendig=zustaendig # nur wenn es das Feld wirklich gibt!
)
```

Wichtig: Wenn dein `CleaningTemplate` dieses Feld nicht hat, crasht es.

Modal nach erfolgreichem Erstellen automatisch schließen

Das passiert bei dir schon durch den `redirect`. 👍

Falls das Modal nach **Validierungsfehlern** offen bleiben soll, setz im Template:

html

 Code kopieren

```
<input type="checkbox" id="modalToggle" style="display: none;"
      {% if form.errors %}checked{% endif %}>
```

Was du mir kurz schicken solltest (damit ich es 100% passend mache)

Poste bitte dein `CleaningTemplate` Model (die Spalten) **und** dein `PutzplanForm` (Felder).
Dann sag ich dir exakt:

- welche Felder du speichern musst,
- wie du die Tabelle korrekt renderst,
- wie `zustaendig` sauber als User-Relation gespeichert wird (User-ID statt Text).



```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True,
index=True)
    wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)
```

Alles klar — mit deinem `CleaningTemplate` -Model kann deine Tabelle so, wie sie aktuell ist, gar nicht funktionieren, weil es keine Spalten `woche`, `von_datum`, `bis_datum`, `aufgabe`, `zustaendig` gibt.

Du hast zwei saubere Wege:

Option 1 (am schnellsten): Template an `CleaningTemplate` anpassen



Du speicherst:

- name (→ Aufgabe)
- description (→ Zeitraum/Infos)
- frequency

Dann ändere **nur die Tabelle** in `putzplan.html` so:

html

 Code kopieren

```
<table>
  <thead>
    <tr>
      <th>Aufgabe</th>
      <th>Beschreibung</th>
      <th>Frequenz</th>
      <th>Aktiv</th>
    </tr>
  </thead>
  <tbody>
    {% for eintrag in putzplan %}
      <tr>
        <td>{{ eintrag.name }}</td>
        <td>{{ eintrag.description }}</td>
        <td>{{ eintrag.frequency }}</td>
        <td>{{ "Ja" if eintrag.is_active else "Nein" }}</td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

Dann wird der neue Eintrag **direkt nach dem Redirect** angezeigt.

Wenn du "Woche/Zeitraum" schöner willst: schreib sie direkt in `description` (machst du ja schon).

Option 2 (richtig für deinen Use-Case): eigenes `CleaningPlanEntry` Model für konkrete Wochen 📌

`CleaningTemplate` klingt nach **Vorlage** (z.B. "Bad putzen wöchentlich").

Für einen echten Putzplan-Eintrag (KW, von/bis, zuständig) brauchst du eine zweite Tabelle, z.B.:

- CLEANING_PLAN_ENTRY :
 - entry_id
 - wg_id
 - woche
 - von_datum
 - bis_datum
 - aufgabe (oder template_id)
 - zustaendig_user_id

Dann bleibt `CleaningTemplate` als Vorlage bestehen und der Plan wird pro Woche erzeugt.

Warum es bei dir “nicht angezeigt” wirkt

Du renderst aktuell:

html

 Code kopieren

```
{{ eintrag.woche }}
{{ eintrag.von_datum.strftime(...) }}
{{ eintrag.aufgabe }}
{{ eintrag.zustaendig }}
```

Aber `eintrag` ist ein `CleaningTemplate` und hat nur `name/description/frequency/...`. Jinja kann das nicht aus dem Nichts anzeigen → es bleibt leer oder es gibt Fehler.

Minimal-Verbesserung in deiner Route (Aktive Templates + Sortierung)

python

 Code kopieren

```
putzplan_eintraege = CleaningTemplate.query.filter_by(is_active=True).order_by(Cl
```

<

>

Bonus: “Zuständig” in deinem aktuellen Model?

In `CleaningTemplate` gibt’s kein Feld dafür. Wenn du “Zuständig” **pro Woche** meinst, gehört das **nicht** in die Template-Tabelle, sondern in eine Plan-Entry-Tabelle (Option 2).

Wenn du willst, sag kurz: Soll `CleaningTemplate` wirklich **der Putzplan selbst** sein (mit Woche/von/bis/zuständig), oder ist es eine **Vorlage**, aus der wöchentliche Einträge entstehen?

Ich kann dir dann das passende Datenmodell + Route + Template sauber zusammenbauen.



es soll nur in template passieren

Alles klar: **dann musst du dein HTML an** `CleaningTemplate` **anpassen**, weil dort nur `name`, `description`, `frequency`, `is_active`, `position` existieren.

1) Route ist schon richtig (commit + redirect) ✓

Dein `redirect(url_for("putzplan"))` sorgt dafür, dass nach dem Erstellen die Seite neu lädt und `CleaningTemplate.query.all()` den neuen Datensatz enthält.

Ich würde nur noch "aktive" und sortiert laden:

python

Code kopieren

```
putzplan_eintraege = (CleaningTemplate.query
    .filter_by(wg_id=1, is_active=True)
    .order_by(CleaningTemplate.position.asc().nullslast(),
              CleaningTemplate.template_id.desc())
    .all())
```

2) Tabelle im Template ändern (damit es wirklich angezeigt wird)

Ersetze deine Tabelle durch diese (passt 1:1 zu deinem Model):

html

Code kopieren

```
<table>
  <thead>
    <tr>
      <th>Aufgabe</th>
```

```

        <th>Beschreibung</th>
        <th>Frequenz</th>
        <th>Position</th>
    </tr>
</thead>
<tbody>
    {% for eintrag in putzplan %}
    <tr>
        <td>{{ eintrag.name }}</td>
        <td>{{ eintrag.description }}</td>
        <td>{{ eintrag.frequency }}</td>
        <td>{{ eintrag.position if eintrag.position is not none else "-" }}</td>
    </tr>
    {% endfor %}
</tbody>
</table>

```

Damit wird der neu erstellte Eintrag sofort nach dem Speichern angezeigt.

3) Modal schließt & Eintrag ist sichtbar

- Bei **Erfolg**: passiert automatisch durch Redirect (Modal zu, Tabelle aktualisiert)
- Bei **Form-Fehlern** (optional): Modal offen lassen:

html

 Code kopieren

```
<input type="checkbox" id="modalToggle" style="display: none;" {% if form.errors
```

Wichtig zu wissen (wegen „Woche/von/bis/zuständig“)

Wenn du diese Infos weiterhin erfassen willst, aber **nur in** `CleaningTemplate` speichern möchtest, dann geht das nur als **Text in** `description` (so machst du's schon).

Dein aktueller „Zuständig“-Input wird aber **nirgendwo gespeichert**, weil das Feld im Model fehlt.

Wenn du „Zuständig“ trotzdem im Template anzeigen willst, hast du 2 Optionen:

- **(einfach)** zuständig in `description` mit reinschreiben
- **(sauber)** Spalte `assigned_to` / `responsible` im `CleaningTemplate` ergänzen

Wenn du willst, sag mir kurz: Soll „Zuständig“ **nur Text** sein oder als **User-Referenz**? Dann gebe ich dir die passende Model-Änderung + Route-Anpassung.



kann man template nicht anpassen?

Nachgedacht für 17s >

Ja — du kannst deine "Template"-Tabelle (CleaningTemplate) anpassen, sodass dein bestehendes HTML (mit Woche / von / bis / zuständig) wieder Sinn ergibt. Aktuell fehlen dafür einfach die Spalten im Model.

Du hast zwei saubere Wege:

Weg 1 (sauber): CleaningTemplate um Spalten erweitern + DB-Migration

1) Model erweitern

python

Code kopieren

```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True, index=True)
    wg_id = db.Column(db.Integer, db.ForeignKey('WG.wg_id'), nullable=False)

    # bisher
    name = db.Column(db.String, nullable=False)           # = aufgabe
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)

    # neu für deinen Screen
    woche = db.Column(db.Integer, nullable=True)
    von_datum = db.Column(db.Date, nullable=True)
    bis_datum = db.Column(db.Date, nullable=True)
    zustandig = db.Column(db.String, nullable=True)      # oder user_id als FK
```

2) Migration ausführen (wenn du Flask-Migrate nutzt)

bash

```
flask db migrate -m "add putzplan fields to cleaning_template"
flask db upgrade
```

3) Route anpassen (damit du auch wirklich speicherst)

python

 Code kopieren

```
from flask import request

if form.validate_on_submit():
    zustandig = request.form.get("zustandig") # weil du ein eigenes <input nam

    new_eintrag = CleaningTemplate(
        wg_id=1,
        name=form.aufgabe.data,
        woche=form.woche.data,
        von_datum=form.von_datum.data,
        bis_datum=form.bis_datum.data,
        zustandig=zustandig,
        frequency="weekly",
        is_active=True
    )
    db.session.add(new_eintrag)
    db.session.commit()
    return redirect(url_for("putzplan"))
```

<

>

Dann kann dein aktuelles HTML **genau so bleiben** (bis auf: du brauchst auch einen `<th>` für die extra Spalte, falls du Tasks drin hast).

Weg 2 (quick & dirty): Nichts an DB ändern, aber "Woche/von/bis" aus description herausparsen

Das geht, aber ist fehleranfällig (Strings parsen, Datumsformat etc.). Würde ich nur nehmen, wenn du wirklich keine Migration machen willst.

Empfehlung

Wenn dein "Putzplan" wirklich Woche/von/bis/zuständig speichern soll: **Weg 1.**

Wenn du willst, kann ich dir auch sagen, wie du "zustaendig" besser als `user_id` (Foreign Key) speicherst, damit es nicht nur Text ist.



ja bitte

Nachgedacht für 16s >

Ja – als **Foreign Key** auf `User` ist deutlich sauberer als ein freies Textfeld. Dann kannst du später z.B. nach Aufgaben pro User filtern, Umbenennungen abfangen usw.

Unten ist ein **sauberer Standard-Weg** (ohne "Username tippen"), plus eine Alternative, falls du unbedingt bei `data1ist` bleiben willst.

Variante A (empfohlen): Dropdown/Select → speichert `user_id` als FK

1) Model: FK + Relationship hinzufügen

Passe `User` -PK ggf. an (`id` vs. `user_id`) und den Tabellennamen (`USER` vs. `users`).

python

Code kopieren

```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True, index=True)
    wg_id = db.Column(db.Integer, db.ForeignKey('WG.wg_id'), nullable=False)

    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)
```

```
responsible_user_id = db.Column(db.Integer, db.ForeignKey('USER.user_id'), nu  
responsible_user = db.relationship('User', foreign_keys=[responsible_user_id]
```

Wenn dein User -Model statt user_id z.B. id hat, dann:

python

 Code kopieren

```
db.ForeignKey('USER.id')
```

2) Migration

Falls du Flask-Migrate nutzt:

bash

 Code kopieren

```
flask db migrate -m "add responsible_user_id to cleaning_template"  
flask db upgrade
```

3) Form: QuerySelectField nutzen (WTForms-SQLAlchemy)

Install (falls noch nicht da):

bash

 Code kopieren

```
pip install wtforms-sqlalchemy
```

Form-Beispiel:

python

 Code kopieren

```
from flask_wtf import FlaskForm  
from wtforms import StringField, DateField, IntegerField, SubmitField  
from wtforms.validators import DataRequired  
from wtforms_sqlalchemy.fields import QuerySelectField  
  
def user_query():  
    return User.query.order_by(User.username)  
  
class PutzplanForm(FlaskForm):  
    woche = IntegerField("Kalenderwoche", validators=[DataRequired()])  
    von_datum = DateField("Von", validators=[DataRequired()])  
    bis_datum = DateField("Bis", validators=[DataRequired()])  
    aufgabe = StringField("Aufgabe", validators=[DataRequired()])
```



```

responsible_user = QuerySelectField(
    "Zuständig",
    query_factory=user_query,
    get_label="username",
    allow_blank=False
)

submit = SubmitField("Speichern")

```

4) Route: FK speichern

python

 Code kopieren

```

@app.route("/putzplan/", methods=['GET', 'POST'])
def putzplan():
    form = PutzplanForm()

    if form.validate_on_submit():
        new_eintrag = CleaningTemplate(
            wg_id=1,
            name=form.aufgabe.data,
            description=f"Week {form.woche.data}: {form.von_datum.data} to {form.
            frequency="weekly",
            responsible_user_id=form.responsible_user.data.user_id # oder .id je
        )
        db.session.add(new_eintrag)
        db.session.commit()
        flash('Eintrag erfolgreich erstellt!', 'success')
        return redirect(url_for("putzplan"))

    putzplan_eintraege = CleaningTemplate.query.all()
    all_users = User.query.all()
    return render_template("putzplan.html", form=form, putzplan=putzplan_eintraeg

```

5) Template: Feld anzeigen + später ausgeben

Im Modal:

html

 Code kopieren

```

<label>Zuständig:</label>
{{ form.responsible_user(style="display:block; margin-top:5px; padding:8px; width

```

In der Tabelle:

html

 Code kopieren

```
<td>
  {% if eintrag.responsible_user %}
    {{ eintrag.responsible_user.username }}
  {% else %}
    -
  {% endif %}
</td>
```

Variante B: Du willst unbedingt `datalist` behalten (trotzdem FK speichern)

Dann brauchst du **zusätzlich** ein hidden Feld `responsible_user_id`, und JS, das beim Tippen den Username auf die User-ID mappt.

HTML:

html

 Code kopieren

```
<input list="users" id="zustaendig_name" placeholder="z.B. Max" style="..." required />
<input type="hidden" name="responsible_user_id" id="responsible_user_id">

<datalist id="users">
  {% for user in all_users %}
    <option value="{{ user.username }}" data-id="{{ user.user_id }}"></option>
  {% endfor %}
</datalist>

<script>
  const input = document.getElementById("zustaendig_name");
  const hidden = document.getElementById("responsible_user_id");
  input.addEventListener("input", () => {
    const opt = Array.from(document.querySelectorAll("#users option"))
      .find(o => o.value === input.value);
    hidden.value = opt ? opt.dataset.id : "";
  });
</script>
```

Route speichern:

python

 Code kopieren

```
responsible_user_id = request.form.get("responsible_user_id") or None

new_eintrag = CleaningTemplate(
    ...,
    responsible_user_id=int(responsible_user_id) if responsible_user_id else None
)
```



Welche Variante solltest du nehmen?

- **Variante A** ist am stabilsten (validiert automatisch, immer eine echte User-ID).
- **Variante B** fühlt sich "freier" an, ist aber fehleranfälliger (Tippschreibfehler → keine ID).

Wenn du mir kurz dein `User`-Model (Tabellenname + PK Feldname) postest, schreibe ich dir die FK-Zeile exakt passend (z.B. `'USER.id'` vs. `'users.id'`).



```
<h1>Putzplan</h1>
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

```
<nav>
```

```

<a href="{{ url_for('dashboard') }}">Start</a>
<a href="{{ url_for('putzplan') }}">Putzplan</a>
<a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
<a href="{{ url_for('activity_board') }}">ActivityBoard</a>
<a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>

```

```

<input type="checkbox" id="modalToggle" style="display:
none;">

```

```

<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
    + Neuen Eintrag erstellen
</label>

```

```

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>

```

```

<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">

```

```

    <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;"> × </label>

```

```

    <h2>Neuen Eintrag erstellen</h2>

```

```

    <h3>Bestimme, wer für was zuständig ist</h3>

```

```

    <form method="POST">

```

```

        {{ form.hidden_tag() }}

```

```

        <div style="margin-bottom: 15px;">

```

```

            <label for="woche">Kalenderwoche:</label>

```

```

            {{ form.woche(style="display: block; margin-top:
5px; padding: 8px; width: 100%;" ) }}

```

```

        </div>

```

```

        <div style="margin-bottom: 15px;">

```

```

            <label for="von_datum">Von:</label>

```

```

            {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;" ) }}

```

```

        </div>

```

```

<div style="margin-bottom: 15px;">
  <label for="bis_datum">Bis:</label>
  {{ form.bis_datum(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="aufgabe">Aufgabe:</label>
  {{ form.aufgabe(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="zustaendig">Zuständig:</label>
  <input list="users" name="zustaendig"
id="zustaendig" style="display: block; margin-top: 5px;
padding: 8px; width: 100%;" placeholder="z.B. Max"
required>
  <datalist id="users">
    {% for user in all_users %}
      <option value="{{ user.username }}"></option>
    {% endfor %}
  </datalist>
  {% if form.zustaendig.errors %}<span style="color:
red;">{{ form.zustaendig.errors[0] }}</span>{% endif %}
</div>

  {{ form.submit() }}
</form>
</div>

<style>
  #modalToggle:checked ~ #modalBackdrop {
    display: block !important;
  }

  #modalToggle:checked ~ #modalContent {
    display: block !important;
  }
</style>

<table>

```

```
<thead>
  <tr>
    <th>Datum</th>
    <th>Aufgabe</th>
    <th>Zuständig</th>
  </tr>
</thead>
<tbody>
  {% for eintrag in putzplan %}
    <tr>
      <td>{{ eintrag.datum }}</td>
      <td>{{ eintrag.aufgabe }}</td>
      <td>{{ eintrag.zustaendig }}</td>
    </tr>
  {% endfor %}
</tbody>
</table>

<style>
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
  }
  th, td {
    border: 1px solid #ddd;
    padding: 8px;
    text-align: left;
  }
  th {
    background-color: #f2f2f2;
  }
}
```

</style> wieso packt er nicht die aufgabe auf die seite, die ich erstellt habe

Nachgedacht für 29s >

Weil dein Template (HTML) Felder anzeigt, die es in deinem DB-Model gar nicht gibt.

Du renderst hier:

```
html
```

 Code kopieren

```
<td>{{ eintrag.datum }}</td>
<td>{{ eintrag.aufgabe }}</td>
<td>{{ eintrag.zustaendig }}</td>
```

Aber dein `CleaningTemplate` hat nur:

- `name`
- `description`
- `frequency`
- `is_active`
- `position`
- ...

👉 Ergebnis: Jinja findet `datum/aufgabe/zustaendig` nicht → es kommt **leer** raus (oder du bekommst je nach Config "UndefinedError").

Und noch dazu: In deiner Route speicherst du die Aufgabe als `name=form.aufgabe.data`, nicht als `aufgabe`.

Quick-Fix (ohne DB ändern): Tabelle an dein Model anpassen

Ersetze die Tabelle so:

```
html
```

 Code kopieren

```
<table>
  <thead>
    <tr>
      <th>Beschreibung</th>
      <th>Aufgabe</th>
      <th>Frequenz</th>
    </tr>
  </thead>
  <tbody>
    {% for eintrag in putzplan %}
    <tr>
      <td>{{ eintrag.description }}</td>
      <td>{{ eintrag.name }}</td>
```

```

        <td>{{ eintrag.frequency }}</td>
    </tr>
{% endfor %}
</tbody>
</table>

```

Dann siehst du den neu erstellten Eintrag **sofort nach dem Redirect**.

Warum "Zuständig" aktuell nie erscheinen kann

Du hast im Model **kein Feld** dafür, und in deiner Route speicherst du den Input `zustaendig` auch **nirgendwo**.

Wenn du es trotzdem sofort sehen willst (ohne DB ändern):

Pack `zustaendig` einfach mit in `description` :

```
python
```

 Code kopieren

```

zustaendig = request.form.get("zustaendig")

new_eintrag = CleaningTemplate(
    wg_id=1,
    name=form.aufgabe.data,
    description=f"KW {form.woche.data}: {form.von_datum.data} bis {form.bis_datum}
    frequency="weekly"
)

```

Dann erscheint es in der Spalte "Beschreibung".

Richtig sauber (DB anpassen): echte Spalte für Zuständig + Datum

Wenn du wirklich **Datum/Zeitraum + Zuständig** als eigene Spalten willst, musst du dein `CleaningTemplate` erweitern (Migration) – z.B.:

- `von_datum` (Date)
- `bis_datum` (Date)
- `responsible_user_id` (FK auf User)

Dann kannst du dein ursprüngliches Tabellenlayout wirklich so anzeigen, wie du es willst.

Wenn du mir dein `User`-Model (Tabellenname + Primary Key Feld) postest, schreibe ich dir die FK-Variante exakt passend (inkl. Migration + Route + Template).



```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True,
index=True)
    wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    week = db.Column(db.Integer, nullable=False)
    von_datum = db.Column(db.DateTime, nullable=False)
    bis_datum = db.Column(db.DateTime, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    assigned_to = db.Column(db.Integer,
db.ForeignKey('USER.user_id'))
```

```
if form.validate_on_submit():
    new_eintrag = CleaningTemplate(
        wg_id=1, # Get from session/logged-in user
        name=form.aufgabe.data,
        description=f"Week {form.woche.data}:
{form.von_datum.data} to {form.bis_datum.data}",
        frequency="weekly"
    )
    db.session.add(new_eintrag)
    db.session.commit()
```

```
<table>
<thead>
<tr>
<th>Datum</th>
<th>Aufgabe</th>
<th>Zuständig</th>
</tr>
</thead>
<tbody>
```

```
{% for eintrag in putzplan %}
<tr>
  <td>{{eintrag.bis_datum}}</td>
  <td>{{ eintrag.von_datum }}</td>
  <td>{{ eintrag.name }}</td>
  <td>{{ eintrag.assigned_to }}</td>
</tr>
{% endfor %}
</tbody>
</table>
```

kannst du die bitte anpassen

Ja — bei dir passt es aktuell an **3 Stellen** nicht zusammen:

1. Im Model fehlen/ändern sich Felder (`frequency` existiert nicht mehr, `week/von_datum/bis_datum/assigned_to` schon).
2. In der Route setzt du diese neuen Felder **nicht** → dadurch sind sie `NULL` /Fehler oder werden nicht gespeichert.
3. Deine Tabelle hat 3 `<th>` , aber 4 `<td>` , und du gibst "Datum/Aufgabe" durcheinander aus.

Hier ist die **konsistente Version** (Model bleibt wie du es gepostet hast):

1) Route anpassen (speichert week + von/bis + assigned_to)

Da dein Input im HTML `name="zustaendig"` ist (Username), aber du in der DB `assigned_to` als **User-ID** speicherst, musst du Username → User holen.

python

 Code kopieren

```
from flask import request, redirect, url_for, flash

@app.route("/putzplan/", methods=["GET", "POST"])
def putzplan():
    form = PutzplanForm()
    all_users = User.query.all()

    if form.validate_on_submit():
        # Username aus dem datalist-input holen
        zustandig_name = request.form.get("zustandig", "").strip()
        user = User.query.filter_by(username=zustandig_name).first()
```

```

if not user:
    flash("WG-Mitglied existiert nicht", "error")
else:
    new_eintrag = CleaningTemplate(
        wg_id=1, # später aus session
        name=form.aufgabe.data,
        description=form.aufgabe.data, # optional: oder leer lassen
        week=form.woche.data,
        von_datum=form.von_datum.data,
        bis_datum=form.bis_datum.data,
        assigned_to=user.user_id
    )
    db.session.add(new_eintrag)
    db.session.commit()
    flash("Eintrag erfolgreich erstellt!", "success")
    return redirect(url_for("putzplan"))

putzplan_eintraege = (CleaningTemplate.query
    .filter_by(wg_id=1, is_active=True)
    .order_by(CleaningTemplate.von_datum.desc())
    .all())

return render_template(
    "putzplan.html",
    form=form,
    putzplan=putzplan_eintraege,
    all_users=all_users
)

```

Wichtig: frequency="weekly" muss raus, weil es in deinem Model nicht existiert.

2) Tabelle im Template korrekt anzeigen

Du willst: Datum(Zeitraum) + Aufgabe + Zuständig.

Zeig von_datum - bis_datum, name, und den Username zum assigned_to.

Variante ohne Relationship (schnell)

Du hast all_users sowieso — wir bauen ein kleines Lookup-Dict in Jinja:

html

 Code kopieren

```
{% set user_map = {} %}
{% for u in all_users %}
    {% set _ = user_map.update({u.user_id: u.username}) %}
{% endfor %}

<table>
  <thead>
    <tr>
      <th>Zeitraum</th>
      <th>Aufgabe</th>
      <th>Zuständig</th>
    </tr>
  </thead>
  <tbody>
    {% for eintrag in putzplan %}
    <tr>
      <td>
        {{ eintrag.von_datum.strftime('%d.%m.%Y') }} - {{ eintrag.bis_datum.strft
        (KW {{ eintrag.week }})}
      </td>
      <td>{{ eintrag.name }}</td>
      <td>{{ user_map.get(eintrag.assigned_to, '-') }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>
```

✓ Jetzt stimmen <th> und <td> (3 und 3) und die Felder passen zum Model.

3) Optional: noch sauberer mit Relationship (empfohlen)

Wenn du statt dem `user_map` lieber direkt `eintrag.assigned_user.username` willst:

Model ergänzen:

python

 Code kopieren

```
assigned_user = db.relationship("User", foreign_keys=[assigned_to])
```

Template:

 Code kopieren

html

```
<td>{{ eintrag.assigned_user.username if eintrag.assigned_user else '-' }}</td>
```

Wenn du mir kurz dein `user`-Model (wie heißt die Tabelle/PK genau?) zeigst, kann ich dir die Relationship-Zeile 100% korrekt auf deinen Tabellennamen machen.



```
@app.route("/putzplan/", methods=['GET', 'POST'])
def putzplan():

    form = PutzplanForm()
    all_users = User.query.all()

    for user in all_users:
        print(f"Benutzer: {user.username}")
    if form.validate_on_submit():
        zustaendig_name = request.form.get("zustaendig",
        "").strip()
        user =
        User.query.filter_by(username=zustaendig_name).first()
        if not user:
            flash('WG-Mitglied existiert nicht', 'error')
            return render_template("putzplan.html",
            form=form, putzplan=[], all_users=all_users)
        else:
            new_eintrag = CleaningTemplate(
                wg_id=1, # später aus session
                name=form.aufgabe.data,
                description=form.aufgabe.data, # optional: oder
            leer lassen
                week=form.woche.data,
                von_datum=form.von_datum.data,
                bis_datum=form.bis_datum.data,
                assigned_to=user.user_id
            )
```

```
db.session.add(new_eintrag)
db.session.commit()
flash("Eintrag erfolgreich erstellt!", "success")
return redirect(url_for("putzplan"))

putzplan_eintraege = (CleaningTemplate.query
                      .filter_by(wg_id=1, is_active=True)

                      .order_by(CleaningTemplate.von_datum.desc())
                      .all())

return render_template("putzplan.html", form=form,
                      putzplan=putzplan_eintraege, all_users=all_users) warum
ist jetzt internal server error?
```

Nachgedacht für 36s >

Ein **Internal Server Error (500)** heißt: In Flask ist irgendwo eine Exception passiert (meistens beim `db.session.commit()` oder beim Rendern des Templates). Mit deinem Code sind das die **3 wahrscheinlichsten Ursachen**:

1) DB-Schema passt nicht zu deinem neuen Model (sehr häufig!)

Du hast `CleaningTemplate` erweitert (`week`, `von_datum`, `bis_datum`, `assigned_to`, evtl. `is_active` schon vorher). Wenn deine Datenbank-Tabelle noch die alten Spalten hat, knallt es beim Insert/Query mit sowas wie:

- `OperationalError: table CLEANING_TEMPLATE has no column named week`
- `no such column: CLEANING_TEMPLATE.von_datum`
- oder `ForeignKey-Probleme`

✅ Fix (mit Flask-Migrate):

```
bash
```

```
flask db migrate -m "add week dates assigned_to to cleaning_template"
flask db upgrade
```

 Code kopieren

Wenn du kein Migrate nutzt und es nur ein Dev-Projekt ist (z.B. SQLite), oft am schnellsten:

- DB löschen (z.B. `app.db`) und neu erstellen / `db.create_all()` laufen lassen
(nur wenn du keine wichtigen Daten verlierst!)

2) request / redirect / url_for / flash nicht importiert

Wenn du oben im File nicht sowas hast, kommt sofort 500:

python

```
from flask import request, redirect, url_for, flash, render_template
```

 Code kopieren

3) Template crasht beim Anzeigen (z.B. strftime auf None)

Falls du im Template sowas machst:

jinja2

```
{{ eintrag.von_datum.strftime('%d.%m.%Y') }}
```

 Code kopieren

und du hast alte Datensätze mit `von_datum = NULL` (aus früheren Inserts), dann knallt es.

✓ Fix:

jinja2

```
{% if eintrag.von_datum and eintrag.bis_datum %}
    {{ eintrag.von_datum.strftime('%d.%m.%Y') }} - {{ eintrag.bis_datum.strftime('%d.%m.%Y') }}
{% else %}
    -
{% endif %}
```

⏪

 Code kopieren

Schnellster Weg zum *exakten* Fehler (ohne raten)

Starte Flask im Debug-Modus, dann siehst du die genaue Exception:

bash

```
flask --app app --debug run
```

 Code kopieren

Oder:

```
python
```

```
app.run(debug=True)
```

 Code kopieren

Und schau in die Konsole / Debug-Seite: die Zeile mit `OperationalError` / `NameError` etc. sagt dir sofort, welche der 3 Ursachen es ist.

Wenn du mir den **kompletten Stacktrace** aus der Konsole (die letzten ~20 Zeilen) hier rein kopierst, sag ich dir in 1 Satz, was es ist, und gebe dir den genauen Fix (Migration vs. Import vs. Template).

    ...

```
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py:22: SAWarning:
relationship 'ShoppingItem.added_by_user' will copy
column USER.user_id to column
SHOPPING_ITEM.added_by, which conflicts with
relationship(s): 'User.added_items' (copies USER.user_id to
SHOPPING_ITEM.added_by). If this is not the intention,
consider if these relationships should be linked with
back_populates, or if viewonly=True should be applied to
one or more if they are read-only. For the less common
case that foreign key constraints are partially overlapping,
the orm.foreign() annotation can be used to isolate the
columns that should be written towards. To silence this
warning, add the parameter 'overlaps="added_items"' to
the 'ShoppingItem.added_by_user' relationship.
(Background on this warning at:
https://sqlalche.me/e/20/qzyx) (This warning originated
from the configure_mappers() process, which was
invoked automatically in response to a user-initiated
operation.)
return cls.query_class(
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
```



```
return cls.query_class(
Benutzer: testuser
[2026-01-02 17:31:29,504] ERROR in app: Exception on
/putzplan/ [POST]
Traceback (most recent call last):
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
    self.dialect.do_execute(
        ~~~~~^
        cursor, str_statement, effective_parameters, context
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
    cursor.execute(statement, parameters)
```

```
~~~~~^  
sqlite3.OperationalError: no such column:  
CLEANING_TEMPLATE.week
```

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

```
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi_app  
    response = self.full_dispatch_request()  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full_dispatch_request  
    rv = self.handle_user_exception(e)  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full_dispatch_request  
    rv = self.dispatch_request()  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch_request  
    return  
self.ensure_sync(self.view_functions[rule.endpoint])  
(*view_args) # type: ignore[no-any-return]
```

```
~~~~~  
~~~~~^  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 111, in  
putzplan  
    putzplan_eintraege = (CleaningTemplate.query.all())  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\query.py", line 2704, in all
```

```
return self._iter().all() # type: ignore
```

```
~~~~~^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\query.py", line 2857, in _iter

```
result: Union[ScalarResult[_T], Result[_T]] =
```

```
self.session.execute(
```

```
~~~~~^
```

```
statement,
```

```
^^^^^^^^
```

```
params,
```

```
^^^^^^
```

```
execution_options={"_sa_orm_load_options":
```

```
self.load_options},
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
)
```

```
^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 2351, in execute

```
return self._execute_internal(
```

```
~~~~~^
```

```
statement,
```

```
^^^^^^^^
```

```
...<4 lines>...
```

```
_add_event=_add_event,
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
)
```

```
^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 2249, in _execute_internal

```
result: Result[Any] =
```

```
compile_state_cls.orm_execute_statement(
```

40/536

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1641, in
_execute_clauseelement

```
ret = self._execute_context(
    dialect,
    ...<8 lines>...
    cache_hit=cache_hit,
)
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1846, in
_execute_context

```
return self._exec_single_context(
    ~~~~~^
    dialect, context, statement, parameters
```

^^
^

```
)
^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1986, in
_exec_single_context

```
self._handle_dbapi_exception(
    ~~~~~^
    e, str_statement, effective_parameters, cursor, context
```

^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

```
)
^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 2363, in
_handle_dbapi_exception

```
raise sqlalchemy_exception.with_traceback(exc_info[2])
from e
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
```

```
    self.dialect.do_execute(
```

```
~~~~~^
```

```
        cursor, str_statement, effective_parameters, context
```

```
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
^^^^^^^^^^^^^^^^^^^^
```

```
)
```

```
^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
```

```
    cursor.execute(statement, parameters)
```

```
~~~~~^
```

```
sqlalchemy.exc.OperationalError: (sqlite3.OperationalError)
```

```
no such column: CLEANING_TEMPLATE.week
```

```
[SQL: SELECT "CLEANING_TEMPLATE".template_id AS
```

```
"CLEANING_TEMPLATE_template_id",
```

```
"CLEANING_TEMPLATE".wg_id AS
```

```
"CLEANING_TEMPLATE_wg_id",
```

```
"CLEANING_TEMPLATE".name AS
```

```
"CLEANING_TEMPLATE_name",
```

```
"CLEANING_TEMPLATE".description AS
```

```
"CLEANING_TEMPLATE_description",
```

```
"CLEANING_TEMPLATE".week AS
```

```
"CLEANING_TEMPLATE_week",
```

```
"CLEANING_TEMPLATE".von_datum AS
```

```
"CLEANING_TEMPLATE_von_datum",
```

```
"CLEANING_TEMPLATE".bis_datum AS
```

```
"CLEANING_TEMPLATE_bis_datum",
```

```
"CLEANING_TEMPLATE".is_active AS
```

```
"CLEANING_TEMPLATE_is_active",
```

```
"CLEANING_TEMPLATE".assigned_to AS
```

```
"CLEANING_TEMPLATE_assigned_to"
```

```
FROM "CLEANING_TEMPLATE"]
```

(Background on this error at:

<https://sqlalche.me/e/20/e3q8>) hier ist die error message

Der Fehler ist jetzt glasklar:

```
sqlite3.OperationalError: no such column: CLEANING_TEMPLATE.week
```

👉 Dein **SQLAlchemy-Model** hat `week`, aber deine **SQLite-Tabelle** `CLEANING_TEMPLATE` in der echten DB hat diese Spalte noch nicht. Du hast also das Model geändert, aber die Datenbank nicht migriert/aktualisiert. Deshalb knallt schon `CleaningTemplate.query.all()`.

Fix (je nachdem, ob dir Daten egal sind)

✅ A) Schnellster Fix (wenn es nur Dev ist und du Daten löschen darfst)

1. DB-Datei löschen (z.B. `app.db` oder wie sie heißt)
2. App starten und `db.create_all()` einmal ausführen (oder dein Init-Code, der Tabellen erstellt).

Wenn du eine Datei hast wie `instance/app.db` oder ähnlich: genau die löschen.

✅ B) Sauberer Fix: Migration (Flask-Migrate)

Wenn du Flask-Migrate nutzt:

```
bash
```

 Code kopieren

```
pip install flask-migrate
```

In deiner App einmalig (falls noch nicht):

```
python
```

 Code kopieren

```
from flask_migrate import Migrate
migrate = Migrate(app, db)
```

Dann:

```
bash
```

 Code kopieren

```
flask db init          # nur beim ersten Mal
flask db migrate -m "add week dates assigned_to to cleaning_template"
```

```
flask db upgrade
```

Danach existiert die Spalte `week` (und `von_datum`, `bis_datum`, `assigned_to`) wirklich in SQLite.

Nebenbei: deine SAWarnings (nicht der 500er, aber solltest du fixen)

Die Warnungen bedeuten: du hast doppelte/überlappende Relationships bei `ShoppingItem`. Lösung ist `back_populates` oder `overlaps=...`.

Beispiel (kurz):

```
python
```

[Code kopieren](#)

```
class User(db.Model):
    assigned_items = db.relationship("ShoppingItem", back_populates="assigned_to_

class ShoppingItem(db.Model):
    assigned_to_user = db.relationship("User", back_populates="assigned_items")
```

Oder (Quick-Silence):

```
python
```

[Code kopieren](#)

```
assigned_to_user = db.relationship("User", overlaps="assigned_items")
```

Warum passiert das genau beim Query?

Weil SQLAlchemy beim `SELECT` die Spalte `week` mit abfragt (laut Trace). SQLite sagt: "kenn ich nicht".

Wenn du mir sagst, wie deine DB-Datei heißt / wo sie liegt (Connection-String aus `SQLALCHEMY_DATABASE_URI`), sag ich dir exakt, welche Datei du löschen musst (Variante A) oder wie du Migration bei euch sauber einbaust (Variante B).




```

@app.route("/putzplan/", methods=['GET', 'POST'])
def putzplan():
    form = PutzplanForm()
    all_users = User.query.all()
    for user in all_users:
        print(f"Benutzer: {user.username}")
    if form.validate_on_submit():
        new_eintrag = CleaningTemplate(
            wg_id=1, # Get from session/logged-in user
            name=form.aufgabe.data,
            description=f"Week {form.woche.data}:
{form.von_datum.data} to {form.bis_datum.data}",
            frequency="weekly"
        )
        db.session.add(new_eintrag)
        db.session.commit()

        flash('Eintrag erfolgreich erstellt!', 'success')
        return redirect(url_for("putzplan"))
    elif form.is_submitted():
        # Form submitted but validation failed
        if form.zustaendig.errors:
            flash('WG-Mitglied existiert nicht', 'error')
        putzplan_eintraege = CleaningTemplate.query.all()
        return render_template("putzplan.html", form=form,
        putzplan=putzplan_eintraege, all_users=all_users)

db.py:
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True,
index=True)
    wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)
    #Beziehungen
    wg = db.relationship('Wg',

```

```

back_populates='cleaning_templates')
    tasks = db.relationship('CleaningTask',
back_populates='template', cascade='all, delete-orphan')

```

```

class CleaningTask(db.Model):
    __tablename__ = 'CLEANING_TASK'
    task_id = db.Column(db.Integer, primary_key=True,
index=True)
    template_id = db.Column(db.Integer,
db.ForeignKey('CLEANING_TEMPLATE.template_id'),
nullable=False)
    assigned_to = db.Column(db.Integer,
db.ForeignKey('USER.user_id'), nullable=False)
    status = db.Column(db.String, nullable=False)
    completed_at = db.Column(db.DateTime)
    notes = db.Column(db.String)
    #Beziehungen
    template = db.relationship('CleaningTemplate',
back_populates='tasks')
    assigned_user = db.relationship('User',
back_populates='cleaning_tasks')

```

```
<h1>Putzplan</h1>
```

```

{% with messages =
get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
                {{ message }}
            </div>
        {% endfor %}
    {% endif %}
{% endwith %}

```

```

<nav>
    <a href="{{ url_for('dashboard') }}">Start</a>
    <a href="{{ url_for('putzplan') }}">Putzplan</a>
    <a href="{{ url_for('innovation_board')

```

```

}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>

<input type="checkbox" id="modalToggle" style="display:
none;">

<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
  + Neuen Eintrag erstellen
</label>

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"></div>

<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
  <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;">×</label>
  <h2>Neuen Eintrag erstellen</h2>
  <h3>Bestimme, wer für was zuständig ist</h3>
  <form method="POST">
    {{ form.hidden_tag() }}

    <div style="margin-bottom: 15px;">
      <label for="woche">Kalenderwoche:</label>
      {{ form.woche(style="display: block; margin-top:
5px; padding: 8px; width: 100%;" ) }}
    </div>

    <div style="margin-bottom: 15px;">
      <label for="von_datum">Von:</label>
      {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;" ) }}
    </div>

    <div style="margin-bottom: 15px;">
      <label for="bis_datum">Bis:</label>

```

```

        {{ form.bis_datum(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="aufgabe">Aufgabe:</label>
        {{ form.aufgabe(style="display: block; margin-top: 5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="zustaendig">Zuständig:</label>
        <input list="users" name="zustaendig"
id="zustaendig" style="display: block; margin-top: 5px;
padding: 8px; width: 100%;" placeholder="z.B. Max"
required>
        <datalist id="users">
            {% for user in all_users %}
                <option value="{{ user.username }}"></option>
            {% endfor %}
        </datalist>
        {% if form.zustaendig.errors %}<span style="color:
red;">{{ form.zustaendig.errors[0] }}</span>{% endif %}
    </div>

    {{ form.submit() }}
</form>
</div>

<style>
    #modalToggle:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggle:checked ~ #modalContent {
        display: block !important;
    }
</style>

<table>
    <thead>
        <tr>
            <th>Datum</th>

```

```

        <th>Aufgabe</th>
        <th>Zuständig</th>
    </tr>
</thead>
<tbody>
    {% for eintrag in putzplan %}
    <tr>
        <td>{{ eintrag.datum }}</td>
        <td>{{ eintrag.aufgabe }}</td>
        <td>{{ eintrag.zustaendig }}</td>
    </tr>
    {% endfor %}
</tbody>
</table>

<style>
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    th, td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }
    th {
        background-color: #f2f2f2;
    }

```

</style> wie mache ich, dass er nach dem erstellen im modal einen eintrag hinzufügt auf der website

Du bist **eigentlich schon 90% da**: Du machst `db.session.commit()` und danach `redirect(url_for("putzplan"))` → das ist genau der Mechanismus, damit nach dem Speichern die Seite neu lädt und den neuen Datensatz anzeigt.

Der Grund, warum du **trotzdem nichts siehst**, ist:

1. Du speicherst in `CleaningTemplate` nur `name/description/frequency`

2. Im HTML versuchst du aber `eintrag.datum`, `eintrag.aufgabe`, `eintrag.zustaendig` anzuzeigen → **diese Felder existieren im Model nicht**
3. "Zuständig" gehört bei dir in `CleaningTask` (`assigned_to`) – du legst aber beim Erstellen **noch keinen Task an**.

So klappt's (mit deinem aktuellen Datenmodell)

✓ Ziel: Nach dem Modal-Speichern

- `CleaningTemplate` wird erstellt (Aufgabe + Zeitraum)
- `CleaningTask` wird erstellt (Zuständig = User)
- Tabelle zeigt: Zeitraum | Aufgabe | Zuständig

1) Route anpassen: Template + Task erstellen

python

 Code kopieren

```
from flask import request, redirect, url_for, flash, render_template

@app.route("/putzplan/", methods=["GET", "POST"])
def putzplan():
    form = PutzplanForm()
    all_users = User.query.all()

    if form.validate_on_submit():
        # Username aus dem datalist-input holen
        zustandig_name = request.form.get("zustandig", "").strip()
        user = User.query.filter_by(username=zustandig_name).first()

        if not user:
            flash("WG-Mitglied existiert nicht", "error")
        else:
            # 1) Template erstellen
            new_template = CleaningTemplate(
                wg_id=1,
                name=form.aufgabe.data,
                description=f"KW {form.woche.data}: {form.von_datum.data} bis {fo
                frequency="weekly",
                is_active=True
            )
```

```

db.session.add(new_template)
db.session.flush() # -> template_id ist jetzt verfügbar, ohne commit

# 2) Task erstellen (Zuständig)
new_task = CleaningTask(
    template_id=new_template.template_id,
    assigned_to=user.user_id,
    status="open",
    notes="Zuständig"
)
db.session.add(new_task)

db.session.commit()
flash("Eintrag erfolgreich erstellt!", "success")
return redirect(url_for("putzplan"))

putzplan_eintraege = (CleaningTemplate.query
    .filter_by(wg_id=1, is_active=True)
    .order_by(CleaningTemplate.template_id.desc())
    .all())

return render_template("putzplan.html", form=form, putzplan=putzplan_eintraeg

```

Wichtig:

- `db.session.flush()` sorgt dafür, dass `new_template.template_id` schon da ist, bevor du den Task anlegst.

2) HTML Tabelle an dein Modell anpassen (richtig anzeigen)

Du willst "Datum / Aufgabe / Zuständig".

Dein Datum steckt aktuell in `description`. Zuständig steckt in `tasks`.

Ersetze deine Tabelle durch diese:

html

 Code kopieren

```

<table>
  <thead>
    <tr>
      <th>Zeitraum</th>
      <th>Aufgabe</th>

```

```

        <th>Zuständig</th>
    </tr>
</thead>
<tbody>
    {% for eintrag in putzplan %}
    <tr>
        <td>{{ eintrag.description }}</td>
        <td>{{ eintrag.name }}</td>
        <td>
            {% if eintrag.tasks and eintrag.tasks[0].assigned_user %}
                {{ eintrag.tasks[0].assigned_user.username }}
            {% else %}
                -
            {% endif %}
        </td>
    </tr>
    {% endfor %}
</tbody>
</table>

```

Damit wird nach dem Speichern **automatisch** der neue Eintrag angezeigt (durch Redirect + neu geladene DB-Daten).

3) Optional: Modal bei Fehlern offen lassen

Falls User nicht existiert oder Validierung fehlschlägt:

html

 Code kopieren

```
<input type="checkbox" id="modalToggle" style="display: none;" {% if form.errors
```

Warum es vorher nicht ging (kurz)

- Du hast zwar gespeichert, aber die Seite zeigt falsche Felder (datum/aufgabe/zustaendig gibt es nicht im Model).
- "Zuständig" wird bei dir über `CleaningTask` abgebildet – du hast aber keinen Task erstellt.

Wenn du mir noch dein `user`-Model (wie heißt die Relationship `cleaning_tasks` genau?) zeigst, kann ich sicherstellen, dass `assigned_user.username` bei dir exakt so funktioniert.



```

class User(db.Model):
    __tablename__ = 'USER'
    user_id = db.Column(db.Integer, primary_key=True,
index=True)
    username = db.Column(db.String, nullable=False,
unique=True)
    password_hash = db.Column(db.String, nullable=False,
unique=True)
    email = db.Column(db.String, nullable=False,
unique=True)
    role = db.Column(db.String, nullable=False)
    wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=True)
    #Beziehungen
    wg = db.relationship('Wg', back_populates='users')
    cleaning_tasks = db.relationship('CleaningTask',
back_populates='assigned_user')
    activities_created = db.relationship('Activity',
back_populates='creator')
    ideas = db.relationship('Idea', back_populates='creator')
    idea_comments = db.relationship('Idea_Comment',
back_populates='user')
    idea_likes = db.relationship('Idea_Like',
back_populates='user')
    added_items = db.relationship('ShoppingItem',
foreign_keys='ShoppingItem.added_by')
    assigned_items = db.relationship('ShoppingItem',
foreign_keys='ShoppingItem.assigned_to')

```

Perfekt – damit passt meine Lösung **genau** (du hast `CleaningTask.assigned_user ↔ User.cleaning_tasks` schon korrekt mit `back_populates`).

Jetzt musst du nur sicherstellen, dass du beim Erstellen **auch wirklich einen** `CleaningTask` anlegst (weil "Zuständig" in deinem Datenmodell über `CleaningTask` läuft) **und** dass dein Template die richtigen Felder rendert.

1) Route: Template + Task erstellen (copy/paste)

python

 Code kopieren

```
from flask import request, redirect, url_for, flash, render_template

@app.route("/putzplan/", methods=["GET", "POST"])
def putzplan():
    form = PutzplanForm()
    all_users = User.query.all()

    if form.validate_on_submit():
        zustandig_name = request.form.get("zustandig", "").strip()
        user = User.query.filter_by(username=zustandig_name).first()

        if not user:
            flash("WG-Mitglied existiert nicht", "error")
        else:
            new_template = CleaningTemplate(
                wg_id=1,
                name=form.aufgabe.data,
                description=f"KW {form.woche.data}: {form.von_datum.data} bis {form.bis_datum.data}",
                frequency="weekly",
                is_active=True
            )
            db.session.add(new_template)
            db.session.flush() # template_id ist jetzt da

            new_task = CleaningTask(
                template_id=new_template.template_id,
                assigned_to=user.user_id,
                status="open",
                notes="Zuständig"
            )
            db.session.add(new_task)

            db.session.commit()
            flash("Eintrag erfolgreich erstellt!", "success")
            return redirect(url_for("putzplan"))

    putzplan_eintraege = (CleaningTemplate.query
                           .filter_by(wg_id=1, is_active=True)
                           .order_by(CleaningTemplate.template_id.desc())
                           .all())
```

```
return render_template("putzplan.html", form=form, putzplan=putzplan_eintraeg
```

2) HTML Tabelle: richtige Felder anzeigen

Deine bisherigen `eintrag.datum` / `eintrag.aufgabe` / `eintrag.zustaendig` gibt's nicht.

Nimm:

- Zeitraum: `eintrag.description` (da steckt bei dir KW+von/bis drin)
- Aufgabe: `eintrag.name`
- Zuständig: über Task → `eintrag.tasks[0].assigned_user.username`

html

 Code kopieren

```
<table>
  <thead>
    <tr>
      <th>Zeitraum</th>
      <th>Aufgabe</th>
      <th>Zuständig</th>
    </tr>
  </thead>
  <tbody>
    {% for eintrag in putzplan %}
    <tr>
      <td>{{ eintrag.description }}</td>
      <td>{{ eintrag.name }}</td>
      <td>
        {% if eintrag.tasks and eintrag.tasks[0].assigned_user %}
          {{ eintrag.tasks[0].assigned_user.username }}
        {% else %}
          -
        {% endif %}
      </td>
    </tr>
    {% endfor %}
  </tbody>
</table>
```

Wenn du später mehrere Tasks pro Template haben willst, zeig sie als Liste:

jinja2

 Code kopieren

```

<td>
  {% for task in eintrag.tasks %}
    {{ task.assigned_user.username }}{% if not loop.last %}, {% endif %}
  {% else %}
    -
  {% endfor %}
</td>

```

3) Kleiner Fix für deine ShoppingItem-Warnings (optional)

Bei dir fehlen `back_populates` für `added_items/assigned_items` → deshalb die `SAWarnings`.

Einfach in `ShoppingItem` entsprechend `back_populates` setzen **oder** `overlaps=...`.
(Nicht nötig für Putzplan, aber die Warnungen nerven.)

Wenn du willst, kann ich dir als nächsten Schritt zeigen, wie du `description` durch echte Spalten (`week`, `von_datum`, `bis_datum`) ersetzt **ohne** alles umzubauen (Migration + Template sauber formatieren).



es zeigt immer noch nichts an

Nachgedacht für 18s >

Wenn **gar nichts** erscheint, dann wird in 99% der Fälle **gar kein Eintrag gespeichert** (oder du fragst ihn direkt wieder weg). Bei dir ist der häufigste Grund:

Hauptursache: `form.validate_on_submit()` ist `False`, weil dein `zustaendig` -Feld nicht aus `WTForms` kommt

Du hast im Template:

html

Code kopieren

```
<input list="users" name="zustaendig" id="zustaendig" ...>
```

...aber wenn dein `PutzplanForm` ein Feld `zustaendig` hat (das deutet `form.zustaendig.errors` an), dann füllt **WTForms** dieses Feld **nur**, wenn du im Template auch `{{ form.zustaendig(...) }}` renderst (oder die `name`-Attribute exakt passen). Sonst bleibt es leer → Validierung schlägt fehl → der Code im `if form.validate_on_submit():` läuft nie → **nichts wird gespeichert**.

Fix 1 (empfohlen): `zustaendig` als echtes WTForms-Feld rendern (mit `datalist`)

Template: ersetze deinen `<input ... name="zustaendig">` Block durch:

html

 Code kopieren

```
<div style="margin-bottom: 15px;">
  <label for="zustaendig">Zuständig:</label>

  {{ form.zustaendig(
    list="users",
    id="zustaendig",
    style="display:block; margin-top:5px; padding:8px; width:100%; ",
    placeholder="z.B. Max"
  ) }}

  <datalist id="users">
    {% for user in all_users %}
      <option value="{{ user.username }}"></option>
    {% endfor %}
  </datalist>

  {% if form.zustaendig.errors %}
    <span style="color:red;">{{ form.zustaendig.errors[0] }}</span>
  {% endif %}
</div>
```

👉 Wichtig: Dann kommt `zustaendig` wirklich in `form.zustaendig.data` an.

Route: nimm dann den Namen aus dem Form (nicht aus `request.form`)

python

 Code kopieren

```
if form.validate_on_submit():
    zustandig_name = (form.zustandig.data or "").strip()
    user = User.query.filter_by(username=zustandig_name).first()
    ...
```

Fix 2 (quick): Validierung umgehen und POST direkt behandeln

Wenn du keine Lust hast, das Form zu ändern:

python

 Code kopieren

```
if request.method == "POST":
    # hier selbst alles prüfen und speichern
```

Dann bist du nicht von `validate_on_submit()` abhängig. (Ist aber weniger sauber als Fix 1.)

Zweiter häufiger Fehler: Du zeigst falsche Felder an

Dein Template zeigt aktuell noch `eintrag.datum / eintrag.aufgabe / eintrag.zustandig` – die gibt's bei `CleaningTemplate` nicht.

Nimm stattdessen (mindestens zum Testen!):

html

 Code kopieren

```
<tbody>
  {% for eintrag in putzplan %}
  <tr>
    <td>{{ eintrag.description }}</td>
    <td>{{ eintrag.name }}</td>
    <td>
      {% if eintrag.tasks %}
        {{ eintrag.tasks[0].assigned_user.username }}
      {% else %}
        -
      {% endif %}
    </td>
```

```
</tr>
{% endfor %}
</tbody>
```

Mini-Debug (10 Sekunden)

Setz irgendwo im Template hin:

html

 Code kopieren

```
<p>Einträge geladen: {{ putzplan|length }}</p>
```

- Wenn da **0** steht → es wird nix gespeichert oder du filterst es weg.
- Wenn da **>0** steht, aber die Tabelle leer wirkt → falsche Felder werden angezeigt.

Wenn du mir dein `PutzplanForm` (die Klasse) kurz hier reinkopierst, passe ich dir Route + Template 1:1 so an, dass das Speichern garantiert triggert und danach direkt ein Eintrag sichtbar ist.



es steht tatsächlich 0

Nachgedacht für 19s >

Wenn `Einträge geladen: 0` ist, heißt das ganz simpel: **in der DB ist (für deine Abfrage) nichts drin**. Und in deinem Fall ist der häufigste Grund:

✅ `form.validate_on_submit()` **läuft nie durch** → es wird **nichts gespeichert** → danach ist die Liste weiterhin 0.

Das passiert bei dir sehr wahrscheinlich, weil dein Feld `zustaendig` **nicht wirklich Teil des WTForms-Forms** ist (du renderst ein eigenes `<input name="zustaendig">`, aber prüfst `form.validate_on_submit() + form.zustaendig.errors`).

1) Sofort-Debug: zeig dir die Form-Fehler an

Pack das in deine Route direkt vor dem Rendern:

python

 Code kopieren

```
if request.method == "POST":
    print("POST angekommen")
    print("form.validate_on_submit():", form.validate_on_submit())
    print("form.errors:", form.errors)
    print("zustandig raw:", request.form.get("zustandig"))
```

Wenn `form.errors` z.B. `{'zustandig': ['This field is required.']}` oder CSRF Fehler enthält, weißt du sofort warum nichts gespeichert wird.

Sehr häufig: `{'csrf_token': ['The CSRF token is missing.']}` → dann fehlt `SECRET_KEY` oder `hidden_tag()` /CSRF stimmt nicht.

2) Fix (empfohlen): mach zustandig zu einem echten WTForms-Feld

PutzplanForm (Beispiel)

python

 Code kopieren

```
from wtforms import StringField
from wtforms.validators import DataRequired

class PutzplanForm(FlaskForm):
    # deine Felder...
    zustandig = StringField("Zuständig", validators=[DataRequired()])
    submit = SubmitField("Speichern")
```

Template: nutze form.zustandig statt eigenem <input>

Ersetze deinen Zuständig-Block durch:

html

 Code kopieren

```
<div style="margin-bottom: 15px;">
    <label for="zustandig">Zuständig:</label>

    {{ form.zustandig(
        list="users",
```



```

        id="zustaendig",
        style="display:block; margin-top:5px; padding:8px; width:100%;",
        placeholder="z.B. Max"
    ) }}
```

```

<datalist id="users">
    {% for user in all_users %}
        <option value="{{ user.username }}"></option>
    {% endfor %}
</datalist>
```

```

{% if form.zustaendig.errors %}
    <span style="color:red;">{{ form.zustaendig.errors[0] }}</span>
{% endif %}
</div>
```

Dann kommt der Wert sauber in `form.zustaendig.data` an und `validate_on_submit()` kann überhaupt `True` werden.

Route: nimm dann den Wert aus dem Form

python

 Code kopieren

```
zustaendig_name = (form.zustaendig.data or "").strip()
```

3) Ganz wichtiger Punkt: du zeigst in der Tabelle falsche Felder an

Du renderst aktuell:

jinja2

 Code kopieren

```

{{ eintrag.datum }}
{{ eintrag.aufgabe }}
{{ eintrag.zustaendig }}
```

Diese Felder gibt es bei `CleaningTemplate` nicht. Stell es (mindestens testweise) auf:

jinja2

 Code kopieren

```

<td>{{ eintrag.description }}</td>
<td>{{ eintrag.name }}</td>
<td>
    {% if eintrag.tasks %}
```

```

        {{ eintrag.tasks[0].assigned_user.username }}
    {% else %}
        -
    {% endif %}
</td>

```

4) Check: CSRF / SECRET_KEY

Wenn in `form.errors` sowas steht: `csrf_token missing`, dann brauchst du (einmalig):

python

 Code kopieren

```
app.config["SECRET_KEY"] = "irgendein-langer-geheimer-string"
```

(oder aus ENV laden)

Wenn du mir **deine** `PutzplanForm` **Klasse** (komplett) postest und sagst, was bei `print("form.errors", form.errors)` rauskommt, kann ich dir den exakten Fix ohne Raten direkt passend machen.

POST angekommen

[2026-01-02 17:54:27,216] ERROR in app: Exception on
/putzplan/ [POST]

Traceback (most recent call last):

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi_app
response = self.full_dispatch_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full_dispatch_request
rv = self.handle_user_exception(e)

File

```

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\app.py", line 902, in dispatch_request
    return
self.ensure_sync(self.view_functions[rule.endpoint])
(**view_args) # type: ignore[no-any-return]

~~~~~
~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\app.py", line 90, in
putzplan
    print("form.validate_on_submit():",
form.validate_on_submit()
                        ^^^^

UnboundLocalError: cannot access local variable 'form'
where it is not associated with a value der fehlercode

```

Der Fehler ist eindeutig:

UnboundLocalError: cannot access local variable 'form' where it is not associated with a value

Das passiert in Python, wenn du **innerhalb der Funktion irgendwo** `form = ...` **nochmal zuweist** (oder `form` in einem inneren Block/try/except überschreibst). Dann behandelt Python `form` als *lokale Variable*, aber du greifst **vor der Zuweisung** darauf zu (bei deinem `print(form.validate_on_submit())`).

✅ Fix (sofort)

1. Stell sicher, dass `form = PutzplanForm()` **vor** jedem Zugriff auf `form` steht und dass du **nirgendwo später** nochmal `form = ...` schreibst.

So sollte es aussehen:

python

 Code kopieren

```

@app.route("/putzplan/", methods=["GET", "POST"])
def putzplan():
    form = PutzplanForm()    # MUSS ganz oben sein

    if request.method == "POST":
        print("POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)
        print("zustaendig raw:", request.form.get("zustaendig"))

    all_users = User.query.all()

    if form.validate_on_submit():
        ...

```

2. Suche in deiner `putzplan()` Funktion nach sowas wie:

- `form = ...` **innerhalb** von `if/else/try`
- `for form in ...` (auch das überschreibt!)
- `except Exception as form:` (auch das überschreibt!)

Schon **eine** solche Stelle reicht.

Warum du "POST angekommen" siehst und dann trotzdem crasht

Weil `print("POST angekommen")` offenbar **vor** der Zeile mit `form.validate_on_submit()` steht — aber `form` ist zu dem Zeitpunkt laut Python "lokal aber noch nicht gesetzt" → Crash.

Danach: der eigentliche Grund, warum keine Einträge kommen

Sobald du den `UnboundLocalError` gefixt hast, wird `form.errors` dir sagen, warum `validate_on_submit()` `False` ist (oft `csrf_token` oder `zustaendig` fehlt, weil du es nicht als WTForms-Feld renderst).

Schnellste Hilfe

Kopier bitte **deine komplette** `putzplan()` **Funktion** so wie sie gerade in `app.py` steht (inkl. Debug-Prints) hier rein. Dann zeige ich dir exakt, **wo** du `form` überschreibst und

gebe dir die korrigierte Version 1:1 zurück.



```
form.validate_on_submit(): False
form.errors: {'datum': ['This field is required.']}
zustaendig raw: testuser
```

Perfekt – jetzt ist klar, warum **0 Einträge** gespeichert werden:

✅ `validate_on_submit()` ist **False**, weil dein WTForms-Form ein Pflichtfeld `datum` hat, aber du sendest **kein** `datum` (du sendest `woche`, `von_datum`, `bis_datum`, `aufgabe`, `zustaendig`).

Darum wird der `if form.validate_on_submit():` -Block nie ausgeführt → nichts wird gespeichert → `putzplan|length` bleibt 0.

Fix: `datum` aus dem Form entfernen ODER im Template mitsenden

Variante A (empfohlen): Entferne `datum` aus `PutzplanForm`

Wenn du eh mit `von_datum` / `bis_datum` arbeitest, brauchst du `datum` nicht.

In deiner `PutzplanForm`:

- lösche das Feld `datum` komplett **oder**
- mach es optional (ohne `DataRequired()`)

Beispiel optional:

python

Code kopieren

```
datum = DateField("Datum") # ohne DataRequired
```

Variante B: Füge `datum` ins Modal ein

Wenn du wirklich ein einzelnes Datum brauchst:

html

 Code kopieren

```
<div style="margin-bottom: 15px;">
  <label for="datum">Datum:</label>
  {{ form.datum(style="display:block; margin-top:5px; padding:8px; width:100%;")
</div>
```

<

>

Danach: Damit "Zuständig" sauber validiert wird

Du sendest `zustaendig raw: testuser`, aber WTForms kennt es nur, wenn du `form.zustaendig` renderst (oder du es nicht validierst).

Also entweder:

- mach `zustaendig` auch als WTForms-Feld und render es (empfohlen)
- oder lass es aus dem Form raus und hol es über `request.form.get("zustaendig")` (wie du es schon tust)

Bonus: Deine Tabelle zeigt noch falsche Felder

`eintrag.datum`, `eintrag.aufgabe`, `eintrag.zustaendig` existieren nicht in `CleaningTemplate`.

Zum Testen nimm:

html

 Code kopieren

```
<td>{{ eintrag.description }}</td>
<td>{{ eintrag.name }}</td>
<td>
  {% if eintrag.tasks %}
    {{ eintrag.tasks[0].assigned_user.username }}
  {% else %}
    -
  {% endif %}
</td>
```

Wenn du mir kurz deine `PutzplanForm` Klasse postest, sage ich dir exakt, welche Felder du löschen/optional machen musst, damit `validate_on_submit()` zuverlässig `True` wird.



```
<tr>
  <td>{{ eintrag.name }}</td>

  <td>{{ eintrag.description }}</td> wie mache ich dass
  hinter der aufgabe (description) die checkbox steht?
```

Du willst die Checkbox **in derselben Zelle direkt hinter der Description** anzeigen. Das machst du einfach, indem du Checkbox + Text in **eine** `<td>` packst (und optional ein `label` drumrum, damit man auf den Text klicken kann).

Variante 1: Checkbox + Description in einer Zelle (einfach)

html

Code kopieren

```
<tr>
  <td>{{ eintrag.name }}</td>

  <td>
    <label style="display:flex; align-items:center; gap:10px; margin:0;">
      <input type="checkbox">
      <span>{{ eintrag.description }}</span>
    </label>
  </td>
</tr>
```

Variante 2: Checkbox soll den Task-Status widerspiegeln (wenn du `eintrag.tasks` hast)

Wenn du pro Template einen Task hast (so wie bei dir geplant), dann:

html

Code kopieren

```

<tr>
  <td>{{ eintrag.name }}</td>

  <td>
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
    <label style="display:flex; align-items:center; gap:10px; margin:0;">
      <input type="checkbox" {% if task and task.status == 'completed' %}checked{%
      <span>{{ eintrag.description }}</span>
    </label>
  </td>
</tr>

```



- disabled = nur Anzeige (kein Klicken).
- Wenn du klickbar machen willst, sag kurz Bescheid, dann gebe ich dir die POST/Fetch-Variante zum Speichern des Status.

Wenn du willst, dass die Checkbox **rechts** vom Text steht, tausche einfach die Reihenfolge:

html

Code kopieren

```

<label style="display:flex; align-items:center; gap:10px;">
  <span>{{ eintrag.description }}</span>
  <input type="checkbox">
</label>

```



```
<h1>Putzplan</h1>
```

```

{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">

```



```

        {{ message }}
    </div>
    {% endfor %}
{% endif %}
{% endwith %}

<nav>
    <a href="{{ url_for('dashboard') }}">Start</a>
    <a href="{{ url_for('putzplan') }}">Putzplan</a>
    <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
    <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
    <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>

<input type="checkbox" id="modalToggle" style="display:
none;">

<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
    + Neuen Eintrag erstellen
</label>

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>

<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
    <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;"> × </label>
    <h2>Neuen Eintrag erstellen</h2>
    <h3>Bestimme, wer für was zuständig ist</h3>
    <form method="POST">
        {{ form.hidden_tag() }}

        <div style="margin-bottom: 15px;">
            <label for="woche">Kalenderwoche:</label>

```

```

        {{ form.woche(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="von_datum">Von:</label>
        {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="bis_datum">Bis:</label>
        {{ form.bis_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="aufgabe">Aufgabe:</label>
        {{ form.aufgabe(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
    </div>

    <div style="margin-bottom: 15px;">
        <label for="zustaendig">Zuständig:</label>

        {{ form.zustaendig(
            list="users",
            id="zustaendig",
            style="display:block; margin-top:5px; padding:8px;
width:100%;",
            placeholder="z.B. Max"
        ) }}

        <datalist id="users">
            {% for user in all_users %}
                <option value="{{ user.username }}"> </option>
            {% endfor %}
        </datalist>

        {% if form.zustaendig.errors %}
            <span style="color:red;">{{
form.zustaendig.errors[0] }}</span>
        {% endif %}

```

```
</div>

    {{ form.submit() }}
</form>
</div>

<style>
    #modalToggle:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggle:checked ~ #modalContent {
        display: block !important;
    }
</style>

<table>
    <thead>
        <tr>
            <th>Aufgabe</th>
            <th>Zeitraum</th>
            <th>Zuständig</th>
        </tr>
    </thead>

    <tbody>
        {% for eintrag in putzplan %}

            <tr>
                <td>
                    <label style="display:flex; align-items:center; gap:10px;
margin:0;">
                        <input type="checkbox">
                        <span>{{ eintrag.name }}</span>
                    </label>
                </td>

                <td>{{ eintrag.description }}</td>

                <td>
                    {% if eintrag.tasks and eintrag.tasks[0].assigned_user
%}
                        {{ eintrag.tasks[0].assigned_user.username }}
                    </td>
                </td>
            </tr>
        </tbody>
    </table>
```

```

        {% else %}
        -
        {% endif %}
    </td>

</tr>
{% endfor %}
</tbody>
</table>

```

```
<p>Einträge geladen: {{ putzplan|length }}</p>
```

```

<style>
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    th, td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }
    th {
        background-color: #f2f2f2;
    }

```

```
</style> wie mache ich diesen skript rein:
```

```
<h1>Putzplan</h1>
```

```

{% with messages =
get_flashed_messages(with_categories=true) %}
    {% if messages %}
        {% for category, message in messages %}
            <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
                {{ message }}
            </div>
        {% endfor %}
    {% endif %}

```

```
{% endwith %}
```

```
<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>
```

```
<input type="checkbox" id="modalToggle" style="display:
none;">
```

```
<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
  + Neuen Eintrag erstellen
</label>
```

```
<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>
```

```
<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
```

```
  <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;"> × </label>
```

```
  <h2>Neuen Eintrag erstellen</h2>
```

```
  <h3>Bestimme, wer für was zuständig ist</h3>
```

```
  <form method="POST">
```

```
    {{ form.hidden_tag() }}
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="woche">Kalenderwoche:</label>
```

```
      {{ form.woche(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
<div style="margin-bottom: 15px;">
  <label for="von_datum">Von:</label>
  {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="bis_datum">Bis:</label>
  {{ form.bis_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="aufgabe">Aufgabe:</label>
  {{ form.aufgabe(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="zustaendig">Zuständig:</label>

  {{ form.zustaendig(
    list="users",
    id="zustaendig",
    style="display:block; margin-top:5px; padding:8px;
width:100%;",
    placeholder="z.B. Max"
  ) }}

  <datalist id="users">
    {% for user in all_users %}
    <option value="{{ user.username }}"> </option>
    {% endfor %}
  </datalist>

  {% if form.zustaendig.errors %}
    <span style="color:red;">{{
form.zustaendig.errors[0] }}</span>
  {% endif %}
</div>

  {{ form.submit() }}
</form>
```

```
</div>

<style>
  #modalToggle:checked ~ #modalBackdrop {
    display: block !important;
  }

  #modalToggle:checked ~ #modalContent {
    display: block !important;
  }
</style>

<table>
  <thead>
    <tr>
      <th>Aufgabe</th>
      <th>Zeitraum</th>
      <th>Zuständig</th>
    </tr>
  </thead>

  <tbody>
    {% for eintrag in putzplan %}

      <tr>
        <td>
          <label style="display:flex; align-items:center; gap:10px;
margin:0;">
            <input type="checkbox">
            <span>{{ eintrag.name }}</span>
          </label>
        </td>

        <td>{{ eintrag.description }}</td>

        <td>
          {% if eintrag.tasks and eintrag.tasks[0].assigned_user
%}
            {{ eintrag.tasks[0].assigned_user.username }}
          {% else %}
            -
          {% endif %}
        </td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

```
</tr>
{% endfor %}
</tbody>
</table>

<p>Einträge geladen: {{ putzplan|length }}</p>

<style>
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
  }
  th, td {
    border: 1px solid #ddd;
    padding: 8px;
    text-align: left;
  }
  th {
    background-color: #f2f2f2;
  }

</style>
```

```
<h1>Putzplan</h1>
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
```



```
{% endif %}  
{% endwith %}
```

```
<nav>  
  <a href="{{ url_for('dashboard') }}">Start</a>  
  <a href="{{ url_for('putzplan') }}">Putzplan</a>  
  <a href="{{ url_for('innovation_board')  
  }}">InnovationBoard</a>  
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>  
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>  
</nav>
```

```
<input type="checkbox" id="modalToggle" style="display:  
none;">
```

```
<label for="modalToggle" style="cursor: pointer; padding:  
10px 20px; background-color: #007bff; color: white;  
border-radius: 5px; display: inline-block;">  
  + Neuen Eintrag erstellen  
</label>
```

```
<div style="position: fixed; top: 0; left: 0; width: 100%;  
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:  
none; z-index: 1;" id="modalBackdrop"> </div>
```

```
<div style="position: fixed; top: 50%; left: 50%; transform:  
translate(-50%, -50%); background-color: white; padding:  
30px; border-radius: 8px; width: 400px; z-index: 2; display:  
none;" id="modalContent">
```

```
  <label for="modalToggle" style="float: right; cursor:  
pointer; font-size: 24px; font-weight: bold;"> × </label>
```

```
  <h2>Neuen Eintrag erstellen</h2>
```

```
  <h3>Bestimme, wer für was zuständig ist</h3>
```

```
  <form method="POST">
```

```
    {{ form.hidden_tag() }}
```

```
    <div style="margin-bottom: 15px;">
```

```
      <label for="woche">Kalenderwoche:</label>
```

```
      {{ form.woche(style="display: block; margin-top:  
5px; padding: 8px; width: 100%;") }}
```

```
    </div>
```

```
<div style="margin-bottom: 15px;">
  <label for="von_datum">Von:</label>
  {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="bis_datum">Bis:</label>
  {{ form.bis_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="aufgabe">Aufgabe:</label>
  {{ form.aufgabe(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="zustaendig">Zuständig:</label>

  {{ form.zustaendig(
    list="users",
    id="zustaendig",
    style="display:block; margin-top:5px; padding:8px;
width:100%;",
    placeholder="z.B. Max"
  ) }}

  <datalist id="users">
    {% for user in all_users %}
    <option value="{{ user.username }}"> </option>
    {% endfor %}
  </datalist>

  {% if form.zustaendig.errors %}
    <span style="color:red;">{{
form.zustaendig.errors[0] }}</span>
  {% endif %}
</div>

{{ form.submit() }}
```

```
</form>
</div>

<style>
  #modalToggle:checked ~ #modalBackdrop {
    display: block !important;
  }

  #modalToggle:checked ~ #modalContent {
    display: block !important;
  }
</style>

<table>
  <thead>
    <tr>
      <th>Aufgabe</th>
      <th>Zeitraum</th>
      <th>Zuständig</th>
    </tr>
  </thead>

  <tbody>
    {% for eintrag in putzplan %}

      <tr>
        <td>
          <label style="display:flex; align-items:center; gap:10px;
margin:0;">
            <input type="checkbox">
            <span>{{ eintrag.name }}</span>
          </label>
        </td>

        <td>{{ eintrag.description }}</td>

        <td>
          {% if eintrag.tasks and eintrag.tasks[0].assigned_user
%}
            {{ eintrag.tasks[0].assigned_user.username }}
          {% else %}
            -
          {% endif %}
        </td>
      </tr>
    {% endfor %}
  </tbody>
</table>
```

```
</td>

</tr>
{% endfor %}
</tbody>
</table>

<p>Einträge geladen: {{ putzplan|length }}</p>

<style>
  table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
  }
  th, td {
    border: 1px solid #ddd;
    padding: 8px;
    text-align: left;
  }
  th {
    background-color: #f2f2f2;
  }

```

</style> wie mache ich in das obige skript das skript rein für eine progress bar, wenn jemand eine aufgabe erledigt hat: <div class="progress-stacked">

```
<div class="progress" role="progressbar" aria-label="Segment one" aria-valuenow="15" aria-valuemin="0" aria-valuemax="100" style="width: 15%">
  <div class="progress-bar"> </div>
</div>
<div class="progress" role="progressbar" aria-label="Segment two" aria-valuenow="30" aria-valuemin="0" aria-valuemax="100" style="width: 30%">
  <div class="progress-bar bg-success"> </div>
</div>
<div class="progress" role="progressbar" aria-label="Segment three" aria-valuenow="20" aria-valuemin="0" aria-valuemax="100" style="width: 20%">
  <div class="progress-bar bg-info"> </div>

```

```
</div>
</div>
```

Du kannst das ganz simpel so machen:

1. **Checkbox** klickbar machen und **Task-Status** speichern (POST)
2. **Progress-Bar** ausrechnen: $\text{completed_tasks} / \text{total_tasks} * 100$
3. Oben auf der Seite anzeigen.

Unten ist eine funktionierende Minimal-Variante (ohne Bootstrap nötig). Wenn du Bootstrap schon nutzt, sag's kurz – dann passe ich die Klassen exakt an.

1) Route: Task per Checkbox toggeln + Seite neu laden

Füge eine zweite Route hinzu:

python

```
@app.route("/putzplan/task/<int:task_id>/toggle", methods=["POST"])
def toggle_cleaning_task(task_id):
    task = CleaningTask.query.get_or_404(task_id)

    # Checkbox sendet "on" wenn gesetzt, sonst fehlt der Key
    is_checked = request.form.get("done") == "on"
    task.status = "completed" if is_checked else "open"
    db.session.commit()

    return redirect(url_for("putzplan"))
```

 Code kopieren

2) In deiner putzplan() Route: Progress Werte berechnen und ans Template geben

Ganz unten **vor** `render_template(...)`:

python

```
putzplan_eintraege = (CleaningTemplate.query
    .filter_by(wg_id=1, is_active=True)
    .order_by(CleaningTemplate.template_id.desc())
    .all())
```

```

all_tasks = CleaningTask.query.join(CleaningTemplate).filter(CleaningTemplate.wg_
total_tasks = len(all_tasks)
completed_tasks = sum(1 for t in all_tasks if t.status == "completed")

progress = int((completed_tasks / total_tasks) * 100) if total_tasks else 0

return render_template(
    "putzplan.html",
    form=form,
    putzplan=putzplan_eintraege,
    all_users=all_users,
    total_tasks=total_tasks,
    completed_tasks=completed_tasks,
    progress=progress
)

```

3) Progress-Bar in dein Template einbauen (oben unter der Nav)

 Code kopieren

Füge direkt nach `<nav>...</nav>` ein:

html

```

<div style="margin: 15px 0;">
  <div style="display:flex; justify-content:space-between; margin-bottom:6px;">
    <strong>Fortschritt</strong>
    <span>{{ completed_tasks }}/{{ total_tasks }} erledigt ({{ progress }}%)</spa
  </div>

  <div style="width:100%; background:#eee; border-radius:10px; overflow:hidden; h
    <div style="width: {{ progress }}%; height:18px; background: #28a745;"></div>
  </div>
</div>

```

Das ist die "Progress-Bar" ohne Bootstrap.

 Code kopieren

4) Checkbox so umbauen, dass sie wirklich einen Task toggelt

In deiner Tabelle: statt einer "toten" Checkbox brauchst du ein kleines Formular pro Task.

Ersetze den Checkbox-Teil in der Aufgabe-Spalte so:

html

```
<td>
  {% set task = eintrag.tasks[0] if eintrag.tasks else None %}

  {% if task %}
    <form method="POST" action="{% url_for('toggle_cleaning_task', task_id=task.t
      <label style="display:flex; align-items:center; gap:10px; margin:0;">
        <input type="checkbox"
          name="done"
          onchange="this.form.submit()"
          {% if task.status == 'completed' %}checked{% endif %}>
        <span>{{ eintrag.name }}</span>
      </label>
    </form>
  {% else %}
    {{ eintrag.name }}
  {% endif %}
</td>
```

Wenn du *genau* dein Bootstrap-Snippet nutzen willst

 Code kopieren

Dann brauchst du Bootstrap CSS im `<head>` (sonst sieht man nichts). Wenn du Bootstrap schon eingebunden hast, sag kurz "ja", dann baue ich dir die Progress-Bar im exakt selben Bootstrap-Style (`progress-stacked` , `progress-bar bg-success` usw.) passend zu deinen progress Werten.



ja

Nice — wenn Bootstrap schon drin ist, baust du die Progress-Bar so ein, dass sie sich automatisch aus deinen **Task-Statuswerten** berechnet.

1) Backend: Werte an Template geben (falls noch nicht)

In deiner `putzplan()` Route vor dem `render_template(...)`:

```
python

all_tasks = (CleaningTask.query
              .join(CleaningTemplate)
              .filter(CleaningTemplate.wg_id == 1)
              .all())

total_tasks = len(all_tasks)
completed_tasks = sum(1 for t in all_tasks if t.status == "completed")
progress = int((completed_tasks / total_tasks) * 100) if total_tasks else 0

return render_template(
    "putzplan.html",
    form=form,
    putzplan=putzplan_eintraege,
    all_users=all_users,
    total_tasks=total_tasks,
    completed_tasks=completed_tasks,
    progress=progress
)
```

2) Bootstrap Progress-Bar (progress-stacked) ins Template einfügen

Füge das direkt nach `<nav>...</nav>` ein:

 Code kopieren

```
html

{% set remaining = 100 - progress %}

<div class="mt-3 mb-4">
    <div class="d-flex justify-content-between align-items-center mb-2">
        <strong>Fortschritt</strong>
        <span class="text-muted">{{ completed_tasks }}/{{ total_tasks }} erledigt ({{
    </div>

    <div class="progress-stacked" style="height: 18px;">
        <!-- erledigt -->
        <div class="progress" role="progressbar"
            aria-label="Erledigt"
```



```

        aria-valuenow="{{ progress }}" aria-valuemin="0" aria-valuemax="100"
        style="width: {{ progress }}%">
    <div class="progress-bar bg-success"></div>
</div>

<!-- offen -->
<div class="progress" role="progressbar"
    aria-label="Offen"
    aria-valuenow="{{ remaining }}" aria-valuemin="0" aria-valuemax="100"
    style="width: {{ remaining }}%">
    <div class="progress-bar bg-secondary"></div>
</div>
</div>
</div>

```

Damit hast du 2 Segmente: grün = erledigt, grau = offen.

3) Checkbox muss den Task-Status ändern (sonst ändert sich die Bar nie)

 Code kopieren

Wenn du pro eintrag genau einen Task hast (dein eintrag.tasks[0]), nutze in der Tabelle diese klickbare Checkbox:

```

html

<td>
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}

    {% if task %}
        <form method="POST" action="{{ url_for('toggle_cleaning_task', task_id=task.t
        <label class="d-flex align-items-center gap-2 m-0">
            <input class="form-check-input m-0"
                type="checkbox"
                name="done"
                onchange="this.form.submit()"
                {% if task.status == 'completed' %}checked{% endif %}>
            <span>{{ eintrag.name }}</span>
        </label>
    </form>
    {% else %}
        {{ eintrag.name }}
    
```

```
{% endif %}
</td>
```

Und die Toggle-Route (falls noch nicht drin):

python

```
@app.route("/putzplan/task/<int:task_id>/toggle", methods=["POST"])
def toggle_cleaning_task(task_id):
    task = CleaningTask.query.get_or_404(task_id)
    task.status = "completed" if request.form.get("done") == "on" else "open"
    db.session.commit()
    return redirect(url_for("putzplan"))
```

 Code kopieren

Wenn du mehrere Tasks pro Eintrag hast (nicht nur [0]), sag kurz Bescheid — dann passe ich dir Tabelle + Fortschrittsberechnung so an, dass **alle** Tasks zählen und pro Eintrag mehrere Checkboxes angezeigt werden.



```
<td>
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}

    {% if task %}
        <form method="POST" action="{{
url_for('toggle_cleaning_task', task_id=task.task_id) }}"
class="m-0">
            <label class="d-flex align-items-center gap-2 m-0">
                <input class="form-check-input m-0"
                    type="checkbox"
                    name="done"
                    onchange="this.form.submit()"
                    {% if task.status == 'completed' %}checked{%
endif %}>
                <span>{{ eintrag.name }}</span>
            </label>
        </form>
    {% else %}
```

```

        {{ eintrag.name }}
    {% endif %}
</td> wo genau mache ich das rein?

```

Das kommt **genau an die Stelle**, wo du in deiner Tabelle aktuell die Aufgabe + Checkbox aus gibst.

Du hast momentan in deiner Tabelle ungefähr sowas:

html

 Code kopieren

```

<tr>
  <td>
    <label ...>
      <input type="checkbox">
      <span>{{ eintrag.name }}</span>
    </label>
  </td>
  <td>{{ eintrag.description }}</td>
  <td>... zuständig ...</td>
</tr>

```

👉 Diesen ersten `<td>...</td>` Block ersetzt du komplett durch den neuen Code.

So sieht der komplette Tabellen-Row dann aus (copy/paste)

html

 Code kopieren

```

<tr>
  <!-- Aufgabe + Checkbox (HIER rein!) -->
  <td>
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}

    {% if task %}
      <form method="POST"
        action="{{ url_for('toggle_cleaning_task', task_id=task.task_id) }}"
        class="m-0">
        <label class="d-flex align-items-center gap-2 m-0">
          <input class="form-check-input m-0"
            type="checkbox"
            name="done"
            onchange="this.form.submit()"

```

```

        {% if task.status == 'completed' %}checked{% endif %}>
        <span>{{ eintrag.name }}</span>
    </label>
</form>
{% else %}
    {{ eintrag.name }}
{% endif %}
</td>

<!-- Zeitraum -->
<td>{{ eintrag.description }}</td>

<!-- Zuständig -->
<td>
    {% if eintrag.tasks and eintrag.tasks[0].assigned_user %}
        {{ eintrag.tasks[0].assigned_user.username }}
    {% else %}
        -
    {% endif %}
</td>
</tr>

```

Wichtig

- Das funktioniert nur, wenn du die Route `toggle_cleaning_task` wirklich in Flask hast (die POST-Route).
- Du brauchst außerdem `{{ form.hidden_tag() }}` **nicht** in diesem Mini-Form, außer du hast CSRF-Schutz global für alle Forms aktiv. Wenn du CSRF aktiv hast, sag kurz Bescheid, dann füge ich dir den CSRF-Token in dieses Mini-Form richtig ein.



`style="width: {{ progress }}%">` property value expected

Das ist **kein Flask/Jinja-Fehler**, sondern nur dein Editor/CSS-Linter (z.B. VS Code) der bei Jinja in `style="..."` meckert, weil er `{{ ... }}` nicht als gültigen CSS-Wert parsen kann („property value expected“).

Zur Laufzeit im Browser ist das **okay**.

Wenn du den Hinweis trotzdem "sauber" machen willst, nimm diese Varianten:

Variante 1: `|int` + Semikolon (meist reicht's)

html

 Code kopieren

```
style="width: {{ progress|int }}%;"
```

und für remaining:

html

 Code kopieren

```
style="width: {{ remaining|int }}%;"
```

Variante 2: Fallback, falls `progress` mal leer ist

html

 Code kopieren

```
style="width: {{ (progress or 0)|int }}%;"
```

Variante 3: Linter beruhigen (optional)

In VS Code kannst du CSS-Validation für HTML/Jinja reduzieren/abschalten, weil er Templates nicht versteht.

Wenn du magst, paste kurz deinen exacten Progress-Bar Block, dann setze ich dir die zwei `style="width: ..."` Zeilen exakt korrekt für deinen Code.



at-rule or selector expected

Das kommt fast immer daher, dass du **HTML/Jinja-Code aus Versehen in einen `<style>...</style>` Block** (oder in eine `.css` Datei) gepackt hast. Ein CSS-Block erwartet nur CSS-Selektoren/Regeln – kein `<div>`, kein `{% ... %}`, kein `{{ ... }}`.



Fix

- Progress-Bar HTML gehört NICHT in `<style>` rein.
- In `<style>` kommt nur CSS wie `table { ... }` oder `.progress { ... }`.

So muss es strukturiert sein

1) HTML/Jinja Progress-Bar (normal im Body, z.B. unter `<nav>`)

html

```
{% set remaining = 100 - progress %}
```

```
<div class="mt-3 mb-4">
  <div class="d-flex justify-content-between align-items-center mb-2">
    <strong>Fortschritt</strong>
    <span class="text-muted">{{ completed_tasks }}/{{ total_tasks }} erledigt ({{
  </div>

  <div class="progress-stacked" style="height: 18px;">
    <div class="progress" role="progressbar" aria-valuenow="{{ progress|int }}"
      aria-valuemin="0" aria-valuemax="100" style="width: {{ progress|int }}%;
      <div class="progress-bar bg-success"></div>
    </div>

    <div class="progress" role="progressbar" aria-valuenow="{{ remaining|int }}"
      aria-valuemin="0" aria-valuemax="100" style="width: {{ remaining|int }}%;
      <div class="progress-bar bg-secondary"></div>
    </div>
  </div>
</div>
```

2) CSS bleibt in `<style>` (wenn du extra was brauchst)

html

```
<style>
  table { width: 100%; border-collapse: collapse; }
  th, td { border: 1px solid #ddd; padding: 8px; text-align: left; }
</style>
```

 Code kopieren

Wo genau ist es bei dir passiert?

Wenn du sowas hast wie:

```
html

<style>
  <div class="progress-stacked"> ... </div>
</style>
```

oder du hast die Progress-Bar in eine `.css` Datei kopiert → dann kommt genau `at-rule or selector expected`.

 Code kopieren

Wenn du mir den Abschnitt um deinen `<style>` Block (10–20 Zeilen davor/danach) hier rein kopierst, sag ich dir sofort, welche Zeile falsch platziert ist.



```
{% set remaining = 100 - progress %}
```

```
<div class="mt-3 mb-4">
  <div class="d-flex justify-content-between align-items-
center mb-2">
    <strong>Fortschritt</strong>
    <span class="text-muted">{{ completed_tasks }}/{{
total_tasks }} erledigt ({{ progress }}%)</span>
  </div>
```

```
<div class="progress-stacked" style="height: 18px;">
  <!-- erledigt -->
  <div class="progress" role="progressbar"
    aria-label="Erledigt"
    aria-valuenow="{{ progress }}" aria-valuemin="0"
aria-valuemax="100"
    style="width: {{ progress|int }}%;">
    <div class="progress-bar bg-success"> </div>
  </div>
```

```
<!-- offen -->
<div class="progress" role="progressbar"
```

```

        aria-label="Offen"
        aria-valuenow="{{ remaining }}" aria-valuemin="0"
    aria-valuemax="100"
    style="width: {{ remaining }}%">
    <div class="progress-bar bg-secondary"> </div>
</div>
</div>
</div>

```

Der Block ist **HTML/Jinja** und ist so grundsätzlich korrekt. Die Meldung „**at-rule or selector expected**“ bekommst du nur, wenn dieser Block **in einem** `<style>...</style>` -Bereich (oder in einer `.css` Datei) gelandet ist.

So baust du ihn richtig ein

1. Füge den Block in dein Template in den Body ein, z.B. direkt nach deinem `<nav>...`

`</nav>` :

html

 Code kopieren

```
<nav> ... </nav>
```

```
{% set remaining = 100 - progress %}
<div class="mt-3 mb-4">
    ...
</div>
```

2. Nicht in den `<style>` -Block packen. In `<style>` kommt nur CSS.

Kleiner Fix in deinem Code

In der zweiten "offen"-Leiste fehlt `|int` und das Semikolon. Mach es so konsistent:

html

 Code kopieren

```
style="width: {{ remaining|int }}%;"
```

Und optional auch `aria-valuenow` :

html

 Code kopieren


```
aria-valuenow="{{ remaining|int }}"
```

Finaler Block (sauber)

html

 Code kopieren

```
{% set remaining = 100 - progress %}
```

```
<div class="mt-3 mb-4">
  <div class="d-flex justify-content-between align-items-center mb-2">
    <strong>Fortschritt</strong>
    <span class="text-muted">{{ completed_tasks }}/{{ total_tasks }} erledigt ({{
  </div>

  <div class="progress-stacked" style="height: 18px;">
    <div class="progress" role="progressbar"
      aria-label="Erledigt"
      aria-valuenow="{{ progress|int }}" aria-valuemin="0" aria-valuemax="100"
      style="width: {{ progress|int }}%;">
      <div class="progress-bar bg-success"></div>
    </div>

    <div class="progress" role="progressbar"
      aria-label="Offen"
      aria-valuenow="{{ remaining|int }}" aria-valuemin="0" aria-valuemax="100"
      style="width: {{ remaining|int }}%;">
      <div class="progress-bar bg-secondary"></div>
    </div>
  </div>
</div>
```

Wenn du mir kurz zeigst, wo du den Block eingefügt hast (ein paar Zeilen davor/danach, inkl. `<style>`), sag ich dir exakt, welche Zeile raus muss bzw. wohin er gehört.



```
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\app.py", line 134, in
<genexpr>
    completed_tasks = sum(1 for t in putzplan_eintraege if
t.status == "completed")
                                ^^^^^^^^^^
AttributeError: 'CleaningTemplate' object has no attribute
'status' CleaningTemplate hat kein status attribut, aber
CleaningTask hat, was soll ich jetzt machen?
```

Genau: status gehört zu CleaningTask, nicht zu CleaningTemplate. Du musst den Fortschritt also über Tasks berechnen (oder über template.tasks), nicht über putzplan_eintraege.

Fix 1 (einfach & sauber): Fortschritt aus allen CleaningTask s berechnen

In deiner putzplan() Route ersetze die fehlerhafte Zeile durch das:

python

 Code kopieren

```
putzplan_eintraege = (CleaningTemplate.query
    .filter_by(wg_id=1, is_active=True)
    .order_by(CleaningTemplate.template_id.desc())
    .all())

tasks = (CleaningTask.query
    .join(CleaningTemplate)
    .filter(CleaningTemplate.wg_id == 1)
    .all())

total_tasks = len(tasks)
completed_tasks = sum(1 for t in tasks if t.status == "completed")
progress = int((completed_tasks / total_tasks) * 100) if total_tasks else 0
```

Und dann wie gehabt ans Template geben:

python

 Code kopieren

```

return render_template(...,
    total_tasks=total_tasks,
    completed_tasks=completed_tasks,
    progress=progress
)

```

Fix 2 (nur über putzplan_eintraege): Tasks pro Template mitzählen

Wenn du lieber ohne extra Query willst:

python

 Code kopieren

```

total_tasks = sum(len(t.tasks) for t in putzplan_eintraege)
completed_tasks = sum(
    1 for tpl in putzplan_eintraege for task in tpl.tasks
    if task.status == "completed"
)
progress = int((completed_tasks / total_tasks) * 100) if total_tasks else 0

```

Tipp

Wenn du auch "open" zählen willst, definiere z.B.:

- "open" und "completed"
- und setz beim Toggle genau diese Werte.

Wenn du magst, poste kurz den Abschnitt deiner putzplan() Route rund um die Progress-Berechnung, dann schreibe ich ihn dir 1:1 korrekt in deinen Code (inkl. wg_id Filter).



```

--#{$prefix}progress-height: #{$progress-height};
@include rfs($progress-font-size, --#{$prefix}progress-
font-size);
--#{$prefix}progress-bg: #{$progress-bg};

```

```
--#{$prefix}progress-border-radius: #{$progress-border-radius};
--#{$prefix}progress-box-shadow: #{$progress-box-shadow};
--#{$prefix}progress-bar-color: #{$progress-bar-color};
--#{$prefix}progress-bar-bg: #{$progress-bar-bg};
--#{$prefix}progress-bar-transition: #{$progress-bar-transition};
```

wie mache ich diese css skript in meine progress bar

Das Snippet ist **kein normales CSS**, sondern **Bootstrap-SCSS** (Sass) – erkennbar an `#{$prefix}` und `@include rfs(...)`. Das kannst du **nicht 1:1** in eine `.css` Datei oder in `<style>` kopieren.

Du hast zwei sinnvolle Wege:

Weg A (einfach): Du überschreibst Bootstrap per normalem CSS (empfohlen)

Du brauchst das SCSS nicht. Du kannst die Progress-Bar so anpassen:

html

```
<style>
  /* Höhe der Progressbar */
  .progress,
  .progress-stacked {
    height: 18px;
    border-radius: 10px;
  }

  /* Hintergrund der Progress-Leiste */
  .progress {
    background-color: #e9ecef;
  }

  /* Übergang (Animation) */
  .progress-bar {
    transition: width .6s ease;
```

```
}  
</style>
```

Das wirkt sofort auf deinen bestehenden Bootstrap-Progress.

 Code kopieren

Weg B (fortgeschritten): SCSS wirklich verwenden (kompilieren)

Nur wenn du Bootstrap selbst als SCSS baust:

1. `bootstrap.scss` in dein Projekt
2. deine Variablen dort setzen
3. SCSS → CSS kompilieren (z.B. mit `sass`)

Beispiel (SCSS):

```
scss  
  
$progress-height: 18px;  
$progress-border-radius: 10px;  
@import "bootstrap/scss/bootstrap";
```

 Code kopieren

Dann:

```
bash  
  
sass custom.scss static/custom.css
```

 Code kopieren

Und im HTML `custom.css` einbinden.

Was du konkret willst („diese Variablen in meine Progressbar“)

Hier ist die **CSS-Übersetzung**, die deinem Snippet am nächsten kommt (ohne SCSS):

```
html  
  
<style>  
  .progress {  
    --bs-progress-height: 18px;  
    --bs-progress-font-size: 0.9rem;  
    --bs-progress-bg: #e9ecef;  
    --bs-progress-border-radius: 10px;  
    --bs-progress-box-shadow: none;  
    --bs-progress-bar-color: #fff;
```

```

    --bs-progress-bar-bg: #198754;
    --bs-progress-bar-transition: width .6s ease;
    height: var(--bs-progress-height);
    border-radius: var(--bs-progress-border-radius);
  }
  .progress-bar {
    transition: var(--bs-progress-bar-transition);
  }
</style>

```

Damit nutzt du **Bootstrap CSS-Variablen** (die `--bs-...`), die Bootstrap 5 bereits versteht.

 Code kopieren

Wenn du mir sagst, welche Bootstrap-Version du nutzt (5.x?) und wie du Bootstrap einbindest (CDN oder lokale Datei), kann ich dir die beste Variante exakt für dein Setup geben.



ich will definitiv bootstrap benutzen

Alles klar — dann ist der wichtigste Punkt:

Dein Snippet ist **Bootstrap-SCSS**, nicht Bootstrap-CSS.

Wenn du Bootstrap "normal" benutzt (CDN oder fertige `bootstrap.min.css`), kannst du **nur CSS** schreiben (oder die **CSS-Variablen** überschreiben). SCSS mit `#{ $prefix }` / `@include` funktioniert dort nicht.

 **So machst du's richtig mit Bootstrap (ohne SCSS-Setup)**

1) Progressbar-HTML bleibt gleich

html

```

<div class="progress-stacked" style="height: 18px;">
  <div class="progress" style="width: {{ progress|int }}%;">
    <div class="progress-bar bg-success"></div>
  </div>

```

```
<div class="progress" style="width: {{ (100-progress)|int }}%;">
  <div class="progress-bar bg-secondary"></div>
</div>
</div>
```

 Code kopieren

2) Bootstrap CSS-Variablen überschreiben (genau dafür ist `--bs-...` da)

Pack das in deinen `<style>` Block oder in deine eigene `static/custom.css` nach dem Bootstrap-Import:

```
html

<style>
  /* Progressbar Bootstrap-Variablen überschreiben */
  .progress {
    --bs-progress-height: 18px;
    --bs-progress-font-size: 0.9rem;
    --bs-progress-bg: #e9ecef;
    --bs-progress-border-radius: 10px;
    --bs-progress-box-shadow: none;
    --bs-progress-bar-color: #fff;
    --bs-progress-bar-transition: width .6s ease;
  }

  /* optional: stacked auch abrunden */
  .progress-stacked {
    border-radius: 10px;
    overflow: hidden;
  }
</style>
```

✅ Das ist der "Bootstrap-Weg", ohne SCSS zu kompilieren.

✅ Wenn du *wirklich* die SCSS-Variante willst (Bootstrap  bauen)

 Code kopieren

Dann brauchst du ein SCSS-Setup (Sass Compiler). Kurz:

1. Install Sass:

```
bash

npm i -D sass bootstrap
```

 Code kopieren

2. custom.scss :

scss

```
$progress-height: 18px;  
$progress-border-radius: 10px;
```

```
@import "bootstrap/scss/bootstrap";
```

3. Kompilieren:

 Code kopieren

bash

```
npx sass custom.scss static/custom.css
```

 Code kopieren

4. In HTML:

html

```
<link rel="stylesheet" href="{% url_for('static', filename='custom.css') %}">
```

 Code kopieren

Welche Option willst du?

- **CDN/Standard Bootstrap CSS** → nimm **CSS-Variablen** (oben, empfohlen)
- **SCSS Build** → nimm Sass-Setup

Wenn du mir kurz sagst, ob du Bootstrap per **CDN** oder als **lokale Datei** eingebunden hast (1 Zeile `<link ...>`), sag ich dir exakt, wo du den CSS-Block platzieren musst.



wie mache ich das in venv

Wenn du Bootstrap mit **SCSS** in deinem Projekt nutzen willst (damit dein SCSS-Snippet funktioniert), brauchst du **Node/Sass** – das hat nichts mit Python- `venv` zu tun. Eine `venv` verwaltet nur Python-Pakete.

Du kannst aber **im gleichen Projektordner** (neben deiner `venv`) Sass/Bootstrap installieren und daraus eine CSS-Datei bauen, die Flask ausliefert.

Variante A (empfohlen): Sass/Bootstrap mit npm im Projekt

1. Terminal im Projektordner öffnen (da wo `app.py` liegt)
2. Node prüfen:

```
bash
```

[Code kopieren](#)

```
node -v
```

```
npm -v
```

3. Bootstrap + Sass installieren:

```
bash
```

[Code kopieren](#)

```
npm init -y
```

```
npm i bootstrap
```

```
npm i -D sass
```

4. SCSS-Datei anlegen: `assets/custom.scss`

```
scss
```

[Code kopieren](#)

```
$progress-height: 18px;
```

```
$progress-border-radius: 10px;
```

```
@import "bootstrap/scss/bootstrap";
```

5. Kompilieren nach Flask `static/` :

```
bash
```

[Code kopieren](#)

```
npx sass assets/custom.scss static/custom.css
```

6. In deinem Base-Template/HTML einbinden (am besten **statt** Bootstrap-CDN oder danach):

```
html
```

[Code kopieren](#)

```
<link rel="stylesheet" href="{% url_for('static', filename='custom.css') %}">
```

Tipp: Autowatch (bei Änderungen automatisch neu bauen):

```
bash
```

[Code kopieren](#)

```
npx sass --watch assets/custom.scss:static/custom.css
```

Variante B: "Nur venv" ohne npm (geht, aber unnötig kompliziert)

Du könntest einen Python-Sass-Compiler nutzen, aber das ist in der Praxis frickelig und nicht der Standard. Für Bootstrap-SCSS ist npm wirklich der richtige Weg.

Wichtig: venv trotzdem aktivieren?

Du kannst die venv parallel aktiv haben, aber npm ist unabhängig.

Windows PowerShell:

```
powershell
```

[Code kopieren](#)

```
.\venv\Scripts\Activate.ps1
```

Dann kannst du trotzdem `npm ...` ausführen (läuft separat).

Wenn du mir sagst, ob du **Windows** oder **Mac** nutzt (bei dir war's teils Mac, aber der Pfad sieht nach Windows aus), gebe ich dir die exakten Befehle inkl. Pfade passend zu deinem Projekt.



```
<style>
.progress {
  --bs-progress-height: 18px;
  --bs-progress-font-size: 0.9rem;
  --bs-progress-bg: #e9ecef;
  --bs-progress-border-radius: 10px;
  --bs-progress-box-shadow: none;
  --bs-progress-bar-color: #fff;
  --bs-progress-bar-bg: #198754;
  --bs-progress-bar-transition: width .6s ease;
  height: var(--bs-progress-height);
  border-radius: var(--bs-progress-border-radius);
}
.progress-bar {
```

```

        transition: var(--bs-progress-bar-transition);
    }
</style>

```

wo packe ich das im code hin in putzplan.html

Pack den `<style>...</style>` Block **einmal** in `putzplan.html` **außerhalb** deiner Tabelle/Form – am besten:

Option A (empfohlen): ganz unten vor dem schließenden `</body>`

Also **nach** deiner Tabelle (und nach deinem bisherigen `table { ... }` CSS), z.B.:

html

 Code kopieren

... deine Tabelle ...

```

<style>
    table {
        width: 100%;
        border-collapse: collapse;
        margin-top: 20px;
    }
    th, td {
        border: 1px solid #ddd;
        padding: 8px;
        text-align: left;
    }
    th {
        background-color: #f2f2f2;
    }
</style>

<!-- HIER direkt darunter einfügen -->
<style>
    .progress {
        --bs-progress-height: 18px;
        --bs-progress-font-size: 0.9rem;
        --bs-progress-bg: #e9ecef;
        --bs-progress-border-radius: 10px;
        --bs-progress-box-shadow: none;
        --bs-progress-bar-color: #fff;
        --bs-progress-bar-bg: #198754;
    }

```

```

    --bs-progress-bar-transition: width .6s ease;
    height: var(--bs-progress-height);
    border-radius: var(--bs-progress-border-radius);
  }
  .progress-bar {
    transition: var(--bs-progress-bar-transition);
  }
</style>

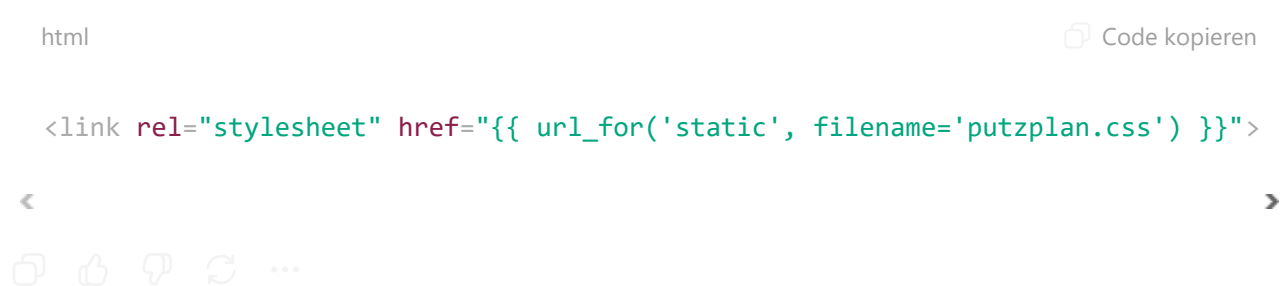
```

Option B: direkt nach dem `<h1>Putzplan</h1>`

Geht auch, ist aber unübersichtlicher, wenn das Template wächst.

Wichtig: Nicht in einen bestehenden `<style>` Block HTML reinmischen. Nur CSS.

Wenn du es "sauber" machen willst: Leg die Regeln in eine Datei `static/putzplan.css` und binde sie oben ein:



ich habe jetzt die sachen eingefügt aber nichts ist passiert

Nachgedacht für 12s >

Wenn "nichts passiert", liegt es fast immer an einem von diesen 3 Punkten:

1) Du nutzt Bootstrap 4.x (oder < 5) → `--bs-...` wirkt nicht

Die `--bs-progress-*` CSS-Variablen gibt's in Bootstrap 5. In Bootstrap 4 passiert damit optisch praktisch nichts.

✅ Lösung: Verwende "normales" CSS (funktioniert in allen Bootstrap-Versionen):

```
html

<style>
  .progress {
    height: 18px !important;
    background-color: #e9ecef !important;
    border-radius: 10px !important;
    overflow: hidden;
  }
  .progress-bar {
    transition: width .6s ease !important;
  }
</style>
```

Das kannst du **zusätzlich** oder **statt** der `--bs-...` Variante nutzen.

 Code kopieren

2) Dein CSS wird von Bootstrap überschrieben → Reihenfolge / Spezifität

Auch wenn `<style>` im Body meist gewinnt: manchmal überschreiben spätere Regeln.

✅ Lösung: `!important` (siehe oben) oder stelle sicher, dass dein CSS **nach** Bootstrap geladen wird (z.B. in `base.html` ganz unten im `<head>` oder in `static/custom.css` nach Bootstrap).

3) Du siehst die Progress-Bar gar nicht / falscher HTML-Block

Nur das CSS einzufügen ändert nichts, wenn die Progress-Bar **nicht** gerendert wird.

✅ Check: Siehst du im HTML wirklich sowas?

```
html

<div class="progress ...">
  <div class="progress-bar ..."></div>
</div>
```

 Code kopieren

Wenn nicht: erst Progress-Bar-HTML einfügen (z.B. direkt unter `<nav>`).

Schnellster Test (sagt dir sofort, ob dein CSS greift)

Setz testweise:

```
html
```

```
<style>
  .progress { outline: 3px solid red !important; }
</style>
```

 Code kopieren

Wenn du **keinen roten Rahmen** siehst, dann:

- Du hast **keine** `.progress` im DOM, oder
- du bist in der falschen Datei / falschem Template, oder
- Bootstrap/HTML ist anders benannt.

Bonus: Bootstrap-Version prüfen (10 Sekunden)

Schau in deinen `<link ... bootstrap ...>`:

- Wenn da `bootstrap@5...` steht → Variablen sollten gehen
- Wenn da `bootstrap@4...` steht → nimm die "normale CSS"-Variante mit `height/background/border-radius`.

Wenn du mir die eine Zeile zeigst, mit der du Bootstrap einbindest (`<link rel="stylesheet" ...>`), sag ich dir exakt, welche Variante du brauchst und wo du sie am saubersten platzierst.



css greift

Okay — wenn dein CSS greift, dann ist das Problem fast sicher: **du erwartest eine Änderung, die deine CSS-Regeln gar nicht verändern** (oder Bootstrap überschreibt genau die Werte, die du ändern willst).

Damit du sofort siehst "es passiert was", mach bitte diese zwei konkreten Änderungen:

1) Progress-Bar-Container abrunden + clipping aktivieren

Bei `progress-stacked` musst du `overflow: hidden;` setzen, sonst sieht man die Rundung oft nicht.

html

 Code kopieren

```
<style>
  .progress-stacked {
    height: 18px !important;
    border-radius: 10px !important;
    overflow: hidden !important;
  }
  .progress {
    height: 18px !important;
    background-color: #e9ecef !important;
  }
  .progress-bar {
    transition: width .6s ease !important;
  }
</style>
```

Wichtig: Das setzt **die sichtbare** Form/Größe. Deine `--bs-...` Variablen können "greifen", aber optisch bleibt es trotzdem gleich, wenn z.B. `progress-stacked` nicht gerundet/clipped ist.

2) Stelle sicher, dass `progress` wirklich `> 0` ist

Wenn `progress` `0` ist, bleibt die grüne Leiste unsichtbar und es wirkt "nichts passiert".

Füge testweise irgendwo neben der Bar ein:

html

 Code kopieren

```
<div>progress={{ progress }}, completed={{ completed_tasks }}, total={{ total_tasks }}
```

<

>

- Wenn `total_tasks` `0` ist → Progress ist immer `0` → nichts sichtbar.
- Wenn `progress` immer `0` bleibt → Checkbox-Toggle speichert nicht / zählt nicht.

3) Häufiger Bug: du setzt `width` am falschen Element

Bei Bootstrap ist die Breite normalerweise auf `.progress-bar` (nicht auf `.progress`).
Dein stacked-Snippet macht zwar Width auf `.progress` , aber wenn das Layout nicht so ist wie Bootstrap erwartet, kann es "komisch" wirken.

Nimm diese robuste Variante (Breite direkt auf die `.progress-bar`):

html

 Code kopieren

```
<div class="progress" style="height:18px; border-radius:10px; overflow:hidden;">
  <div class="progress-bar bg-success" style="width: {{ progress|int }}%;"></div>
</div>
```

Wenn du stacked willst:

html

 Code kopieren

```
<div class="progress-stacked" style="height:18px; border-radius:10px; overflow:hi
  <div class="progress">
    <div class="progress-bar bg-success" style="width: {{ progress|int }}%;"></di
  </div>
  <div class="progress">
    <div class="progress-bar bg-secondary" style="width: {{ (100-progress)|int }}%'
  </div>
</div>
```

Sag mir kurz, was bei dir angezeigt wird:

progress= ? completed=? total=?

Dann sag ich dir sofort, ob das Problem **Frontend** (CSS/HTML) oder **Backend** (Tasks zählen / Status speichern) ist.

ich will, dass der fortschritt in der bar angezeigt wird. wie mache ich das?

Du willst Text (z.B. "40%") direkt in der Progress-Bar sehen. Das geht mit Bootstrap ganz einfach: Du packst den Text in die `.progress-bar`.

Variante 1: Prozent in der grünen Leiste anzeigen (empfohlen)

html

 Code kopieren

```
<div class="progress" style="height: 18px; border-radius: 10px; overflow: hidden;
  <div class="progress-bar bg-success"
    role="progressbar"
    style="width: {{ progress|int }}%;"
    aria-valuenow="{{ progress|int }}" aria-valuemin="0" aria-valuemax="100">
    {{ progress|int }}%
  </div>
</div>
```

Damit steht der Text in der Bar.

Optional: Text zentrieren + gut lesbar machen

html

 Code kopieren

```
<div class="progress" style="height: 22px; border-radius: 10px; overflow: hidden;
  <div class="progress-bar bg-success d-flex align-items-center justify-content-c
    role="progressbar"
    style="width: {{ progress|int }}%;"
    aria-valuenow="{{ progress|int }}" aria-valuemin="0" aria-valuemax="100">
    {{ progress|int }}%
  </div>
</div>
```

Variante 2: Text immer sichtbar, auch bei 0–5% (wenn die grüne Bar zu schmal ist)

Dann legst du den Text über die Bar:

html

 Code kopieren

```

<div class="position-relative progress" style="height: 22px; border-radius: 10px;
  <div class="progress-bar bg-success"
    role="progressbar"
    style="width: {{ progress|int }}%;"
    aria-valuenow="{{ progress|int }}" aria-valuemin="0" aria-valuemax="100">
  </div>

  <div class="position-absolute top-50 start-50 translate-middle fw-semibold">
    {{ progress|int }}%
  </div>
</div>

```



Wo einfügen?

Genau dort, wo du aktuell deine Progress-Bar anzeigst (unter `<nav>`). Ersetze deinen bisherigen Progress-Bar-Block durch eine der Varianten.

Wenn du willst, kann ich dir auch die Version machen, die zusätzlich `completed/total` in der Bar zeigt (z.B. "3/7 · 43%").



```

<div style="margin-bottom: 15px;">
  <label for="idea_description">Beschreibung:
</label>
  {{ form.description(style="display: block; margin-top: 5px; padding: 8px; width: 100%; height: 100px;") }}
</div>

```

wie mache ich, dass er im eingabefeld oben links zu schreiben startet und die box ausfüllt, und nicht über den boxrand hinaus, sondern eine neue zeile anfängt

Du brauchst dafür ein **Textarea** (mehrzeiliges Feld). Dann startet der Cursor automatisch **oben links**, und Text bricht in die nächste Zeile um (statt "über den Rand" zu laufen).

1) In deinem WTForms: `TextAreaField` verwenden

python

```

from wtforms import TextAreaField
from wtforms.validators import DataRequired

description = TextAreaField("Beschreibung", validators=[DataRequired()])

```

 Code kopieren

2) Im Template: so rendern (mit Umbruch + Scroll)

html

```

<div style="margin-bottom: 15px;">
  <label for="idea_description">Beschreibung:</label>
  {{ form.description(
    style="display:block; margin-top:5px; padding:8px; width:100%; height:100px
  ) }}
</div>

```

<

>

Wichtig:

 Code kopieren

- TextAreaField erzeugt <textarea> → automatisch mehrzeilig, Cursor oben links.
- Text läuft nicht "über den Rand", sondern macht Zeilenumbruch. Wenn es mehr wird als die Höhe, gibt's Scroll (oder du erlaubst resize).

Falls es trotzdem ein normales Input ist

Dann ist es ein StringField. Den musst du wirklich zu TextAreaField ändern – ein <input> kann nicht mehrzeilig sein.



```

def innovation_board():
    form = InnovationForm()

    if request.method == 'POST':
        print("Innovation eingereicht")
        print("form.validate_on_submit():",
              form.validate_on_submit())
        print("form.errors:", form.errors)

    all_users = User.query.all()

```

```

        if form.validate_on_submit():
            flash("Innovation erfolgreich eingereicht!",
                "success")
            user =
            User.query.filter_by(username="testuser").first()
            new_idea = Idea(
                wg_id=1,
                created_by=user.user_id,
                title=form.title.data,
                description=form.description.data,
                created_at=db.func.now()
            )
            db.session.add(new_idea)
            db.session.commit()

            return redirect(url_for("innovation_board")) wie
            mache ich, dass der user angezeigt wird, der die idee
            erstellt hat?

```

Damit der **User** angezeigt wird, der die Idee erstellt hat, brauchst du zwei Dinge:

1. Relationship zwischen `Idea` und `User`
2. Im Template dann z.B. `idea.creator.username` ausgeben

1) Models: Relationship richtig setzen

Idea Model (wichtig: creator Relationship)

python

```

class Idea(db.Model):
    __tablename__ = "IDEA"
    idea_id = db.Column(db.Integer, primary_key=True)
    wg_id = db.Column(db.Integer, db.ForeignKey("WG.wg_id"), nullable=False)

    created_by = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=False)
    title = db.Column(db.String, nullable=False)
    description = db.Column(db.String, nullable=False)
    created_at = db.Column(db.DateTime, nullable=False)

```

```
creator = db.relationship("User", back_populates="ideas", foreign_keys=[creat
```

 Code kopieren

User Model (du hast ideas schon, aber so sollte es aussehen)

python

```
class User(db.Model):
    __tablename__ = "USER"
    user_id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String, nullable=False, unique=True)
    # ...

    ideas = db.relationship("Idea", back_populates="creator", foreign_keys="Idea.
```

<

>

 Code kopieren

2) Route: Ideen laden und ans Template geben

python

```
from sqlalchemy.orm import joinedload

@app.route("/innovation_board/", methods=["GET", "POST"])
def innovation_board():
    form = InnovationForm()

    if form.validate_on_submit():
        user = User.query.filter_by(username="testuser").first() # später aus se
        new_idea = Idea(
            wg_id=1,
            created_by=user.user_id,
            title=form.title.data,
            description=form.description.data,
            created_at=db.func.now()
        )
        db.session.add(new_idea)
        db.session.commit()
        flash("Innovation erfolgreich eingereicht!", "success")
        return redirect(url_for("innovation_board"))

    ideas = (Idea.query
```

```

        .filter_by(wg_id=1)
        .options(joinedload(Idea.creator))
        .order_by(Idea.created_at.desc())
        .all()

```

```

return render_template("innovation_board.html", form=form, ideas=ideas)

```

3) Template: Creator anzeigen

html

```

{% for idea in ideas %}
    <div>
        <h3>{{ idea.title }}</h3>
        <p>{{ idea.description }}</p>
        <small>Erstellt von: {{ idea.creator.username }}</small>
    </div>
{% endfor %}

```

 Code kopieren

Wenn du aktuell beim Ausgeben einen Fehler bekommst (z.B. `NoneType has no attribute username`), sag kurz, wie dein `Idea` Model genau aussieht (Spaltennamen + Tabellennamen), dann passe ich dir die Relationship exakt an.



kannst du user aus session machen bei user =?

Ja. Wenn du nach dem Login die `user_id` in der Session speicherst, kannst du den User in `innovation_board()` so holen.

1) Beim Login `user_id` in die Session schreiben

python

 Code kopieren

```
from flask import session

# nach erfolgreichem Login:
session["user_id"] = user.user_id
session["wg_id"] = user.wg_id # optional, aber praktisch
```

2) In innovation_board() den User aus der Session holen

python

 Code kopieren

```
from flask import session, redirect, url_for, flash

@app.route("/innovation_board/", methods=["GET", "POST"])
def innovation_board():
    form = InnovationForm()

    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    user = User.query.get(user_id) # oder db.session.get(User, user_id) bei SQLA

    if form.validate_on_submit():
        new_idea = Idea(
            wg_id=user.wg_id, # statt fix 1
            created_by=user.user_id,
            title=form.title.data,
            description=form.description.data,
            created_at=db.func.now()
        )
        db.session.add(new_idea)
        db.session.commit()
        flash("Innovation erfolgreich eingereicht!", "success")
        return redirect(url_for("innovation_board"))

    ideas = Idea.query.filter_by(wg_id=user.wg_id).order_by(Idea.created_at.desc())
    return render_template("innovation_board.html", form=form, ideas=ideas)
```

<

>

Tipp: kleines Helper-Funktionchen (optional)

python

 Code kopieren

```
def current_user():
    uid = session.get("user_id")
    return User.query.get(uid) if uid else None
```

Dann in jeder Route:

python

 Code kopieren

```
user = current_user()
if not user: ...
```

Wenn du willst, kann ich dir auch zeigen, wie du das als `@login_required` Decorator baust, damit du das nicht in jeder Route wiederholen musst.



wie mache ich einen löschen button für die ideen cards?

So baust du einen Löschen-Button für jede Idea-Card (sauber + sicher):

1) Route zum Löschen (POST, nicht GET)

python

```
from flask import session, redirect, url_for, flash, abort
from functools import wraps
```

```
def login_required(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        if not session.get("user_id"):
            flash("Bitte zuerst einloggen.", "error")
            return redirect(url_for("login"))
        return fn(*args, **kwargs)
    return wrapper

@app.route("/innovation_board/idea/<int:idea_id>/delete", methods=["POST"])
@login_required
def delete_idea(idea_id):
```



```

idea = Idea.query.get_or_404(idea_id)

current_user_id = session.get("user_id")

# optional: nur Ersteller darf löschen (oder Admin)
if idea.created_by != current_user_id:
    abort(403)

db.session.delete(idea)
db.session.commit()
flash("Idee gelöscht.", "success")
return redirect(url_for("innovation_board"))

```

Wenn du Admins erlauben willst: `or user.role == "admin"` (dafür müsstest du den User aus DB holen).

2) Button in der Card (Template)

 Code kopieren

In deiner `for idea in ideas` Schleife:

html

```

<form method="POST"
      action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
      style="display:inline;">
  <button type="submit"
        class="btn btn-sm btn-danger"
        onclick="return confirm('Idee wirklich löschen?');">
    Löschen
  </button>
</form>

```

 Code kopieren

3) Wenn du CSRF aktiviert hast (Flask-WTF)

Dann muss in jedes Mini-Form ein Token, sonst schlägt `POST` fehl.

Einfach in das Delete-Form:

html

```
<form method="POST" action="{ url_for('delete_idea', idea_id=idea.idea_id) }" >
  {{ csrf_token() }}
  <button type="submit" class="btn btn-sm btn-danger"
    onclick="return confirm('Idee wirklich löschen?');">Löschen</button>
</form>
```

 Code kopieren


Falls `csrf_token()` bei dir nicht verfügbar ist, sag kurz, wie du CSRF gerade nutzt (z.B. `form.hidden_tag()`), dann gebe ich dir die passende Variante.

Wenn du mir kurz sagst, wie dein Primary Key bei `Idea` heißt (`idea_id` ?), passe ich die `url_for(... idea_id=...)` Zeile exakt an dein Model an.



```
def innovation_board():
    form = InnovationForm()

    #user_id = session.get("user_id")
    #if not user_id:
    #    flash("Bitte zuerst einloggen.", "error")
    #    return redirect(url_for("login"))

    #user = User.query.get(user_id)

    if request.method == 'POST':
        print("Innovation eingereicht")
        print("form.validate_on_submit():",
              form.validate_on_submit())
        print("form.errors:", form.errors)

    all_users = User.query.all()

    if form.validate_on_submit():
        flash("Innovation erfolgreich eingereicht!",
              "success")

    user =
```

User.query.filter_by(username="testuser").first() #To be replaced with current_user() or session user

```

    new_idea = Idea(
        wg_id=1, #wg_id = user.wg_id
        created_by=user.user_id,
        title=form.title.data,
        description=form.description.data,
        created_at=db.func.now()
    )
    db.session.add(new_idea)
    db.session.commit()

    return redirect(url_for("innovation_board"))
else:
    flash("Fehler beim Einreichen der Innovation. Bitte
    überprüfen Sie die Eingaben.", "error")
    ideas = (Idea.query
        .filter_by(wg_id=1) #wg_id = user.wg_id
        .options(joinedload(Idea.creator))
        .order_by(Idea.created_at.desc())
        .all())
    #ideas =
    Idea.query.filter_by(wg_id=user.wg_id).order_by(Idea.created_at.desc()).all()
    return render_template("innovationboard.html",
    form=form, all_users=all_users,ideas=ideas) wieso wird
    hier immer "Fehler beim Einreichen der Innovation. Bitte
    überprüfen Sie die Eingaben." angezeigt

```

Mein HTML: {% with messages = get_flashed_messages(with_categories=true) %}

```

    {% if messages %}
        {% for category, message in messages %}
            <div class="alert-message {% if category == 'success' %}alert-success{% else %}alert-error{% endif %}"
            style="padding: 15px; margin-bottom: 20px; border-radius: 5px;" >
                {{ message }}
            </div>
        {% endfor %}
    {% endif %}
{% endwith %}

```

Weil dein `else: immer` ausgeführt wird, sobald `form.validate_on_submit()` **False** ist – und das ist es auch bei **jedem GET-Request** (also beim normalen Seitenaufruf/Redirect). Dadurch flashst du den Fehler praktisch ständig.

Was passiert gerade?

- GET `/innovation_board/` → `validate_on_submit()` ist False → du landest im `else:` → Fehler-Flash erscheint.
- Nach erfolgreichem POST machst du `redirect(url_for("innovation_board"))` → neuer GET → wieder `else:` → Fehler-Flash erscheint, obwohl alles ok war.

Fix: Fehler nur bei POST flashen

Ändere deine Logik so:

python

 Code kopieren

```
@app.route("/innovation_board/", methods=["GET", "POST"])
def innovation_board():
    form = InnovationForm()
    all_users = User.query.all()

    if request.method == "POST":
        print("Innovation eingereicht")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            user = User.query.filter_by(username="testuser").first()
            new_idea = Idea(
                wg_id=1,
                created_by=user.user_id,
                title=form.title.data,
                description=form.description.data,
                created_at=db.func.now()
            )
            db.session.add(new_idea)
            db.session.commit()
            flash("Innovation erfolgreich eingereicht!", "success")
            return redirect(url_for("innovation_board"))
```

```

else:
    flash("Fehler beim Einreichen der Innovation. Bitte überprüfen Sie di

ideas = (Idea.query
        .filter_by(wg_id=1)
        .options(joinedload(Idea.creator))
        .order_by(Idea.created_at.desc())
        .all())

return render_template("innovationboard.html", form=form, all_users=all_users

```

✅ Jetzt kommt der Fehler nur, wenn wirklich ein POST abgesendet wurde und die Validierung scheitert.

Extra Tipp (weil du debug schon druckst)

Wenn du wissen willst, *warum* es fehlschlägt, schau auf:

python

 Code kopieren

```
print(form.errors)
```

Häufige Ursachen:

- fehlender CSRF Token (dann brauchst du `{{ form.hidden_tag() }}` im `<form>`)
- ein required Feld fehlt (z.B. `title` / `description`)

Wenn du magst, poste einmal `form.errors` aus einem fehlgeschlagenen POST, dann sag ich dir exakt welches Feld fehlt.



wie mache ich meinen löschen button so, dass er das design von der card annimmt

Du willst, dass der **Löschen-Button** optisch "zur Card passt" (also nicht wie ein fremder roter Klotz wirkt). Mit Bootstrap geht das am besten so:

Option A (meistens am schönsten): "Trash"-Icon + btn-outline-danger (unauffällig)

In deiner Card-Header-Zeile (z.B. rechts oben):

html

 Code kopieren

```
<form method="POST"
      action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
      class="ms-auto">
  {{ csrf_token() if csrf_token else '' }}
  <button type="submit"
          class="btn btn-sm btn-outline-danger"
          onclick="return confirm('Idee wirklich löschen?');">
    🗑️ Löschen
  </button>
</form>
```

Wenn du Bootstrap Icons nutzt, statt Emoji:

html

 Code kopieren

```
<button type="submit" class="btn btn-sm btn-outline-danger">
  <i class="bi bi-trash"></i>
</button>
```

Option B: Button wie "Link" (passt oft perfekt in Cards)

html

 Code kopieren

```
<form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}">
  {{ csrf_token() if csrf_token else '' }}
  <button type="submit"
          class="btn btn-link text-danger p-0"
          onclick="return confirm('Idee wirklich löschen?');">
    Löschen
  </button>
</form>
```



Das sieht aus wie ein Card-Action-Link, nicht wie ein dicker Button.

Option C: Button in Card-Footer (klassisches Card-Design)

html

 Code kopieren

```
<div class="card-footer d-flex justify-content-end">
  <form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
    {{ csrf_token() if csrf_token else '' }}
    <button type="submit" class="btn btn-sm btn-outline-danger"
      onclick="return confirm('Idee wirklich löschen?');">
      Löschen
    </button>
  </form>
</div>
```

<

>

Wichtig: Card-Layout damit's "eingebettet" wirkt

Wenn du oben einen Header hast, mach ihn flex:

html

 Code kopieren

```
<div class="card-header d-flex align-items-center">
  <strong>{{ idea.title }}</strong>
  <form ... class="ms-auto">...</form>
</div>
```

Wenn du mir einen kurzen Ausschnitt deiner Card (HTML) zeigst (Header/Body/Footer), setze ich dir den Button exakt an die beste Stelle + passende Klassen.



```
<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: outline-block; margin-top:
20px;">
  + Neue Idee
```

</label> wie mache ich es dass der button oben rechts angezeigt wird

Du hast das Label aktuell einfach "normal im Flow". Damit es **oben rechts** sitzt, gibt's zwei saubere Varianten:

Variante A (am einfachsten): Flex-Container über h1 + Button

Pack h1 und dein Label in einen gemeinsamen Wrapper:

html

```
<div style="display:flex; justify-content:space-between; align-items:center; marg
  <h1 style="margin:0;">InnovationBoard</h1>

  <label for="modalToggle"
    style="cursor:pointer; padding:10px 20px; background-color:#007bff; colo
    + Neue Idee
  </label>
</div>
```



✓ Das ist responsiv und bleibt sauber.

Code kopieren

Nebenbei: display: outline-block; ist ein Tippfehler — richtig ist inline-block .

Variante B: Absolut oben rechts (fest an Ecke)

Wenn du es wirklich "oben rechts im View" willst:

html

```
<label for="modalToggle"
  style="position:fixed; top:20px; right:20px; cursor:pointer; padding:10px
  + Neue Idee
</label>
```



✓ Bleibt immer sichtbar beim Scrollen.

Code kopieren

Wenn du sagst, ob der Button **in der Seite oben rechts** (Variante A) oder **immer fixed im Fenster** (Variante B) sein soll, kann ich dir das direkt passend zu deinem bestehenden Layout (nav + h1) einsetzen.



```
<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>
```

```
<div style="display:flex; justify-content:space-between;
align-items:center; margin-bottom: 20px;">
  <h1 style="margin:0;">InnovationBoard</h1>
```

```
<label for="modalToggle"
      style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;">
  + Neue Idee
</label>
</div>
```

```
<h2>Teile deine Ideen zur Verbesserung des WG-
Lebens</h2>
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;" >
        {{ message }}
      </div>
```

```

    {% endfor %}
  {% endif %}
{% endwith %}

```

```

<div class="ideas-list">
  {% for idea in ideas %}
    <div class="idea-card">
      <form method="POST"
        action="{{ url_for('delete_idea',
idea_id=idea.idea_id) }}"
        style="display:inline;">
        <button type="submit"
          class="btn btn-link text-danger p-0"
          onclick="return confirm('Idee wirklich löschen?');">
            🗑️
        </button>
      </form>
      </form>
      <h3>{{ idea.title }}</h3>
      <p>{{ idea.description }}</p>
      <small>Erstellt von {{ idea.creator.username }} am {{
idea.created_at.strftime('%d.%m.%Y') }}</small>
    </div>
  {% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
  {% endfor %}

```

```

<input type="checkbox" id="modalToggle" style="display:
none;">

```

```

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>
<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
  <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold; margin-top:
10px;"> × </label>

```

```

<h2>Neue Idee einreichen</h2>
<h3>Schlage eine neue Idee für Deine WG vor.</h3>
<form method="POST">
  {{ form.hidden_tag() }}
  <div style="margin-bottom: 15px;">
    <label for="idea_title">Idee:</label>
    {{ form.title(style="display: block; margin-top: 5px;
padding: 8px; width: 100%;") }}
  </div>
  <div style="margin-bottom: 15px;">
    <label for="idea_description">Beschreibung:</label>
    {{ form.description(
      style="display:block; margin-top:5px;
padding:8px; width:100%; height:100px; resize: vertical;
white-space: pre-wrap;"
    ) }}
  </div>
  {{ form.submit() }}
</form>

</div>
<style>
  #modalToggle:checked ~ #modalBackdrop {
    display: block !important;
  }

  #modalToggle:checked ~ #modalContent {
    display: block !important;
  }
</style>

<table>
  <thead>
    <tr>
      <th>Titel</th>
      <th>Beschreibung</th>
      <th>Erstellt von</th>
      <th>Erstellt am</th>
    </tr>
  </thead>
  <tbody>
    {% for idea in ideas %}
    <tr>

```

```
<td>{{ idea.title }}</td>
<td>{{ idea.description }}</td>
<td>{{ idea.creator.username }}</td>
<td>{{ idea.created_at.strftime('%d.%m.%Y') }}
</td>
</tr>
{% else %}
<tr>
    <td colspan="4">Noch keine Ideen eingereicht. Sei
der Erste!</td>
</tr>
{% endfor %}
</tbody>
</table>
```

```
<style>
.ideas-list {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    margin-bottom: 30px;
}
.idea-card {
    background-color: #736d6d;
    border: 1px solid #000000;
    border-radius: 8px;
    padding: 15px;
    width: calc(33% - 20px);
    box-sizing: border-box;
}
.idea-card h3 {
    margin-top: 0;
}
</style>
```

```
<style>
table {
    width: 100%;
    border-collapse: collapse;
    margin-top: 20px;
}
th, td {
    border: 1px solid #ddd;
```

```
padding: 8px;
text-align: left;
}
th {
background-color: #f2f2f2;
}
```

</style> wie mache ich es, dass die beschreibung nicht aus der card geht. gerade geht sie über den rahmen der idea card hinaus

Das passiert, wenn der Text **keinen Umbruch** macht (z.B. bei langen Wörtern/URLs) oder wenn die Card keine Regeln fürs Wrapping hat. Fix das mit CSS für die Beschreibung (und optional für die ganze Card).

✅ 1) CSS: Text in der Card umbrechen + lange Wörter/Links brechen

Füg in deinen `.idea-card` Styles hinzu:

```
html

<style>
.idea-card p {
margin: 0.5rem 0;
white-space: pre-wrap;      /* Zeilenumbrüche aus Textarea behalten */
overflow-wrap: anywhere;    /* bricht sehr lange Wörter/URLs */
word-break: break-word;     /* Fallback */
}
</style>
```

✅ 2) Zusätzlich (optional): sicherstellen, dass die Card selbst ni

 Code kopieren

```
html

<style>
.idea-card {
overflow: hidden; /* falls irgendwo doch was übersteht */
}
</style>
```

 Code kopieren

Kleiner HTML-Fix bei dir (wichtig)

Du hast bei deinem Delete-Button ein **doppeltes** `</form>` :

```
html

<form ...>
  <button ...>🗑️</button>
</form>
</form> <!-- <- das muss weg -->
```

Lösch das zweite `</form>` , sonst kann Layout komisch werden.

 Code kopieren

Wenn du willst, dass die Beschreibung nach z.B. 5 Zeilen abgeschnitten wird (mit "..."), sag Bescheid — dann gebe ich dir die CSS-Variante mit `line-clamp` .



die box ist jetzt unnötig groß, die fixe ich das? ich will, dass es sich nach dem content anpasst

Deine Card ist "unnötig groß", weil du ihr eine **feste Breite** gibst (`width: calc(33% - 20px)`) und/oder weil sie als Block-Element die volle "Zeilenhöhe" mitnimmt. Wenn du willst, dass die Card **nur so hoch ist wie der Inhalt**, musst du vor allem sicherstellen, dass **keine feste Höhe/min-height** gesetzt ist (hast du nicht) – und dass die Card im Flex-Container **nicht stretched** wird.

✅ Fix: Flex-Items nicht auf gleiche Höhe "strecken"

Setz bei `.ideas-list` :

```
html

<style>
  .ideas-list {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
```

```
margin-bottom: 30px;
align-items: flex-start; /* <- wichtig: Cards bleiben so hoch wie Content */
}
</style>
```

✓ Fix: Card soll "shrink to content" in der Höhe

 Code kopieren

Das ist Standard, aber wir stellen sicher:

```
html

<style>
  .idea-card {
    background-color: #736d6d;
    border: 1px solid #000000;
    border-radius: 8px;
    padding: 15px;
    box-sizing: border-box;

    height: auto;          /* Standard, aber explizit */
    align-self: flex-start; /* extra sicher */
  }
</style>
```

Wenn du meinst: „unnötig breit“ statt „hoch“

 Code kopieren

Dann ist der Grund `width: calc(33% - 20px);`. Wenn du willst, dass die Card **nur so breit ist wie ihr Inhalt**, nimm stattdessen:

```
CSS

.idea-card {
  width: fit-content;
  max-width: 33%;
}
```

Oder (meist besser): du lässt 3 Spalten, aber passt es responsive an:

 Code kopieren

```
CSS

.idea-card { flex: 1 1 calc(33.333% - 20px); }
@media (max-width: 900px) { .idea-card { flex-basis: calc(50% - 20px); } }
```

```
@media (max-width: 600px) { .idea-card { flex-basis: 100%; } }
```

 Code kopieren

Sag kurz: Meinst du **zu hoch** oder **zu breit**? Dann gebe ich dir genau die passende CSS-Zeile (ohne Optionen).



die card sieht jetzt so aus, aber ich möchte dass sie sich an den content anpasst und nicht so groß ist

Nachgedacht für 39s >

Das sieht **sehr stark nach kaputtem HTML-Layout** aus (nicht nach "CSS macht die Card groß"). In deinem Code oben sind **zwei typische Ursachen**, die genau so ein Ergebnis produzieren:

1. Du hast ein **doppeltes** `</form>` (das zerschießt den DOM).
2. Du hast den Wrapper `<div class="ideas-list">` **vermutlich nicht sauber geschlossen** – dann "verschluckt" die erste Card den Rest der Seite (Modal, Tabelle, Styles ...) und wird riesig.

1) Korrigiere zuerst die Struktur (wichtig!)

Ersetze deinen gesamten Ideen-Block durch diesen (sauber geschlossen, kein extra `</form>`):

```
html
```

```
<div class="ideas-list">
  {% for idea in ideas %}
    <div class="idea-card">

      <div class="d-flex justify-content-end">
        <form method="POST"
              action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
              class="m-0">
          <button type="submit"
                  class="btn btn-link text-danger p-0"
                  onclick="return confirm('Idee wirklich löschen?');">
            
          </button>
        </form>
      </div>

      <h3>{{ idea.title }}</h3>
      <p class="idea-desc">{{ idea.description }}</p>
      <small>Erstellt von {{ idea.creator.username }} am {{ idea.created_at.strftime('%Y-%m-%d %H:%M') }}</small>
    </div>
  {% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
  {% endfor %}
</div>
```



➡ Achte drauf: ganz am Ende steht `</div>` für `.ideas-list`.

2) CSS so setzen, dass die Card wirklich "nur so groß wie Inhalt" ist

Dein `width: calc(33% - 20px)` macht sie breit (aber nicht hoch). Wenn du `width: 100%` setzt, richtet sie sich nach Content:

```
html
```

```
<style>
  .ideas-list{
    display: flex;
```

```
flex-wrap:wrap;
gap:20px;
align-items:flex-start;
}

.idea-card{
  display:inline-block;
  width: fit-content;      /* passt sich Inhalt an */
  max-width: 420px;       /* damit sie nicht riesig breit wird */
  background-color:#736d6d;
  border:1px solid #000;
  border-radius:8px;
  padding:15px;
  box-sizing:border-box;
}

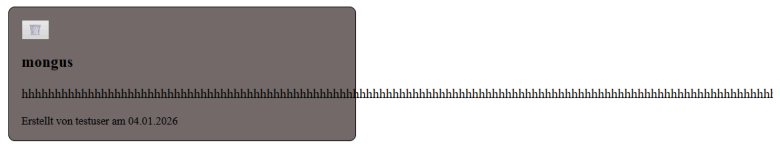
.idea-desc{
  white-space: pre-wrap;
  overflow-wrap: anywhere;
  word-break: break-word;
}
</style>
```

Warum ich sicher bin, dass es HTML ist

Wenn eine Card "komisch viel leeren Raum" hat, obwohl du keine `height` gesetzt hast, ist es fast immer: **nicht geschlossene Tags / falsche Verschachtelung** → die ( Code kopieren plötzlich viel mehr Elemente als gedacht (auch wenn sie z.B. durch Modal "unsichtbar" wirken).

Wenn du danach immer noch ein Problem siehst: schick mir kurz den HTML-Teil **ab** `<div class="ideas-list">` **bis kurz nach dem Modal**, dann sag ich dir genau, welcher Tag noch offen ist.





das sieht jetzt so aus

Ich sehe zwei Dinge im Screenshot:

1. Deine Card wird **super breit**, weil in der Beschreibung ein **sehr langes Wort ohne Leerzeichen** steht (hhhhhhhh...).
2. Du hast wahrscheinlich noch `width: fit-content;` drin – das sorgt dafür, dass die Card sich **an die Wortlänge anpasst** (und bei "Endlos-Wörtern" eben endlos breit wird).

✓ Fix: Card-Breite begrenzen + lange Wörter umbrechen

Entferne `width: fit-content;` und nutze stattdessen dieses CSS:

html

<style>

```
.ideas-list{
  display: flex;
  flex-wrap: wrap;
  gap: 20px;
  align-items: flex-start;
}
```

```
.idea-card{
  background-color: #736d6d;
  border: 1px solid #000;
  border-radius: 8px;
  padding: 15px;
  box-sizing: border-box;
```

```
/* wichtig: nicht fit-content! */
```

```
flex: 0 1 520px; /* Card ist "bis zu" 520px breit, schrumpft aber */
```

```
max-width: 100%; /* nie breiter als Bildschirm */
```

```
min-width: 0; /* wichtig in Flexbox, damit Text wirklich umbrechen darf
```

```
}
```

```

/* Text in der Beschreibung umbrechen, auch ohne Leerzeichen */
.idea-desc{
  white-space: pre-wrap;
  overflow-wrap: anywhere; /* bricht lange Strings/URLs */
  word-break: break-word; /* Fallback */
}
</style>

```

Und in deinem HTML die Beschreibung so (damit die CSS-Klasse greift):

html

```
<p class="idea-desc">{{ idea.description }}</p>
```

 Code kopieren

Warum das jetzt passiert

Ein String wie hhhhhhhhhhhh... hat **keine Leerzeichen**, also kann der Browser ohne `overflow-wrap: anywhere` nicht umbrechen. Und mit `width: fit-content` wächst die Card einfach mit.

Wenn du willst, dass die Cards **immer 3 pro Reihe** sind (statt "bis 520px"), sag kurz Bescheid – dann geb ich dir die `flex-basis: calc(33.33% - 20px)` Variante plus Wrap-Fix.

    ...



wie mache ich es, dass die cards im zentrum vom bildschirm sind

Damit die Cards **zentriert** sind, musst du den Container `.ideas-list` so einstellen, dass er die Items in der Zeile zentriert.

Füge/ändere dein CSS so:

html

```

<style>
  .ideas-list{
    display:flex;
    flex-wrap:wrap;
    gap:20px;

    justify-content: center; /* <-- zentriert die Cards horizontal */
    align-items: flex-start;

    /* optional: bisschen Luft links/rechts */
    padding: 20px;
  }
</style>

```

Wenn du zusätzlich willst, dass der ganze Bereich nicht über die volle Breite geht (sieht meist schöner aus), pack die Liste in einen Wrapper:

 Code kopieren

```

html

<div class="ideas-wrapper">
  <div class="ideas-list"> ...cards... </div>
</div>

<style>
  .ideas-wrapper{
    max-width: 1100px;
    margin: 0 auto;          /* <-- zentriert den gesamten Block */
  }
</style>

```

Kurz: justify-content: center; zentriert die Cards, margin: 0 auto; zentriert den Wrapper.

 Code kopieren

    ...

```

<div class="ideas-list">
  {% for idea in ideas %}
    <div class="idea-card">
      <div class="d-flex justify-content-end">

```

```

{% if idea.likes|length > 0 %}
    <span title="{{ idea.likes|length }} Likes"> 👍 {{
idea.likes|length }}</span>
{% endif %}
<div class="d-flex justify-content-end">
    <form method="POST"
        action="{{ url_for('delete_idea',
idea_id=idea.idea_id) }}"
        class="m-0">
        <button type="submit"
            class="btn btn-link text-danger p-0"
            onclick="return confirm('Idee wirklich
löschen?');">
            🗑️
        </button>
    </form>
</div>

<h3>{{ idea.title }}</h3>
<p class="idea-desc">{{ idea.description }}</p>
<small>Erstellt von {{ idea.creator.username }} am {{
idea.created_at.strftime('%d.%m.%Y') }}</small>
</div>
{% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
{% endfor %}
</div> ich würde gerne auch likes hinzufügen, wie geht
das?

```

Du bist schon nah dran – du zeigst `idea.likes|length` ja bereits an. Jetzt brauchst du nur:

1. Like-Model + Relationship
2. Route zum Liken/Unlikem
3. Button/Form in der Card
4. Optional: ein User darf nur 1 Like pro Idee

Unten ist eine saubere Standardlösung (Toggle: klicken = like/unlike).

1) Model: Idea_Like (falls du es nicht schon hast)

(Du hast `idea_likes` im User, daher hast du vermutlich schon sowas. Wenn nicht:)

python

 Code kopieren

```

class Idea_Like(db.Model):
    __tablename__ = "IDEA_LIKE"
    like_id = db.Column(db.Integer, primary_key=True)

    idea_id = db.Column(db.Integer, db.ForeignKey("IDEA.idea_id"), nullable=False)
    user_id = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=False)

    created_at = db.Column(db.DateTime, server_default=db.func.now())

    idea = db.relationship("Idea", back_populates="likes")
    user = db.relationship("User", back_populates="idea_likes")

    __table_args__ = (db.UniqueConstraint("idea_id", "user_id", name="uq_idea_use

```

Und in Idea :

python

 Code kopieren

```

class Idea(db.Model):
    # ...
    likes = db.relationship("Idea_Like", back_populates="idea", cascade="all, del

```

Und in User hast du schon:

python

 Code kopieren

```
idea_likes = db.relationship("Idea_Like", back_populates="user")
```

Wenn du Idea_Like schon hast, prüf nur, ob UniqueConstraint drin ist (damit kein Doppel-Like möglich ist).

2) Route: Like toggeln (POST)

python

 Code kopieren

```

from flask import session, redirect, url_for, flash
from sqlalchemy.exc import IntegrityError

```

```
@app.route("/ideas/<int:idea_id>/like", methods=["POST"])
```

```

def toggle_like(idea_id):
    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    existing = Idea_Like.query.filter_by(idea_id=idea_id, user_id=user_id).first()

    if existing:
        db.session.delete(existing)    # unlike
    else:
        db.session.add(Idea_Like(idea_id=idea_id, user_id=user_id)) # like

    try:
        db.session.commit()
    except IntegrityError:
        db.session.rollback()

    return redirect(url_for("innovation_board"))

```

3) Template: Like-Button in der Card

Wichtig: Dein HTML hat gerade ein Problem: `h3/p/small` liegen in deinem `d-flex justify-content-end` Container – dadurch wird das Layout kaputt.
Mach oben eine **Header-Zeile** nur für Like + Delete, danach Content.

So:

html

 Code kopieren

```

<div class="ideas-list">
    {% for idea in ideas %}
        <div class="idea-card">

            <div class="d-flex justify-content-between align-items-center mb-2">
                <!-- Like -->
                <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_i
                    <button type="submit" class="btn btn-sm btn-outline-primary">
                        👍 {{ idea.likes|length }}
                    </button>
                </form>
            
```



```

<!-- Delete -->
<form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_i
    <button type="submit" class="btn btn-link text-danger p-0"
        onclick="return confirm('Idee wirklich löschen?');">
        
    </button>
</form>
</div>

<h3 class="mb-1">{{ idea.title }}</h3>
<p class="idea-desc mb-2">{{ idea.description }}</p>
<small class="text-muted">Erstellt von {{ idea.creator.username }} am {{ id

</div>
{% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
{% endfor %}
</div>

```

4) Optional: Button anders anzeigen, wenn der User schon geliked hat

Dafür gibst du aus der Route ein Set der gelikten Idea-IDs an das Template:

Route:

python

 Code kopieren

```

user_id = session.get("user_id")
liked_ids = set()

if user_id:
    liked_ids = {l.idea_id for l in Idea_Like.query.filter_by(user_id=user_id).all()}

return render_template(..., liked_ids=liked_ids)

```

◀

▶

Template:

html

 Code kopieren

```
<button type="submit"
  class="btn btn-sm {{ 'btn-primary' if idea.idea_id in liked_ids else 'btn-outli
  👍 {{ idea.likes|length }}
</button>
```

Wenn du mir kurz zeigst, wie dein `Idea_Like` Model aktuell aussieht (falls vorhanden),
 passe ich dir den Code 1:1 an deine Tabellennamen/PKs an, damit es sofort läuft.



Traceback (most recent call last):

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
 am\Web-Development-Project\venv\Lib\site-
 packages\flask\app.py", line 1511, in wsgi_app
 response = self.full_dispatch_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
 am\Web-Development-Project\venv\Lib\site-
 packages\flask\app.py", line 919, in full_dispatch_request
 rv = self.handle_user_exception(e)

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
 am\Web-Development-Project\venv\Lib\site-
 packages\flask\app.py", line 917, in full_dispatch_request
 rv = self.dispatch_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
 am\Web-Development-Project\venv\Lib\site-
 packages\flask\app.py", line 902, in dispatch_request
 return

self.ensure_sync(self.view_functions[rule.endpoint])

(**view_args) # type: ignore[no-any-return]

~~~~~

~~~~~ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 266, in
 post_comment
 if content:
 ^^^^^^^

UnboundLocalError: cannot access local variable 'content'
 where it is not associated with a value

```
@app.route("/ideas/<int:idea_id>/comment", methods=
["POST"])
```

```
def post_comment(idea_id):
```

```
    form = CommentForm()
    #user_id = session.get("user_id")
    #if not user_id:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    if request.method == "POST":
        print("Kommentar POST angekommen")
        print("form.validate_on_submit():",
form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            content = request.form.get("content", "").strip()
            if content:
                new_comment = Idea_Comment(
                    idea_id=idea_id,
                    user_id="testuser", #To be replaced with
user_id=user_id
                    content=content,
                    created_at=db.func.now()
                )
                db.session.add(new_comment)
                db.session.commit()
                flash("Kommentar hinzugefügt.", "success")
            else:
                flash("Kommentar darf nicht leer sein.", "error")
```

```
    return redirect(url_for("innovation_board"),form=form)
```

Der Fehler kommt, weil `content` **nur innerhalb** von

python

```
if form.validate_on_submit():
    content = ...
```

 Code kopieren

gesetzt wird – aber du prüfst danach **immer** `if content:` . Wenn `validate_on_submit()` `False` ist, existiert `content` gar nicht → `UnboundLocalError` .

Außerdem ist deine Einrückung falsch: `if content:` muss **in** den Block rein oder du musst `content` vorher initialisieren.

Hier ist eine saubere, funktionierende Version (mit deinem Setup):

python

```
@app.route("/ideas/<int:idea_id>/comment", methods=["POST"])
def post_comment(idea_id):
    form = CommentForm()

    print("Kommentar POST angekommen")
    print("form.validate_on_submit():", form.validate_on_submit())
    print("form.errors:", form.errors)

    if form.validate_on_submit():
        content = (form.content.data or "").strip() # <-- WTForms nutzen

        if content:
            new_comment = Idea_Comment(
                idea_id=idea_id,
                user_id=1, # TODO: session.get("user_id")
                content=content,
                created_at=db.func.now()
            )
            db.session.add(new_comment)
            db.session.commit()
            flash("Kommentar hinzugefügt.", "success")
        else:
            flash("Kommentar darf nicht leer sein.", "error")
    else:
        flash("Kommentar konnte nicht gespeichert werden.", "error")
```

```
return redirect(url_for("innovation_board"))
```

2 wichtige Hinweise zu deinem Code

1. `user_id="testuser"` ist falsch, weil `user_id` bei dir ein **Integer FK** (`USER.user_id`) ist.
→ Nimm `user_id=session.get("user_id")` oder testweise `user_id=1`  Code kopieren
2. `return redirect(url_for("innovation_board"), form=form)` bringt nichts: `redirect` kann keine Template-Parameter mitnehmen.
→ Du machst `Redirect` und lädst die Daten im `innovation_board()` neu.

Damit es überhaupt validiert: `CommentForm` muss `content` haben und im HTML gerendert werden

Beispiel `CommentForm` :

python

```
class CommentForm(FlaskForm):
    content = TextAreaField("Kommentar", validators=[DataRequired()])
    submit = SubmitField("Senden")
```

 Code kopieren

Und im Template im Kommentar-Form:

html

```
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}">
    {{ form.hidden_tag() }}
    {{ form.content(class="form-control", rows="2") }}
    <button class="btn btn-sm btn-primary mt-2" type="submit">Kommentieren</button>
</form>
```

 Code kopieren

Wenn du mir `CommentForm` + dein Kommentar-HTML zeigst, passe ich es dir 1:1 korrekt an.



```

<label for="modalToggleComment{{ idea.idea_id }}"
      style="cursor:pointer; padding:5px 10px;
background-color:#28a745; color:white; border-radius:5px;
display:inline-block; font-size: 0.8rem;">
    + Kommentar
</label>
<form method="POST" action="{{
url_for('post_comment', idea_id=idea.idea_id) }}" class="m-
0">
    <input type="text" name="comment_text"
placeholder="Kommentar hinzufügen..." required
      style="padding: 5px; width: 70%;">
    <button type="submit" class="btn btn-sm btn-
outline-secondary">Kommentieren</button>

</form> wie mache ich es, dass man die kommentare
öffnen kann von einer idea card und selbst eins verfassen
kann?

```

Du brauchst dafür (wie bei deinem Modal) pro Idea eine **eigene "aufklappbare" Kommentar-Sektion**. Das geht super ohne JS mit `details/summary` (am einfachsten) **oder** mit deinem Checkbox-Modal-Pattern. Ich zeig dir beide – nimm Variante A, die ist am saubersten.

Variante A: `details/summary` (aufklappen + schreiben + Kommentare anzeigen)

Direkt in deiner Idea-Card:

html

```

<details class="mt-2">
  <summary style="cursor:pointer; color:#28a745; font-weight:600;">
    💬 Kommentare ({{ idea.comments|length if idea.comments else 0 }})
  </summary>

  <!-- bestehende Kommentare -->

```

```

<div style="margin-top:10px;">
  {% for c in idea.comments %}
    <div style="border-top:1px solid #ddd; padding-top:6px; margin-top:6px;">
      <small><strong>{{ c.user.username }}</strong> · {{ c.created_at.strftime(
      <div>{{ c.content }}</div>
    </div>
  {% else %}
    <small class="text-muted">Noch keine Kommentare.</small>
  {% endfor %}
</div>

<!-- Kommentar schreiben -->
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
  {{ form.hidden_tag() if form else '' }}
  <div class="d-flex gap-2 mt-2">
    <input type="text"
      name="content"
      class="form-control"
      placeholder="Kommentar hinzufügen..."
      required>
    <button type="submit" class="btn btn-sm btn-outline-secondary">Senden</butt
  </div>
</form>
</details>

```

Wichtig: Dein Backend muss dann `request.form.get("content")` lesen (nicht `comment_text`).

Also in `post_comment`:

```
python
```

```
content = (request.form.get("content") or "").strip()
```

 Code kopieren

Variante B: Checkbox-Toggle (wie dein Modal, aber unter der Card)

Wenn du lieber bei deinem Pattern bleibst:

```
html
```

```
<input type="checkbox" id="toggleComments{{ idea.idea_id }}" style="display:none;
```

```

<label for="toggleComments{{ idea.idea_id }}"
      class="btn btn-sm btn-outline-success mt-2">
  + Kommentar / Anzeigen
</label>

<div class="comments-box mt-2" id="commentsBox{{ idea.idea_id }}">
  <!-- Kommentare -->
  {% for c in idea.comments %}
    <div class="border-top pt-2 mt-2">
      <small><strong>{{ c.user.username }}</strong> · {{ c.created_at.strftime('%
      <div>{{ c.content }}</div>
    </div>
  {% else %}
    <small class="text-muted">Noch keine Kommentare.</small>
  {% endfor %}

  <!-- schreiben -->
  <form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}">
    <div class="d-flex gap-2">
      <input type="text" name="content" class="form-control" placeholder="Komment
      <button type="submit" class="btn btn-sm btn-outline-secondary">Senden</butt
    </div>
  </form>
</div>

<style>
  /* default zu */
  #toggleComments{{ idea.idea_id }} ~ #commentsBox{{ idea.idea_id }} { display: n
  /* offen */
  #toggleComments{{ idea.idea_id }}:checked ~ #commentsBox{{ idea.idea_id }} { di
</style>

```

Achtung: Diese CSS-Regel funktioniert nur, wenn `input`, `label`, `div` echte Geschwister sind (gleiche Ebene in der Card).

Backend: Kommentare beim Laden mitgeben (sonst zeigt er nichts)

Damit `idea.comments` funktioniert, brauchst du die Relationship und in de  Code kopieren
`joinedload :`

python


```
ideas = (Idea.query
        .filter_by(wg_id=1)
        .options(joinedload(Idea.creator),
                 joinedload(Idea.comments).joinedload(Idea_Comment.user))
        .order_by(Idea.created_at.desc())
        .all())
```

Und im Model:

 Code kopieren

python

```
class Idea(db.Model):
    comments = db.relationship("Idea_Comment", back_populates="idea", cascade="all, delete, delete-orphan")

class Idea_Comment(db.Model):
    idea = db.relationship("Idea", back_populates="comments")
    user = db.relationship("User") # oder back_populates, wenn du willst
```

<

>

 Code kopieren

Mini-Fix zu deinem aktuellen HTML

Du nutzt `name="comment_text"` , aber mein/typischer Backend-Code erwartet `content` .
Entscheide dich für eins:

- HTML: `name="content"` 
- Backend: `request.form.get("content")`

Wenn du mir kurz sagst, wie dein `Idea_Comment` Model heißt (Spalten + Relationships) und wie du `ideas` aktuell lädst, passe ich dir Query + Template exakt an dein Projekt an, damit Kommentare sofort angezeigt und gespeichert werden.

    ...

<details class="mt-2">

<summary style="cursor:pointer; color:#28a745; font-weight:600;">

 Kommentare ({{ idea.comments|length if

```

idea.comments else 0 }})
</summary>

<div style="margin-top:10px;">
  {% for c in idea.comments %}
    <div style="border-top:1px solid #ddd;
padding-top:6px; margin-top:6px;">
      <small> <strong>{{ c.user.username }}
</strong> · {{ c.created_at.strftime('%d.%m.%Y %H:%M') }}
</small>

      <div>{{ c.content }}</div>
    </div>
  {% else %}
    <small class="text-muted">Noch keine
Kommentare.</small>
  {% endfor %}
</div>

<form method="POST" action="{{
url_for('post_comment', idea_id=idea.idea_id) }}" class="m-
0">
  <input type="text" name="comment_text"
placeholder="Kommentar hinzufügen..." required
style="padding: 5px; width: 70%;">
  <button type="submit" class="btn btn-sm btn-
outline-secondary">Kommentieren</button>

</form>
</div>

<h3 class="mb-1">{{ idea.title }}</h3>
<p class="idea-desc mb-2">{{ idea.description }}</p>
<small class="text-muted">Erstellt von {{
idea.creator.username }} am {{
idea.created_at.strftime('%d.%m.%Y') }}</small>

</div>
{% else %}
  <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
{% endfor %}
</div>

```

kannst du mir diesen code anpassen an variante A

Ja — dein Snippet ist fast Variante A, aber es hat 3 Probleme:

1. `</details>` fehlt (und bei dir steht ein extra `</div>` drin)
2. Du verwendest im Form `name="comment_text"` → ich würde auf `name="content"` umstellen (sauberer).
3. Der Kommentarbereich muss **innerhalb der Card** bleiben, aber **vor** dem restlichen Content oder an der Stelle, wo du ihn willst.

Hier ist die **saubere Variante A** (copy/paste), passend zu deinem Aufbau:

html

 Code kopieren

```
<div class="idea-card">

  <!-- Kommentar-Block (Variante A: details/summary) -->
  <details class="mt-2">
    <summary style="cursor:pointer; color:#28a745; font-weight:600;">
      💡 Kommentare ({{ idea.comments|length if idea.comments else 0 }})
    </summary>

    <!-- bestehende Kommentare -->
    <div style="margin-top:10px;">
      {% for c in idea.comments %}
        <div style="border-top:1px solid #ddd; padding-top:6px; margin-top:6px;">
          <small>
            <strong>{{ c.user.username }}</strong>
            · {{ c.created_at.strftime('%d.%m.%Y %H:%M') }}
          </small>
          <div>{{ c.content }}</div>
        </div>
      {% else %}
        <small class="text-muted">Noch keine Kommentare.</small>
      {% endfor %}
    </div>

    <!-- Kommentar schreiben -->
    <form method="POST"
      action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
      style="margin-top:10px;">
      <div style="display:flex; gap:10px; align-items:center;">
        <input type="text"
          name="content"
          placeholder="Kommentar hinzufügen..."
```

```

        required
        style="padding: 5px; width: 100%;">
<button type="submit" class="btn btn-sm btn-outline-secondary">
    Kommentieren
</button>
</div>
</form>
</details>

<!-- Card Content -->
<h3 class="mb-1">{{ idea.title }}</h3>
<p class="idea-desc mb-2">{{ idea.description }}</p>
<small class="text-muted">
    Erstellt von {{ idea.creator.username }} am {{ idea.created_at.strftime('%d.%
</small>

</div>

```

Wichtig: Backend muss das passende Feld lesen

Da wir jetzt `name="content"` nutzen, nimm in `post_comment` :

python

 Code kopieren

```
content = (request.form.get("content") or "").strip()
```

Wenn du lieber `comment_text` behalten willst, ändere nur die Zeile im HTML zurück auf `name="comment_text"` **und** im Backend `get("comment_text")` .

Wenn du mir kurz sagst, ob `idea.comments` bei dir schon wirklich gefüllt wird (Relationship + Query mit `joinedload`), kann ich dir auch die Query fix dazuschreiben, damit die Kommentare wirklich angezeigt werden.



```

form.validate_on_submit(): False
form.errors: {'csrf_token': ['The CSRF token is missing.']}

```

Du sendest den Kommentar per **eigenem** `<form>` , aber du fügst **kein CSRF Token** mit – deshalb ist `validate_on_submit()` `False`.

✅ Fix: CSRF Token ins Kommentar-Form einbauen

Wenn du Flask-WTF/CSRF nutzt, musst du in **jedes POST-Form** eins von diesen reinpacken:

Variante A (am saubersten): `hidden_tag()` vom `CommentForm`

Wenn du im Template ein `comment_form` übergibst (empfohlen):

html

 Code kopieren

```
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}">
  {{ comment_form.hidden_tag() }}
  <input type="text" name="content" class="form-control" required>
  <button type="submit" class="btn btn-sm btn-outline-secondary">Kommentieren</button>
</form>
```

<

>

Dann muss deine Route beim Rendern `comment_form` mitgeben:

python

 Code kopieren

```
return render_template("innovationboard.html",
                      form=form,
                      comment_form=CommentForm(),
                      ideas=ideas,
                      all_users=all_users)
```

Und in `post_comment()` nutzt du auch `CommentForm()` .

Variante B (quick): globales `csrf_token()` ins Form

Falls `csrf_token()` in deinem Template verfügbar ist:

html

 Code kopieren

```
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  <input type="text" name="content" class="form-control" required>
```

```
<button type="submit" class="btn btn-sm btn-outline-secondary">Kommentieren</button>
</form>
```

Wichtig

Du kannst **nicht** `form.hidden_tag()` (vom `InnovationForm`) für Kommentare nutzen – jedes Form braucht seinen eigenen Token (bzw. denselben Token, aber als hidden field im jeweiligen Form).

Wenn du mir kurz sagst, ob du im Template bereits `{{ form.hidden_tag() }}` irgendwo nutzt (ja, im Modal), dann ist CSRF aktiv. Nimm am besten **Variante B** direkt in deinem Kommentar-Form, dann funktioniert es sofort.



```
<!-- Kommentar schreiben -->
<form method="POST" action="{{
url_for('post_comment', idea_id=idea.idea_id) }}">
  <input type="hidden" name="csrf_token" value="{{
csrf_token() }}">
  <input type="text" name="content" class="form-control"
required>
  <button type="submit" class="btn btn-sm btn-outline-
secondary">Kommentieren</button>

</form>
</details> kannst du es mir hier anpassen
```

Klar — hier ist dein Kommentar-Form **sauber angepasst** (Bootstrap-Layout, CSRF drin, alles innerhalb von `<details>`). Copy/paste:

```
html
```

```

<!-- Kommentar schreiben -->
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

  <div class="d-flex gap-2">
    <input type="text"
      name="content"
      class="form-control"
      placeholder="Kommentar hinzufügen..."
      required>
    <button type="submit" class="btn btn-sm btn-outline-secondary">
      Kommentieren
    </button>
  </div>
</form>
</details>

```



Wichtig (Backend muss passen)

 Code kopieren

In `post_comment()` musst du dann lesen:

python

```
content = (request.form.get("content") or "").strip()
```

 Code kopieren

Wenn du danach immer noch CSRF-Probleme bekommst, sag kurz, ob du `CSRFProtect(app)` verwendest oder nur `FlaskForm` — dann passe ich es 100% passend an.



der sagt ich hätte kein csrf token

Nachgedacht für 30s >

Wenn du **den Hidden-Input mit `csrf_token` drin hast** und der Server trotzdem sagt *"CSRF token is missing"*, dann ist es fast immer eins von diesen beiden Dingen:

1. `csrf_token()` ist bei dir im Template gar nicht verfügbar / rendert leer → dann wird ein leeres Feld gesendet.
2. Dein Kommentar-Form wird vom Browser nicht als eigenes Form abgeschickt, weil irgendwo im HTML ein verschachteltes/kaputtes `<form>` ist (HTML erlaubt **keine nested forms**). Dann "verschluckt" der Browser Inputs – und dein `csrf_token` landet nicht im Request.

Am stabilsten ist deshalb: **nutze ein echtes `CommentForm` und rendere `hidden_tag()`** (so wie du es beim Modal schon machst). Dann ist CSRF garantiert richtig.

✅ Fix (robust): `CommentForm.hidden_tag()` verwenden

1) Route `innovation_board: comment_form` ans Template geben

python

 Code kopieren

```
@app.route("/innovation_board/", methods=["GET", "POST"])
def innovation_board():
    form = InnovationForm()
    comment_form = CommentForm() # <-- neu

    # ... dein POST für neue Idee ...

    ideas = (Idea.query
              .filter_by(wg_id=1)
              .options(joinedload(Idea.creator),
                      joinedload(Idea.comments).joinedload(Idea_Comment.user))
              .order_by(Idea.created_at.desc())
              .all())

    return render_template(
        "innovationboard.html",
        form=form,
        comment_form=comment_form, # <-- neu
        ideas=ideas
    )
```

2) Template: Kommentar-Form so (Variante A / details)

Ersetze deinen Kommentar-Form-Block damit:

html

 Code kopieren


```
<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
    {{ comment_form.hidden_tag() }}

    <div class="d-flex gap-2">
        {{ comment_form.content(class="form-control", placeholder="Kommentar hinzufüg
        <button type="submit" class="btn btn-sm btn-outline-secondary">Kommentieren</
    </div>
</form>
```

Wichtig: In `CommentForm` muss das Feld `content` heißen:

python

 Code kopieren

```
class CommentForm(FlaskForm):
    content = TextAreaField("Kommentar", validators=[DataRequired()])
    submit = SubmitField("Senden")
```

3) `post_comment` Route: Form benutzen (nicht `request.form.get(...)`)

python

 Code kopieren

```
@app.route("/ideas/<int:idea_id>/comment", methods=["POST"])
def post_comment(idea_id):
    form = CommentForm()

    print("POST form keys:", list(request.form.keys())) # Debug

    if form.validate_on_submit():
        content = (form.content.data or "").strip()

        new_comment = Idea_Comment(
            idea_id=idea_id,
            user_id=session.get("user_id", 1), # TODO: session
            content=content,
            created_at=db.func.now()
        )
        db.session.add(new_comment)
        db.session.commit()
        flash("Kommentar hinzugefügt.", "success")
    else:
        print("form.errors:", form.errors)
        flash("Kommentar konnte nicht gespeichert werden.", "error")
```

```
return redirect(url_for("innovation_board"))
```

🔥 Ganz wichtiger Debug-Schritt (zeigt sofort, ob der Token wirklich mitkommt)

In `post_comment` hast du oben schon:

python

 Code kopieren

```
print("POST form keys:", list(request.form.keys()))
```

Wenn dort **kein** `csrf_token` auftaucht, dann ist dein HTML sehr wahrscheinlich kaputt (z.B. irgendwo ein offenes/verschachteltes `<form>`).

Nebenbei: Auch deine Like/Delete Forms brauchen CSRF

Wenn du `CSRFProtect(app)` nutzt, müssen **alle** POST-Forms (Delete/Like/Comment) ein Token haben.

Wenn du mir den HTML-Ausschnitt **von einer kompletten Card** (inkl. Like-Form, Delete-Form, Comments-Details) schickst, kann ich dir sagen, ob bei dir irgendwo ein `<form>` falsch verschachtelt ist (das ist der häufigste Grund, warum der Token "verschwindet").



```
Kommentar POST angekommen
form.validate_on_submit(): False
form.errors: {'csrf_token': ['The CSRF token is missing.']}
```

Nachgedacht für 36s >

Das heißt: im Request kommt wirklich kein Feld `csrf_token` an. (Nicht "ungültig", sondern **fehlt**.)

Dann liegt's fast immer an **(A)** du renderst keinen echten Token ins Kommentar-Form **oder** **(B)** dein HTML ist kaputt (z.B. verschachtelte/vergessene `</form>`), sodass der Browser dein Hidden-Feld gar nicht mit abschickt.

Der robusteste Fix: **Benutze für Kommentare ein echtes `CommentForm` und rendere `hidden_tag()`** (wie du's beim Modal machst). Dann ist CSRF garantiert drin.

1) innovation_board Route: comment_form ans Template geben

python

 Code kopieren

```
@app.route("/innovation_board/", methods=["GET", "POST"])
def innovation_board():
    form = InnovationForm()
    comment_form = CommentForm() # <-- neu

    # ... Idee speichern wie bisher ...

    ideas = (Idea.query
              .filter_by(wg_id=1)
              .options(joinedload(Idea.creator),
                      joinedload(Idea.comments).joinedload(Idea_Comment.user))
              .order_by(Idea.created_at.desc())
              .all())

    return render_template(
        "innovationboard.html",
        form=form,
        comment_form=comment_form, # <-- neu
        ideas=ideas
    )
```

2) Template: Kommentar-Form mit `comment_form.hidden_tag()`

So (Variante A mit `<details>`), und **wichtig**: wir geben dem Feld pro Idea eine eindeutige `id` , damit es nicht 20× "content" heißt.

html

 Code kopieren

```

<form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
      {{ comment_form.hidden_tag() }}

<div class="d-flex gap-2">
    {{ comment_form.content(
        id="content-" ~ idea.idea_id,
        class="form-control",
        placeholder="Kommentar hinzufügen..."
    ) }}
    <button type="submit" class="btn btn-sm btn-outline-secondary">
        Kommentieren
    </button>
</div>
</form>

```

<

>

3) post_comment Route: CommentForm validieren und form.content.data nutzen

python

 Code kopieren

```

@app.route("/ideas/<int:idea_id>/comment", methods=["POST"])
def post_comment(idea_id):
    form = CommentForm()

    print("POST keys:", list(request.form.keys())) # Debug

    if form.validate_on_submit():
        content = (form.content.data or "").strip()

        new_comment = Idea_Comment(
            idea_id=idea_id,
            user_id=session.get("user_id", 1), # später session user
            content=content,
            created_at=db.func.now()
        )
        db.session.add(new_comment)
        db.session.commit()
        flash("Kommentar hinzugefügt.", "success")
    else:
        print("form.errors:", form.errors)

```

```
flash("Kommentar konnte nicht gespeichert werden.", "error")

return redirect(url_for("innovation_board"))
```

Wenn es danach IMMER NOCH "missing" sagt

Dann ist es zu 99% **HTML kaputt / Forms verschachtelt** (z.B. irgendwo ein extra `</form>` oder ein Form um die ganze Card).

Check dafür sofort den Debug-Print:

python

 Code kopieren

```
print("POST keys:", list(request.form.keys()))
```

- Wenn da **kein** `csrf_token` drinsteht → dein Token ist nicht im tatsächlich abgeschickten Form → HTML/Forms reparieren.
- Wenn `csrf_token` drinsteht, aber Fehler bleibt → dann wäre es eher "invalid" (anderes Problem).

Wenn du magst: paste mir den kompletten HTML-Block **einer** Idea-Card (Like-Form, Delete-Form, Comments-Details). Ich markiere dir dann exakt, wo ein Tag/ `</form>` falsch sitzt.



```
<details class="mt-2">
  <summary style="cursor:pointer; color:#28a745; font-weight:600;">
    💬 Kommentare ({{ idea.comments|length if
idea.comments else 0 }})
  </summary>

  <!-- bestehende Kommentare -->
  <div style="margin-top:10px;">
    {% for c in idea.comments %}
      <div style="border-top:1px solid #ddd; padding-top:6px; margin-top:6px;">
        <small>
          <strong>{{ c.user.username }}</strong>
```

```

        · {{ c.created_at.strftime('%d.%m.%Y %H:%M') }}
    </small>
    <div>{{ c.content }}</div>
</div>
{% else %}
    <small class="text-muted">Noch keine Kommentare.
</small>
{% endfor %}
</div>

<!-- Kommentar schreiben -->
<form method="POST"
    action="{{ url_for('post_comment',
idea_id=idea.idea_id) }}"
    style="margin-top:10px;">
    <div style="display:flex; gap:10px; align-
items:center;">
        <input type="text"
            name="content"
            placeholder="Kommentar hinzufügen..."
            required
            style="padding: 5px; width: 100%;">
        <button type="submit" class="btn btn-sm btn-
outline-secondary">
            Kommentieren
        </button>
    </div>
</form>
</details>

```

Dein Kommentar-Form hat **keinen CSRF-Token** drin – deshalb "missing".

Da du **kein WTForms-Field** in diesem Form nutzt, kannst du den Token trotzdem manuell einfügen, aber nur, wenn `csrf_token()` bei dir wirklich verfügbar ist. Sicherer ist: `CommentForm.hidden_tag()`.

Ich gebe dir beide funktionierenden Varianten:

 **Variante 1 (empfohlen):** `CommentForm.hidden_tag()` (100% zuverlässig)

Template (ersetze dein `<form>...</form>` durch das hier)

html

```
<form method="POST"
      action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
      style="margin-top:10px;">
  {{ comment_form.hidden_tag() }}

  <div style="display:flex; gap:10px; align-items:center;">
    {{ comment_form.content(
      id="content-" ~ idea.idea_id,
      class="form-control",
      placeholder="Kommentar hinzufügen..."
    ) }}
    <button type="submit" class="btn btn-sm btn-outline-secondary">
      Kommentieren
    </button>
  </div>
</form>
```

Route `innovation_board: comment_form` mitgeben

python

```
return render_template("innovationboard.html",
                      form=form,
                      comment_form=CommentForm(),
                      ideas=ideas,
                      all_users=all_users)
```

`post_comment: CommentForm()` validieren

 Code kopieren

python

```
form = CommentForm()
if form.validate_on_submit():
    content = (form.content.data or "").strip()
    ...
```

 Code kopieren

Variante 2 (schnell): Hidden-CSRF per `csrf_token()`

Nur wenn `csrf_token()` bei dir **wirklich** rendert (nicht leer):

html

```
<form method="POST"
      action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
      style="margin-top:10px;">
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">

  <div style="display:flex; gap:10px; align-items:center;">
    <input type="text"
          name="content"
          placeholder="Kommentar hinzufügen..."
          required
          style="padding: 5px; width: 100%;">
    <button type="submit" class="btn btn-sm btn-outline-secondary">
      Kommentieren
    </button>
  </div>
</form>
```

 **Debug** (damit wir sicher sind, dass der Token wirklich ~~gesendet~~ wird)

 Code kopieren

In `post_comment` einmal kurz:

python

```
print("POST keys:", list(request.form.keys()))
```

 Code kopieren

Wenn du danach **kein** `csrf_token` in den Keys siehst, dann ist der Token immer noch nicht im abgeschickten Form (oder das Form wird nicht so abgeschickt wie du denkst).

Wenn du willst, schick mir kurz deine `CommentForm` Klasse (falls vorhanden). Dann passe ich Variante 1 exakt auf deine Felder an.



kommentarspalte größer machen wie geht das?

Meinst du das **Eingabefeld zum Schreiben** (im `<details>`), oder die **Anzeige-Spalte** der bestehenden Kommentare?

Hier sind beide Lösungen:

1) Eingabefeld größer machen (breiter + höher)

Variante A: Einzeiliges Input höher machen

html

```
<input type="text" name="content"
      class="form-control"
      placeholder="Kommentar hinzufügen..."
      style="height: 44px; font-size: 1rem;">
```

 Code kopieren

Variante B (empfohlen): Mehrzeiliges Textfeld (`textarea`)

Dann können Leute längere Kommentare schreiben:

html

```
<textarea name="content"
          class="form-control"
          rows="3"
          placeholder="Kommentar hinzufügen..."
          style="resize: vertical;"></textarea>
```

Wenn du WTForms nutzt:

 Code kopieren

html

```
{{ comment_form.content(class="form-control", rows="3", style="resize:vertical;)"
```

 Code kopieren

2) Kommentarbereich (Anzeige) größer machen

Z.B. mehr "Platz" und begrenzte Höhe mit Scroll:

html

```
<div style="margin-top:10px; max-height: 200px; overflow-y: auto; padding-right:
  {% for c in idea.comments %}
    ...
  {% endfor %}
</div>
```



Code kopieren

Sag kurz: Willst du das Feld zum Tippen größer (hoch/breit) oder die Liste der Kommentare?



```
<button type="submit" class="btn btn-link text-danger p-0"
        style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;"
        onclick="return confirm('Idee wirklich
löschen?');"> diesen button oben rechts in der card
anzeigen?
```

Ja — setz den Button in einen **Header-Bereich der Card** und richte den Header per Flex nach rechts aus.

1) In der Card ganz oben ein Header-Div einfügen

html

```
<div class="idea-card">

  <div class="d-flex justify-content-end">
    <form method="POST" action="{% url_for('delete_idea', idea_id=idea.idea_id) %}
```

```

        <button type="submit"
            class="btn btn-link text-danger p-0"
            style="cursor:pointer; padding:10px 20px; background-color:#007bff;
            onclick="return confirm('Idee wirklich löschen?');">
            Löschen
        </button>
    </form>
</div>

<!-- Rest der Card -->
<h3 class="mb-1">{{ idea.title }}</h3>
<p class="idea-desc mb-2">{{ idea.description }}</p>
<small class="text-muted">Erstellt von {{ idea.creator.username }} am {{ idea.c

</div>

```

Damit ist der Button **oben rechts** innerhalb der Card.

2) Optional: absolut oben rechts (wirklich Ecke)

 Code kopieren

Wenn du willst, dass er exakt in der Ecke "klebt":

css

```

.idea-card { position: relative; }
.idea-card .delete-btn { position: absolute; top: 10px; right: 10px; }

```

 Code kopieren

Und HTML:

html

```

<form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}" c
    <button type="submit" ...>Löschen</button>
</form>

```

Sag kurz, ob du **Flex (Variante 1)** oder **absolute Ecke (Variante 2)** willst.

 Code kopieren

    ...

```

<div class="ideas-list">
  {% for idea in ideas %}
    <div class="idea-card">

      <div class="d-flex justify-content-between align-
items-center mb-2">
        <!-- Like -->
        <form method="POST" action="{{
url_for('toggle_like', idea_id=idea.idea_id) }}" class="m-0">
          <button type="submit" class="btn btn-sm btn-
outline-primary"
            style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;">
            👍 {{ idea.likes|length }}
          </button>
        </form>

        <!-- Delete -->
        <div class="d-flex justify-content-end">
          <form method="POST" action="{{
url_for('delete_idea', idea_id=idea.idea_id) }}" class="m-0">

            <button type="submit" class="btn btn-link text-
danger p-0"
              style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;"
              onclick="return confirm('Idee wirklich
löschen?');">
              🗑️
            </button>
          </form>
        </div>
      </div> wo packe ich das hier rein
    </div>
  {% endfor %}
</div>

```

Du packst es **ganz oben in die Card**, also **direkt nach** `<div class="idea-card">` und **vor** `h3 / description / comments`.

Bei dir ist es schon fast richtig – du brauchst nur **keinen extra** `div d-flex justify-content-end` um das Delete-Form, weil der äußere Header (`justify-content-between`) bereits links/rechts trennt.

So soll der **komplette Header-Block** in deiner Card aussehen (copy/paste):

html

```
<div class="idea-card">

  <!-- HEADER: Like links, Delete rechts -->
  <div class="d-flex justify-content-between align-items-center mb-2">

    <!-- Like (links) -->
    <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}">
      <button type="submit"
        class="btn btn-sm btn-outline-primary"
        style="cursor:pointer; padding:10px 20px; background-color:#007bff;"
        <img alt="thumbs up icon" data-bbox="198 428 215 445" style="vertical-align: middle;"/> {{ idea.likes|length }}
      </button>
    </form>

    <!-- Delete (rechts) -->
    <form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}">
      <button type="submit"
        class="btn btn-link text-danger p-0"
        style="cursor:pointer; padding:10px 20px; background-color:#007bff;"
        onclick="return confirm('Idee wirklich löschen?');">
        <img alt="trash icon" data-bbox="198 642 215 659" style="vertical-align: middle;"/>
      </button>
    </form>

  </div>

  <!-- danach kommt dein Card-Inhalt -->
  <h3 class="mb-1">{{ idea.title }}</h3>
  <p class="idea-desc mb-2">{{ idea.description }}</p>
  <small class="text-muted">Erstellt von {{ idea.creator.username }} am {{ idea.c

  <!-- comments etc... -->

</div>
```

Wichtig: Falls bei dir CSRF aktiv ist, musst du in **beide** Forms noch ein Token rein (Like + Delete), sonst bekommst du wieder "CSRF token missing":

html

```
<input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

 Code kopieren

Wenn du willst, füge ich dir die CSRF-Zeile direkt korrekt in beide Forms ein

 Code kopieren

    ...

```
<form method="POST" action="{{ url_for('delete_idea',
idea_id=idea.idea_id) }}" class="m-0">
```

```
    <button type="submit" class="btn btn-link text-
danger p-0"
        style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;"
```

```
        class ="btn btn-link text-danger p-0"
        onclick="return confirm('Idee wirklich
löschen?');">
```



```
</button>
```

```
</form>
```

```
</div> ich möchte, dass delete gegenüber von like ist
```

Damit **Delete gegenüber von Like** steht, brauchst du einen gemeinsamen **Header-Container mit Flex** (`justify-content-between`). Dann ist Like links und Delete rechts.

Ersetze deinen Block durch das hier:

html

```
<div class="d-flex justify-content-between align-items-center mb-2">
```

```
    <!-- Like (links) -->
```

```
    <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}"
```

```
        <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
```

```

<button type="submit" class="btn btn-sm btn-outline-primary">
  👍 {{ idea.likes|length }}
</button>
</form>

<!-- Delete (rechts) -->
<form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
  <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
  <button type="submit" class="btn btn-sm btn-outline-danger"
    onclick="return confirm('Idee wirklich löschen?');">
    🗑️
  </button>
</form>
</div>

```

2 Mini-Fixes bei deinem aktuellen Code

- Du hattest 2× `class` im Button → nur **einmal** erlaubt. 📄 Code kopieren
- Du brauchst **kein extra** `<div class="d-flex justify-content-end">` um das Delete-Form, sonst kann's das Layout stören. ➤

Damit ist Delete zuverlässig rechts und Like links.



ohne csrf token

Okay — dann einfach die gleichen Forms **ohne** die hidden `csrf_token` Inputs. (Nur wichtig: Das klappt nur, wenn du CSRF global deaktiviert hast oder diese Routes davon ausgenommen sind.)

html

```

<div class="d-flex justify-content-between align-items-center mb-2">
  <!-- Like (links) -->
  <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}"
    <button type="submit" class="btn btn-sm btn-outline-primary">
      👍 {{ idea.likes|length }}
    </button>
  </form>
  <!-- Delete (rechts) -->
  <form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
    <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
    <button type="submit" class="btn btn-sm btn-outline-danger"
      onclick="return confirm('Idee wirklich löschen?');">
        🗑️
      </button>
    </form>
</div>

```

```

</form>

<!-- Delete (rechts) -->
<form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
  <button type="submit"
    class="btn btn-sm btn-outline-danger"
    onclick="return confirm('Idee wirklich löschen?');">
    
  </button>
</form>
</div>

```

Wenn du CSRF **doch** aktiviert hast, bekommst du bei Like/Delete wieder "CSRF token missing". Dann sag kurz Bescheid, dann zeig ich dir, wie du **nur diese beiden Routes** von CSRF ausnimmst.

 Code kopieren



ich will, dass das delete button oben rechts in der card angezeigt wird, nicht unter dem like

Du hast ihn **unter** dem Like, weil dein Header-Container vermutlich **nicht horizontal** ist (z.B. weil er fehlt, oder weil er durch CSS/HTML-Struktur kaputt ist). Mach es "idiotensicher": **Card** = `position: relative`, Delete-Form **absolut** oben rechts. Dann sitzt es immer dort – egal was links passiert.

1) CSS hinzufügen

html

```

<style>
  .idea-card{
    position: relative;          /* nötig für absolute Position */
  }
  .delete-btn{
    position: absolute;
    top: 12px;
    right: 12px;
    z-index: 5;
  }
</style>

```

2) Delete-Button im HTML so platzieren (direkt am Anfang der Card)

html

```

<div class="idea-card">

  <!-- Delete oben rechts -->
  <form method="POST"
        action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
        class="delete-btn m-0">
    <button type="submit"
            class="btn btn-sm btn-outline-danger"
            onclick="return confirm('Idee wirklich löschen?');">
      🗑️
    </button>
  </form>

  <!-- Like (bleibt normal oben links oder wo du willst) -->
  <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}"
        <button type="submit" class="btn btn-sm btn-outline-primary">
      👍 {{ idea.likes|length }}
    </button>
  </form>

  <!-- Rest -->
  <details class="mt-2"> ... </details>
  <h3 class="mb-1">{{ idea.title }}</h3>
  <p class="idea-desc mb-2">{{ idea.description }}</p>
  <small class="text-muted">Erstellt von {{ idea.creator.username }} am {{ idea.c

```

</div>

✅ Ergebnis: Delete ist **immer** oben rechts in der Card, unabhängig von Like/Kommentar-Layout.

Wenn du willst, dass Like links oben und Delete rechts oben **in einer Zeile** sitzen (ohne absolute), sag kurz Bescheid – dann mache ich dir die Flex-Header-Variante  Code kopieren deinem aktuellen HTML (ohne dass es wieder untereinander rutscht).



```
<div class="ideas-list">
  {% for idea in ideas %}
    <div class="idea-card">

      <div class="d-flex justify-content-between align-items-center mb-2">
        <!-- Like -->
        <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}" class="m-0">

          <button type="submit" class="btn btn-sm btn-outline-primary"
            style="cursor:pointer; padding:10px 20px; background-color:#007bff; color:white; border-radius:5px; display:inline-block;">
            👍 {{ idea.likes|length }}
          </button>
        </form>
      </div>
      <!-- Delete -->
    </div class="d-flex justify-content-end align-items-center mb-2">
    <form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}" class="m-0">

      <button type="submit" class="btn btn-sm btn-outline-danger"
        style="cursor:pointer; padding:10px 20px;
```

```
background-color:#dc3545; color:white; border-radius:5px;
display:inline-block;"
        class="delete-btn m-0"
        onclick="return confirm('Idee wirklich
löschen?');">
        🗑️
    </button>
</form>
</div> warum jetzt kommentare außerhalb der card
```

Weil dein HTML **kaputt verschachtelt** ist: Du hast

```
html
```

```
</div class="d-flex ...">
```

 Code kopieren

Das ist **kein gültiger Closing-Tag** und du schließt/öffnest damit die `div`s falsch. Dadurch endet deine `.idea-card` an der falschen Stelle und alles danach (Kommentare) landet "außerhalb".

Außerdem hast du das Delete- `div` **außerhalb** der Card platziert (weil du die Card vorher schon schließt).

✅ Fix: Card richtig öffnen/ schließen + Delete/Like im selben Header

Ersetze deinen ganzen Block (bis vor Kommentare/Titel) durch das hier:

```
html
```

```
<div class="ideas-list">
    {% for idea in ideas %}
        <div class="idea-card">

            <div class="d-flex justify-content-between align-items-center mb-2">
                <!-- Like (links) -->
                <form method="POST" action="{% url_for('toggle_like', idea_id=idea.idea_i
                    <button type="submit"
                        class="btn btn-sm btn-outline-primary"
                        style="cursor:pointer; padding:10px 20px; background-color:#007
                        👍 {{ idea.likes|length }}
            </div>
        </div>
    </div>
```

```

        </button>
    </form>

    <!-- Delete (rechts) -->
    <form method="POST" action="{% url_for('delete_idea', idea_id=idea.id) %}">
        <button type="submit"
            class="btn btn-sm btn-outline-danger"
            style="cursor:pointer; padding:10px 20px; background-color:#dc3545;"
            onclick="return confirm('Idee wirklich löschen?');">
            
        </button>
    </form>
</div>

<!-- HIER kommen dann Kommentare/Titel/Beschreibung innerhalb der card -->
<!-- z.B. details, h3, p ... -->

</div>
{% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
{% endfor %}
</div>

```

Wichtig:

- **Kein** `</div class="...">` → das ist falsch. Ein Closing-Tag ist immer nur `</div>`.
- `</div>` für `.idea-card` kommt **ganz am Ende**, nachdem Kommentare/Titel usw. drin stehen.

 Code kopieren

Bonus: Du hast `class` doppelt im Button

Du hattest sowas wie `class="btn..."` und nochmal `class="delete-btn..."`. In HTML gilt nur eins (das zweite überschreibt). Wenn du beide willst, schreib sie zusammen:

```
html
```

```
class="btn btn-sm btn-outline-danger delete-btn m-0"
```

 Code kopieren

Wenn du mir deinen kompletten Card-Inhalt (inkl. `<details>` Kommentare) postest, setze ich dir die Tags exakt richtig, damit nichts mehr "rausfällt".



flash("Kommentar hinzugefügt.", "success") kann ich es auch so machen, dass er nach einer zeit verschwindet

Ja – das machst du am einfachsten im Template mit ein bisschen JavaScript, das die Flash-Box nach z.B. 3–5 Sekunden ausblendet.

1) Gib deinen Flash-Divs eine Klasse/ID

Dein Flash-Block ist schon da. Ergänzt nur eine Klasse (z.B. `flash-message`):

html

```
{% with messages = get_flashed_messages(with_categories=true) %}
{% if messages %}
    {% for category, message in messages %}
        <div class="alert-message flash-message {% if category == 'success' %}alert
            style="padding:15px; margin-bottom:20px; border-radius:5px;">
            {{ message }}
        </div>
    {% endfor %}
{% endif %}
{% endwith %}
```

2) Script hinzufügen (z.B. ganz unten im Template)

 Code kopieren

html

```
<script>
    setTimeout(() => {
        document.querySelectorAll('.flash-message').forEach(el => {
            el.style.transition = "opacity 0.5s ease";
            el.style.opacity = "0";
            setTimeout(() => el.remove(), 600);
        });
    }, 3500); // 3.5 Sekunden
</script>
```

✓ Ergebnis: Flash-Meldungen verschwinden automatisch nach 3,5 Sekunden (mit Fade-out).

 Code kopieren

Wenn du lieber willst, dass nur "success" verschwindet, aber "error" bleibt, sag kurz Bescheid, dann passe ich das Script an.

    ...

kein javascript

Ohne JavaScript kannst du es **nicht wirklich "nach X Sekunden"** ausblenden – CSS allein hat keinen Timer, der ein Element nach einer festen Zeit zuverlässig entfernt (nur Animationen/Übergänge, aber die Box bleibt im DOM und es ist je nach Browser/Barrierefreiheit nicht wirklich "weg").

Was du **ohne JS** machen kannst:

Option A: Manuell schließbar (ohne JS)

Du baust einen "X"-Button, der per Checkbox/CSS das Element ausblendet.

html

```
{% for category, message in messages %}
<input type="checkbox" id="close-{{ loop.index }}" style="display:none;">

<div class="flash-wrapper">
  <div class="alert-message {% if category == 'success' %}alert-success{% else %}alert-error{% endif %}">
    <label for="close-{{ loop.index }}" class="flash-close">×</label>
    {{ message }}
  </div>
</div>

<style>
  /* wenn checkbox gecheckt, verstecken */
  #close-{{ loop.index }}:checked + .flash-wrapper { display:none; }

  .alert-message{
    padding:15px;
```

```

        margin-bottom: 20px;
        border-radius: 5px;
        position: relative;
    }
    .flash-close{
        position: absolute;
        top: 6px;
        right: 10px;
        cursor: pointer;
        font-weight: bold;
    }
</style>
{% endfor %}

```

✓ Kein JS, User kann die Meldung schließen.

Option B: Nur einmal anzeigen (verschwindet beim nächsten Request)

Flash-Messages sind sowieso "one-time": nach Redirect/Reload sind sie weg.

Wenn du also nach dem Kommentar ein `redirect` machst (machst du ja), ist die Meldung beim nächsten Seitenwechsel weg.  Code kopieren

Wenn du wirklich "nach 3 Sekunden automatisch weg" willst, geht das praktisch nur mit **JavaScript** (oder sehr umständlich serverseitig mit Meta-Refresh/Redirects, was UX kaputt macht).



wie mache ich es, dass diese delete buttons nicht im modal drin sind?

Die Delete-Buttons sind im Modal sichtbar, weil deine Cards (und damit die Buttons) **trotz Backdrop noch "oben drüber" liegen** – das ist ein klassisches **z-index / stacking context** Problem.

Du hast:

- Backdrop z-index: 1
- Modal z-index: 2

Aber deine Delete-Buttons haben sehr wahrscheinlich **einen höheren z-index** (z.B. durch `position: absolute + z-index: 5`), deshalb erscheinen sie **im Modal**.

✅ Fix (am saubersten): Modal immer über alles legen

Setz Backdrop/Modal deutlich höher:

html

 Code kopieren

```
<style>
  #modalBackdrop { z-index: 1000 !important; }
  #modalContent { z-index: 1001 !important; }
</style>
```

Dann kann kein Card-Button mehr "reinragen".

✅ Extra Fix: Delete-Button z-index runter (optional)

Wenn du Delete absolut positionierst, nimm einen niedrigen Wert:

css

 Code kopieren

```
.idea-card { position: relative; }
.delete-btn { position: absolute; top: 12px; right: 12px; z-index: 1; }
```

Wichtig: **Nicht** z-index: 999 o.ä.

✅ Bootstrap-kompatible Werte (falls du Bootstrap nutzt)

Bootstrap Modals liegen i.d.R. bei `z-index: 1050+` . Dann passt:

CSS

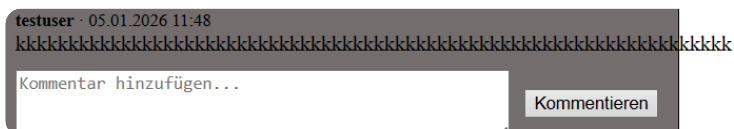
 Code kopieren

```
#modalBackdrop { z-index: 1050 !important; }  
#modalContent  { z-index: 1055 !important; }
```

Warum das passiert (kurz)

Dein Modal hat `z-index: 2` , aber deine Delete-Buttons haben vermutlich einen höheren `z-index` (oder ein eigenes `stacking context`). Deshalb "schweben" sie über dem Modal.

Wenn du mir deine Modal-CSS Zeilen (Backdrop + Content) und den Delete-Button CSS (`.delete-btn`) zeigst, kann ich dir die perfekten Zahlen direkt passend einsetzen.



wie fixe ich, dass der kommentar über den rand hinausgeht

Das passiert wieder wegen **sehr langen Strings ohne Leerzeichen** (wie `kkkkkkkk...`).

Lösung: beim Kommentar-Text **Word-Wrapping** erzwingen.

✅ 1) Gib dem Kommentar-Text eine Klasse

Wo du den Kommentar ausgibst, z.B.:

html

 Code kopieren

```
<div class="comment-text">{{ c.content }}</div>
```

✅ 2) CSS dazu (bricht auch endlose Wörter/URLs)

html

 Code kopieren

```
<style>
  .comment-text{
    white-space: pre-wrap;      /* Zeilenumbrüche behalten */
    overflow-wrap: anywhere;    /* bricht lange Wörter */
    word-break: break-word;     /* Fallback */
    max-width: 100%;
  }
</style>
```

Damit geht der Kommentar **nicht mehr über den Rand hinaus**, sondern bricht sauber um.

Optional: Wenn der ganze Kommentar-Container zu breit ist

Falls dein Kommentarbereich ein Flex-Container ist, setz zusätzlich:

css

 Code kopieren

```
.comment-item { min-width: 0; }
```

(Bei Flexbox verhindert `min-width: auto` manchmal den Umbruch.)

Wenn du mir den HTML-Schnipsel zeigst, wie du Kommentare renderst (`for c in idea.comments`), kann ich dir die Klassen direkt an die richtige Stelle setzen.



```
<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>

<div style="display:flex; justify-content:space-between;
align-items:center; margin-bottom: 20px;">
  <h1 style="margin:0;">ActivityBoard</h1>
```

```

<label for="modalToggleNewActivity"
  style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;">
  + Neue Aktivität
</label>
</div>

```

```

<h2>Teile deine Aktivitäten und Erlebnisse in der
WG</h2>

```

```

{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;" >
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}

```

```

<div class="activities-list">
  {% for activity in activities %}
    <div class="activity-card">
      <h3 class="mb-1">{{ activity.title }}</h3>
      <p class="activity-desc mb-2">{{ activity.description }}
</p>
      <small class="text-muted">Gepostet von {{
activity.user.username }} am {{
activity.created_at.strftime('%d.%m.%Y %H:%M') }}
</small>
    </div>
  {% else %}
    <p>Noch keine Aktivitäten geteilt. Sei der/die Erste!
</p>
  {% endfor %}
</div>

```

```

<!-- Modal for New Activity -->
<input type="checkbox" id="modalToggleNewActivity"
style="display:none;">

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>
<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
    <label for="modalToggleNewActivity" style="float: right;
cursor: pointer; font-size: 24px; font-weight: bold; margin-
top: 10px;"> x </label>
    <h2>Neue Aktivität vorschlagen</h2>
    <h3>Schlage eine neue Aktivität für Deine WG vor.
</h3>
    <form method="POST">
        {{ form.hidden_tag() }}
        <div style="margin-bottom: 15px;">
            <label for="activity_title">Aktivität:</label>
            {{ form.title(style="display: block; margin-top: 5px;
padding: 8px; width: 100%;") }}
        </div>
        <div style="margin-bottom: 15px;">
            <label for="activity_description">Beschreibung:
</label>
            {{ form.description(
                style="display:block; margin-top:5px;
padding:8px; width:100%; height:100px; resize: vertical;
white-space: pre-wrap;"
            ) }}
        </div>
        {{ form.submit() }}
    </form>
</div>

<style>
    #modalToggleNewIdea:checked ~ #modalBackdrop {
        display: block !important;
    }

```

```
#modalToggleNewIdea:checked ~ #modalContent {  
  display: block !important;  
}  
  
#modalBackdrop { z-index: 1000 !important; }  
#modalContent { z-index: 1001 !important; }  
</style> wieso kann ich nicht den button "neue aktivität"  
klicken?
```

Du kannst ihn nicht klicken, weil dein CSS die falsche Checkbox-ID anspricht.

Du hast:

html

```
<input type="checkbox" id="modalToggleNewActivity" ...>
```

 Code kopieren

Aber dein CSS nutzt:

css

```
#modalToggleNewIdea:checked ~ #modalBackdrop { ... }  
#modalToggleNewIdea:checked ~ #modalContent { ... }
```

 Code kopieren

➡ modalToggleNewIdea gibt es hier nicht, deshalb wird Backdrop/Modal nie eingeblendet.

✅ Fix: CSS-Selektoren auf modalToggleNewActivity ändern

html

```
<style>  
  #modalToggleNewActivity:checked ~ #modalBackdrop {  
    display: block !important;  
  }  
  
  #modalToggleNewActivity:checked ~ #modalContent {  
    display: block !important;  
  }  
</style>
```

```
#modalBackdrop { z-index: 1000 !important; }  
#modalContent { z-index: 1001 !important; }  
</style>
```

Dann funktioniert der Button sofort.

 Code kopieren

Kleiner Zusatz-Check

Dein Label ist korrekt:

html

```
<label for="modalToggleNewActivity">+ Neue Aktivität</label>
```

 Code kopieren

Also nach dem CSS-Fix sollte es gehen.

    ...

time = db.Column(db.DateTime) wie mache ich es, dass
man die uhrzeit auswählen kann?

Du brauchst dafür in deinem Formular ein **DateTime-Input** (oder getrennt Date + Time).
Im Browser ist am einfachsten: `<input type="datetime-local">` .

1) WTForms Feld anlegen

python

```
from wtforms.fields import DateTimeLocalField  
from wtforms.validators import DataRequired  
  
time = DateTimeLocalField(  
    "Uhrzeit",  
    format="%Y-%m-%dT%H:%M",  
    validators=[DataRequired()]  
)
```

2) Im Template rendern

 Code kopieren

html

```
<label>Uhrzeit:</label>
{{ form.time(type="datetime-local", class="form-control") }}
```

 Code kopieren

Damit kann man Datum + Uhrzeit auswählen.

Alternative: Nur Uhrzeit auswählen (ohne Datum)

Dann nimm ein normales `StringField` und render als `type="time"` :

Form:

python

```
from wtforms import StringField
from wtforms.validators import DataRequired

time = StringField("Uhrzeit", validators=[DataRequired()])
```

Template:

 Code kopieren

html

```
{{ form.time(type="time", class="form-control") }}
```

 Code kopieren

Dann musst du im Backend das Datum separat setzen oder zusammenbauen.

Wichtig bei SQLite/DB

`DateTimeLocalField` liefert ein Python- `datetime` → kannst du direkt in `db.Column(db.DateTime)` speichern:

python

```
new_activity.time = form.time.data
```

 Code kopieren

Wenn du mir sagst, ob du **Datum+Uhrzeit** oder **nur Uhrzeit** willst, passe ich dir das exakt in dein `ActivityForm` + Route ein.



C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py:22: SAWarning: relationship 'ShoppingItem.added_by_user' will copy column USER.user_id to column SHOPPING_ITEM.added_by, which conflicts with relationship(s): 'User.added_items' (copies USER.user_id to SHOPPING_ITEM.added_by). If this is not the intention, consider if these relationships should be linked with back_populates, or if viewonly=True should be applied to one or more if they are read-only. For the less common case that foreign key constraints are partially overlapping, the orm.foreign() annotation can be used to isolate the columns that should be written towards. To silence this warning, add the parameter 'overlaps="added_items"' to the 'ShoppingItem.added_by_user' relationship.

(Background on this warning at:

<https://sqlalche.me/e/20/qzyx>) (This warning originated from the `configure_mappers()` process, which was invoked automatically in response to a user-initiated operation.)

```
return cls.query_class(
```

C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py:22: SAWarning: relationship 'ShoppingItem.assigned_to_user' will copy column USER.user_id to column SHOPPING_ITEM.assigned_to, which conflicts with relationship(s): 'User.assigned_items' (copies USER.user_id to SHOPPING_ITEM.assigned_to). If this is not the intention, consider if these relationships should be linked with back_populates, or if viewonly=True should be applied to one or more if they are read-only. For the less common case that foreign key constraints are partially overlapping, the orm.foreign() annotation can be used to isolate the columns that should be written towards. To silence this warning, add the parameter 'overlaps="assigned_items"' to the

189/536

```
am\Web-Development-Project\venv\Lib\site-  
packages\flask\app.py", line 919, in full_dispatch_request  
    rv = self.handle_user_exception(e)  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\flask\app.py", line 917, in full_dispatch_request  
    rv = self.dispatch_request()  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\flask\app.py", line 902, in dispatch_request  
    return  
self.ensure_sync(self.view_functions[rule.endpoint])  
(**view_args) # type: ignore[no-any-return]  
  
~~~~~  
~~~~~ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\app.py", line 320, in  
activity_board  
    .all()  
    ~~~ ^^  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\sqlalchemy\orm\query.py", line 2704, in all  
    return self._iter().all() # type: ignore  
        ~~~~~ ^ ^  
File  
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\sqlalchemy\orm\query.py", line 2857, in _iter  
    result: Union[ScalarResult[_T], Result[_T]] =  
self.session.execute(  
  
~~~~~ ^  
    statement,  
    ^ ^ ^ ^ ^ ^ ^ ^ ^ ^  
    params,  
    ^ ^ ^ ^ ^ ^ ^ ^  
    execution_options={"_sa_orm_load_options":
```

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 2351, in
execute

```
return self._execute_internal(
    ~~~~~^
    statement,
    ^^^^^^
    ...<4 lines>...
    _add_event=_add_event,
    ^^^^^^^^^^^^^^^^^^
)
^
```

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 2249, in
_execute_internal

```
result: Result[Any] =
compile_state_cls.orm_execute_statement(
```

[illegible]

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\context.py", line 306, in
orm_execute_statement
    result = conn.execute(
```

```

        statement, params or {},
    execution_options=execution_options
)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1419, in execute
    return meth(
        self,
        distilled_parameters,
        execution_options or NO_OPTIONS,
    )
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\sql\elements.py", line 527, in _execute_on_connection
    return connection._execute_clauseelement(
~~~~~^
        self, distilled_params, execution_options

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^
    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1641, in _execute_clauseelement
    ret = self._execute_context(
        dialect,
        ...<8 lines>...
        cache_hit=cache_hit,
    )
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\engine\base.py", line 1846, in _execute_context
    return self._exec_single_context(

```


packages\sqlalchemy\engine\default.py", line 952, in
do_execute

cursor.execute(statement, parameters)

~~~~~^

sqlalchemy.exc.OperationalError: (sqlite3.OperationalError)

no such column: ACTIVITY.date

```
[SQL: SELECT "ACTIVITY".activity_id AS
"ACTIVITY_activity_id", "ACTIVITY".wg_id AS
"ACTIVITY_wg_id", "ACTIVITY".created_by AS
"ACTIVITY_created_by", "ACTIVITY".title AS
"ACTIVITY_title", "ACTIVITY".description AS
"ACTIVITY_description", "ACTIVITY".date AS
"ACTIVITY_date", "ACTIVITY".location AS
"ACTIVITY_location", "ACTIVITY".max_participants AS
"ACTIVITY_max_participants", "ACTIVITY".status AS
"ACTIVITY_status", "ACTIVITY".created_at AS
"ACTIVITY_created_at", "ACTIVITY".updated_at AS
"ACTIVITY_updated_at", "USER_1".user_id AS
"USER_1_user_id", "USER_1".username AS
"USER_1_username", "USER_1".password_hash AS
"USER_1_password_hash", "USER_1".email AS
"USER_1_email", "USER_1".role AS "USER_1_role",
"USER_1".wg_id AS "USER_1_wg_id"
FROM "ACTIVITY" LEFT OUTER JOIN "USER" AS "USER_1"
ON "USER_1".user_id = "ACTIVITY".created_by
WHERE "ACTIVITY".wg_id = ? ORDER BY
"ACTIVITY".created_at DESC]
```

[parameters: (1,)]

(Background on this error at:

<https://sqlalche.me/e/20/e3q8>)

127.0.0.1 - - [05/Jan/2026 13:54:17] "GET /activityboard/  
HTTP/1.1" 500 - warum kein ACTIVITY.date, ich habe aber  
in meiner db.py ein date column

Der Fehler heißt nicht „dein Model hat keine Spalte“, sondern: **deine SQLite-Tabelle ACTIVITY in der Datenbank hat physisch keine Spalte date**.

Bei SQLAlchemy gilt: **Model-Code ≠ Datenbank-Schema**.

Wenn du nachträglich `date = db.Column(...)` in `db.py` ergänzt hast, wird SQLite die

Tabelle **nicht automatisch ändern**. Deshalb kommt beim SELECT: `no such column: ACTIVITY.date`.

## ✅ Was du jetzt tun musst (3 mögliche Wege)

### Weg 1 (am schnellsten im Uni-Projekt): DB neu erstellen

Wenn du keine wichtigen Daten behalten musst:

1. App stoppen
2. Deine SQLite Datei löschen (z.B. `app.db` / `database.db`)
3. App starten und einmal ausführen:

```
python
```

```
with app.app_context():  
    db.create_all()
```

 Code kopieren

Dann ist `ACTIVITY.date` wirklich in der DB.

### Weg 2: Migration (sauber): Flask-Migrate

Wenn du DB nicht löschen willst, brauchst du Migrations:

```
bash
```

```
pip install flask-migrate
```

 Code kopieren

Dann (im Projektordner, venv aktiv):

```
bash
```

```
flask db init  
flask db migrate -m "add date to activity"  
flask db upgrade
```

 Code kopieren

Und in deiner App:

```
python
```

```
from flask_migrate import Migrate  
migrate = Migrate(app, db)
```

 Code kopieren

### Weg 3: Manuell per SQL ALTER TABLE (nur für 1 Spalte ok)

SQLite kann ADD COLUMN :

sql

```
ALTER TABLE ACTIVITY ADD COLUMN date DATETIME;
```

 Code kopieren

Das kannst du z.B. in DB Browser for SQLite ausführen.

### Check, ob die Spalte wirklich fehlt

In SQLite:

sql

```
PRAGMA table_info(ACTIVITY);
```

 Code kopieren

Wenn date nicht drin steht → genau das ist das Problem.

### Kleiner Zusatz: Name date

date ist nicht ideal als Spaltenname (kann verwirren). Besser z.B. event\_time oder start\_time . Aber der aktuelle Fehler kommt **nur** vom nicht aktualisierten DB-Schema.

Wenn du mir sagst, ob du deine DB löschen darfst oder Daten behalten musst, sag ich dir den besten Weg und die exakten Befehle/Code für dein Projektsetup.

    ...

p(s): 'User.assigned\_items' (copies USER.user\_id to SHOPPING\_ITEM.assigned\_to). If this is not the intention, consider if these relationships should be linked with back\_populates, or if viewonly=True should be applied to one or more if they are read-only. For the less common case that foreign key constraints are partially overlapping,



<https://sqlalche.me/e/20/qzyx> (This warning originated from the `configure_mappers()` process, which was invoked automatically in response to a user-initiated operation.)

```
return render_template("activityboard.html", form=form,
```

```

activities=activities, all_users=all_users)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\templating.py", line 150, in
render_template
    return _render(app, template, context)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\templating.py", line 131, in _render
    rv = template.render(context)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\jinja2\environment.py", line 1295, in render
    self.environment.handle_exception()
    ~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\jinja2\environment.py", line 942, in
handle_exception
 raise rewrite_traceback_stack(source=source)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-
Project\templates\activityboard.html", line 67, in top-level
template code
 {{ form.date(style="display: block; margin-top: 5px;
padding: 8px; width: 100%;") }}
 ^^^^^^^^^^^^^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\jinja2\utils.py", line 92, in from_obj
 if hasattr(obj, "jinja_pass_arg"):
    ~~~~~^
    ~~~~~^
jinja2.exceptions.UndefinedError: 'docs.forms.ActivityForm
object' has no attribute 'date'

```

Du hast jetzt zwei verschiedene Probleme im Log:

## 1) Der echte Crash (500):

```
UndefinedError: 'ActivityForm object' has no attribute 'date'
```

In deinem Template steht:

```
jinja2
```

```
{{ form.date(...) }}
```

 Code kopieren

Aber dein `ActivityForm` hat **kein Feld** namens `date`. Deshalb knallt Jinja.

✓ Fix: Entweder:

### A) Feld in `ActivityForm` hinzufügen (empfohlen)

Wenn du Datum/Uhrzeit auswählen willst, nenn es z.B. `date` oder `time` – aber es muss genau so heißen wie im Template.

Beispiel (Datum+Uhrzeit als `datetime-local`):

```
python
```

```
from flask_wtf import FlaskForm
from wtforms.fields import DateTimeLocalField
from wtforms.validators import DataRequired

class ActivityForm(FlaskForm):
 title = StringField("Titel", validators=[DataRequired()])
 description = TextAreaField("Beschreibung", validators=[DataRequired()])
 date = DateTimeLocalField("Datum & Uhrzeit", format="%Y-%m-%dT%H:%M", validate_on_submit=True)
 submit = SubmitField("Aktivität erstellen")
```

Und im Template:

```
jinja2
```

```
{{ form.date(type="datetime-local", style="display:block; margin-top:5px; padding:
```

 Code kopieren

## B) Oder Template an dein Feld anpassen

Wenn dein Form-Feld z.B. `time` heißt, dann muss es heißen:

```
jinja2
```

```
{{ form.time(...) }}
```

 Code kopieren

## 2) Die SAWarning (kein Crash, nur Warnung):

```
overlaps="assigned_items" / overlaps="added_items"
```

Das ist nur SQLAlchemy, das sagt: du hast doppelte Relationships auf dieselben FK-Spalten.

Wenn du es sauber machen willst, in `ShoppingItem` :

```
python
```

```
assigned_to_user = db.relationship("User", foreign_keys=[assigned_to], overlaps="added_by_user"
added_by_user = db.relationship("User", foreign_keys=[added_by], overlaps="assigned_to_user")
```

◀

 Code kopieren ▶

Oder du verknüpfst korrekt mit `back_populates` . Aber: **das verursacht nicht den 500er.**

## Kurz gesagt

Dein Fehler ist simpel: **Template erwartet** `form.date` , **dein ActivityForm hat es nicht.**

Wenn du mir deinen aktuellen `ActivityForm` zeigst, passe ich dir exakt an, welche Zeile du ergänzen musst (und ob du Datum+Uhrzeit oder nur Uhrzeit willst).

    ...

wie würde ich rangehen, wenn ich ein optionales maximale teilnehmer zahl feld einfügen möchte

So würde ich's machen (sauber + optional):

## 1) Datenbank: Spalte nullable machen

python

```
max_participants = db.Column(db.Integer, nullable=True)
```

 Code kopieren

Wenn die Tabelle schon existiert: Migration oder DB neu erstellen (wie bei `ACTIVITY.date` ).

## 2) WTForms: Optionales Integer-Feld

python

```
from wtforms import IntegerField
from wtforms.validators import Optional, NumberRange

class ActivityForm(FlaskForm):
 # ...
 max_participants = IntegerField(
 "Max. Teilnehmer",
 validators=[Optional(), NumberRange(min=1, max=999)]
)
```

`Optional()` ist wichtig, sonst wird leeres Feld als Fehler gewertet.

 Code kopieren

## 3) Template: Input rendern (optional mit Placeholder)

html

```
<div style="margin-bottom: 15px;">
 <label>Max. Teilnehmer (optional)</label>
 {{ form.max_participants(
 style="display:block; margin-top:5px; padding:8px; width:100%; ",
 placeholder="z.B. 10"
) }}
 {% if form.max_participants.errors %}
 {{ form.max_participants.errors[0] }}
```

```
{% endif %}
</div>
```

 Code kopieren

## 4) Route: Wert speichern (None wenn leer)

In deiner POST-Logik:

```
python

new_activity = Activity(
 # ...
 max_participants=form.max_participants.data # None, wenn leer
)
db.session.add(new_activity)
db.session.commit()
```

 Code kopieren

## 5) Anzeige: Nur zeigen, wenn gesetzt

```
jinja2

{% if activity.max_participants %}
 <small>Max: {{ activity.max_participants }} Teilnehmer</small>
{% endif %}
```

 Code kopieren

Wenn du mir sagst, ob du **Datum/Uhrzeit** schon als Feld hast und wie dein `Activity` - Model genau aussieht, kann ich dir den kompletten `ActivityForm` + Route snippet im selben Stil wie dein Projekt zusammenbauen.



wie würde eine funktion aussehen, die das datum in google calendar oder apple calendar macht?

Wenn du "aus deiner Web-App heraus ins Google/Apple Calendar" willst, gibt's praktisch 2 sinnvolle Wege:

## Weg A (am einfachsten, ohne OAuth): Kalender-Link + .ics Download

Du erzeugst einen Link "In Google Calendar öffnen" und/oder eine **ICS-Datei** (funktioniert für Apple Calendar, Outlook, Google Import).

### 1) Google Calendar Link generieren

python

```
from urllib.parse import urlencode

def google_calendar_url(title, start_dt, end_dt, details="", location=""):
 # start_dt/end_dt: naive datetime in local time -> besser in UTC umwandeln, h
 fmt = "%Y%m%dT%H%M%S"
 dates = f"{start_dt.strftime(fmt)}/{end_dt.strftime(fmt)}"

 params = {
 "action": "TEMPLATE",
 "text": title,
 "dates": dates,
 "details": details,
 "location": location,
 }
 return "https://calendar.google.com/calendar/render?" + urlencode(params)
```

Im Template:

html

```
In Google Calendar
```

 Code kopieren

### 2) ICS erzeugen (Apple Calendar & Import überall)

python

```
from flask import Response
from datetime import timezone

def dt_to_ics(dt):
```

```

am besten UTC
dt_utc = dt.replace(tzinfo=timezone.utc)
return dt_utc.strftime("%Y%m%dT%H%M%SZ")

def build_ics(uid, title, start_dt, end_dt, description="", location=""):
 return "\r\n".join([
 "BEGIN:VCALENDAR",
 "VERSION:2.0",
 "PRODID:-//WG Planner//DE",
 "BEGIN:VEVENT",
 f"UID:{uid}",
 f"DTSTART:{dt_to_ics(start_dt)}",
 f"DTEND:{dt_to_ics(end_dt)}",
 f"SUMMARY:{title}",
 f"DESCRIPTION:{description}",
 f"LOCATION:{location}",
 "END:VEVENT",
 "END:VCALENDAR",
 ""
])

@app.route("/activities/<int:activity_id>/ics")
def activity_ics(activity_id):
 activity = Activity.query.get_or_404(activity_id)

 ics = build_ics(
 uid=f"activity-{activity.activity_id}@wgplanner",
 title=activity.title,
 start_dt=activity.date, # oder activity.time
 end_dt=activity.date, # ggf. + timedelta(hours=2)
 description=activity.description or "",
 location=activity.location or "",
)

 return Response(
 ics,
 mimetype="text/calendar",
 headers={"Content-Disposition": f'attachment; filename="activity-{activity_id}.ics"'},
)

```

Im Template:

html



```
📅 In 1
```



Code kopieren

Das ist der beste Einstieg, weil:

- kein Login bei Google nötig
- Apple Calendar kann .ics direkt öffnen
- Google kann .ics importieren oder du nutzt den Google-Link

Code kopieren

## Weg B (voll integriert, "wirklich in Google Calendar eintragen"): Google Calendar API + OAuth

Dann brauchst du:

- Google Cloud Projekt
- OAuth Consent Screen
- Credentials
- Zugriffstoken pro User speichern
- Event via API erstellen

Das ist deutlich mehr Setup. Wenn du das willst, sag Bescheid – dann geb ich dir das konkrete Flask-Setup (google-auth, oauthlib, callback routes, token storage).

## Was ich von dir brauche, damit ich's 1:1 passend mache

- Heißt dein Feld `activity.date` oder `activity.time` ?
  - Wie lang soll eine Aktivität dauern (z.B. 2 Stunden)?
- Dann setze ich dir `end_dt = start_dt + timedelta(...)` direkt ein.



ist migrating git sicher? was passiert genau beim migraten?

Mit "migrating" in eurem Flask/SQLAlchemy-Projekt meinst du sehr wahrscheinlich **Datenbank-Migrationen** (Flask-Migrate/Alembic), nicht Git-Migration. Das ist

grundsätzlich sicher – wenn du verstehst, was ausgeführt wird und ein Backup hast.

## Was passiert beim “Migraten” genau?

**Kurz:** Du änderst dein SQLAlchemy-Model → Alembic erzeugt daraus ein Skript → beim Upgrade wird dieses Skript als SQL auf die DB angewendet.

Typischer Ablauf:

1. Du änderst Modelle ( `db.py` )
2. `flask db migrate -m "..."`
  - Alembic vergleicht **Modelle vs. aktuelles DB-Schema**
  - erzeugt eine Datei in `migrations/versions/...py`
  - da stehen Operationen wie `op.add_column(...)`, `op.create_table(...)`,  
`op.drop_column(...)`
3. `flask db upgrade`
  - führt diese Operationen auf der DB aus
  - schreibt den aktuellen Stand in eine Tabelle `alembic_version`

## Ist das “sicher”?

✅ Ja, wenn...

- du die erzeugte Migrationsdatei **kurz anschaust** (sehr wichtig!)
- du vorher ein **Backup** machst (bei SQLite: DB-Datei kopieren)
- du im Team eine klare Reihenfolge hast: **Pull** → **venv** → **upgrade**

## ⚠️ Risiken / wann es schiefgehen kann

- **Datenverlust**, wenn eine Migration `drop_column`, `drop_table` macht
- **Autogenerate liegt falsch** (z.B. bei Umbenennungen: Alembic denkt “alte Spalte löschen, neue anlegen”)
- **Mehrere Leute erstellen Migrationen parallel**, dann gibt’s Konflikte / Reihenfolgeprobleme
- SQLite kann nicht alles (z.B. manche Alter-Table-Operationen werden intern als “Tabelle neu bauen” umgesetzt)

## “Ist migrating git sicher?”

Migrationen sind **nur Dateien im Repo** (Python-Skripte).

Das “Gefährliche” ist nicht Git, sondern das, was beim `upgrade` auf der **DB** passiert.

- Git: speichert Migration-Skripte → sicher
- DB: wird verändert → hier brauchst du Backup/Review

## Best Practice für euch (Team + GitHub)

1. migrations/ ins Repo committen (nicht ignorieren)
2. Jeder macht nach Pull:
  - `pip install -r requirements.txt`
  - `flask db upgrade`
3. Vor großen Änderungen: DB sichern
  - SQLite: einfach `app.db` kopieren

## Quick Backup (SQLite)

Wenn eure DB eine Datei ist (z.B. `app.db`):

- kopieren: `app.db` → `app_backup.db`
- oder über Terminal:
  - Windows: `copy app.db app_backup.db`

## Was du immer checken solltest

Öffne die generierte Datei unter `migrations/versions/...py` und guck:

- Ist es nur `add_column` / `create_table` ? → meist safe
- Siehst du `drop_*` ? → dann aufpassen (Datenverlust möglich)

Wenn du willst: Paste mir den Inhalt deiner zuletzt erzeugten `migrations/versions/...py` Datei (oder die `upgrade()` Funktion daraus). Dann sag ich dir ganz konkret, ob sie "safe" ist und was sie in eurer DB verändern wird.



```
new_activity = Activity(
 wg_id=1, # wg_id = user.wg_id
 created_by=user.user_id,
 title=form.title.data,
 description=form.description.data,
 date=form.date.data,
 updated_at=form.updated_at.data,
 location=form.location.data,
 participants=
 max_participants=form.max_participants.data,
 created_at=db.func.now())
```

) wie würde ich hier participants darstellen, wenn jeder einzeln joinen kann

Wenn "jeder einzeln joinen kann", dann ist participants= **keine einzelne Spalte** (kein String/Int), sondern eine **Beziehung (Many-to-Many)** zwischen Activity und User .

Der Standard-Ansatz: **Join-Tabelle** ACTIVITY\_PARTICIPANT (oder ActivityParticipant ), in der jede Teilnahme als eigener Datensatz steht.

## 1) Datenmodell: Join-Tabelle

python

 Code kopieren

```
class ActivityParticipant(db.Model):
 __tablename__ = "ACTIVITY_PARTICIPANT"
 id = db.Column(db.Integer, primary_key=True)
 activity_id = db.Column(db.Integer, db.ForeignKey("ACTIVITY.activity_id"), nullable=False)
 user_id = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=False)
 joined_at = db.Column(db.DateTime, server_default=db.func.now())

 activity = db.relationship("Activity", back_populates="participants")
 user = db.relationship("User", back_populates="joined_activities")

 __table_args__ = (db.UniqueConstraint("activity_id", "user_id", name="uq_acti
```

&lt;

&gt;

In Activity:

python

 Code kopieren

```
class Activity(db.Model):
 __tablename__ = "ACTIVITY"
 activity_id = db.Column(db.Integer, primary_key=True)
 # ... deine Felder ...
 max_participants = db.Column(db.Integer, nullable=True)

 participants = db.relationship(
 "ActivityParticipant",
 back_populates="activity",
```

```
 cascade="all, delete-orphan"
)
```

In User :

python

 Code kopieren

```
class User(db.Model):
 __tablename__ = "USER"
 user_id = db.Column(db.Integer, primary_key=True)
 # ...

 joined_activities = db.relationship("ActivityParticipant", back_populates="us
```



## 2) Beim Erstellen der Activity: participants= setzt du NICHT

Du erstellst die Activity wie gehabt – und wenn der Ersteller automatisch teilnehmen soll, fügst du **nach dem Commit** einen Participant-Eintrag hinzu:

python

 Code kopieren

```
new_activity = Activity(
 wg_id=1,
 created_by=user.user_id,
 title=form.title.data,
 description=form.description.data,
 date=form.date.data,
 location=form.location.data,
 max_participants=form.max_participants.data,
 created_at=db.func.now()
)
db.session.add(new_activity)
db.session.commit()

Optional: Ersteller joined automatisch
db.session.add(ActivityParticipant(activity_id=new_activity.activity_id, user_id=
db.session.commit()
```



updated\_at=form.updated\_at.data würde ich weglassen und serverseitig setzen (z.B. onupdate), sonst ist es oft None/kaputt.

### 3) Join-Route: User klickt "Join"

python

 Code kopieren

```
@app.route("/activities/<int:activity_id>/join", methods=["POST"])
def join_activity(activity_id):
 user_id = session.get("user_id") # oder testuser
 activity = Activity.query.get_or_404(activity_id)

 # schon dabei?
 exists = ActivityParticipant.query.filter_by(activity_id=activity_id, user_id=
 if exists:
 flash("Du bist schon beigetreten.", "error")
 return redirect(url_for("activity_board"))

 # max prüfen (nur wenn gesetzt)
 current_count = ActivityParticipant.query.filter_by(activity_id=activity_id).
 if activity.max_participants and current_count >= activity.max_participants:
 flash("Aktivität ist voll.", "error")
 return redirect(url_for("activity_board"))

 db.session.add(ActivityParticipant(activity_id=activity_id, user_id=user_id))
 db.session.commit()
 flash("Du bist beigetreten!", "success")
 return redirect(url_for("activity_board"))
```

Optional: /leave analog (Datensatz löschen).

### 4) Template: Teilnehmer anzeigen

Anzahl:

jinja2

 Code kopieren

```
{% set count = activity.participants|length %}
<small>{{ count }}{% if activity.max_participants %}/{{ activity.max_participants
```

Liste:

jinja2

 Code kopieren

```

 {% for p in activity.participants %}
 {{ p.user.username }}
 {% endfor %}

```

Join-Button:

jinja2

 Code kopieren

```
<form method="POST" action="{{ url_for('join_activity', activity_id=activity.acti
 <button class="btn btn-sm btn-outline-primary" type="submit">Join</button>
</form>
```

<

>

### Kurzantwort zu deiner Zeile `participants=`

Du würdest **keine Spalte** `participants` in `Activity` befüllen. Stattdessen speicherst du Teilnehmer in der Join-Tabelle `ActivityParticipant`.

Wenn du willst, poste kurz dein aktuelles `Activity`-Model (Felder + Relationships), dann baue ich dir den exakten Code passend zu deinen Tabellennamen (`ACTIVITY`, `USER`, etc.) zusammen.

    ...

kann man das auch ohne `activityParticipant` machen?  
Einfach vom user

Ja – **ohne extra** `ActivityParticipant`-**Model** geht das auch. Du brauchst trotzdem *irgendeine* Join-Tabelle in der DB, aber die kannst du als **“pure association table”** definieren (ohne eigene Klasse). Das ist die übliche “einfacher” Variante.

 **Variante: Many-to-Many nur mit Association Table**

## 1) Join-Tabelle (kein Model)

python

 Code kopieren

```
activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer, db.ForeignKey("ACTIVITY.activity_id"), p
 db.Column("user_id", db.Integer, db.ForeignKey("USER.user_id"), primary_key=T
)
```

&lt;

&gt;

## 2) Relationship in Activity und User

python

 Code kopieren

```
class Activity(db.Model):
 __tablename__ = "ACTIVITY"
 activity_id = db.Column(db.Integer, primary_key=True)
 # ... deine Felder ...
 max_participants = db.Column(db.Integer, nullable=True)

 participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
)

class User(db.Model):
 __tablename__ = "USER"
 user_id = db.Column(db.Integer, primary_key=True)
 # ...

 joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
)
```

## 3) Beim Erstellen: participants NICHT setzen

Du speicherst die Activity normal:

python

 Code kopieren



```

new_activity = Activity(
 wg_id=1,
 created_by=user.user_id,
 title=form.title.data,
 description=form.description.data,
 date=form.date.data,
 location=form.location.data,
 max_participants=form.max_participants.data,
 created_at=db.func.now()
)
db.session.add(new_activity)
db.session.commit()

```

Optional: Ersteller automatisch beitreten lassen:

python

 Code kopieren

```

new_activity.participants.append(user)
db.session.commit()

```

## 4) Join-Route: User klickt "Join"

python

 Code kopieren

```

@app.route("/activities/<int:activity_id>/join", methods=["POST"])
def join_activity(activity_id):
 user_id = session.get("user_id", 1) # später richtig
 user = User.query.get_or_404(user_id)
 activity = Activity.query.get_or_404(activity_id)

 if user in activity.participants:
 flash("Du bist schon beigetreten.", "error")
 return redirect(url_for("activity_board"))

 # max prüfen
 if activity.max_participants and len(activity.participants) >= activity.max_p
 flash("Aktivität ist voll.", "error")
 return redirect(url_for("activity_board"))

 activity.participants.append(user)
 db.session.commit()

```

```
flash("Du bist beigetreten!", "success")
return redirect(url_for("activity_board"))
```

Leave wäre:

python

 Code kopieren

```
activity.participants.remove(user)
db.session.commit()
```

## 5) Template: Teilnehmer anzeigen

jinja2

 Code kopieren

```
<small>
 {{ activity.participants|length }}
 {% if activity.max_participants %}/{% activity.max_participants %}{% endif %}
 Teilnehmer
</small>
```

## Wann brauchst du doch ein ActivityParticipant -Model?

Nur wenn du **zusätzliche Infos pro Teilnahme** speichern willst, z.B. `joined_at` , `status` , `role` , `comment` . Ohne das reicht die Association Table komplett.

Wenn du willst, poste dein `Activity` -Model kurz, dann schreibe ich dir die Join-Tabelle exakt mit deinen Tabellennamen/PKs rein.

    ...

```
activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer,
db.ForeignKey("ACTIVITY.activity_id"), primary_key=True),
 db.Column("user_id", db.Integer,
```

```
db.ForeignKey("USER.user_id", primary_key=True),
) wo packe ich das hin?
```

Pack die `db.Table(...)` in die gleiche Datei wie deine Models (bei dir `db.py`), oberhalb der Klassen `Activity` und `User`, damit beide sie beim Import schon kennen.

### ✓ Typischer Aufbau in `db.py`

python

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()

✓ HIER hin (global, oben in der Datei)
activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer, db.ForeignKey("ACTIVITY.activity_id"), p
 db.Column("user_id", db.Integer, db.ForeignKey("USER.user_id"), primary_key=T
)

class User(db.Model):
 __tablename__ = "USER"
 user_id = db.Column(db.Integer, primary_key=True)
 # ...
 joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
)

class Activity(db.Model):
 __tablename__ = "ACTIVITY"
 activity_id = db.Column(db.Integer, primary_key=True)
 # ...
 participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
)
```



## Wichtig danach

- Du musst das DB-Schema anpassen (Migration oder DB neu).  
Sonst existiert die Tabelle `ACTIVITY_PARTICIPANTS` in SQLite nicht.

Wenn du aktuell keine Migration nutzt und es dir egal ist, kannst du (wie bei dir oft) die SQLite-Datei löschen und `db.create_all()` neu laufen lassen.

 Code kopieren



Jetzt mal eine generelle Frage zu git. Wenn ich beispielsweise jetzt das activityboard auf eine branch mache und mein Partner dann die Registrierung bzw. das Login etc auf einem anderen macht, wir aber beide auf `db.py` zugreifen müssen, wie würde man das lösen? Oder gibt es am Ende keine Probleme beim mergen auf Main?

Wenn ihr beide parallel arbeitet und **beide** `db.py` **anfassen müsst**, ist das ganz normal in Git – aber ihr solltet ein paar Regeln haben, sonst gibt's Merge-Stress und (wichtiger) DB-Schema-Chaos.

## Was passiert beim Mergen?

- **Git-Merge:** `db.py` ist eine Textdatei. Wenn ihr an **verschiedenen Stellen** editiert, merged Git meist automatisch.
- Wenn ihr **dieselben Zeilen/denselben Block** ändert (z.B. beide ändern `class User`), gibt's einen **Merge-Conflict**, den ihr manuell auflöst.
- **DB-Schema:** Das ist der eigentliche Knackpunkt. Selbst wenn Git "sauber" merged, kann die lokale SQLite/DB bei euch unterschiedlich sein → dann gibt's Runtime-Errors wie „no such column“.

## So löst man es sauber (empfohlen)

1) `db.py` darf ruhig gemeinsam geändert werden – aber koordiniert

- Sprecht euch ab: **wer baut welche Modelle/Spalten**
- Haltet Änderungen klein und logisch getrennt (ein Feature = ein Commit)

2) Nutzt Migrationen (Flask-Migrate/Alembic) und committet sie ins Repo

Das ist der wichtigste Punkt, damit am Ende **keiner** "no such column" hat.

Workflow im Team:

1. Jeder arbeitet in Feature-Branch.
2. Wenn du `db.py` änderst → du machst **Migration**:
  - `flask db migrate -m "add activity date"`
  - `flask db upgrade`
3. Committe **beides**:
  - `db.py`
  - `migrations/versions/*.py`
4. Partner pulled/merged später und macht:
  - `flask db upgrade`→ DB wird automatisch auf den richtigen Stand gebracht.

**Ohne Migrationen** müsst ihr ständig DB löschen/neu erstellen, und beim Merge kommt Chaos.

### 3) "Nur eine Person erstellt Migrationen" (Option für Anfänger)

Für Uni-Projekte oft am stressfreisten:

- Beide ändern `db.py` in ihren Branches,
- aber Migrationen macht nur **eine Person**, wenn's auf `main` zusammengeführt wird.  
Nachteil: kurzfristig arbeiten manche gegen ein DB-Schema, das lokal noch nicht passt.

### Wenn ihr keine Migrationen nutzen wollt (quick & dirty)

Dann müsst ihr akzeptieren:

- Nach Merge auf `main` wird oft jemand **die DB löschen** und `db.create_all()` neu laufen lassen.
- Das ist ok, wenn ihr keine wichtigen Daten persistieren müsst.

Dann unbedingt:

- SQLite DB-Datei **niemals committen**
- Seed/Testdaten über ein Script erzeugen (optional)

### Praktische Git-Regeln, damit `db.py` nicht eskaliert

- Jeder macht **kleine PRs** (nicht 2000 Zeilen auf einmal)
- Ein Feature pro Branch

- Vor dem Merge: `git pull` / `git merge main` in den Feature-Branch, Konflikte dort lösen
- Bei Konflikten in `db.py` : gemeinsam 5 Minuten drüberschauen, sauber zusammenführen

## Antwort auf deine Frage „Gibt es am Ende keine Probleme beim Mergen?“

Git-seitig: **meistens ok**, außer ihr ändert dieselben Stellen → dann Conflict, aber lösbar.

DB-seitig: **Probleme sind sehr wahrscheinlich**, wenn ihr **keine Migrationen** habt. Dann hat jeder eine andere DB und nach dem Merge fehlen Spalten/Tabellen.

### Empfehlung für euch (realistisch und stabil)

1. Flask-Migrate einrichten
2. `migrations/` ins Repo
3. Jeder macht Migration, wenn er Schema ändert
4. Nach Pull/Merge immer `flask db upgrade`

Wenn du willst, schreib mir kurz:

- Nutzt ihr SQLite?
- Habt ihr schon `migrations/` Ordner?

Dann gebe ich dir den exakten "einmal einrichten" Schritt-für-Schritt für euer Projekt.



```
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py:22: SAWarning:
relationship 'ShoppingItem.added_by_user' will copy
column USER.user_id to column
SHOPPING_ITEM.added_by, which conflicts with
relationship(s): 'User.added_items' (copies USER.user_id to
SHOPPING_ITEM.added_by). If this is not the intention,
consider if these relationships should be linked with
back_populates, or if viewonly=True should be applied to
one or more if they are read-only. For the less common
case that foreign key constraints are partially overlapping,
```

the `orm.foreign()` annotation can be used to isolate the columns that should be written towards. To silence this warning, add the parameter `'overlaps="added_items"'` to the `'ShoppingItem.added_by_user'` relationship.

(Background on this warning at:

<https://sqlalche.me/e/20/qzyx>) (This warning originated from the `configure_mappers()` process, which was invoked automatically in response to a user-initiated operation.)

```
return cls.query_class(
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py:22: SAWarning:
relationship 'ShoppingItem.assigned_to_user' will copy
column USER.user_id to column
SHOPPING_ITEM.assigned_to, which conflicts with
relationship(s): 'User.assigned_items' (copies USER.user_id
to SHOPPING_ITEM.assigned_to). If this is not the
intention, consider if these relationships should be linked
with back_populates, or if viewonly=True should be
applied to one or more if they are read-only. For the less
common case that foreign key constraints are partially
overlapping, the orm.foreign() annotation can be used to
isolate the columns that should be written towards. To
silence this warning, add the parameter
'overlaps="assigned_items"' to the
'ShoppingItem.assigned_to_user' relationship. (Background
on this warning at: https://sqlalche.me/e/20/qzyx) (This
warning originated from the configure_mappers()
process, which was invoked automatically in response to a
user-initiated operation.)
```

```
return cls.query_class(
[2026-01-06 13:52:21,872] ERROR in app: Exception on
/activityboard/ [GET]
```

Traceback (most recent call last):

```
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\mapper.py", line 2531, in
get_property
```

```
return self._props[key]
~~~~~^ ^ ^ ^ ^
```

KeyError: 'join\_activity'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

```

File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full_dispatch_request
    rv = self.handle_user_exception(e)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full_dispatch_request
    rv = self.dispatch_request()
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch_request
    return
self.ensure_sync(self.view_functions[rule.endpoint])
(**view_args) # type: ignore[no-any-return]

~~~~~
~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 291, in activity_board
    all_users = User.query.all()
                ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask_sqlalchemy\model.py", line 22, in __get__
    return cls.query_class(
           ~~~~~^
 cls, session=cls.__fsa__.session() # type: ignore[arg-

```



type]

```
^^
^^
)
^
```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\query.py", line 276, in \_\_init\_\_  
self.\_set\_entities(entities)  
~~~~~^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\query.py", line 289, in \_set\_entities  
coercions.expect(  
~~~~~^  
roles.ColumnsClauseRole,  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
...<2 lines>...  
post\_inspect=True,  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
)  
^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\sql\coercions.py", line 389, in expect  
insp.\_post\_inspect

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\util\langhelpers.py", line 1338, in \_\_get\_\_  
obj.\_\_dict\_\_[self.\_\_name\_\_] = result = self.fget(obj)  
~~~~~^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\mapper.py", line 2733, in \_post\_inspect

```
self._check_configure()
```

```
~~~~~^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\sqlalchemy\orm\mapper.py", line 2410, in  
_check_configure
```

```
    _configure_registries({self.registry}, cascade=True)
```

```
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^  
^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\mapper.py", line 4227, in
_configure_registries
```

```
 _do_configure_registries(registries, cascade)
```

```
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^  
^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\sqlalchemy\orm\mapper.py", line 4268, in  
_do_configure_registries
```

```
    mapper._post_configure_properties()
```

```
~~~~~^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\mapper.py", line 2427, in
_post_configure_properties
```

```
 prop.init()
```

```
~~~~~^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr  
am\Web-Development-Project\venv\Lib\site-  
packages\sqlalchemy\orm\interfaces.py", line 595, in init  
self.do_init()
```

```
~~~~~^ ^
```

File

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
```

```

packages\sqlalchemy\orm\relationships.py", line 1660, in
do_init
 self._generate_backref()
    ~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\relationships.py", line 2144, in
_generate_backref
    self._add_reverse_property(self.back_populates)

~~~~~^
^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\relationships.py", line 1591, in
_add_reverse_property
 other = self.mapper.get_property(key,
_configure_mappers=False)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\mapper.py", line 2533, in
get_property
 raise sa_exc.InvalidRequestError(
...<3 lines>...
) from err
sqlalchemy.exc.InvalidRequestError: Mapper
'Mapper[User(USER)]' has no property 'join_activity'. If this
property was indicated from other mappers or configure
events, ensure registry.configure() has been called.
127.0.0.1 - - [06/Jan/2026 13:52:21] "GET /activityboard/
HTTP/1.1" 500 - was ist hier der fehler?

```

Der entscheidende Teil ist:

```
InvalidRequestError: Mapper 'User(USER)' has no property 'join_activity'
```

Heißt: **Irgendein** `db.relationship(..., back_populates="join_activity")` **zeigt auf ein Attribut, das es in `User` nicht gibt.**

Das ist ein *Model-Config Fehler*, deshalb crasht schon `User.query.all()` (SQLAlchemy versucht beim ersten Query alle Mapper zu konfigurieren).

## ✓ Wo genau ist der Fehler?

Sehr wahrscheinlich hast du in **Activity** (oder in der Many-to-Many Beziehung) sowas stehen wie:

python

 Code kopieren

```
participants = db.relationship("User", secondary=activity_participants,
 back_populates="join_activity")
```

oder irgendwo:

python

 Code kopieren

```
db.relationship("User", back_populates="join_activity")
```

Aber in deiner `User` -Klasse existiert kein:

python

 Code kopieren

```
join_activity = ...
```

## ✓ Fix (wie es richtig heißen sollte)

Wenn du Many-to-Many so machen willst:

### Association Table:

python

 Code kopieren

```
activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer, db.ForeignKey("ACTIVITY.activity_id"), p
 db.Column("user_id", db.Integer, db.ForeignKey("USER.user_id"), primary_key=T
)
```

In **Activity**:

python

```
participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
)
```

**In User :**

python

 Code kopieren

```
joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
)
```

**Wichtig:** `back_populates` Namen müssen auf beiden Seiten existieren und exakt gleich geschrieben sein.

## Was du jetzt tun sollst (schnell)

1. Suche in `db.py` nach `join_activity` (Strg+F).
2. Wo du es findest:
  - entweder **umbenennen** auf `joined_activities`
  - oder in `user` eine Relationship mit genau diesem Namen anlegen (aber besser: konsistent benennen)

## Nebenbei: SAWarnings zu ShoppingItem (nicht der Crash)

Die Warnungen sind unabhängig. Die kannst du später mit `overlaps=...` oder sauberem `back_populates` beheben. Der 500er kommt rein von `join_activity`.

Wenn du mir die `Activity` - und `User` -Relationship-Blöcke aus `db.py` (nur die paar Zeilen mit `participants` / `joined_...` / `back_populates` ) hier rein kopierst, sag ich dir exakt, welche Zeile falsch ist und wie du sie 1:1 ersetzen musst.

    ...

```

class Activity(db.Model):
 __tablename__ = 'ACTIVITY'
 activity_id = db.Column(db.Integer, primary_key=True,
index=True)
 wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
 created_by = db.Column(db.Integer,
db.ForeignKey('USER.user_id'), nullable=False)
 title = db.Column(db.String, nullable=False)
 description = db.Column(db.String)
 date = db.Column(db.DateTime, nullable=False)
 location = db.Column(db.String)
 max_participants = db.Column(db.Integer,
nullable=True)

 participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
) #participants

 created_at = db.Column(db.DateTime,
server_default=db.func.now())
 updated_at = db.Column(db.DateTime, nullable=False)
#new time varivable as to not migrate
#Beziehungen
 wg = db.relationship('Wg', back_populates='activities')
 creator = db.relationship('User',
back_populates='activities_created')

 joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
) #joined_activities
meine activity class

```

Der Fehler steckt direkt in deiner `Activity`-Klasse: Du hast `joined_activities` fälschlicherweise in `Activity` definiert.

`joined_activities` gehört in die `User`-Klasse, nicht in `Activity`.

Aktuell sagst du sinngemäß:

- `Activity` hat `participants` (`User`) ✓
- `Activity` hat auch `joined_activities` (`Activity`) ✗ → das ergibt keinen Sinn und verwirrt SQLAlchemy beim Mapping (und führt zu solchen "property fehlt"-Fehlern).

## ✓ So muss es aussehen

### 1) `Activity` (nur `participants`, kein `joined_activities`)

python

 Code kopieren

```
class Activity(db.Model):
 __tablename__ = 'ACTIVITY'
 activity_id = db.Column(db.Integer, primary_key=True, index=True)
 wg_id = db.Column(db.Integer, db.ForeignKey('WG.wg_id'), nullable=False)
 created_by = db.Column(db.Integer, db.ForeignKey('USER.user_id'), nullable=False)
 title = db.Column(db.String, nullable=False)
 description = db.Column(db.String)
 date = db.Column(db.DateTime, nullable=False)
 location = db.Column(db.String)
 max_participants = db.Column(db.Integer, nullable=True)

 participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
)

 created_at = db.Column(db.DateTime, server_default=db.func.now())
 updated_at = db.Column(db.DateTime, nullable=False)

 wg = db.relationship('Wg', back_populates='activities')
 creator = db.relationship('User', back_populates='activities_created')
```

### 2) `User` (hier kommt `joined_activities` rein)

python

 Code kopieren

```

class User(db.Model):
 __tablename__ = 'USER'
 user_id = db.Column(db.Integer, primary_key=True, index=True)
 # ...

 joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
)

```

## Warum kam dann "User has no property join\_activity"?

Weil irgendwo in deinem Code (wahrscheinlich in `User` oder in einer anderen Relationship) noch `back_populates="join_activity"` steht **oder** du hast dich beim Namen vertippt.

✅ Mach bitte einmal **Strg+F** nach `join_activity` in deinem Projekt.

Wenn du das findest: ersetzen durch `joined_activities` (oder umgekehrt konsistent).

## Noch 2 wichtige Hinweise

1. `updated_at = db.Column(..., nullable=False)` wird dir beim Insert Probleme machen, wenn du keinen Wert setzt.

Besser:

python

 Code kopieren

```
updated_at = db.Column(db.DateTime, server_default=db.func.now(), onupdate=db.fun
```

&lt;

&gt;

Dann musst du es nicht im Form setzen.

2. Nach Model-Änderungen: DB-Schema anpassen (Migration oder DB neu). Sonst kommen wieder "no such column / no such table".



Wenn du mir noch kurz deine `User`-Klasse (den Teil mit `activities/relationships`) postest, sag ich dir exakt, welche Zeile mit `join_activity` falsch ist und wie sie aussehen muss.



```
activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer,
db.ForeignKey("ACTIVITY.activity_id"), primary_key=True),
 db.Column("user_id", db.Integer,
db.ForeignKey("USER.user_id"), primary_key=True),
)
```

```
class User(db.Model):
 __tablename__ = 'USER'
 user_id = db.Column(db.Integer, primary_key=True,
index=True)
 username = db.Column(db.String, nullable=False,
unique=True)
 password_hash = db.Column(db.String, nullable=False,
unique=True)
 email = db.Column(db.String, nullable=False,
unique=True)
 role = db.Column(db.String, nullable=False)
 wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=True)
 #Beziehungen
 wg = db.relationship('Wg', back_populates='users')
 cleaning_tasks = db.relationship('CleaningTask',
back_populates='assigned_user')
 activities_created = db.relationship('Activity',
back_populates='creator')
 ideas = db.relationship('Idea',
back_populates='creator',foreign_keys='Idea.created_by')
 idea_comments = db.relationship('Idea_Comment',
back_populates='user')
 idea_likes = db.relationship('Idea_Like',
back_populates='user')
 added_items = db.relationship('ShoppingItem',
foreign_keys='ShoppingItem.added_by')
```

```

 assigned_items = db.relationship('ShoppingItem',
 foreign_keys='ShoppingItem.assigned_to')
 joined_activities = db.relationship(
 "Activity",
 secondary=participants,
 back_populates="participants"
) #joined_activities muss activity_participants in user
 rein?

```

Ja — in User **muss** secondary=activity\_participants **stehen**, und bei dir ist es gerade doppelt falsch:

1. Du nutzt secondary=participants → diese Variable gibt's nicht (und participants ist außerdem der Name der Relationship in Activity, nicht die Tabelle).
2. back\_populates="participants" ist korrekt, **aber nur**, wenn Activity.participants genauso heißt (tut es).

✅ So musst du es schreiben:

python

```

joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants, # ✅ das ist die db.Table(...)
 back_populates="participants"
)

```

Also dein kompletter Block (nur der Teil) korrekt:

 Code kopieren

python

```

activity_participants = db.Table(
 "ACTIVITY_PARTICIPANTS",
 db.Column("activity_id", db.Integer, db.ForeignKey("ACTIVITY.activity_id"), p
 db.Column("user_id", db.Integer, db.ForeignKey("USER.user_id"), primary_key=T
)

class User(db.Model):
 __tablename__ = 'USER'
 user_id = db.Column(db.Integer, primary_key=True, index=True)
 # ... rest ...

```

```
joined_activities = db.relationship(
 "Activity",
 secondary=activity_participants,
 back_populates="participants"
)
```

Und in Activity muss das Gegenstück so bleiben:

python

```
participants = db.relationship(
 "User",
 secondary=activity_participants,
 back_populates="joined_activities"
)
```

 Code kopieren

## Danach wichtig

- Wenn du das vorher falsch drin hattest, starte den Server neu.
- Und: Die Tabelle `ACTIVITY_PARTICIPANTS` muss in der DB existieren (Migration oder DB neu), sonst kommt später "no such table".

Wenn du willst, poste noch kurz den Relationship-Teil aus deiner `Activity`-Klasse (nur `participants + creator`), dann checke ich, ob alle `back_populates` Namen 100% matchen.



```
se to a user-initiated operation.)
return cls.query_class(
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgra
m\Web-Development-Project\venv\Lib\site-
packages\flask_sqlalchemy\model.py:22: SAWarning:
relationship 'ShoppingItem.assigned_to_user' will copy
column USER.user_id to column
SHOPPING_ITEM.assigned_to, which conflicts with
relationship(s): 'User.assigned_items' (copies USER.user_id
to SHOPPING_ITEM.assigned_to). If this is not the
intention, consider if these relationships should be linked
with back_populates, or if viewonly=True should be
```

applied to one or more if they are read-only. For the less common case that foreign key constraints are partially overlapping, the `orm.foreign()` annotation can be used to isolate the columns that should be written towards. To silence this warning, add the parameter `'overlaps="assigned_items"'` to the `'ShoppingItem.assigned_to_user'` relationship. (Background on this warning at: <https://sqlalche.me/e/20/qzyx>) (This warning originated from the `configure_mappers()` process, which was invoked automatically in response to a user-initiated operation.)

```
return cls.query_class(
[2026-01-06 14:05:32,408] ERROR in app: Exception on
/activity/1/join_activity [POST]
Traceback (most recent call last):
 File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
 self.dialect.do_execute(
        ~~~~~^
        cursor, str_statement, effective_parameters, context

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^
)
^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
    cursor.execute(statement, parameters)

~~~~~^
sqlite3.IntegrityError: UNIQUE constraint failed:
ACTIVITY_PARTICIPANTS.activity_id,
ACTIVITY_PARTICIPANTS.user_id
```

The above exception was the direct cause of the following exception:

Traceback (most recent call last):

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi\_app  
response = self.full\_dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full\_dispatch\_request  
rv = self.handle\_user\_exception(e)

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full\_dispatch\_request  
rv = self.dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch\_request  
return

self.ensure\_sync(self.view\_functions[rule.endpoint])  
(\*view\_args) # type: ignore[no-any-return]

~~~~~  
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 344, in join\_activity  
db.session.commit()

~~~~~^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\scoping.py", line 597, in commit  
return self.\_proxied.commit()

~~~~~^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 2030, in commit

```
trans.commit(_to_root=True)
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^
File "<string>", line 2, in commit
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\state_changes.py", line 137, in
_go
    ret_value = fn(self, *arg, **kw)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 1311, in
commit
    self._prepare_impl()
~~~~~^ ^
File "<string>", line 2, in _prepare_impl
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\state_changes.py", line 137, in
_go
 ret_value = fn(self, *arg, **kw)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 1286, in
_prepare_impl
 self.session.flush()
~~~~~^ ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 4331, in flush
    self._flush(objects)
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 4466, in _flush
 with util.safe_reraise():
~~~~~^ ^
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\util\langhelpers.py", line 224, in
__exit__
    raise exc_value.with_traceback(exc_tb)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 4427, in _flush
    flush_context.execute()
    ~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\unitofwork.py", line 466, in
execute
 rec.execute(self)
    ~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\unitofwork.py", line 591, in
execute
    self.dependency_processor.process_saves(uow, states)

~~~~~
~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\dependency.py", line 1195, in
process_saves
 self._run_crud(
    ~~~~~^
        uowcommit, secondary_insert, secondary_update,
secondary_delete

    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram
```

```
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\dependency.py", line 1258, in
_run_crud
```

```
    connection.execute(statement, secondary_insert)
```

```
~~~~~^
^
```

```
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1419, in
execute
```

```
 return meth(
 self,
 distilled_parameters,
 execution_options or NO_OPTIONS,
)
```

```
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\sql\elements.py", line 527, in
_execute_on_connection
```

```
 return connection._execute_clauseelement(
```

```
~~~~~^
    self, distilled_params, execution_options
```

```
^
^
```

```
)
```

```
^
```

```
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1641, in
_execute_clauseelement
```

```
    ret = self._execute_context(
        dialect,
        ...<8 lines>...
        cache_hit=cache_hit,
    )
```

```
File
```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
```



```
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1846, in
_execute_context
    return self._exec_single_context(
        ~~~~~^
 dialect, context, statement, parameters

^^
^
)
 ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1986, in
_exec_single_context
 self._handle_dbapi_exception(
        ~~~~~^
        e, str_statement, effective_parameters, cursor, context

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 2363, in
_handle_dbapi_exception
    raise sqlalchemy_exception.with_traceback(exc_info[2])
from e
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
    self.dialect.do_execute(
        ~~~~~^
 cursor, str_statement, effective_parameters, context

^^
^^
)
```

```

^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
 cursor.execute(statement, parameters)

~~~~~^
sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError)
UNIQUE constraint failed:
ACTIVITY_PARTICIPANTS.activity_id,
ACTIVITY_PARTICIPANTS.user_id
[SQL: INSERT INTO "ACTIVITY_PARTICIPANTS" (activity_id,
user_id) VALUES (?, ?)]
[parameters: (1, 1)]
(Background on this error at:
https://sqlalche.me/e/20/gkpj)
127.0.0.1 - - [06/Jan/2026 14:05:32] "POST
/activity/1/join_activity HTTP/1.1" 500 -

heißt das, ich muss unique constraints hinzufügen und
dann die db migraten?

```

Nein — das Gegenteil 😊: **der Unique-Constraint ist bereits da** (weil deine Join-Tabelle `ACTIVITY_PARTICIPANTS` einen **Composite Primary Key** hat: `activity_id + user_id`).

Die Fehlermeldung bedeutet einfach:

✅ **Der User (`user_id=1`) ist schon bei Activity (`activity_id=1`) gejoint**

und du versuchst ihn **nochmal** einzutragen → daher `UNIQUE constraint failed`.

Du musst dafür **nichts migraten**. Du musst in deiner Join-Route verhindern, dass doppelt gejoint wird (oder den Fehler abfangen).

✅ **Fix in der Route: vorher prüfen, ob schon dabei**

python

```

@app.route("/activity/<int:activity_id>/join_activity", methods=["POST"])
def join_activity(activity_id):
    user_id = session.get("user_id", 1) # später richtig
    user = User.query.get_or_404(user_id)
    activity = Activity.query.get_or_404(activity_id)

    # ✓ schon Teilnehmer?
    if user in activity.participants:
        flash("Du bist schon beigetreten.", "error")
        return redirect(url_for("activity_board"))

    # ✓ max participants prüfen (optional)
    if activity.max_participants and len(activity.participants) >= activity.max_p
        flash("Aktivität ist voll.", "error")
        return redirect(url_for("activity_board"))

    activity.participants.append(user)
    db.session.commit()
    flash("Beigetreten!", "success")
    return redirect(url_for("activity_board"))

```



Code kopieren

## ✓ Optional: trotzdem sicher abfangen (falls Race Condition)

python

```

from sqlalchemy.exc import IntegrityError

try:
    activity.participants.append(user)
    db.session.commit()
except IntegrityError:
    db.session.rollback()
    flash("Du bist schon beigetreten.", "error")

```

Code kopieren

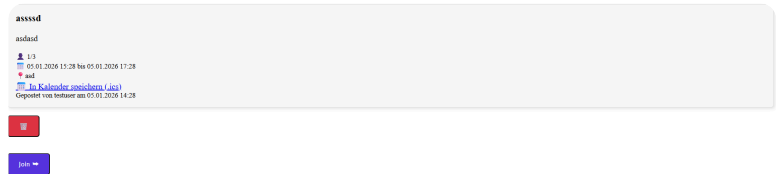
## Warum das bei dir passiert

Entweder:

- du klickst "Join" zweimal, oder

- du joinst automatisch beim Erstellen **und** joinst danach nochmal per Button.

Wenn du willst: ich kann dir auch kurz zeigen, wie du im Template den Join-Button ausblendest, wenn der User schon Teilnehmer ist.



wie mache ich es, dass diese beiden buttons unter der card (delete und join) in der card drinne sind (löschen rechts am rande der card, join drunten unten links?

Du musst **Join + Delete** in die `.activity-card` reinziehen und die Card als "Position-Container" machen. Dann kannst du **Delete oben rechts** und **Join unten links** "ankleben".

## 1) HTML-Struktur (Buttons IN der Card)

Pack die Buttons direkt **am Anfang** (Delete) bzw. **am Ende** (Join) innerhalb deiner `activity-card`:

html

```
<div class="activity-card">

  <!-- Delete oben rechts -->
  <form method="POST" action="{ { url_for('delete_activity', activity_id=activity.
    <button type="submit" class="btn btn-sm btn-danger"
      onclick="return confirm('Aktivität wirklich löschen?');">🗑️</button>
  </form>

  <h3 class="mb-1">{ { activity.title }}</h3>
  <p class="activity-desc mb-2">{ { activity.description }}</p>

  <small class="text-muted">
    Gepostet von { { activity.user.username }} am { { activity.created_at.strftime(
  </small>
```

```

    <!-- Join unten links -->
    <form method="POST" action="{ url_for('join_activity', activity_id=activity.ac
      <button type="submit" class="btn btn-sm btn-primary">Join →</button>
    </form>

</div>

```

Wichtig: beide `<form>` müssen **innerhalb** der `div.activity-card` sein.  
Bei dir sind sie aktuell außerhalb, deshalb landen sie unter der Card.

 Code kopieren

## 2) CSS: Card "position: relative" + Buttons absolut positionieren

CSS

```

.activity-card {
  position: relative;      /* wichtig */
  padding: 16px;
  border-radius: 10px;
  border: 1px solid #ddd;
  margin-bottom: 16px;
}

/* Delete oben rechts */
.activity-card .delete-form {
  position: absolute;
  top: 12px;
  right: 12px;
  margin: 0;
}

/* Join unten links */
.activity-card .join-form {
  position: absolute;
  left: 12px;
  bottom: 12px;
  margin: 0;
}

/* Damit Text nicht unter den Buttons "drunter läuft" */
.activity-card h3 {

```

```
padding-right: 60px; /* Platz für Delete Button */
}
.activity-card {
padding-bottom: 60px; /* Platz für Join Button */
}
```

## Mini-Check (häufigster Fehler)

Wenn es trotzdem "unter der Card" bleibt, ist fast immer:

- ein falsches `</div>` zu früh (Card wird geschlossen, bevor Forms kommen)
- oder du hast die Buttons außerhalb des `{% for %}` -Blocks

 Code kopieren

Wenn du willst, poste kurz deinen kompletten `{% for activity in activities %}` Block (nur den), dann markiere ich dir exakt, wo die schließenden `</div>` hinmüssen.



join unten rechts in der card (siehe bild)

Alles klar — dann nur die Position fürs Join ändern: **bottom + right** statt **bottom + left**.

## HTML bleibt gleich (Join-Form in der Card)

html

```
<div class="activity-card">
  <!-- Delete oben rechts -->
  <form method="POST" action="{% url_for('delete_activity', activity_id=activity.
    <button type="submit" class="btn btn-sm btn-danger"
      onclick="return confirm('Aktivität wirklich löschen?');">🗑️ </button>
  </form>

  <h3 class="mb-1">{% activity.title %}</h3>
  <p class="activity-desc mb-2">{% activity.description %}</p>

  <small class="text-muted">
    Gepostet von {% activity.user.username %} am {% activity.created_at.strftime(
  </small>
```

```
<!-- Join unten rechts -->
<form method="POST" action="{ url_for('join_activity', activity_id=activity.ac
    <button type="submit" class="btn btn-sm btn-primary">Join →</button>
</form>
</div>
```

## CSS (Join unten rechts)

CSS

```
.activity-card {
    position: relative;
    padding: 16px;
    border-radius: 10px;
    border: 1px solid #ddd;
    margin-bottom: 16px;

    padding-bottom: 60px; /* Platz für Join unten */
}

/* Delete oben rechts */
.activity-card .delete-form {
    position: absolute;
    top: 12px;
    right: 12px;
    margin: 0;
}

/* Join unten rechts */
.activity-card .join-form {
    position: absolute;
    right: 12px; /* ✅ rechts */
    bottom: 12px; /* ✅ unten */
    margin: 0;
}

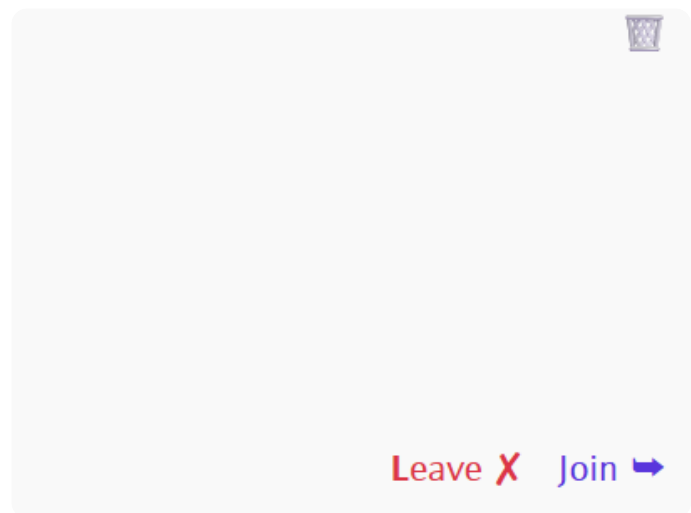
/* Platz für Delete, damit Titel nicht drunter läuft */
.activity-card h3 {
    padding-right: 60px;
}
```

Damit sitzt:

- 🗑️ oben rechts
- Join → unten rechts innerhalb der Card

Wenn Join trotzdem "rausrutscht", ist fast immer ein zu frühes `</div>` (Card wird zu früh geschlossen).

 Code kopieren



als ich leave eingefügt habe, rendern die buttons nicht mehr richtig. wie fixe ich das?

Das passiert fast immer aus 2 Gründen:

1. Beide Buttons (Join/Leave) werden gleichzeitig gerendert und liegen dann übereinander / drücken Layout kaputt.
2. Du hast zwei absolute Buttons unten rechts, die sich überlagern (weil beide `position:absolute; right:12px; bottom:12px` haben).

✅ Fix: Rendere entweder Join oder Leave (nicht beide) und gib ihnen unterschiedliche Klassen (oder gleiche Klasse, weil es eh nur 1 Button ist).

## 1) Template: Join ODER Leave (sauber)

Du brauchst irgendwo `current_user_id` aus der Session, z.B. in der Route:

python



```

return render_template("activityboard.html",
                        activities=activities,
                        current_user_id=session.get("user_id"))

```

 Code kopieren

Dann im Template:

jinja2

```

{% set is_joined = (current_user_id and (activity.participants | selectattr('user

<form method="POST"
    action="{% if is_joined %}{{ url_for('leave_activity', activity_id=activity
    class="action-form">
    <button type="submit"
        class="btn btn-sm {% if is_joined %}btn-outline-danger{% else %}btn-pri
        {% if is_joined %}Leave ✕{% else %}Join →{% endif %}
    </button>
</form>

```

&lt;

&gt;

Damit existiert unten rechts nur ein Button, also kein Chaos.

 Code kopieren

## 2) CSS: nur eine "Action"-Position unten rechts

CSS

```

.activity-card { position: relative; padding: 16px; padding-bottom: 64px; }

.activity-card .action-form{
    position: absolute;
    right: 12px;
    bottom: 12px;
    margin: 0;
}

```

 Code kopieren

## Alternative (wenn du beide Buttons zeigen willst)

Dann musst du sie nebeneinander positionieren:

html

```

<div class="card-actions">
  <!-- leave -->
  <form ...><button class="btn btn-sm btn-outline-danger">Leave ✕</button></form>
  <!-- join -->
  <form ...><button class="btn btn-sm btn-primary">Join →</button></form>
</div>

```


[Code kopieren](#)

CSS

```

.activity-card .card-actions{
  position: absolute;
  right: 12px;
  bottom: 12px;
  display: flex;
  gap: 10px;
  margin: 0;
}
.activity-card .card-actions form{ margin:0; }

```

[Code kopieren](#)

## Warum bei dir das Trash-Icon "oben rechts" noch da ist, aber die Buttons unten komisch wirken

Weil die unteren Buttons vermutlich **außerhalb** des Cards-Containers landen oder sich **überlagern** (zwei absolute Elemente auf gleicher Position).

Wenn du mir den aktuellen Block aus deiner `activity-card` (nur den HTML Teil mit Join/Leave/Delete) hier reinkopierst, sag ich dir exakt, wo ein `</div>` zu früh ist oder welche Klasse du doppelt benutzt.



```

<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
  }}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>

```

```
<a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>
```

```
<div style="display:flex; justify-content:space-between;
align-items:center; margin-bottom: 20px;">
  <h1 style="margin:0;">ActivityBoard</h1>
```

```
  <label for="modalToggleNewActivity"
    style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;
display:inline-block;">
    + Neue Aktivität
  </label>
</div>
```

```
<h2>Teile deine Aktivitäten und Erlebnisse in der
WG</h2>
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;" >
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

```
<div class="activities-list">
  {% for activity in activities %}
    <div class="activity-card">
      <form method="POST" action="{{
url_for('delete_activity', activity_id=activity.activity_id) }}"
class="delete-form">
        <button type="submit" class="delete-btn"

        onclick="return confirm('Aktivität wirklich
löschen?');">  </button>
```

```

</form>

<h3 class="mb-1">{{ activity.title }}</h3>
<p class="activity-desc mb-2">{{ activity.description }}
</p>
{% if activity.max_participants %}
  <small> 👤 {{activity.participants|length}}/{{
activity.max_participants }} </small>
{% endif %}
<br>

<small class="text-muted"> 📅 {{
activity.date.strftime('%d.%m.%Y %H:%M') }} bis {{
activity.updated_at.strftime('%d.%m.%Y %H:%M') }}
</small>
{% if activity.location %}
  <br>
  <small class="text-muted"> 📍 {{ activity.location }}
</small>
{% endif %}
<br>

<a href="{{ url_for('activity_ics',
activity_id=activity.activity_id) }}"> 📅 In Kalender
speichern (.ics)</a>
<br>
<small class="text-muted" style="margin-top: 20px;
margin-bottom: 5px;">Gepostet von {{
activity.creator.username }} am {{
activity.created_at.strftime('%d.%m.%Y %H:%M') }}
</small>
<br>

<form method="POST" action="{{
url_for('join_activity', activity_id=activity.activity_id) }}"
class="m-0">

  <button type="submit" class="join-btn"

      onclick="return confirm('Aktivität wirklich
beitreten?');">
    Join ➡
  </button>

```

```

</form>

<form method="POST" action="{{
url_for('leave_activity', activity_id=activity.activity_id) }}"
class="m-0">

    <button type="submit" class="leave-btn"

        onclick="return confirm('Von der Aktivität
wirklich abmelden?');">
        Leave X
    </button>
</form>

</div>

{% else %}
    <p>Noch keine Aktivitäten geteilt. Sei der/die Erste!
</p>
{% endfor %}

</div>

<!-- Modal for New Activity -->
<input type="checkbox" id="modalToggleNewActivity"
style="display:none;">

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>
<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
    <label for="modalToggleNewActivity" style="float: right;
cursor: pointer; font-size: 24px; font-weight: bold; margin-
top: 10px;"> x </label>
    <h2>Neue Aktivität vorschlagen</h2>
    <h3>Schlage eine neue Aktivität für Deine WG vor.
</h3>

```

```

<form method="POST">
  {{ form.hidden_tag() }}
  <div style="margin-bottom: 15px;">
    <label for="activity_title">Aktivität:</label>
    {{ form.title(style="display: block; margin-top: 5px;
padding: 8px; width: 100%;") }}
  </div>
  <div style="margin-bottom: 15px;">
    <label for="activity_description">Beschreibung:
</label>
    {{ form.description(
      style="display:block; margin-top:5px;
padding:8px; width:100%; height:100px; resize: vertical;
white-space: pre-wrap;"
    ) }}
  </div>
  <div style="margin-bottom: 15px;">
    <label for="activity_date">Am / um</label>
    {{ form.date(style="display: block; margin-top: 5px;
padding: 8px; width: 100%;") }}
  </div>
  <div style="margin-bottom: 15px;">
    <label for="activity_date">Bis zum / um</label>
    {{ form.updated_at(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
  </div>

  <div style = "margin-bottom: 15px;">
    <label for="activity_location">Ort:</label>
    {{ form.location(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
  </div>

  <div style="margin-bottom: 15px;">
    <label>Max. Teilnehmer (optional):</label>

    {{ form.max_participants(
      style="display:block; margin-top:5px;
padding:8px; width:100%;",
      placeholder="z.B. 10"
    ) }}

```

```
{% if form.max_participants.errors %}
    <span style="color:red;">{{
form.max_participants.errors[0] }}</span>
{% endif %}

</div>

{{ form.submit() }}

</form>
</div>

<style>
    #modalToggleNewActivity:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggleNewActivity:checked ~ #modalContent {
        display: block !important;
    }

    #modalBackdrop { z-index: 1000 !important; }
    #modalContent { z-index: 1001 !important; }
</style>

<style>
    .activities-list {
        display: flex;
        flex-direction: column;
        gap: 15px;
    }

    .activity-card {
        position: relative;
        border: 1px solid #ddd;
        border-radius: 5px;
        padding: 15px;
        background-color: #f9f9f9;
        box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
    }

    .activity-card .delete-btn {
        position: absolute;
```

```
top: 10px;
right: 10px;
background-color: transparent;
border: none;
color: #dc3545;
font-size: 18px;
cursor: pointer;
}

.activity-card .join-btn {
  position: absolute;
  bottom: 12px;
  right: 12px;
  background-color: transparent;
  border: none;
  color: #5935dc;
  font-size: 18px;
  cursor: pointer;
}

.activity-card .leave-btn {
  position: absolute;
  bottom: 12px;
  right: 80px;
  background-color: transparent;
  border: none;
  color: #dc3545;
  font-size: 18px;
  cursor: pointer;
}

.activity-card h3 {
  padding-right: 60px; /* Platz für Delete schaffen */
}

.activity-card{
  padding-bottom: 40px; /* Platz für Join schaffen */
}

.activity-desc {
  white-space: pre-wrap; /* Zeilenumbrüche behalten */
  overflow-wrap: anywhere; /* Lange Wörter
```



```

        umbrechen */
        word-break: break-word; /* Fallback */
        max-width: 100%;
    }
</style> hier mein ganzer html code

```

Dein Layout ist fast richtig – das “kaputte Rendering” kommt daher, dass **Join + Leave immer beide gerendert werden** und du sie nur “nebeneinander schiebst” ( `right: 12px` und `right: 80px` ). Wenn der Text/Font anders ist (oder Buttonbreiten variieren), “springen” sie und wirken wie im Screenshot.

✅ Sauberer Fix: **Nur EINEN Button unten rechts anzeigen** (Join *oder* Leave) + optional daneben ein zweiter Platzhalter.

## 1) Route: gib `current_user_id` ans Template

In deiner `activity_board` Route:

python

```

return render_template(
    "activityboard.html",
    form=form,
    activities=activities,
    current_user_id=session.get("user_id") # oder testuser id
)

```

 Code kopieren

## 2) Template: setze `is_joined` und zeige Join ODER Leave

Ersetze deine beiden Forms (join + leave) durch DAS hier:

jinja2

```

{% set is_joined = current_user_id and (activity.participants | selectattr('user_

<form method="POST"
    action="{% if is_joined %}{{ url_for('leave_activity', activity_id=activity
    class="action-form">

```

```

<button type="submit"
      class="{% if is_joined %}leave-btn{% else %}join-btn{% endif %}"
      onclick="return confirm('{% if is_joined %}Von der Aktivität wirklich a
      {% if is_joined %}Leave X{% else %}Join ➡{% endif %}'
    </button>
</form>

```

Damit existiert immer nur ein Button unten rechts → kein Überlappen, kein "komisches Layout".

 Code kopieren

### 3) CSS: nur noch eine Position unten rechts

Füge das hinzu/ändere es:

CSS

```

.activity-card { padding-bottom: 56px; } /* bisschen mehr Platz unten */

.activity-card .action-form {
  position: absolute;
  bottom: 12px;
  right: 12px;
  margin: 0;
}

.activity-card .join-btn,
.activity-card .leave-btn {
  background-color: transparent;
  border: none;
  font-size: 18px;
  cursor: pointer;
}

.activity-card .join-btn { color: #5935dc; }
.activity-card .leave-btn { color: #dc3545; }

```

Und lösche deine alten `.join-btn { position: absolute... }` und `.leave-btn { position: absolute... }` Blöcke, sonst hast du doppelte Regeln.

 Code kopieren

**Bonus: Wenn du *beide* Buttons sichtbar haben willst (nebeneinander stabil)**

Dann pack sie in einen Container unten rechts:

html

```
<div class="card-actions">
  <form ...><button class="leave-btn">Leave ✕</button></form>
  <form ...><button class="join-btn">Join ➡</button></form>
</div>
```

CSS

```
.activity-card .card-actions{
  position:absolute;
  right:12px;
  bottom:12px;
  display:flex;
  gap:14px;
  align-items:center;
}
.activity-card .card-actions form{ margin:0; }
```

 Code kopieren

Wenn du mir sagst, ob du `user_id` schon in der Session hast (und wie er heißt), passe ich dir die `is_joined` -Zeile exakt so an, dass sie bei dir sofort funktioniert.



ich meine, die farben werden nicht gerendert (also die box mit der farbe und dem join emoji drin) nur das emoji und der text wird gerendert

Ahhh ok — dann ist das kein Position-Problem, sondern **CSS**: Du setzt bei `.join-btn` und `.leave-btn` aktuell

CSS

```
background-color: transparent;  
border: none;
```

Damit **kann keine farbige "Box"** erscheinen (egal welches Button-HTML du hast). Du renderst dann nur Text/Emoji → genau das, was du siehst.

✅ Fix: gib den Buttons wieder **background + padding + border-radius** (und nicht transparent).

## Variante A (einfach, ohne Bootstrap-Klassen)

Ersetze deine Button-CSS so:

CSS

```
.activity-card .join-btn,  
.activity-card .leave-btn {  
  position: absolute;  
  bottom: 12px;  
  font-size: 16px;  
  cursor: pointer;  
  
  padding: 8px 14px;          /* ✅ Box */  
  border-radius: 8px;         /* ✅ runde Ecken */  
  border: 1px solid transparent;  
  color: white;               /* ✅ Textfarbe */  
}  
  
/* Join unten rechts */  
.activity-card .join-btn {  
  right: 12px;  
  background-color: #0d6efd;  /* ✅ blau */  
  border-color: #0d6efd;  
}  
  
/* Leave daneben (links vom Join) */  
.activity-card .leave-btn {  
  right: 110px;               /* Abstand anpassen */  
  background-color: #dc3545;  /* ✅ rot */  
  border-color: #dc3545;  
}
```

👉 Wichtig: Entferne/überschreibe die alten Regeln mit `background-color: transparent;`  
`border: none;` .

## Variante B (Bootstrap richtig nutzen – am saubersten)

 Code kopieren

Dann brauchst du gar kein eigenes Button-Design:

### HTML

html

```
<button type="submit" class="btn btn-sm btn-primary join-btn">Join →</button>
<button type="submit" class="btn btn-sm btn-danger leave-btn">Leave ✕</button>
```

 Code kopieren

### CSS nur für die Position

css

```
.activity-card .join-btn { position:absolute; right:12px; bottom:12px; }
.activity-card .leave-btn { position:absolute; right:110px; bottom:12px; }
```

 Code kopieren

Wenn die Bootstrap-Farben **trotzdem** nicht kommen, ist Bootstrap nicht geladen.

## Check: Ist Bootstrap überhaupt geladen?

Wenn du `class="btn btn-primary"` nutzt und es bleibt trotzdem "nur Text", dann fehlt wahrscheinlich im `<head>` :

html

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.c
```

<

 Code kopieren

(oder du nutzt lokal Bootstrap, dann muss der Pfad stimmen)

### Kurz gesagt

- Dein CSS macht die Buttons absichtlich "transparent" → deshalb keine Box.
- Gib ihnen `padding + background-color + border-radius` zurück **oder** nutze Bootstrap `btn btn-primary/btn-danger` .

Wenn du mir sagst, ob du Bootstrap per CDN oder lokal einbindest, sag ich dir exakt, wo der Link hin muss und wie du prüfst, ob er wirklich geladen wird.



```
.activities-list {  
    display: flex;  
    flex-direction: column;  
    gap: 15px;  
} wie mache ich die card kleiner? gerade ist sie sehr lang
```

Deine Cards sind lang, weil `.activities-list` als **column** läuft → jede Card nimmt automatisch die **volle Breite** ein (Block-Element). Mach sie kleiner, indem du **Breite begrenzt** und **zentrierst**.

### Schnellste Lösung: max-width + margin auto

CSS

```
.activities-list{  
    display: flex;  
    flex-direction: column;  
    gap: 15px;  
    align-items: center;          /* ✅ Cards zentrieren */  
}  
  
.activity-card{  
    width: 100%;  
    max-width: 700px;           /* ✅ hier kleiner machen */  
}
```

Wenn du es noch schmaler willst: `max-width: 520px;` oder so.

 Code kopieren

### Alternative: Cards wie "Kacheln" (mehrspaltig)

Wenn du später 2–3 pro Zeile willst:

CSS

```

.activities-list{
  display: flex;
  flex-wrap: wrap;
  gap: 15px;
  justify-content: center;
}

.activity-card{
  width: 420px;           /* feste Card-Breite */
  max-width: 100%;
}

```

Das reicht schon, damit sie nicht mehr so "sehr lang" über den ganzen Screen geht.



 Code kopieren

wie kann ich nur die activities leaven, in denen ich drin bin?

Du musst prüfen, ob der aktuelle User in `activity.participants` drin ist – dann zeigst du nur dort den "Leave" Button (und optional blendest "Join" aus).

## 1) In der Route: `current_user_id` ans Template geben

python

```

return render_template(
    "activityboard.html",
    activities=activities,
    form=form,
    current_user_id=session.get("user_id")
)

```

 Code kopieren

## 2) Im Template: "is\_joined" berechnen + Buttons bedingt rendern

Ersetze deine Join/Leave-Forms durch:

jinja2

```
{% set is_joined = current_user_id and (activity.participants
    | selectattr('user_id', 'equalto', current_user_id)
    | list | length > 0) %}

{% if is_joined %}
    <!-- Leave nur wenn du drin bist -->
    <form method="POST" action="{{ url_for('leave_activity', activity_id=activity.a
        <button type="submit" class="leave-btn"
            onclick="return confirm('Von der Aktivität wirklich abmelden?');">
        Leave X
    </button>
</form>
{% else %}
    <!-- Join nur wenn du NICHT drin bist -->
    <form method="POST" action="{{ url_for('join_activity', activity_id=activity.ac
        <button type="submit" class="join-btn"
            onclick="return confirm('Aktivität wirklich beitreten?');">
        Join ➡
    </button>
</form>
{% endif %}
```



✓ Ergebnis: Du kannst nur bei Activities "leaven", bei denen du auch Teilnehmer bist.

### 3) Sicherheit in der Leave-Route (damit man nicht "fremd" kann) Code kopieren

python

```
@app.route("/activity/<int:activity_id>/leave_activity", methods=["POST"])
def leave_activity(activity_id):
    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    user = User.query.get_or_404(user_id)
```



```

activity = Activity.query.get_or_404(activity_id)

if user not in activity.participants:
    flash("Du bist nicht Teilnehmer dieser Aktivität.", "error")
    return redirect(url_for("activity_board"))

activity.participants.remove(user)
db.session.commit()
flash("Du hast die Aktivität verlassen.", "success")
return redirect(url_for("activity_board"))

```

Wenn du willst, kann ich dir auch zeigen, wie du statt "Join/Leave" einfach  Code kopieren **Button** renderst, der je nach Status automatisch "Join" oder "Leave" macht (optisch am saubersten).



ja mach diesen button

Alles klar — dann machst du **einen einzigen Button**, der je nach Teilnahme **Join** oder **Leave** ausführt (und auch optisch anders aussieht).

## 1) Route: `current_user_id` ans Template

python

```

return render_template(
    "activityboard.html",
    activities=activities,
    form=form,
    current_user_id=session.get("user_id")
)

```

 Code kopieren

## 2) Template: EIN Button (Join oder Leave automatisch)

Ersetze deine beiden Forms durch diesen Block:

```

jinja2

{% set is_joined = current_user_id and (activity.participants
    | selectattr('user_id', 'equalto', current_user_id)
    | list | length > 0) %}

<form method="POST"
    action="{% if is_joined %}{{ url_for('leave_activity', activity_id=activity
    class="m-0 action-form">

    <button type="submit"
        class="action-btn {% if is_joined %}action-leave{% else %}action-join{%
        onclick="return confirm('{% if is_joined %}Von der Aktivität wirklich a
        {% if is_joined %}Leave X{% else %}Join ➡{% endif %}
    </button>
</form>

```

### 3) CSS: Position unten rechts + Farben als "Box"

 Code kopieren

CSS

```

.activity-card{
    position: relative;
    padding: 15px;
    padding-bottom: 60px; /* Platz für Button unten */
}

/* Form unten rechts "ankleben" */
.activity-card .action-form{
    position: absolute;
    right: 12px;
    bottom: 12px;
    margin: 0;
}

/* Button-Box */
.activity-card .action-btn{
    padding: 8px 14px;
    border-radius: 8px;
}

```

```

border: 1px solid transparent;
color: #fff;
font-size: 16px;
cursor: pointer;
}

/* Join (blau) */
.activity-card .action-join{
  background: #0d6efd;
  border-color: #0d6efd;
}

/* Leave (rot) */
.activity-card .action-leave{
  background: #dc3545;
  border-color: #dc3545;
}

```

## Wichtig

- Entferne deine alten `.join-btn` / `.leave-btn` CSS-Regeln (sonst überschreiben die wieder die Styles).
- Wenn du Bootstrap nutzt, kannst du statt eigener Farben auch `btn btn-primary` / `btn btn-danger` nehmen — aber so klappt's ohne Abhängigkeiten.  Code kopieren

Wenn du willst, passe ich dir das so an, dass der Button **verschwindet**, sobald die Aktivität voll ist (`max_participants`).



```

@app.route("/activityboard/", methods=['GET', 'POST'])
def activity_board():
    form = ActivityForm()

    all_users = User.query.all()

    if request.method == "POST":
        print("Aktivität hinzufügen POST angekommen")
        print("form.validate_on_submit():",

```

```

form.validate_on_submit()
print("form.errors:", form.errors)

if form.validate_on_submit():
    flash("Aktivität erfolgreich hinzugefügt!", "success")

    user =
    User.query.filter_by(username="testuser").first() # To be
    replaced with current_user() or session user
    new_activity = Activity(
        wg_id=1, # wg_id = user.wg_id
        created_by=user.user_id,
        title=form.title.data,
        description=form.description.data,
        date=form.date.data,
        updated_at=form.updated_at.data,
        location=form.location.data,

        max_participants=form.max_participants.data,
        created_at=db.func.now()
    )
    db.session.add(new_activity)
    db.session.commit()

    return redirect(url_for("activity_board"))
else:
    flash("Fehler beim Hinzufügen der Aktivität. Bitte
    überprüfen Sie die Eingaben.", "error")

activities = (Activity.query
    .filter_by(wg_id=1) # wg_id = user.wg_id
    .options(joinedload(Activity.creator))
    .order_by(Activity.created_at.desc())
    .all())

return render_template("activityboard.html", form=form,
activities=activities, all_users=all_users,
current_user_id=session.get("user_id")) hier current user?

```

Ja — `current_user_id=session.get("user_id")` ist genau der richtige Ansatz, aber aktuell nutzt du beim Erstellen noch `"testuser"`. Du kannst das komplett auf Session

umstellen.

So sieht's sauber aus (mit Login-Check):

python

```
@app.route("/activityboard/", methods=["GET", "POST"])
def activity_board():
    form = ActivityForm()

    # ✓ current user aus session
    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    user = User.query.get(user_id)
    if not user:
        flash("User nicht gefunden. Bitte erneut einloggen.", "error")
        return redirect(url_for("login"))

    all_users = User.query.all()

    if request.method == "POST":
        print("Aktivität hinzufügen POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            new_activity = Activity(
                wg_id=user.wg_id or 1,          # besser: user.wg_id
                created_by=user.user_id,
                title=form.title.data,
                description=form.description.data,
                date=form.date.data,
                updated_at=form.updated_at.data,
                location=form.location.data,
                max_participants=form.max_participants.data,
                created_at=db.func.now()
            )
            db.session.add(new_activity)
            db.session.commit()
```

```
        flash("Aktivität erfolgreich hinzugefügt!", "success")
        return redirect(url_for("activity_board"))
    else:
        flash("Fehler beim Hinzufügen der Aktivität. Bitte überprüfen Sie die

activities = (Activity.query
              .filter_by(wg_id=user.wg_id or 1)
              .options(joinedload(Activity.creator),
                      joinedload(Activity.participants)) # ☒ wichtig fürs
              .order_by(Activity.created_at.desc())
              .all())

return render_template(
    "activityboard.html",
    form=form,
    activities=activities,
    all_users=all_users,
    current_user_id=user.user_id
)
```

## Warum `joinedload(Activity.participants)` ?

Damit dein Template-Check `is_joined` (ob der User Teilnehmer ist) **ohne N+1 Queries** funktioniert und zuverlässig ist.

Wenn du noch kein Login-System hast: du kannst statt `session.get("user_id")` vorübergehend `user_id = 1` setzen — aber dann funktioniert "Join/Leave nur für mich" natürlich nur für diesen User.





warum ist der leave button jetzt so lang?

Der Button wird so "lang", weil er als **Flex-Item gestreckt wird**. In deinem Card-Header/Container hast du irgendwo (typisch) `display:flex + align-items: stretch` (Default). Wenn dein Button/Form in diesem Flex-Container sitzt, bekommt er automatisch die volle Höhe der Zeile/Card → genau der rote Balken.

✅ Fix: Button/Form aus dem Flex-Flow rausnehmen oder Flex-Stretch verhindern.

## Fix A (empfohlen): absolute Position unten rechts + "inline" Button

Stell sicher, dass Form/Btn so aussehen:

html

```
<form method="POST" ... class="action-form">
  <button type="submit" class="action-btn action-leave">Leave X</button>
</form>
```

 Code kopieren

Und CSS:

css

```
.activity-card { position: relative; padding-bottom: 60px; }

/* Form unten rechts */
.activity-card .action-form{
  position: absolute;
  right: 12px;
  bottom: 12px;
```

```

margin: 0;
}

/* Button darf NICHT strecken */
.activity-card .action-btn{
  display: inline-block;      /* ✓ verhindert Stretch */
  width: auto;                /* ✓ */
  height: auto;               /* ✓ */
  padding: 8px 14px;          /* ✓ */
  border-radius: 8px;
  white-space: nowrap;        /* ✓ bleibt "klein" */
}

```

## Fix B: Wenn dein Button in einem Flex-Container bleiben soll

 Code kopieren

Dann setze beim Container:

CSS

```

.activity-card .d-flex{
  align-items: flex-start; /* statt stretch */
}

```

 Code kopieren

Oder direkt beim Button/Form:

CSS

```

.activity-card .action-form,
.activity-card .action-btn{
  align-self: flex-start; /* ✓ nicht volle Höhe */
}

```

 Code kopieren

## Quick Check

Wenn du irgendwo sowas hast:

CSS

```

.action-btn { height: 100%; } /* oder */
.action-btn { display: block; width: 100%; }

```

 Code kopieren



→ das verursacht ebenfalls "Riesenbutton". Dann entfernen/überschreiben mit  
`height:auto; width:auto; display:inline-block; .`

Wenn du mir kurz deinen aktuellen HTML-Block vom Button + die CSS-Regeln für  
`.action-form`, `.action-btn`, `.activity-card` (und evtl. den Flex-Container drumherum)  
 schickst, sag ich dir genau welche Zeile bei dir den Stretch auslöst.



```
<style>
.activities-list {
  display: flex;
  flex-direction: column;
  gap: 15px;
  align-items: flex-start;
}

.activity-card {

  position: relative;
  border: 1px solid #ddd;
  border-radius: 5px;
  padding: 15px;
  padding-bottom: 60px;
  background-color: #f9f9f9;
  box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
  width: 100%;
  max-width: 500px;
}
```

```
.activity-card .action-form{  
  position: absolute;  
  right: 12px;  
  bottom: 12px;  
  margin: 0;  
}
```

```
.activity-card .action-btn {  
  position: absolute;  
  top: 10px;  
  right: 10px;  
  background-color: rgb(89, 55, 175);  
  border: none;  
  color: #dc3545;  
  font-size: 18px;  
  cursor: pointer;  
}
```

```
.activity-card .action-join {  
  position: absolute;  
  bottom: 12px;  
  right: 12px;  
  background-color: rgb(89, 55, 175);  
  border-color: black;  
  border: 1px solid black;  
  border-radius: 5px;  
  color: white;  
  font-size: 18px;  
  cursor: pointer;  
  display: inline-block;  
  width: auto;  
  height: auto;  
  padding: 8px 14px;  
  white-space: nowrap;  
}
```

```
.activity-card .action-leave {  
  position: absolute;  
  bottom: 12px;  
  right: 12px;  
  background-color: rgb(218, 84, 84);  
  border: 1px solid black;
```

```

border-radius: 5px;
color: white;
font-size: 18px;
cursor: pointer;
}

.activity-card h3 {
padding-right: 60px; /* Platz für Delete schaffen */

}

.activity-card{
padding-bottom: 40px; /* Platz für Join schaffen */
}

.activity-desc {
white-space: pre-wrap; /* Zeilenumbrüche behalten */
overflow-wrap: anywhere; /* Lange Wörter
umbrechen */
word-break: break-word; /* Fallback */
max-width: 100%;
}

```

sieht jetzt so aus

Du hast gerade **doppelt "absolute positioning"** drin – einmal auf dem **Form** ( `.action-form` ) und nochmal auf dem **Button** ( `.action-btn` , `.action-join` , `.action-leave` ). Dadurch "entkoppelt" sich der Button vom Form/Flow und landet komisch (bei dir oben / außerhalb / falsche Farbe etc.).

Außerdem überschreibt `.action-btn` bei dir vieles (Top/Right, Farbe, etc.) und kann dir die Join/Leave-Styles kaputt machen.

✅ Ziel: Nur das Form wird unten rechts absolut positioniert. Der Button bleibt normal (inline-block), ohne `position: absolute` .

## Mach's so (minimaler Fix)

1) Lösche/ersetze komplett diesen Block (der ist falsch für deinen Action-Button):

CSS

```
.activity-card .action-btn {  
  position: absolute;  
  top: 10px;  
  right: 10px;  
  background-color: rgb(89, 55, 175);  
  border: none;  
  color: #dc3545;  
  font-size: 18px;  
  cursor: pointer;  
}
```

## 2) Vereinheitliche Join + Leave (ohne position absolute!)

 Code kopieren

Ersetze `.action-join` und `.action-leave` durch:

CSS

```
.activity-card .action-btn{  
  display: inline-block;  
  width: auto;  
  height: auto;  
  padding: 8px 14px;  
  border-radius: 6px;  
  border: 1px solid black;  
  color: white;  
  font-size: 18px;  
  cursor: pointer;  
  white-space: nowrap;  
}  
  
.activity-card .action-join{  
  background-color: rgb(89, 55, 175);  
}  
  
.activity-card .action-leave{  
  background-color: rgb(218, 84, 84);  
}
```

## 3) Lass nur das Form unten rechts positionieren (hast du schon korrekt):

CSS

```
.activity-card .action-form{
  position: absolute;
  right: 12px;
  bottom: 12px;
  margin: 0;
}
```

 Code kopieren

## 4) Kleiner Bug in deinem CSS: du hast padding-bottom zweimal

Du hast:

CSS

```
.activity-card { padding-bottom: 60px; ... }
...
.activity-card { padding-bottom: 40px; }
```

 Code kopieren

Der zweite überschreibt den ersten. Lass **nur eins** stehen, z.B.:

CSS

```
.activity-card{ padding-bottom: 60px; }
```

 Code kopieren

## HTML muss dazu passen

Dein Button sollte so aussehen:

html

```
<form method="POST" ... class="action-form">
  <button type="submit" class="action-btn {% if is_joined %}action-leave{% else %}
    {% if is_joined %}Leave X{% else %}Join ➡{% endif %}
  </button>
</form>
```

<

>

 Code kopieren

## Warum war's "komisch"?

Weil `position: absolute` am Button ihn **aus dem Form herauszieht** und zusätzlich dein `.action-btn` ihn nach **top/right** zwingt.

Wenn du die oben genannten CSS-Änderungen machst, sitzt der Button stabil **unten rechts in der Card** und hat wieder eine normale "Box".

Wenn du willst, kann ich dir auch kurz zeigen, wie du links oben den Delete-Button machst, ohne dass er mit dem Action-Button kollidiert.



```
<small class="text-muted" style="margin-top: 20px;
margin-bottom: 5px;">Gepostet von {{
activity.creator.username }} am {{
activity.created_at.strftime('%d.%m.%Y %H:%M') }}
</small>
<br>
    {% set is_joined = current_user_id and
(activity.participants
    | selectattr('user_id', 'equalto', current_user_id)
    | list | length > 0) %}

    <form method="POST"
        action="{% if is_joined %}{{ url_for('leave_activity',
activity_id=activity.activity_id) }}{% else %}{{
url_for('join_activity', activity_id=activity.activity_id) }}{%
endif %}"
        class="m-0 action-form">

        <button type="submit"
            class="action-btn {% if is_joined %}action-
leave{% else %}action-join{% endif %}"
            onclick="return confirm('{% if is_joined %}Von
der Aktivität wirklich abmelden?{% else %}Aktivität wirklich
beitreten?{% endif %}');">
            {% if is_joined %}Leave ✕{% else %}Join ➡{% endif
%}

        </button>
    </form>
    <small> {{for activity.participant in
```

activity.participants}} {{ activity.participant.username }}{%  
endfor %}</small> wie zeige ich hier richtige alle namen  
der activity participants an

Du bist fast richtig dran — du musst in Jinja einfach **über** activity.participants  
iterieren (das sind User -Objekte), und nicht activity.participant .

So geht's korrekt:

jinja2

```
<small class="text-muted">
    Teilnehmer:
    {% if activity.participants and activity.participants|length > 0 %}
        {% for u in activity.participants %}
            {{ u.username }}{% if not loop.last %}, {% endif %}
        {% endfor %}
    {% else %}
        keine
    {% endif %}
</small>
```

## Optional: schöner (mit Icon + Anzahl)

 Code kopieren

jinja2

```
<small class="text-muted">
    👤 {{ activity.participants|length }}
    {% if activity.participants|length > 0 %}
        .
        {% for u in activity.participants %}
            {{ u.username }}{% if not loop.last %}, {% endif %}
        {% endfor %}
    {% endif %}
</small>
```

Wichtig: Damit das ohne extra DB-Queries gut läuft, solltest du in deiner Route beim  
Laden der Activities auch `joinedload(Activity.participants)` drin haben  Code kopieren  
schon/ich hatte es empfohlen).



ich sehe die user nicht, die beigetreten sind

## Nachgedacht für 15s >

Wenn du keine Teilnehmer angezeigt bekommst, obwohl du "Join" drückst, dann liegt's fast immer an einem von diesen 3 Punkten:

1. Join-Route speichert nicht den richtigen User (Session)
2. Many-to-Many Beziehung ist falsch verdrahtet (secondary/back\_populates)
3. Activities werden ohne Participants geladen / oder du zeigst die falsche Beziehung an

Ich geb dir die sichere Variante, die garantiert funktioniert.

## 1) Models richtig verdrahten (WICHTIG)

### Association-Table (einmal, global, oberhalb der Klassen)

python

```
activity_participants = db.Table(
    "ACTIVITY_PARTICIPANTS",
    db.Column("activity_id", db.Integer, db.ForeignKey("ACTIVITY.activity_id"), p
    db.Column("user_id", db.Integer, db.ForeignKey("USER.user_id"), primary_key=T
)
```

<

>

## Activity

 Code kopieren

python

```
class Activity(db.Model):
    __tablename__ = "ACTIVITY"
    activity_id = db.Column(db.Integer, primary_key=True)

    # ...

    participants = db.relationship(
        "User",
```



```

        secondary=activity_participants,
        back_populates="joined_activities"
    )

```

## User

 Code kopieren

python

```

class User(db.Model):
    __tablename__ = "USER"
    user_id = db.Column(db.Integer, primary_key=True)

    joined_activities = db.relationship(
        "Activity",
        secondary=activity_participants, # ☒ hier MUSS activity_participants s
        back_populates="participants"
    )

```



⚠ Entferne in Activity unbedingt diese falsche Zeile, falls du sie noch hast  Code kopieren

python

```

joined_activities = db.relationship("Activity", secondary=activity_participants,

```

 Code kopieren


joined\_activities gehört nur in User, nicht in Activity.

## 2) Join-Route: nur hinzufügen, wenn nicht schon drin

python

```

@app.route("/activity/<int:activity_id>/join_activity", methods=["POST"])
def join_activity(activity_id):
    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    activity = Activity.query.get_or_404(activity_id)
    user = User.query.get_or_404(user_id)

    if user not in activity.participants:

```

```

        activity.participants.append(user)
        db.session.commit()
        flash("Du bist beigetreten.", "success")
    else:
        flash("Du bist schon beigetreten.", "info")

    return redirect(url_for("activity_board"))

```

### 3) ActivityBoard: participants mitladen

 Code kopieren

python

```

activities = (Activity.query
              .filter_by(wg_id=1)
              .options(joinedload(Activity.creator),
                      joinedload(Activity.participants)) # ☒ wichtig
              .order_by(Activity.created_at.desc())
              .all())

```

 Code kopieren

### 4) Template: Teilnehmer anzeigen (so ist es korrekt)

jinja2

```

<small class="text-muted">
    Teilnehmer:
    {% if activity.participants and activity.participants|length > 0 %}
        {% for u in activity.participants %}
            {{ u.username }}{% if not loop.last %}, {% endif %}
        {% endfor %}
    {% else %}
        keine
    {% endif %}
</small>

```

 Code kopieren

**Schneller Debug (damit du sofort weißt ob die Liste überhaupt gefüllt ist)**

Pack **einmal** in dein Template:

jinja2

```
<p>DEBUG participants count: {{ activity.participants|length }}</p>
```

 Code kopieren

Wenn da nach Join immer **0** steht → dann ist es **Model/Join-Route** (nicht Template).

Wenn du willst, paste mir kurz deine aktuelle `join_activity` Route + die `User` und `Activity Relationship`-Teile, dann sag ich dir exakt, welche Zeile bei dir gerade verhindert, dass Teilnehmer gespeichert/geladen werden.

    ...

da steht 1

Perfekt — **wenn** `activity.participants|length = 1` ist, dann ist der Join in der DB angekommen 

Dass du **trotzdem keine Namen siehst**, liegt dann fast sicher an **deinem Template-Block** (oder daran, dass der Text "unsichtbar" ist durch CSS/Farbe).

Mach's so (kopier 1:1):

jinja2

```
<p>DEBUG count: {{ activity.participants|length }}</p>
```

```
<small class="text-muted">
```

```
  Teilnehmer:
```

```
  {% for u in activity.participants %}
```

```
    [{{ u.username }}]{% if not loop.last %}, {% endif %}
```

```
  {% endfor %}
```

```
</small>
```

Wenn du jetzt `[username]` siehst → dein vorheriger Code war ka  Code kopieren  
(Syntaxfehler / falsche Schleife).

Du hattest nämlich das hier:

jinja2

```
{{for activity.participant in activity.participants}} ...
```

 Code kopieren

Das ist kein gültiges Jinja (muss `{% for ... %}` sein und `u` in `activity.participants`).

## Falls du *immer noch* nichts siehst (obwohl `count=1`):

Dann ist es ein **CSS/Style Problem** (Textfarbe identisch zum Hintergrund oder overflow).

Teste sofort mit knalligem Inline-Style:

jinja2

```
<div style="color: red; font-weight: bold;">
  Teilnehmer:
  {% for u in activity.participants %}
    {{ u.username }}
  {% endfor %}
</div>
```

Wenn das sichtbar ist, dann liegt's an deinem `small / text-muted` Styling.  Code kopieren

## Häufige Ursache in deinem Code: falsche Einrückung / außerhalb der card

Achte, dass der Teilnehmer-Block **innerhalb** von `<div class="activity-card"> ...`  
`</div>` steht, nicht dahinter.

Wenn du mir den exakten Abschnitt der Card (ab `<div class="activity-card">` bis `</div>`) postest, sag ich dir sofort, ob der Teilnehmer-Block aus Versehen außerhalb gerendert wird oder vom CSS "weggedrückt" wird.

    ...

```
{% if activity.max_participants %}
  <small>  {{activity.participants|length}}/{{
```

```

activity.max_participants }} </small>
{%else%}
<small class="text-muted">
  👤 {{ activity.participants|length }}
  {% if activity.participants|length > 0 %}
    .
    {% for u in activity.participants %}
      {{ u.username }}{% if not loop.last %}, {% endif %}
    {% endfor %}
  {% endif %}
</small>
{% endif %} ist dieser code korrekt dann?

```

Fast — **logisch korrekt**, aber du hast einen kleinen "Design/Info"-Bug:

- Wenn `max_participants` gesetzt ist, zeigst du **nur** 👤 1/5 , aber **keine Namen**.
- Wenn `max_participants` *nicht* gesetzt ist, zeigst du Anzahl **und** Namen.

Wenn du in **beiden Fällen** die Namen anzeigen willst, mach's so:

jinja2

```

<small class="text-muted">
  {% if activity.max_participants %}
    👤 {{ activity.participants|length }}/{{ activity.max_participants }}
  {% else %}
    👤 {{ activity.participants|length }}
  {% endif %}

  {% if activity.participants|length > 0 %}
    .
    {% for u in activity.participants %}
      {{ u.username }}{% if not loop.last %}, {% endif %}
    {% endfor %}
  {% endif %}
</small>

```

Wenn du die Namen **NUR** anzeigen willst, wenn es keine max gibt

Dann ist dein Code grundsätzlich ok — ich würde nur kleine Syntax-Sauberung  Code kopieren

jinja2

```
{% if activity.max_participants %}
  <small>👤 {{ activity.participants|length }}/{{ activity.max_participants }}</small>
{% else %}
  <small class="text-muted">
    👥 {{ activity.participants|length }}
    {% if activity.participants|length > 0 %}
      .
      {% for u in activity.participants %}
        {{ u.username }}{% if not loop.last %}, {% endif %}
      {% endfor %}
    {% endif %}
  </small>
{% endif %}
```



Beide Varianten sind gültiges Jinja.

 Code kopieren



```
<div class="shopping-list">
  {% for item in shopping_list %}
    <div class="shopping-item-card">
      <form method="POST" action="{{
url_for('delete_shopping_item', item_id=item.item_id) }}"
class="delete-form">
        <button type="submit" class="delete-btn"
        onclick="return confirm('Einkaufsartikel wirklich
löschen?');">🗑️ </button>
      </form>
    </div>

    <h3 class="mb-1">{{ item.name }}</h3>
    <p class="item-quantity mb-2">Menge: {{
item.quantity }}</p>
    <small class="text-muted">Hinzugefügt von: {{
item.added_by }} am {{ item.added_at.strftime('%d.%m.%Y
%H:%M') }}</small>

    {%endfor %}
</div>
```

```

</div>
<form method = "POST" action="{{ url_for('einkaufsplan')
}}">
    {{ form.hidden_tag() }}
    <div>
        {{ form.item.label }}<br>
        {{ form.item(size=32) }}<br>
        {% for error in form.item.errors %}
            <span style="color: red;">{{ error }}</span>
        {% endfor %}
    </div>
    <div>
        {{ form.quantity.label }}<br>
        {{ form.quantity() }}<br>
        {% for error in form.quantity.errors %}
            <span style="color: red;">{{ error }}</span>
        {% endfor %}
    </div>
    <div>
        {{ form.submit() }}
    </div>
</form> wieso zeigt er nicht meine einkäufe

```

Dein HTML ist **kaputt verschachtelt**: Du machst die Card **zu**, bevor du `h3/p/small` renderst – dadurch landen die Inhalte **außerhalb** der Card und das Loop-Markup ist durcheinander. Außerdem hast du ein extra `</div>`.

Mach es so (sauber & korrekt verschachtelt):

jinja2

```

<div class="shopping-list">
    {% for item in shopping_list %}
        <div class="shopping-item-card">

            <form method="POST"
                action="{{ url_for('delete_shopping_item', item_id=item.item_id) }}"
                class="delete-form">

                <button type="submit" class="delete-btn"
                    onclick="return confirm('Einkaufsartikel wirklich löschen?');"> ✖

            </form>
        </div>
    </div>

```

```

<h3 class="mb-1">{{ item.name }}</h3>
<p class="item-quantity mb-2">Menge: {{ item.quantity }}</p>

<small class="text-muted">
  Hinzugefügt von:
  {% if item.added_by_user %}
    {{ item.added_by_user.username }}
  {% else %}
    {{ item.added_by }}
  {% endif %}
  am {{ item.added_at.strftime('%d.%m.%Y %H:%M') }}
</small>

</div>
{% else %}
  <p>Noch keine Einkäufe eingetragen.</p>
{% endfor %}
</div>

```

Und danach dein Formular (wie du es schon hast).

## Wenn danach immer noch nichts kommt

Dann ist `shopping_list` wahrscheinlich leer. Füge kurz zum Debug in dein Template ein:

```

jinja2

<p>DEBUG: items={{ shopping_list|length }}</p>

```

 Code kopieren

Wenn da `0` steht, liegt's in der Route (Query/Filter). Dann schick mir kurz deine

◀ einkaufsplan Route + dein `ShoppingItem` Model, dann sag ich dir genau welchen Filter ▶ du fixen musst.

    ...

(venv)

C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project> flask run

\* Debug mode: off



WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.

\* Running on http://127.0.0.1:5000

Press CTRL+C to quit

[2026-01-08 17:42:23,978] ERROR in app: Exception on /einkaufsplan/ [GET]

Traceback (most recent call last):

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi\_app  
response = self.full\_dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full\_dispatch\_request  
rv = self.handle\_user\_exception(e)

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full\_dispatch\_request  
rv = self.dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch\_request  
return

self.ensure\_sync(self.view\_functions[rule.endpoint])

(\*\*view\_args) # type: ignore[no-any-return]

~~~~~  
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 421, in einkaufsplan

return render\_template("einkaufsplan.html", form=form,  
shopping\_items=shopping\_items) #wg\_id = user.wg\_id

^ ^ ^ ^ ^ ^ ^ ^ ^ ^

UnboundLocalError: cannot access local variable  
'shopping\_items' where it is not associated with a value

Der Fehler ist eindeutig: `shopping_items` wird in deiner Route nicht in jedem Code-Pfad gesetzt. Du gibst es am Ende an `render_template(...)` weiter, aber mindestens bei GET (oder bei einem fehlgeschlagenen POST) wird die Variable nie initialisiert → `UnboundLocalError`.

## Fix (minimal + sauber): `shopping_items` immer vorher setzen

So muss deine Route ungefähr aussehen:

python

```
@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = ShoppingForm()

    # ✓ IMMER initialisieren (auch bei GET!)
    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1)           # später user.wg_id
                      .order_by(ShoppingItem.added_at.desc())
                      .all())

    if form.validate_on_submit():
        user_id = session.get("user_id") # oder testuser tmp
        if not user_id:
            flash("Bitte zuerst einloggen.", "error")
            return redirect(url_for("login"))

        new_item = ShoppingItem(
            wg_id=1,
            name=form.item.data,
            quantity=form.quantity.data,
            added_by=user_id,
            added_at=db.func.now()
        )
        db.session.add(new_item)
        db.session.commit()
        flash("Einkauf hinzugefügt!", "success")
        return redirect(url_for("einkaufsplan"))
```

```
#  hier existiert shopping_items garantiert  
return render_template("einkaufsplan.html", form=form, shopping_items=shoppin
```

## Wichtig: Template-Variable muss übereinstimmen

 Code kopieren

Du renderst in Python `shopping_items=shopping_items`, also muss dein Template auch so iterieren:

```
jinja2
```

```
{% for item in shopping_items %}
```

 Code kopieren

Nicht `shopping_list`.

## Warum passiert das?

Du hast vermutlich `shopping_items = ...` nur **innerhalb** von `if request.method == "POST":` oder `if form.validate_on_submit():` stehen. Bei GET wird es dann nie gesetzt.

Wenn du mir kurz deine komplette `einkaufsplan()` -Route postest, kann ich dir exakt zeigen, wo der Code-Pfad ist, der `shopping_items` überspringt.

    ...

```
(venv)
```

```
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgra  
m\Web-Development-Project> flask run
```

```
* Debug mode: off
```

```
WARNING: This is a development server. Do not use it in a  
production deployment. Use a production WSGI server  
instead.
```

```
* Running on http://127.0.0.1:5000
```

```
Press CTRL+C to quit
```

[2026-01-08 17:47:18,056] ERROR in app: Exception on  
/einkaufsplan/ [GET]

Traceback (most recent call last):

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 1511, in wsgi\_app  
response = self.full\_dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full\_dispatch\_request  
rv = self.handle\_user\_exception(e)

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full\_dispatch\_request  
rv = self.dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch\_request  
return

self.ensure\_sync(self.view\_functions[rule.endpoint])

(\*view\_args) # type: ignore[no-any-return]

~~~~~  
~~~~~ ^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 428, in  
einkaufsplan

return render\_template("einkaufsplan.html", form=form,  
shopping\_items=shopping\_items) #wg\_id = user.wg\_id

@app.route("/einkaufsplan/", methods=["GET", "POST"])  
def einkaufsplan():

form = EinkaufsplanForm()

if request.method == 'POST':

print("Einkaufsplan POST angekommen")

print("form.validate\_on\_submit():",

```

form.validate_on_submit()
print("form.errors:", form.errors)

shopping_items = (ShoppingItem.query
                  .filter_by(wg_id=1)      # TODO: später
user.wg_id
                  .order_by(ShoppingItem.added_at.desc())
                  .all())

if form.validate_on_submit():
    user_id = session.get("user_id",1) #Temporary set
to 1 for testing
    #if not user_id:
    #flash("Bitte zuerst einloggen.", "error")
    #return redirect(url_for("login"))

    new_item = ShoppingItem(
        wg_id=1, #wg_id = user.wg_id
        added_by=user_id, #To be replaced with
user.user_id
        name=form.item.data,
        quantity=form.quantity.data,
        added_at=db.func.now()
    )
    db.session.add(new_item)
    db.session.commit()

    flash("Artikel zum Einkaufsplan hinzugefügt!",
"success")

    return render_template("einkaufsplan.html", form=form,
shopping_items=shopping_items) #wg_id = user.wg_id

```

Der Fehler ist klar: **bei GET wird shopping\_items nie gesetzt**, weil du es **nur innerhalb von** `if request.method == 'POST':` definierst. Dann kommst du am Ende zu `render_template(... shopping_items=shopping_items)` → Variable existiert nicht → `UnboundLocalError`.

✅ **Fix: shopping\_items IMMER oben holen (außerhalb vom POST-Block)**

So bitte (kopier 1:1):

python

```
@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    # ☒ IMMER laden (GET + POST)
    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1)
                      .order_by(ShoppingItem.added_at.desc())
                      .all())

    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            user_id = session.get("user_id", 1) # temp

            new_item = ShoppingItem(
                wg_id=1,
                added_by=user_id,
                name=form.item.data,
                quantity=form.quantity.data,
                added_at=db.func.now()
            )
            db.session.add(new_item)
            db.session.commit()

            flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
            return redirect(url_for("einkaufsplan")) # ☒ damit Liste direkt akt

    return render_template("einkaufsplan.html", form=form, shopping_items=shoppin
```

## Warum redirect nach dem Speichern?

Damit du nach POST wieder einen GET bekommst (PRG-Pattern) und **die Liste frisch** aus der DB geladen wird, ohne „Form resubmit“-Probleme.



## Template: Schleife muss exakt so heißen

Code kopieren

Du übergibst `shopping_items=...`, also im HTML:

```
jinja2
```

```
{% for item in shopping_items %}
```

Code kopieren

Nicht `shopping_list`.

Wenn du willst, kann ich dir auch direkt zeigen, wie du statt `added_by` (nur ID) den **Username** anzeigst ( `item.added_by_user.username` ), sofern du die Relationship schon hast.



er zeigt trotzdem noch 0 einträge, wieso

### Nachgedacht für 43s >

Wenn immer 0 Einträge angezeigt werden, obwohl du speicherst, sind es fast immer **diese 2 Ursachen**:

1. `form.validate_on_submit()` ist doch **False** → es wird gar nichts gespeichert
2. Du speicherst in eine andere **SQLite-Datei** als die, aus der du liest (sehr häufig bei `sqlite:///...`)

Hier ist der schnellste Weg, das in 2 Minuten sicher zu prüfen.

## 1) Prüfen, ob wirklich gespeichert wird (Debug-Prints)

Füge in deiner Route direkt nach dem Commit ein:

```
python
```

```
db.session.commit()
```

```
print("✅ gespeichert, item_id =", new_item.item_id)
```

```
print("✅ count wg=1 =", ShoppingItem.query.filter_by(wg_id=1).count())
```

Und vor dem validate:

```
python

print("validate:", form.validate_on_submit())
print("errors:", form.errors)
```

 Code kopieren

➔ Wenn `validate: False` → dann kommt es nicht zum Insert (und deswegen bleibt es 0).

Typische Gründe:

- CSRF fehlt → `{{ form.hidden_tag() }}` muss im **richtigen** `<form>` sein
- Field-Namen passen nicht (z.B. `form.item` existiert nicht / falscher Name im Form)

## 2) Wahrscheinlichster Grund: du nutzt 2 verschiedene SQLite DB Dateien

Bei SQLite passiert das schnell, je nachdem von wo du startest.

Füge **einmal** in die Route (oder beim App-Start) ein:

```
python

print("DB URL:", db.engine.url)
```

 Code kopieren

➔ Dann siehst du z.B. sowas wie:

- `sqlite:///app.db` (relativ)
- oder `sqlite:///Users/.../instance/app.db`
- oder `sqlite:///instance/app.db`

Wenn du aus Versehen zwei DB-Dateien hast, speicherst du in A, liest aber aus B → Ergebnis immer 0.

✅ Fix: Nutze eine **absolute** DB-Path-Config, z.B.:

```
python

import os
basedir = os.path.abspath(os.path.dirname(__file__))
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite://" + os.path.join(basedir, "app.db")
```

 Code kopieren



### 3) Mini-Check im Template

Pack kurz irgendwo rein:

```
jinja2
```

```
<p>DEBUG: {{ shopping_items|length }} items</p>
```

 Code kopieren

- Wenn das 0 bleibt → DB/Insert Problem (oben)
- Wenn das >0 ist, aber du siehst keine Cards → HTML/CSS/Loop Problem

Wenn du mir **die Ausgabe** von:

- `validate: + errors:`
- und DB URL:

postest, sag ich dir sofort **welcher der beiden Fälle** es ist und den konkreten Fix in deinem Projekt.



Einkaufsplan POST angekommen  
form.validate\_on\_submit(): True  
form.errors: {}

✅ gespeichert, item\_id = 10

✅ count wg=1 = 10

Super — das heißt: **DB speichert korrekt** (10 Einträge sind drin). ✅

Wenn du trotzdem **0 im Template** siehst, dann ist es **kein Datenbank-Problem**, sondern eines von diesen beiden:

1. Du zeigst im Template **die falsche Variable** an ( `shopping_list` statt `shopping_items` )
2. Du "überschreibst"/versteckst die Ausgabe durch **kaputtes HTML** (Loop falsch verschachtelt)

## 1) Template: nutze exakt `shopping_items`

Du übergibst in der Route:

```
python
```

```
render_template(..., shopping_items=shopping_items)
```

 Code kopieren

Also muss dein Template so iterieren:

```
jinja2
```

```
<p>DEBUG template count: {{ shopping_items|length }}</p>
```

```
<div class="shopping-list">
```

```
  {% for item in shopping_items %}
```

```
    <div class="shopping-item-card">
```

```
      <h3 class="mb-1">{{ item.name }}</h3>
```

```
      <p class="item-quantity mb-2">Menge: {{ item.quantity }}</p>
```

```
      <small class="text-muted">
```

```
        Hinzugefügt von: {{ item.added_by }} am {{ item.added_at.strftime('%d.%m.%Y') }}</small>
```

```
      <form method="POST" action="{{ url_for('delete_shopping_item', item_id=item.id) }}">
```

```
        <button type="submit" class="delete-btn">
```

```
          onclick="return confirm('Einkaufsartikel wirklich löschen?');"> 
```

```
      </form>
```

```
    </div>
```

```
  {% else %}
```

```
    <p>Noch keine Einkäufe eingetragen.</p>
```

```
  {% endfor %}
```

```
</div>
```

**Wichtig:** Du hattest vorher `shopping_list` benutzt – wenn du das noch irgendwo hast, bleibt die Liste leer.

 Code kopieren

## 2) Typischer HTML-Fehler in deinem Code

Du hattest `</div>` und `</h3>` usw. **außerhalb** der Card/Loop – das kann dazu führen, dass die Cards nicht sichtbar sind bzw. Layout "zerbricht".

Mit dem Block oben ist die Verschachtelung sauber.

## Sofort-Test (1 Zeile)

Pack ganz oben in `einkaufsplan.html` :

```
jinja2
```

```
<h2>DEBUG: {{ shopping_items|length }}</h2>
```

 Code kopieren

- Wenn da **10** steht → Schleife/HTML war das Problem.
- Wenn da **0** steht → dann übergibst du im Template noch den falschen Namen (oder du renderst eine andere Seite als du denkst).

Wenn du willst, poste kurz den oberen Teil deines `einkaufsplan.html` (bis inklusive der for-loop), dann sag ich dir genau, wo der Variablenname/HTML gerade falsch ist.



`assigned_to=random.randint(1, 3)` wie randomisiere ich,  
wer zuständig für den einkauf ist

Wenn du **zufällig** einen User aus deiner WG wählen willst, mach das **nicht** mit `randint(1,3)` (IDs sind selten "1..3" sauber). Nimm lieber einen zufälligen User aus der DB.

## Variante A (empfohlen): random aus den WG-Usern

```
python
```

 Code kopieren

```
import random
```

```
users = User.query.filter_by(wg_id=1).all() # später: user.wg_id aus session
if not users:
    assigned_to = None # oder Fehler/flash
else:
    assigned_to = random.choice(users).user_id
```

Dann beim Erstellen:

```
python
```

 Code kopieren

```

new_item = ShoppingItem(
    wg_id=1,
    added_by=user_id,
    assigned_to=assigned_to,
    name=form.item.data,
    quantity=form.quantity.data,
    added_at=db.func.now()
)

```

## Variante B: direkt aus DB "random row" (ohne alle zu laden)

### SQLite:

python

 Code kopieren

```

from sqlalchemy.sql import func

u = User.query.filter_by(wg_id=1).order_by(func.random()).first()
assigned_to = u.user_id if u else None

```

✅ Wichtig: Wenn du nur WG-Mitglieder nehmen willst, **immer nach wg\_id filtern**, sonst wird irgendein User aus der ganzen DB zuständig.







```

<form method = "POST" action="{{ url_for('einkaufsplan')
}}">
    {{ form.hidden_tag() }}
    <div>
        {{ form.item(size=32) }}<br>
        {% for error in form.item.errors %}
            <span style="color: red;">[{{ error }}]</span>
        {% endfor %}

        {{ form.quantity() }}<br>
        {% for error in form.quantity.errors %}
            <span style="color: red;">[{{ error }}]</span>

```

```
        {% endfor %}
    </div>
    <div>
        {{ form.submit() }}
    </div>
</form> wie packe ich die nebeneinander
```

Du kannst die Felder am einfachsten **nebeneinander** packen mit Flexbox.

## Variante 1 (clean, ohne Bootstrap)

html

 Code kopieren

```
<form method="POST" action="{{ url_for('einkaufsplan') }}">
    {{ form.hidden_tag() }}

    <div style="display:flex; gap:12px; align-items:flex-start;">
        <div style="flex:2;">
            {{ form.item(size=32, style="width:100%;") }}
            {% for error in form.item.errors %}
                <div style="color:red;">{{ error }}</div>
            {% endfor %}
        </div>

        <div style="flex:1;">
            {{ form.quantity(style="width:100%;") }}
            {% for error in form.quantity.errors %}
                <div style="color:red;">{{ error }}</div>
            {% endfor %}
        </div>

        <div style="align-self:flex-end;">
            {{ form.submit() }}
        </div>
    </div>
</form>
```

## Variante 2 (mit Bootstrap, falls du es nutzt)

html

 Code kopieren

```

<form method="POST" action="{{ url_for('einkaufsplan') }}">
  {{ form.hidden_tag() }}

  <div class="d-flex gap-2 align-items-end">
    <div class="flex-grow-1">
      {{ form.item(class="form-control") }}
      {% for error in form.item.errors %}
        <div class="text-danger small">{{ error }}</div>
      {% endfor %}
    </div>

    <div style="width:140px;">
      {{ form.quantity(class="form-control") }}
      {% for error in form.quantity.errors %}
        <div class="text-danger small">{{ error }}</div>
      {% endfor %}
    </div>

    {{ form.submit(class="btn btn-primary") }}
  </div>
</form>

```

Wenn du sagst, ob du Bootstrap wirklich aktiv eingebunden hast, sag ich dir die beste Variante für euer Layout.



```
from sqlalchemy.sql import func
```

```

u =
User.query.filter_by(wg_id=1).order_by(func.random()).first(
)
assigned_to = u.user_id if u else None wo packe ich diesen
block rein

```

Den Block packst du in deine `/einkaufsplan/` -Route genau dort rein, wo du `new_item = ShoppingItem(...)` baust – also nachdem du `user_id` bestimmt hast und bevor du

ShoppingItem(...) erstellst.

Beispiel (einfach copy/paste in deinen `if form.validate_on_submit():` -Block):

python

 Code kopieren

```
from sqlalchemy.sql import func

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1)
                      .order_by(ShoppingItem.added_at.desc())
                      .all())

    if request.method == "POST":
        if form.validate_on_submit():
            user_id = session.get("user_id", 1) # temp

            # ☒ HIER rein: random Verantwortlichen wählen (aus der WG)
            u = User.query.filter_by(wg_id=1).order_by(func.random()).first()
            assigned_to = u.user_id if u else None

            new_item = ShoppingItem(
                wg_id=1,
                added_by=user_id,
                assigned_to=assigned_to, # ☒ random
                name=form.item.data,
                quantity=form.quantity.data,
                added_at=db.func.now()
            )
            db.session.add(new_item)
            db.session.commit()
            flash("Artikel hinzugefügt!", "success")
            return redirect(url_for("einkaufsplan"))

    return render_template("einkaufsplan.html", form=form, shopping_items=shoppin
```



☒ Wichtig:

- `from sqlalchemy.sql import func` **einmal oben** in deiner Datei importieren (nicht in jedem Request nötig).
- Das funktioniert nur, wenn dein `ShoppingItem` auch wirklich ein Feld `assigned_to` hat (FK auf `USER.user_id`).

Wenn du willst, kann ich dir auch zeigen, wie du im Template dann direkt den **Username** des zufällig zugewiesenen Users anzeigen lässt.



es geht nicht

def einkaufsplan():

form = EinkaufsplanForm()

```
shopping_items = (ShoppingItem.query
                  .filter_by(wg_id=1)      # TODO: später
user.wg_id
                  .order_by(ShoppingItem.item_id)
                  .all())
```

if request.method == 'POST':

```
print("Einkaufsplan POST angekommen")
print("form.validate_on_submit():",
form.validate_on_submit())
print("form.errors:", form.errors)
```

```
if form.validate_on_submit():
    user_id = session.get("user_id",1) #Temporary set
to 1 for testing
    #if not user_id:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    u =
User.query.filter_by(wg_id=1).order_by(func.random()).first(
)
```



```

        assigned_to = u.user_id if u else None #Temporary
assigned_to random user for testing

        new_item = ShoppingItem(
            wg_id=1, #wg_id = user.wg_id
            added_by=user_id, #To be replaced with
            user.user_id
            name=form.item.data,
            quantity=form.quantity.data,
            assigned_to=assigned_to #Temporary assigned_to
            random user_id between 1 and 3 for testing

        )
        db.session.add(new_item)
        db.session.commit()
        print("gespeichert, item_id =", new_item.item_id)
        print("count wg= 1 =",
            ShoppingItem.query.filter_by(wg_id=1).count())

        flash("Artikel zum Einkaufsplan hinzugefügt!",
            "success")
        return redirect(url_for("einkaufsplan"))

    return render_template("einkaufsplan.html", form=form,
        shopping_items=shopping_items)

```

Ich sehe den Fehler sofort: **dein** `new_item = ShoppingItem(...)` **Block ist falsch eingerückt** (und dadurch werden `db.session.add(...)` / `commit()` ggf. außerhalb des `if form.validate_on_submit():` ausgeführt oder Python wirft/verschluckt Logikfehler). Außerdem fehlt sehr wahrscheinlich `func import`.

So muss es aussehen (korrekt eingerückt + mit Import):

## 1) Oben in deiner Datei (app.py) einmal:

```
python
```

```
from sqlalchemy.sql import func
```

 Code kopieren

## 2) Route korrekt (copy/paste):

python

```
@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1)
                      .order_by(ShoppingItem.item_id)
                      .all())

    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            user_id = session.get("user_id", 1) # temp

            u = User.query.filter_by(wg_id=1).order_by(func.random()).first()
            assigned_to = u.user_id if u else None

            new_item = ShoppingItem(
                wg_id=1,
                added_by=user_id,
                name=form.item.data,
                quantity=form.quantity.data,
                assigned_to=assigned_to
            )

            db.session.add(new_item)
            db.session.commit()

            print("gespeichert, item_id =", new_item.item_id)
            print("count wg=1 =", ShoppingItem.query.filter_by(wg_id=1).count())

            flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
            return redirect(url_for("einkaufsplan"))

    return render_template("einkaufsplan.html", form=form, shopping_items=shoppin
```

&lt;

&gt;

## Wenn du jetzt immer noch sagst "geht nicht":

Sag mir bitte **welche Fehlermeldung** du bekommst (Traceback).

Die zwei häufigsten danach wären:

**A) NameError: func is not defined**

 Code kopieren

→ Import fehlt (oben)

**B) TypeError: 'assigned\_to' is an invalid keyword argument for ShoppingItem**

→ Dein `ShoppingItem` Model hat kein Feld `assigned_to` (oder heißt anders).

Dann poste kurz deine `ShoppingItem` Klasse, dann passe ich es exakt an.

    ...

```

user_id = session.get("user_id",1) #Temporary set to 1 for
testing
    #if not user_id:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    u =
    User.query.filter_by(wg_id=1).order_by(func.random()).first(
)
        assigned_to = u.user_id if u else None #Temporary
assigned_to random user for testing

u.user_id (user_id ist "any")

```

Ahh—du bekommst den Hinweis/Fehler, weil `u` **manchmal** `None` **ist** (oder dein Type-Checker sieht: `u` kann `None` sein), und dann ist `u.user_id` nicht sicher. Laufzeit-sicher hast du es schon fast richtig gemacht – du musst nur **vorher prüfen**, ob überhaupt ein User gefunden wurde.

 **So ist es 100% safe**

python

```
u = User.query.filter_by(wg_id=1).order_by(func.random()).first()

if u is None:
    flash("Keine WG-Mitglieder gefunden – kann niemanden zufällig zuweisen.", "error")
    return redirect(url_for("einkaufsplan"))

assigned_to = u.user_id
```

 Code kopieren

## Alternative in einer Zeile (auch ok)

python

```
u = User.query.filter_by(wg_id=1).order_by(func.random()).first()
assigned_to = u.user_id if u is not None else None
```

 Code kopieren

## Warum ist `u` evtl. `None`?

Weil diese Query nichts findet:

python

```
User.query.filter_by(wg_id=1)
```

 Code kopieren

Das passiert z.B. wenn:

- deine User `wg_id = NULL` haben (noch nicht zugewiesen)
- oder deine WG-ID ist nicht 1
- oder es gibt (noch) keine User in der Tabelle

## Mini-Debug (einmal ausgeben)

python

```
print("users in wg=1:", User.query.filter_by(wg_id=1).count())
```

 Code kopieren

Wenn das 0 ist, dann ist klar warum.

## Wenn du “irgendeinen User” nehmen willst (ohne `wg_id` Filter)

Nur zum Test:

```
python
```

```
u = User.query.order_by(func.random()).first()
assigned_to = u.user_id if u else None
```

 Code kopieren

Wenn du mir kurz sagst, was `print("users in wg=1:", ...)` ausgibt, sag ich dir direkt, ob dein Problem die `wg_id` -Zuordnung ist und wie du das sauber mit Session-User + `user.wg_id` machst.

    ...

users in wg=1: 0

Alles klar: **deshalb geht's nicht.**

Dein Random-Pick macht `filter_by(wg_id=1)`, aber **es gibt 0 User mit `wg_id=1`** → `u = None` → niemand kann zugewiesen werden.

**Fix: User haben (noch) keine `wg_id=1`**

Du musst dafür sorgen, dass eure User beim Join/Registrieren eine `wg_id` bekommen.

**Schnelltest (ohne WG-Filter) – damit du siehst, dass Random an sich klappt**

```
python
```

```
u = User.query.order_by(func.random()).first()
assigned_to = u.user_id if u else None
```

 Code kopieren

Wenn das funktioniert, ist bestätigt: das Problem ist nur `wg_id`.

## Richtige Lösung (WG-ID aus Session-User)

Wenn der eingeloggte User eine WG hat:

```
python
```

```
user_id = session.get("user_id", 1)
current_user = User.query.get(user_id)

wg_id = current_user.wg_id # <- muss gesetzt sein!
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
assigned_to = u.user_id if u else None
```

 Code kopieren

Wenn `current_user.wg_id` `None` ist → dann ist der User noch keiner WG zugeordnet.

## Einmalig "reparieren" (Test): setze wg\_id bei deinen Usern

Wenn du erstmal schnell testen willst, setz in deiner DB für testuser wg\_id=1:

```
python

u = User.query.filter_by(username="testuser").first()
u.wg_id = 1
db.session.commit()
```

 Code kopieren

Oder für alle User:

```
python

User.query.update({User.wg_id: 1})
db.session.commit()
```

 Code kopieren

(Bei SQLite/SQLAlchemy klappt das.)

## Empfehlung fürs Projekt

- Beim **WG erstellen**: creator bekommt `wg_id`
- Beim **WG beitreten**: setze `user.wg_id = wg_id`
- Dann klappt dein Random-Assign zuverlässig.

Wenn du mir kurz sagst, wie ihr aktuell WG erstellt/joint (Route/Logik), kann ich dir genau zeigen, wo du `user.wg_id = ...` setzen musst.



```

    return cls.query_class(
Einkaufsplan POST angekommen
form.validate_on_submit(): True
form.errors: {}
users in wg=1: 1
[2026-01-08 19:07:04,060] ERROR in app: Exception on
/einkaufsplan/ [POST]
Traceback (most recent call last):
  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
    self.dialect.do_execute(
    ~~~~~^
 cursor, str_statement, effective_parameters, context

^^
^^^^^^^^^^^^^^^^^^^^
)
 ^
 File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
 cursor.execute(statement, parameters)

~~~~~^
sqlite3.IntegrityError: NOT NULL constraint failed:
SHOPPING_ITEM.wg_id

```

The above exception was the direct cause of the following exception:

```

Traceback (most recent call last):
  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\app.py", line 1511, in wsgi_app
    response = self.full_dispatch_request()

```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 919, in full\_dispatch\_request  
rv = self.handle\_user\_exception(e)

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 917, in full\_dispatch\_request  
rv = self.dispatch\_request()

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\app.py", line 902, in dispatch\_request  
return  
self.ensure\_sync(self.view\_functions[rule.endpoint])  
(\*\*view\_args) # type: ignore[no-any-return]

~~~~~  
~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 437, in einkaufsplan  
db.session.commit()

~~~~~^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\scoping.py", line 597, in commit
return self._proxied.commit()

~~~~~^ ^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 2030, in commit  
trans.commit(\_to\_root=True)

~~~~~^ ^ ^ ^ ^ ^ ^ ^ ^ ^

File "<string>", line 2, in commit

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-


```

packages\sqlalchemy\orm\state_changes.py", line 137, in
_go
    ret_value = fn(self, *arg, **kw)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 1311, in
commit
    self._prepare_impl()
    ~~~~~^ ^
File "<string>", line 2, in _prepare_impl
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\state_changes.py", line 137, in
_go
    ret_value = fn(self, *arg, **kw)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 1286, in
_prepare_impl
    self.session.flush()
    ~~~~~^ ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 4331, in flush
    self._flush(objects)
    ~~~~~^ ^ ^ ^ ^ ^ ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\session.py", line 4466, in _flush
    with util.safe_reraise():
        ~~~~~^ ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\util\langhelpers.py", line 224, in
__exit__
    raise exc_value.with_traceback(exc_tb)
File

```

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\session.py", line 4427, in _flush
flush_context.execute()

~~~~~^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\unitofwork.py", line 466, in  
execute

rec.execute(self)

~~~~~^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\unitofwork.py", line 642, in
execute

util.preloaded.orm_persistence.save_obj(

~~~~~  
~^

self.mapper,

^^^^^^^^

uow.states\_for\_mapper\_hierarchy(self.mapper, False,  
False),

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^  
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

uow,

^^^^

)

^

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\sqlalchemy\orm\persistence.py", line 93, in  
save\_obj

\_emit\_insert\_statements(

~~~~~^

base_mapper,

^^^^^^^^

...<3 lines>...

insert,

[illegible]

```
        dialect,
...<8 lines>...
        cache_hit=cache_hit,
    )
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1846, in
_execute_context
    return self._exec_single_context(
        ~~~~~^
        dialect, context, statement, parameters

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^
    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1986, in
_exec_single_context
    self._handle_dbapi_exception(
        ~~~~~^
        e, str_statement, effective_parameters, cursor, context

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 2363, in
_handle_dbapi_exception
    raise sqlalchemy_exception.with_traceback(exc_info[2])
from e
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\base.py", line 1967, in
_exec_single_context
    self.dialect.do_execute(
```

```

~~~~~^
cursor, str_statement, effective_parameters, context

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^
)
^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\engine\default.py", line 952, in
do_execute
    cursor.execute(statement, parameters)

~~~~~^
sqlalchemy.exc.IntegrityError: (sqlite3.IntegrityError) NOT
NULL constraint failed: SHOPPING_ITEM.wg_id
[SQL: INSERT INTO "SHOPPING_ITEM" (wg_id, added_by,
assigned_to, name, quantity) VALUES (?, ?, ?, ?, ?)]
[parameters: (None, 1, 1, 'dsfdsf', 8)]
(Background on this error at:
https://sqlalche.me/e/20/gkpj)
127.0.0.1 - - [08/Jan/2026 19:07:04] "POST /einkaufsplan/
HTTP/1.1" 500 -

```

Der Trace ist eindeutig:

NOT NULL constraint failed: SHOPPING_ITEM.wg_id
und in den INSERT-Parametern steht: (None, 1, 1, 'dsfdsf', 8)

Heißt: **du übergibst beim Speichern** `wg_id=None`, obwohl die DB-Spalte `wg_id` **NOT NULL** ist.

Das passiert typischerweise, wenn du inzwischen sowas machst wie:

- `wg_id = current_user.wg_id`
aber `current_user.wg_id` **ist None**, oder
- du hast `wg_id` als Variable benutzt und sie ist nicht gesetzt.

✅ Fix: Setze `wg_id` **explizit** und prüfe es, bevor du speicherst.

Robuste Version deiner POST-Logik (mit Check)

python

```
from sqlalchemy.sql import func

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1)
                      .order_by(ShoppingItem.item_id)
                      .all())

    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            user_id = session.get("user_id", 1)

            current_user = User.query.get(user_id)
            wg_id = current_user.wg_id if current_user else None

            # ✅ WICHTIG: wg_id darf nicht None sein
            if wg_id is None:
                flash("Du bist keiner WG zugeordnet (wg_id fehlt).", "error")
                return redirect(url_for("einkaufsplan"))

            print("users in wg:", User.query.filter_by(wg_id=wg_id).count())

            u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
            assigned_to = u.user_id if u else None

            new_item = ShoppingItem(
                wg_id=wg_id,                    # ✅ nicht None
                added_by=user_id,
                assigned_to=assigned_to,
                name=form.item.data,
                quantity=form.quantity.data,
            )
```

```

        db.session.add(new_item)
        db.session.commit()

        flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
        return redirect(url_for("einkaufsplan"))

    return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Warum ist `wg_id` bei dir `None`?

Du hast zwar jetzt 1 User in `wg=1` (dein Debug), aber der eingeloggte User (`session["user_id"]`) hat vermutlich `wg_id=None`.

Quick Debug (einmal)

python

```

print("session user_id:", user_id)
print("current_user:", current_user.username if current_user else None)
print("current_user.wg_id:", wg_id)

```

 Code kopieren

Wenn `current_user.wg_id` `None` ist:

➡️ musst du beim Login/Join WG `user.wg_id = 1` **setzen** (oder die richtige WG).

Quick-Fix zum Testen (einmalig)

Wenn du nur testen willst:

python

```

current_user = User.query.get(user_id)
current_user.wg_id = 1
db.session.commit()

```

 Code kopieren

Dann sollte das Speichern sofort funktionieren.

Wenn du willst, poste kurz deinen **Login / WG-Join Code**, dann sag ich dir genau, **wo** du `user.wg_id` sauber setzen solltest (damit es nicht nur "Test-Fix" ist).



wie zeige ich hier den namen an von der id

Wenn du in der Tabelle nur eine **User-ID** hast (z.B. `item.assigned_to` oder `item.added_by`) und du willst **den Namen** anzeigen, gibt's 2 saubere Wege.

✅ **Beste Lösung: Relationship nutzen (dann kannst du direkt `.username` anzeigen)**

1) In `ShoppingItem` (`db.py`) Beziehungen hinzufügen

python

```
class ShoppingItem(db.Model):
    __tablename__ = "SHOPPING_ITEM"
    item_id = db.Column(db.Integer, primary_key=True)
    wg_id = db.Column(db.Integer, db.ForeignKey("WG.wg_id"), nullable=False)

    added_by = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=False)
    assigned_to = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=True)

    name = db.Column(db.String, nullable=False)
    quantity = db.Column(db.Integer)
    added_at = db.Column(db.DateTime)

    # Beziehungen:
    added_by_user = db.relationship("User", foreign_keys=[added_by], backref="shopping_item_added_by")
    assigned_to_user = db.relationship("User", foreign_keys=[assigned_to], backref="shopping_item_assigned_to")
```

2) Dann im Template:

jinja2

```
<small>Hinzugefügt von: {{ item.added_by_user.username }}</small>

{% if item.assigned_to_user %}
    <small>Zuständig: {{ item.assigned_to_user.username }}</small>
{% else %}
```



```
<small>Zuständig: -</small>
{% endif %}
```

 Code kopieren

Alternative (ohne Relationship): dict mapping in Route bauen

Wenn du **db.py** nicht anfassen willst:

Route:

python

```
users = User.query.filter_by(wg_id=1).all()
user_map = {u.user_id: u.username for u in users}
return render_template("einkaufsplan.html", form=form, shopping_items=shopping_it
```

 Code kopieren

Template:

jinja2

```
<small>Hinzugefügt von: {{ user_map.get(item.added_by, "Unbekannt") }}</small>
<small>Zuständig: {{ user_map.get(item.assigned_to, "-") }}</small>
```

 Code kopieren

Wenn du mir sagst, **welche ID genau** du meinst (assigned_to oder added_by oder etwas anderes), passe ich dir den konkreten HTML-Block 1:1 an deine Datei an.



wie mache ich, dass ein user der wg randomisiert
zuständig gemacht wird

So machst du es **sauber**: du wählst zufällig einen User aus derselben WG und setzt dessen user_id in assigned_to .

 In deiner /einkaufsplan/ Route (beim Erstellen des Items)

python

```

from sqlalchemy.sql import func

user_id = session.get("user_id") # eingeloggter User
current_user = User.query.get(user_id)
wg_id = current_user.wg_id

# zufälliger User aus derselben WG
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
assigned_to = u.user_id if u else None

new_item = ShoppingItem(
    wg_id=wg_id,
    added_by=user_id,
    name=form.item.data,
    quantity=form.quantity.data,
    assigned_to=assigned_to
)
db.session.add(new_item)
db.session.commit()

```

Wichtig (sonst klappt's nicht)

- `current_user.wg_id` darf nicht `None` sein (User muss einer WG zugeordnet sein).
- In der WG muss mindestens 1 User existieren (sonst wird `u=None`).

 Code kopieren

Falls du NICHT willst, dass der Ersteller sich selbst zugewiesen bekommt

python

```

u = (User.query
    .filter(User.wg_id == wg_id, User.user_id != user_id)
    .order_by(func.random())
    .first())

assigned_to = u.user_id if u else user_id # fallback: dann doch er selbst

```

Wenn du mir kurz sagst, ob der Ersteller ausgeschlossen werden soll, kann Variante direkt passend geben.

 Code kopieren


kann ich irgendwie eine funktion mit button erstellen, die confirmt, dass dieses item schon gekauft wurde und es durchstreicht ohne status attribut?

Nachgedacht für 30s >

Ja – **geht**, aber du musst dich entscheiden, ob es **nur optisch** (ohne DB, verschwindet nach Reload) oder **persistent** (merkt sich "gekauft") sein soll.

Variante 1 (nur Template/CSS, kein DB-Status)

Durchstreichen klappt, aber nach F5 ist es wieder weg.

In deiner Item-Card:

jinja2

```
<label class="buy-row">
  <input type="checkbox" class="buy-toggle">
  <span class="buy-text">{{ item.name }} ({{ item.quantity }})</span>
</label>

<button class="btn btn-sm btn-outline-success"
  type="button"
  onclick="document.querySelector('#buy_{{ item.item_id }}').click()">
  ✓ Gekauft
</button>
```

Besser (sauber): Checkbox + CSS, ohne Button-„Trick“:

jinja2

```
<label class="buy-row">
  <input type="checkbox" class="buy-toggle" id="buy_{{ item.item_id }}">
  <span class="buy-text">{{ item.name }} ({{ item.quantity }})</span>
</label>
```

CSS (z.B. unten in einkaufsplan.html in <style>):

 Code kopieren

CSS

```
.buy-row { display: flex; gap: 10px; align-items: center; }
.buy-toggle:checked + .buy-text {
  text-decoration: line-through;
  opacity: 0.6;
}
```

➡ Vorteil: kein Backend/DB nötig. Nachteil: nicht gespeichert.

📋 Code kopieren

Variante 2 (persistent, ohne Status-Spalte in SHOPPING_ITEM)

Empfohlen

Du machst eine **separate Tabelle**, z.B. `SHOPPING_PURCHASED`. Das ist **kein Status-Attribut** am Item, aber speichert den Zustand.

1) Model (db.py)

python

```
class ShoppingPurchased(db.Model):
    __tablename__ = "SHOPPING_PURCHASED"
    item_id = db.Column(db.Integer, db.ForeignKey("SHOPPING_ITEM.item_id"), primary_key=True)
    purchased_at = db.Column(db.DateTime, server_default=db.func.now())

    item = db.relationship("ShoppingItem", backref=db.backref("purchased", uselist=False))
```

<

>

📋 Code kopieren

2) Route zum Toggle

python

```
@app.route("/shopping/<int:item_id>/toggle_purchased", methods=["POST"])
def toggle_purchased(item_id):
    item = ShoppingItem.query.get_or_404(item_id)

    row = ShoppingPurchased.query.get(item_id)
    if row:
        db.session.delete(row) # wieder "offen"
    else:
        db.session.add(ShoppingPurchased(item_id=item_id)) # "gekauft"
```

```
db.session.commit()
return redirect(url_for("einkaufsplan"))
```

3) Template: Button + Durchstreichen

 Code kopieren

jinja2

```
<form method="POST" action="{% url_for('toggle_purchased', item_id=item.item_id)
  <button type="submit"
    class="btn btn-sm {% if item.purchased %}btn-secondary{% else %}btn-suc
    onclick="return confirm('Als gekauft markieren?');">
    {% if item.purchased %}↶ Rückgängig{% else %}✓ Gekauft{% endif %}
  </button>
</form>
```

```
<h3 class="{% if item.purchased %}text-decoration-line-through opacity-75{% endif
  {{ item.name }}
</h3>
```

<

>

onclick="return confirm(...)" ist der einfachste "Confirm". Ohne irgendein JS bekommst du leider keinen echten Confirm-Dialog hin.

 Code kopieren

Wenn du mir sagst, ob du es **persistent** willst (Variante 2) oder **nur optisch** (Variante 1), passe ich dir deinen aktuellen `einkaufsplan.html` Block exakt so an, dass es bei dir sofort läuft.



```
<nav>
  <a href="{% url_for('dashboard') %}">Start</a>
  <a href="{% url_for('putzplan') %}">Putzplan</a>
  <a href="{% url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{% url_for('activity_board') %}">ActivityBoard</a>
  <a href="{% url_for('einkaufsplan') %}">Einkauf</a>
</nav>
```

```
<h1>Einkaufsplan</h1>
```

```
<h2>Gemeinsame Einkaufsliste für die WG</h2>
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;" >
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

```
<h2>Neuen Einkaufsartikel hinzufügen</h2>
```

```
<form method = "POST" action="{{ url_for('einkaufsplan')
}}" >
  {{ form.hidden_tag() }}
  <div class="item-forms">
    {{ form.item(size=32) }}<br>
    {% for error in form.item.errors %}
      <span style="color: red;">[{{ error }}]</span>
    {% endfor %}

    {{ form.quantity() }}<br>
    {% for error in form.quantity.errors %}
      <span style="color: red;">[{{ error }}]</span>
    {% endfor %}
    <div class = "submit-btn">
      {{ form.submit() }}
    </div>
  </div>
</form>
<
<div class="shopping-list">
  {% for item in shopping_items %}
    <div class="shopping-item-card">
      <form method="POST" action="{{
url_for('delete_shopping_item', item_id=item.item_id) }}"
```

```

class="delete-form">
  <button type="submit" class="delete-btn"

      onclick="return confirm('Einkaufsartikel wirklich
löschen?');"> 🗑️
    </button>
  </form>

  <h3 class="mb-1">{{ item.name }}</h3>
  <p class="item-quantity mb-2">Menge: {{
item.quantity }}</p>

  <small class="text-muted">
    Hinzugefügt von:
    {% if item.added_by_user %}
      {{ item.added_by_user.username }}
    {% else %}
      {{ item.added_by }}
    {% endif %}
  </small>
  <br>

  {% if item.assigned_to_user %}
    <small class = "assigned_to">{{
item.assigned_to_user.username }}</small>

    {% endif %}

  </div>
  {% else %}
    <p>Noch keine Einkäufe eingetragen.</p>

  {% endfor %}
</div>

<p>DEBUG: items={{ shopping_list|length }}</p>

<style>
  .shopping-list {
    display: flex;
    flex-wrap: wrap;
    gap: 20px;
    align-items: flex-start;

```

```
justify-content:center;
}

.shopping-item-card {
  border: 1px solid #161414;
  border-radius: 5px;
  padding: 15px;
  margin-bottom: 15px;
  position: relative;
  background-color: #f9f9f9;
  box-shadow: inset 2px 2px 5px rgba(0,0,0,0.05);
  width: 100%;
  max-width: 500px;
  box-sizing: border-box;
}

.item-forms{
  display: flex;
  gap: 10px;
  align-items: flex-start;
  margin-bottom: 20px;
}

.delete-btn {
  position: absolute;
  right: 12px;
  top: 12px;
  margin: 0;
  background-color: tomato;
  cursor: pointer;
}

.delete-btn:hover {
  color: red;
}

.item-forms .submit-btn{
  cursor: pointer;
  padding: 10px 20px;
  background-color: #007bff;
  color: white;
  border-radius: 5px;
  display: inline-block;
```



```
}

.item-quantity {
  font-weight: bold;
}

.assigned_to {
  position: absolute;
  bottom: 12px;
  right: 12px;
  font-style: italic;
  color: #555;
  font-size: large;
  border: black;
  padding: 2px 6px;
  background-color: #e0e0e0;
  border-radius: 5px;
  margin-top: auto;
}
```

</style> kannst du mir ein schönes css design entwerfen, etwa wie bei figma

Nachgedacht für 13s >

Klar — ich gebe dir ein **Figma-Style (clean, modern, soft shadow, rounded corners)** Design, das du **direkt** in `einkaufsplan.html` unten in dein `<style>` packen kannst.

Vorher 2 Mini-Bugs in deinem HTML (sonst wirkt es "komisch"):

1. Du hast **ein einzelnes** `<` vor `<div class="shopping-list">` → das **löschen**.
2. Debug: `shopping_list|length` existiert bei dir nicht → nutz `shopping_items|length`.

✓ Drop-in CSS (Figma-Look)

Einfach dein bestehendes `<style>` ersetzen durch dieses hier:

html

 Code kopieren

```
<style>
:root{
  --bg: #f6f7fb;
  --surface: #ffffff;
  --text: #111827;
  --muted: #6b7280;
  --border: #e5e7eb;

  --primary: #2563eb;
  --primary-hover: #1d4ed8;
  --danger: #ef4444;
  --danger-hover: #dc2626;

  --radius: 14px;
  --shadow: 0 10px 25px rgba(17, 24, 39, 0.08);
  --shadow-sm: 0 6px 16px rgba(17, 24, 39, 0.08);
}

body{
  background: var(--bg);
  color: var(--text);
  font-family: system-ui, -apple-system, Segoe UI, Roboto, Arial, sans-serif;
  line-height: 1.4;
}

/* NAV */
nav{
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
  margin: 18px auto 18px;
  max-width: 980px;
  padding: 10px;
  background: var(--surface);
  border: 1px solid var(--border);
  border-radius: var(--radius);
  box-shadow: var(--shadow-sm);
}
nav a{
  text-decoration: none;
  color: var(--muted);
  padding: 8px 12px;
  border-radius: 10px;
```

```
    transition: 0.15s ease;
    font-weight: 600;
    font-size: 14px;
  }
  nav a:hover{
    background: #eef2ff;
    color: var(--primary);
  }

  /* PAGE HEADINGS */
  h1, h2{
    max-width: 980px;
    margin: 14px auto;
    padding: 0 6px;
  }
  h1{ font-size: 28px; letter-spacing: -0.02em; }
  h2{ font-size: 16px; color: var(--muted); font-weight: 600; }

  /* FLASH */
  .alert-message{
    max-width: 980px;
    margin: 10px auto 16px;
    border-radius: var(--radius);
    border: 1px solid var(--border);
    padding: 12px 14px;
    box-shadow: var(--shadow-sm);
  }
  .alert-success{
    background: #ecfdf5;
    border-color: #a7f3d0;
    color: #065f46;
  }
  .alert-error{
    background: #fef2f2;
    border-color: #fecaca;
    color: #7f1d1d;
  }

  /* FORM */
  form{
    max-width: 980px;
    margin: 0 auto;
    padding: 0 6px;
```

```
}
```

```
.item-forms{  
  display: grid;  
  grid-template-columns: 1fr 140px 160px;  
  gap: 12px;  
  align-items: end;  
  background: var(--surface);  
  border: 1px solid var(--border);  
  border-radius: var(--radius);  
  padding: 14px;  
  box-shadow: var(--shadow-sm);  
  margin-bottom: 18px;  
}
```

```
/* inputs (WTFORMS rendert meistens input/select) */
```

```
.item-forms input[type="text"],  
.item-forms input[type="number"],  
.item-forms input,  
.item-forms select{  
  width: 100%;  
  padding: 10px 12px;  
  border-radius: 12px;  
  border: 1px solid var(--border);  
  background: #fafafa;  
  outline: none;  
  font-size: 14px;  
  transition: 0.15s ease;  
}  
  
.item-forms input:focus,  
.item-forms select:focus{  
  background: #fff;  
  border-color: rgba(37, 99, 235, 0.45);  
  box-shadow: 0 0 0 4px rgba(37, 99, 235, 0.12);  
}
```

```
/* submit (WTFORMS submit ist meist input[type=submit]) */
```

```
.item-forms input[type="submit"],  
.item-forms button[type="submit"]{  
  width: 100%;  
  padding: 10px 12px;  
  border-radius: 12px;  
  border: 0;
```

```
background: var(--primary);
color: #fff;
font-weight: 700;
cursor: pointer;
transition: 0.15s ease;
}
.item-forms input[type="submit"]:hover,
.item-forms button[type="submit"]:hover{
background: var(--primary-hover);
transform: translateY(-1px);
}

/* SHOPPING LIST GRID */
.shopping-list{
max-width: 980px;
margin: 0 auto;
padding: 0 6px 30px;
display: grid;
grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
gap: 14px;
}

.shopping-item-card{
position: relative;
background: var(--surface);
border: 1px solid var(--border);
border-radius: var(--radius);
padding: 14px 14px 48px;
box-shadow: var(--shadow);
transition: 0.15s ease;
}
.shopping-item-card:hover{
transform: translateY(-2px);
box-shadow: 0 14px 30px rgba(17,24,39,0.10);
}

.shopping-item-card h3{
margin: 0 0 8px;
font-size: 18px;
padding-right: 44px; /* Platz für Delete */
}

.item-quantity{
```

```
margin: 0 0 8px;
font-weight: 700;
color: var(--text);
}

.text-muted{
  color: var(--muted);
  font-size: 13px;
}

/* Delete button */
.delete-btn{
  position: absolute;
  top: 10px;
  right: 10px;
  width: 34px;
  height: 34px;
  border-radius: 10px;
  border: 1px solid #fecaca;
  background: #fff;
  cursor: pointer;
  display: flex;
  align-items: center;
  justify-content: center;
  transition: 0.15s ease;
}

.delete-btn:hover{
  background: #fef2f2;
  border-color: #fca5a5;
  transform: translateY(-1px);
}

/* Assigned badge bottom-right */
.assigned_to{
  position: absolute;
  bottom: 12px;
  right: 12px;
  padding: 6px 10px;
  border-radius: 999px;
  background: #eef2ff;
  color: #3730a3;
  font-weight: 700;
  font-size: 13px;
```

```
border: 1px solid #c7d2fe;
max-width: 75%;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
}

/* Responsive: Form stacked */
@media (max-width: 700px){
  .item-forms{
    grid-template-columns: 1fr;
  }
}
</style>
```

2 kleine HTML-Anpassungen (optional, aber hilft)

1. Das < entfernen:

html

 Code kopieren

```
<!-- < <-- das weg -->
<div class="shopping-list">
```

2. Debug fix:

jinja2

 Code kopieren

```
<p>DEBUG: items={{ shopping_items|length }}</p>
```

Wenn du willst, mache ich dir als nächstes auch:

- "gekauft" Optik (durchgestrichen + graue Card) **ohne DB**
- oder "gekauft" **persistent** (mit extra Tabelle), inkl. Button rechts unten.



File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr

```

am\Web-Development-Project\app.py", line 422, in
einkaufsplan
    wg_id = current_user.wg_id #Temporary set to 1 for
testing
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'wg_id'

```

Der Fehler heißt ganz simpel: `current_user` ist `None`, also ist niemand eingeloggt (oder du setzt `current_user` gar nicht), und dann knallt `current_user.wg_id`.

Du hast 3 saubere Lösungen (je nachdem wie du Login machst):

✓ Lösung A (du nutzt Session-Login mit `session["user_id"]`)

Dann holst du den User aus der DB und prüfst ihn:

python

 Code kopieren

```

from flask import session, redirect, url_for, flash
from sqlalchemy.sql import func

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    user_id = session.get("user_id")
    if not user_id:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    current_user = User.query.get(user_id)
    if not current_user:
        flash("User nicht gefunden. Bitte erneut einloggen.", "error")
        session.pop("user_id", None)
        return redirect(url_for("login"))

    wg_id = current_user.wg_id
    if not wg_id:
        flash("Du bist keiner WG zugeordnet.", "error")
        return redirect(url_for("dashboard"))

```



```

shopping_items = (ShoppingItem.query
                  .filter_by(wg_id=wg_id)
                  .order_by(ShoppingItem.item_id)
                  .all())

if form.validate_on_submit():
    # random Verantwortlicher aus derselben WG
    u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
    assigned_to = u.user_id if u else None

    new_item = ShoppingItem(
        wg_id=wg_id,
        added_by=current_user.user_id,
        assigned_to=assigned_to,
        name=form.item.data,
        quantity=form.quantity.data,
        added_at=db.func.now()
    )
    db.session.add(new_item)
    db.session.commit()
    flash("Artikel hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

✅ Damit ist `current_user` nie `None`, weil du vorher prüfst.

✅ **Lösung B (du nutzt Flask-Login: `from flask_login import current_user`)**

Dann musst du **Login erforderlich** machen:

python

 Code kopieren

```

from flask_login import login_required, current_user
from sqlalchemy.sql import func

```

```

@app.route("/einkaufsplan/", methods=["GET", "POST"])
@login_required
def einkaufsplan():

```

```

form = EinkaufsplanForm()

wg_id = current_user.wg_id
if not wg_id:
    flash("Du bist keiner WG zugeordnet.", "error")
    return redirect(url_for("dashboard"))

shopping_items = (ShoppingItem.query
                  .filter_by(wg_id=wg_id)
                  .order_by(ShoppingItem.item_id)
                  .all())

if form.validate_on_submit():
    u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
    assigned_to = u.user_id if u else None

    new_item = ShoppingItem(
        wg_id=wg_id,
        added_by=current_user.user_id,
        assigned_to=assigned_to,
        name=form.item.data,
        quantity=form.quantity.data,
        added_at=db.func.now()
    )
    db.session.add(new_item)
    db.session.commit()
    flash("Artikel hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Wenn `current_user` bei dir `None` ist, dann nutzt du Flask-Login noch nicht richtig oder bist nicht eingeloggt.

✓ Lösung C (du willst erstmal testen ohne Login)

Dann setz `wg_id` hart, aber nicht über `current_user`:



python

Code kopieren

```
wg_id = 1 # TEST
```

Warum passiert das bei dir gerade?

Du hast sowas wie:

```
python
```

 Code kopieren

```
wg_id = current_user.wg_id
```

aber `current_user` **wurde nicht gesetzt**, oder du bist nicht eingeloggt → `None` .

Wenn du mir sagst, ob du **Session** nutzt (`session["user_id"]`) oder **Flask-Login**, sag ich dir exakt welche Variante du nehmen sollst (und wo du `user_id` beim Login speichern musst).



```
def einkaufsplan():

    form = EinkaufsplanForm()

    if request.method == 'POST':

        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():",
              form.validate_on_submit())
        print("form.errors:", form.errors)

        shopping_items = (ShoppingItem.query
                          .filter_by(wg_id=1)      # TODO: später
                          .order_by(ShoppingItem.item_id)
                          .all())

        user_id = session.get("user_id")
        #if not user_id:
            #flash("Bitte zuerst einloggen.", "error")
        current_user = User.query.get(user_id)
```

```

    #if not current_user:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    wg_id = 1 #Temporary set to 1 for testing
    #wg_id = current_user.wg_id

    if form.validate_on_submit():
        #Temporary set to 1 for testing
        #if not user_id:
            #flash("Bitte zuerst einloggen.", "error")
            #return redirect(url_for("login"))

        #print("users in wg:",
        User.query.filter_by(wg_id=wg_id).count())

        u =
        User.query.filter_by(wg_id=1).order_by(func.random()).first(
        )

        assigned_to = u.user_id if u else None #Temporary
        assigned_to random user for testing
        #print("users in wg=1:",
        User.query.filter_by(wg_id=wg_id).count())

        new_item = ShoppingItem(
            wg_id=1, #wg_id = user.wg_id
            added_by=user_id, #To be replaced with
            user.user_id
            name=form.item.data,
            quantity=form.quantity.data,
            assigned_to=assigned_to #Temporary assigned_to
            random user_id between 1 and 3 for testing

        )
        db.session.add(new_item)

        db.session.commit()
        #print("gespeichert, item_id =", new_item.item_id)
        #print("count wg=1 =",
        ShoppingItem.query.filter_by(wg_id=1).count())

        flash("Artikel zum Einkaufsplan hinzugefügt!",
        "success")
        return redirect(url_for("einkaufsplan"))

```

```
        return render_template("einkaufsplan.html", form=form,
                               shopping_items=shopping_items)
```

Du hast gerade 3 Probleme in deinem Code:

1. `shopping_items` wird nur im POST-Zweig gesetzt → bei GET ist die Variable nicht definiert ⇒ Fehler/0 Einträge.
2. Indentation ist kaputt: `db.session.add(new_item)` / `commit()` liegen **außerhalb** von `if form.validate_on_submit()` (und teils sogar außerhalb der richtigen Blocktiefe).
3. Du setzt `wg_id=1`, fragst aber teilweise `current_user` ab – wenn `user_id` fehlt, ist `current_user=None`.

Hier ist eine saubere Version (mit `wg_id=1` als Test), die **GET + POST korrekt** macht und `random assigned_to` setzt:

python

```
from sqlalchemy.sql import func
from flask import session, redirect, url_for, flash, render_template, request

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    # TEST: später wg_id = current_user.wg_id
    wg_id = 1

    # IMMER laden (damit GET auch funktioniert)
    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=wg_id)
                      .order_by(ShoppingItem.item_id)
                      .all())

    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        user_id = session.get("user_id")
        if not user_id:
            flash("Bitte zuerst einloggen.", "error")
```

```

        return redirect(url_for("login"))

# optional: check user exists
current_user = User.query.get(user_id)
if not current_user:
    flash("User nicht gefunden. Bitte erneut einloggen.", "error")
    return redirect(url_for("login"))

if form.validate_on_submit():
    # random User aus derselben WG (wg_id)
    u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
    assigned_to = u.user_id if u else None

    new_item = ShoppingItem(
        wg_id=wg_id,
        added_by=user_id,
        assigned_to=assigned_to,
        name=form.item.data,
        quantity=form.quantity.data,
        added_at=db.func.now()
    )

    db.session.add(new_item)
    db.session.commit()

    flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))
else:
    flash("Bitte Eingaben prüfen.", "error")

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Wichtig (Template)

Stell sicher, dass du **auch wirklich** `shopping_items` **iterierst**:

```
jinja2
```

```
{% for item in shopping_items %}
```

 Code kopieren

und nicht irgendwo noch `shopping_list` verwendest (das führt sonst zu "0").

Wenn du willst, kann ich dir auch direkt zeigen, wie du später `wg_id` sauber aus `current_user.wg_id` ziehst (sobald Login + WG-Zuordnung stehen).

 Code kopieren

ja bitte

Alles klar — so ziehst du `wg_id` sauber aus dem eingeloggten User (Session), ohne überall `wg_id=1` zu hardcoden.

1) Beim Login: `user_id` in die Session schreiben

In deiner Login-Route (wenn Login erfolgreich):

```
python
```

```
session["user_id"] = user.user_id
```

 Code kopieren

Optional auch:

```
python
```

```
session["wg_id"] = user.wg_id
```

 Code kopieren

(ist praktisch, aber nicht zwingend)

2) Kleine Helfer-Funktion: aktuellen User holen

Pack das z.B. oben in `app.py` :

```
python
```

```
from flask import session
from functools import wraps
from flask import redirect, url_for, flash
```

```
def get_current_user():
    user_id = session.get("user_id")
    if not user_id:
```

```

        return None
    return User.query.get(user_id)

```

Optional (wenn du "Login required" willst):

 Code kopieren

python

```

def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not session.get("user_id"):
            flash("Bitte zuerst einloggen.", "error")
            return redirect(url_for("login"))
        return view(*args, **kwargs)
    return wrapped

```

 Code kopieren

3) Einkaufsplan-Route mit `current_user.wg_id`

So ist es am saubersten:

python

```

from sqlalchemy.sql import func
from flask import request, render_template, flash, redirect, url_for

@app.route("/einkaufsplan/", methods=["GET", "POST"])
# @login_required # optional, wenn du den Decorator nutzt
def einkaufsplan():
    form = EinkaufsplanForm()

    current_user = get_current_user()
    if not current_user:
        flash("Bitte zuerst einloggen.", "error")
        return redirect(url_for("login"))

    wg_id = current_user.wg_id
    if not wg_id:
        flash("Du bist keiner WG zugeordnet.", "error")
        return redirect(url_for("dashboard"))

    # immer laden (GET und nach Fehlern auch POST)
    shopping_items = (ShoppingItem.query

```



```

        .filter_by(wg_id=wg_id)
        .order_by(ShoppingItem.item_id)
        .all())

if request.method == "POST":
    if form.validate_on_submit():
        # random User aus derselben WG
        u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
        assigned_to = u.user_id if u else None

        new_item = ShoppingItem(
            wg_id=wg_id,
            added_by=current_user.user_id,
            assigned_to=assigned_to,
            name=form.item.data,
            quantity=form.quantity.data,
            added_at=db.func.now()
        )

        db.session.add(new_item)
        db.session.commit()

        flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
        return redirect(url_for("einkaufsplan"))
    else:
        flash("Bitte Eingaben prüfen.", "error")

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

4) Warum hattest du NoneType has no attribute wg_id?

Weil `current_user = User.query.get(user_id)` `None` ist, wenn:

- `session["user_id"]` gar nicht gesetzt ist, oder
- der User nicht existiert / DB zurückgesetzt wurde.

Mit dem obigen Check fängst du das sauber ab.

 Code kopieren

Wenn du mir kurz sagst, wo du aktuell `session["user_id"]` setzt (Login-Route), kann ich dir die 3–5 Zeilen exakt passend zu deinem Login-Code einbauen.



seit dem neuen design zeigt er mir nicht mehr an, wer assigned-to ist

Das liegt fast immer an **einem von diesen 2 Punkten**:

1. Du renderst im Template noch `item.assigned_to` (das ist nur die **ID**) – oder
2. die **Relationship** `assigned_to_user` ist nicht (richtig) definiert / nicht mitgeladen – dann ist sie im Template `None`.

Hier ist die saubere Lösung.

1) In deinem `ShoppingItem` Model: Relationships sauber definieren

python

```
class ShoppingItem(db.Model):
    __tablename__ = "SHOPPING_ITEM"
    item_id = db.Column(db.Integer, primary_key=True)
    wg_id = db.Column(db.Integer, db.ForeignKey("WG.wg_id"), nullable=False)

    added_by = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=False)
    assigned_to = db.Column(db.Integer, db.ForeignKey("USER.user_id"), nullable=True)

    name = db.Column(db.String, nullable=False)
    quantity = db.Column(db.Integer, nullable=False)
    added_at = db.Column(db.DateTime, server_default=db.func.now())

    added_by_user = db.relationship("User", foreign_keys=[added_by], backref="add")
    assigned_to_user = db.relationship("User", foreign_keys=[assigned_to], backref="assign")
```

Wichtig: `foreign_keys=[...]` damit SQLAlchemy weiß, welche FK-Spalte zu welcher Beziehung gehört.

Code kopieren

2) In der Route: `joinedload` nutzen (damit es sicher geladen ist)

python

```

from sqlalchemy.orm import joinedload

shopping_items = (ShoppingItem.query
    .filter_by(wg_id=wg_id)
    .options(
        joinedload(ShoppingItem.added_by_user),
        joinedload(ShoppingItem.assigned_to_user),
    )
    .order_by(ShoppingItem.item_id)
    .all()
)

```

 Code kopieren

3) Im Template: assigned_to sauber ausgeben

jinja2

```

{% if item.assigned_to_user %}
    <small class="assigned_to">Zuständig: {{ item.assigned_to_user.username }}</small>
{% else %}
    <small class="assigned_to">Zuständig: -</small>
{% endif %}

```

 Code kopieren

4) Mini-Debug (wenn's immer noch nicht angezeigt wird)

Füge kurz in deiner Route hinzu:

python

```

for i in shopping_items[:3]:
    print(i.item_id, i.assigned_to, i.assigned_to_user.username if i.assigned_to_

```

 Code kopieren

Wenn da `assigned_to` eine Zahl ist, aber `assigned_to_user` `None`, dann stimmt die Relationship-Definition (Punkt 1) noch nicht oder die `USER.user_id` passt nicht.

Wenn du mir einmal deine aktuelle `ShoppingItem` -Klasse (komplett) schickst, sag ich dir exakt, welche Zeile bei dir fehlt/anders ist.



```
5 None None
6 None None
7 None None
```

Nachgedacht für 24s >

Das Debug sagt eindeutig: In deiner DB steht bei diesen Items `assigned_to = NULL` (also gar niemand ist zugewiesen).

css

Code kopieren

```
5 None None
6 None None
7 None None
```

- erste `None` = `item.assigned_to` (ID) ist `NULL`
- zweite `None` = `item.assigned_to_user` kann dann natürlich auch nicht aufgelöst werden

Warum passiert das?

Dein "random User"-Block liefert in dem Moment **keinen User** zurück (also `u = None`) oder du hast die Items schon früher erstellt, als noch niemand in `wg_id=1` war → dann bleiben sie dauerhaft `NULL`.

1) Prüfe sofort, ob dein Random-User wirklich gefunden wird (POST)

Füge vor dem `new_item = ...` das hier ein:

python

Code kopieren

```
u = User.query.filter_by(wg_id=1).order_by(func.random()).first()
print("random user u =", u)
```

```
print("random user id =", u.user_id if u else None)
assigned_to = u.user_id if u else None
```

Wenn da `None` rauskommt, ist dein Problem: **User haben wahrscheinlich `wg_id != 1` oder `wg_id` ist `NULL`.**

➔ Quick-Check:

python

 Code kopieren

```
print("users in wg=1:", User.query.filter_by(wg_id=1).count())
print("all users:", [(x.user_id, x.username, x.wg_id) for x in User.query.all()])
```



2) Damit neue Items IMMER assigned_to bekommen (und nicht None)

Wenn du willst: "immer irgendwer aus der WG", dann so:

python

 Code kopieren

```
wg_id = 1 # später: current_user.wg_id

users = User.query.filter_by(wg_id=wg_id).all()
if not users:
    flash("Keine User in dieser WG gefunden - kann niemanden zuweisen.", "error")
    return redirect(url_for("einkaufsplan"))

u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
assigned_to = u.user_id
```



3) Deine alten Items haben assigned_to=NULL → die musst du einmal "reparieren"

Sonst werden die in der UI immer leer bleiben.

python

 Code kopieren

```
from sqlalchemy.sql import func

wg_id = 1
```

```

items = ShoppingItem.query.filter_by(wg_id=wg_id, assigned_to=None).all()

for it in items:
    u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
    it.assigned_to = u.user_id if u else None

db.session.commit()
print("fixed items:", len(items))

```

4) Template-Ausgabe ist dann so korrekt

jinja2

 Code kopieren

```

{% if item.assigned_to_user %}
    <small class="assigned_to">Zuständig: {{ item.assigned_to_user.username }}</small>
{% else %}
    <small class="assigned_to">Zuständig: -</small>
{% endif %}

```

Wenn du willst, schick mir kurz deine `User`-Tabelle (1–2 Zeilen aus `print("all users:", ...)`) – dann sag ich dir sofort, warum dein Random-Query manchmal `None` liefert (fast immer: `wg_id` bei Usern ist `NULL` oder nicht 1).



```

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():

```

```

    form = EinkaufsplanForm()
    wg_id = 1
    shopping_items = (ShoppingItem.query
                      .filter_by(wg_id=1) # TODO: später
    user.wg_id

```

```

        .options(joinedload(ShoppingItem.assigned_to_user),
                  joinedload(ShoppingItem.added_by_user))

```

```

        .order_by(ShoppingItem.item_id)
        .all())
    for i in shopping_items[:3]:
        print(i.item_id, i.assigned_to,
i.assigned_to_user.username if i.assigned_to_user else
None)

    if request.method == 'POST':

        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():",
form.validate_on_submit())
        print("form.errors:", form.errors)

        user_id = session.get("user_id", wg_id) #Temporary
set to 1 for testing
        #if not user_id:
            #flash("Bitte zuerst einloggen.", "error")
        current_user = User.query.get(user_id)
        #if not current_user:
            #flash("Bitte zuerst einloggen.", "error")
            #return redirect(url_for("login"))
        #Temporary set to 1 for testing
        #wg_id = current_user.wg_id

    if form.validate_on_submit():
        #Temporary set to 1 for testing
        #if not user_id:
            #flash("Bitte zuerst einloggen.", "error")
            #return redirect(url_for("login"))

        #print("users in wg:",
User.query.filter_by(wg_id=wg_id).count())

        u =
User.query.filter_by(wg_id=wg_id).order_by(func.random()).
first()
        print("random user u =", u)
        print("random user id =", u.user_id if u else None)
        assigned_to = u.user_id if u else None
        #Temporary assigned_to random user for testing
        #print("users in wg=1:",
User.query.filter_by(wg_id=wg_id).count())

```

```

        new_item = ShoppingItem(
            wg_id=wg_id, #wg_id = user.wg_id
            added_by=user_id, #To be replaced with
            user.user_id
            name=form.item.data,
            quantity=form.quantity.data,
            assigned_to=assigned_to #Temporary assigned_to
            random user_id between 1 and 3 for testing

        )
        db.session.add(new_item)

        db.session.commit()
        #print("gespeichert, item_id =", new_item.item_id)
        #print("count wg=1 =",
        ShoppingItem.query.filter_by(wg_id=1).count())

        flash("Artikel zum Einkaufsplan hinzugefügt!",
            "success")
        return redirect(url_for("einkaufsplan"))

    return render_template("einkaufsplan.html", form=form,
        shopping_items=shopping_items) #wg_id = user.wg_id

schreibe mir einfach diesen code um zum testen und fertig

```

Klar — hier ist deine Route **sauber umgeschrieben fürs Testen**, ohne unnötige Variablen/Fehlerquellen, mit:

- `wg_id = 1` fest
- `user_id` fallback auf 1 (Testuser)
- Random `assigned_to` aus `wg_id=1`
- **GET** lädt immer die Items (auch nach POST via redirect)
- Debug prints für Random-User + erste Items

python

```

from sqlalchemy.orm import joinedload
from sqlalchemy.sql import func

```



```
@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    # TEST: feste WG
    wg_id = 1

    # TEST: fester eingeloggter User (falls session leer ist)
    user_id = session.get("user_id") or 1

    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            # Random zuständiger User aus derselben WG
            u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
            print("random user u =", u)
            print("random user id =", u.user_id if u else None)

            if not u:
                flash("Keine User in WG gefunden - assigned_to bleibt leer.", "error")
                return redirect(url_for("einkaufsplan"))

            new_item = ShoppingItem(
                wg_id=wg_id,
                added_by=user_id,
                assigned_to=u.user_id,
                name=form.item.data,
                quantity=form.quantity.data,
            )
            db.session.add(new_item)
            db.session.commit()

            print("✅ gespeichert, item_id =", new_item.item_id)
            print("✅ count wg=1 =", ShoppingItem.query.filter_by(wg_id=wg_id).count())

            flash("Artikel zum Einkaufsplan hinzugefügt!", "success")
            return redirect(url_for("einkaufsplan"))
        else:
            flash("Fehler: Bitte Eingaben prüfen.", "error")
```

```

# GET (und nach POST-Redirect): Items laden
shopping_items = (
    ShoppingItem.query
    .filter_by(wg_id=wg_id)
    .options(
        joinedload(ShoppingItem.assigned_to_user),
        joinedload(ShoppingItem.added_by_user),
    )
    .order_by(ShoppingItem.item_id.desc())
    .all()
)

# Debug: erste 3 Items prüfen
for i in shopping_items[:3]:
    print(
        "ITEM:",
        i.item_id,
        "assigned_to=",
        i.assigned_to,
        "assigned_name=",
        (i.assigned_to_user.username if i.assigned_to_user else None),
    )

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Wenn du damit **immer noch** `assigned_name=None` siehst, dann liegt's **nicht** am Template, sondern daran, dass:

- `User.wg_id` nicht 1 ist (oder NULL) **oder**
- deine Relationships `assigned_to_user` / `added_by_user` in `ShoppingItem` nicht korrekt definiert sind (ForeignKey + relationship).

Wenn du willst, poste kurz deine `ShoppingItem` -Model-Klasse (nur die Spalten + relationships), dann sag ich dir in 10 Sekunden, ob das passt.



File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\app.py", line 443, in

```
einkaufsplan
    assigned_to=u.user_id #Temporary assigned_to random
    user_id between 1 and 3 for testing
    ^^^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'user_id'
```

Der Fehler bedeutet ganz simpel: `u` ist `None`, weil diese Query **keinen User findet**:

python

```
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
```

 Code kopieren

Also gibt es aktuell **keinen User mit** `wg_id = 1` (oder `wg_id` ist bei deinen Usern `NULL`).

Fix (minimal, sicher, ohne Crash)

Bau eine **Guard-If** ein, bevor du `u.user_id` benutzt:

python

```
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
```

```
if not u:
```

```
    flash("Es gibt keine Nutzer in dieser WG (wg_id=1). Bitte User einer WG zuwei
    return redirect(url_for("einkaufsplan"))
```

```
    assigned_to = u.user_id
```



Dann in `new_item`:

 Code kopieren

python

```
new_item = ShoppingItem(
    wg_id=wg_id,
    added_by=user_id,
    name=form.item.data,
    quantity=form.quantity.data,
    assigned_to=assigned_to
)
```

Warum passiert das bei dir?

 Code kopieren

Du hast schon mal geloggt: `users in wg=1: 0` → das war genau das Problem.

D. h. deine User sind wahrscheinlich so:

- `wg_id = None` (Standard)
- oder `wg_id` ist eine andere Zahl als `1`

Schnell testen / reparieren (1 User in WG=1 setzen)

Einmalig irgendwo (z.B. in einer Test-Route oder Python Shell) ausführen:

```
python
```

```
u = User.query.filter_by(username="testuser").first()
u.wg_id = 1
db.session.commit()
```

 Code kopieren

Danach funktioniert `filter_by(wg_id=1)` .

Wenn du willst, schick mir kurz den Output von:

```
python
```

```
print([(u.user_id, u.username, u.wg_id) for u in User.query.all()])
```

 Code kopieren

Dann sag ich dir sofort, welche `wg_id` du tatsächlich benutzen musst.

    ...

```
flash("Es gibt keine Nutzer in dieser WG (wg_id=1). Bitte
User einer WG zuweisen.", "error")
```

Genau — die Meldung heißt: In deiner DB gibt es aktuell keinen User mit `wg_id = 1` .

Deshalb liefert

```
python
```

```
User.query.filter_by(wg_id=1).first()
```

 Code kopieren

einfach `None` , und dann kann man keinen Random-User ziehen.

Schnellste Lösung (Testuser in WG 1 setzen)

Variante A: Flask Shell (empfohlen)

Im Projektordner (mit aktivem venv):

```
bash
```

```
flask shell
```

 Code kopieren

Dann:

```
python
```

```
from app import db
from db import User # oder wo dein User-Model liegt
```

```
u = User.query.filter_by(username="testuser").first()
print(u.user_id, u.username, u.wg_id)
```

```
u.wg_id = 1
db.session.commit()
```

```
print("Jetzt:", u.user_id, u.username, u.wg_id)
```

Danach sollte dein Random-Select funktionieren.

 Code kopieren

Alternative: Alle User (die noch None haben) in WG 1 packen (nur fürs Testen)

```
python
```

```
users = User.query.filter(User.wg_id.is_(None)).all()
for u in users:
    u.wg_id = 1
db.session.commit()
```

 Code kopieren

Warum du das brauchst

Dein Code macht:

```
python
```

```
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
```

 Code kopieren

Wenn **kein** User diese `wg_id` hat → `u = None` → Fehlermeldung / Flash.

Bonus: In deinem Code die `wg_id` korrekt aus dem eingeloggten User holen

Sobald Login/Sessions sauber sind:

python

```
user_id = session.get("user_id")
current_user = User.query.get(user_id)
wg_id = current_user.wg_id
```

 Code kopieren

Dann random:

python

```
u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
```

 Code kopieren

Wenn du willst, poste mir kurz den Output von:

python

```
print([(u.user_id, u.username, u.wg_id) for u in User.query.all()])
```

 Code kopieren

Dann sag ich dir direkt, ob du wirklich `wg_id=1` nutzen solltest oder ob deine WG-IDs anders sind.



```
print([(u.user_id, u.username, u.wg_id) for u in
User.query.all()])
```

 wohin soll ich das packen

Am besten **nicht** in deine Route, sondern kurz in die **Flask Shell** (sauberster Weg).

Variante 1 (empfohlen): Flask Shell

1. Terminal im Projektordner (wo auch `app.py` / `wsgi.py` liegt) und venv aktivieren
2. Dann:

```
bash
```

```
flask shell
```

 Code kopieren

3. In der Shell:

```
python
```

```
from db import User # oder: from app import User (je nachdem wo dein Model impo
print([(u.user_id, u.username, u.wg_id) for u in User.query.all()]])
```

 Code kopieren ,

Wenn `from db import User` nicht geht, probier:

```
python
```

```
from app import db
from db import User
```

 Code kopieren

oder

```
python
```

```
from app import User
```

 Code kopieren

(je nachdem wie dein Projekt aufgebaut ist)

Variante 2: Kurz in der Route (nur zum Debuggen)

Ganz oben in `einkaufsplan()` nach dem Query:

```
python
```

```
print([(u.user_id, u.username, u.wg_id) for u in User.query.all()]])
```

 Code kopieren

Dann Seite einmal laden → Ausgabe steht im Terminal.

👉 Danach wieder rausnehmen, sonst spammt es dein Terminal jedes Mal.



```
@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():

    form = EinkaufsplanForm()

    wg_id = 1 #Temporary set to 1 for testing

    user_id = session.get("user_id",1)

    if request.method == 'POST':

        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():",
form.validate_on_submit())
        print("form.errors:", form.errors)


        if form.validate_on_submit():
            #Temporary set to 1 for testing
            #if not user_id:
                #flash("Bitte zuerst einloggen.", "error")
                #return redirect(url_for("login"))

            #print("users in wg:",
User.query.filter_by(wg_id=wg_id).count())

            u =
User.query.filter_by(wg_id=wg_id).order_by(func.random()).
first()
            assigned_to = u.user_id if u else None #Temporary
assigned_to random user for testing
            #print("users in wg=1:",
User.query.filter_by(wg_id=wg_id).count())

            new_item = ShoppingItem(
wg_id=wg_id, #wg_id = user.wg_id
added_by=user_id, #To be replaced with
```



```

user.user_id
    name=form.item.data,
    quantity=form.quantity.data,
    assigned_to=assigned_to #Temporary assigned_to
random user_id between 1 and 3 for testing

)
db.session.add(new_item)
db.session.commit()
flash("Artikel erfolgreich hinzugefügt!", "success")
return redirect(url_for("einkaufsplan"))
#print("gespeichert, item_id =", new_item.item_id)
#print("count wg=1 =",
ShoppingItem.query.filter_by(wg_id=1).count())

shopping_items = (ShoppingItem.query
                  .filter_by(wg_id=wg_id) # TODO: später
user.wg_id

.options(joinedload(ShoppingItem.assigned_to_user),
joinedload(ShoppingItem.added_by_user))
        .order_by(ShoppingItem.item_id.desc())
        .all())

return render_template("einkaufsplan.html", form=form,
shopping_items=shopping_items) mach bitte dass es
funktioniert

```

Klar — dein Code scheitert gerade an zwei typischen Dingen:

1. **Wenn es keinen User mit `wg_id=1` gibt, ist `u = None` → dann crasht es später oder `assigned_to` bleibt leer.**
2. **Wenn das Formular nicht validiert, wird trotzdem weiter gerendert, aber du willst dann sauber die Seite mit den bestehenden Items anzeigen + Fehlermeldung.**

Hier ist eine **saubere funktionierende Version** (1:1 ersetzbar). Sie:

- lädt die Items immer (GET + POST)
- speichert nur bei `validate_on_submit()`
- weist **random** einen User aus der WG zu

- zeigt eine **Flash-Fehlermeldung**, wenn keine User in der WG sind
- zeigt Flash bei Formularfehlern

python

 Code kopieren

```
from sqlalchemy.orm import joinedload
from sqlalchemy.sql import func

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    wg_id = 1 # TODO später: wg_id = current_user.wg_id
    user_id = session.get("user_id") or 1 # testing fallback

    # Liste IMMER laden (damit sie auch bei Form-Fehlern angezeigt wird)
    shopping_items = (
        ShoppingItem.query
        .filter_by(wg_id=wg_id)
        .options(
            joinedload(ShoppingItem.assigned_to_user),
            joinedload(ShoppingItem.added_by_user),
        )
        .order_by(ShoppingItem.item_id.desc())
        .all()
    )

    # POST: speichern
    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            # Random User aus der WG auswählen
            u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()

            if not u:
                flash("Es gibt keine Nutzer in dieser WG (wg_id=1). Bitte User ei
                return render_template("einkaufsplan.html", form=form, shopping_i

            new_item = ShoppingItem(
                wg_id=wg_id,
```

```

        added_by=user_id,
        assigned_to=u.user_id,
        name=form.item.data,
        quantity=form.quantity.data,
    )

    db.session.add(new_item)
    db.session.commit()

    flash("Artikel erfolgreich hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))

# Formular abgeschickt, aber ungültig
flash("Bitte überprüfe deine Eingaben.", "error")

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Wenn du *garantiert* vermeiden willst, dass der Ersteller sich selbst zugewiesen wird

Ersetze die Random-Zeile durch:

python

 Code kopieren

```

u = (User.query
    .filter(User.wg_id == wg_id, User.user_id != user_id)
    .order_by(func.random())
    .first())

```

Und falls dann niemand übrig bleibt (weil nur 1 User in der WG ist), kannst du fallbacken auf "sich selbst".

Wenn du mir kurz dein `ShoppingItem`-Model zeigst (Spalten + Relationships), kann ich dir auch direkt sagen, ob `assigned_to` nullable sein muss bzw. warum bei alten Einträgen `assigned_to_user` manchmal `None` bleibt.

    ...

```

class ShoppingItem(db.Model):
    __tablename__ = 'SHOPPING_ITEM'

```

```

        item_id = db.Column(db.Integer, primary_key=True,
index=True)
        wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
        added_by = db.Column(db.Integer,
db.ForeignKey('USER.user_id'), nullable=False)
        assigned_to = db.Column(db.Integer,
db.ForeignKey('USER.user_id'))
        name = db.Column(db.String, nullable=False)
        quantity = db.Column(db.String)
        #Beziehungen
        wg = db.relationship('Wg',
back_populates='shopping_items')
        added_by_user = db.relationship('User', foreign_keys=
[added_by])
        assigned_to_user = db.relationship('User', foreign_keys=
[assigned_to], back_populates='assigned_items')

```

Danke — dein `ShoppingItem` ist fast richtig. Dass bei alten Einträgen `assigned_to_user` als `None` kommt, liegt zu **99%** daran, dass in der DB bei diesen Items `assigned_to` **NULL** ist (oder auf eine `user_id` zeigt, die es nicht gibt).

Damit du das sauber und ohne weitere Überraschungen hast, mach bitte diese 2 Anpassungen:

1) Beziehungen in User korrekt + ohne Warnungen

In `User` hast du schon:

python

 Code kopieren

```

assigned_items = db.relationship('ShoppingItem', foreign_keys='ShoppingItem.assigned_to_user_id')
added_items = db.relationship('ShoppingItem', foreign_keys='ShoppingItem.added_by_user_id')

```

◀

▶

Dann ergänze in `ShoppingItem` noch `back_populates`, damit SQLAlchemy beide Seiten sauber koppelt:

python

 Code kopieren

```

class ShoppingItem(db.Model):
    __tablename__ = 'SHOPPING_ITEM'

```

```

item_id = db.Column(db.Integer, primary_key=True, index=True)
wg_id = db.Column(db.Integer, db.ForeignKey('WG.wg_id'), nullable=False)
added_by = db.Column(db.Integer, db.ForeignKey('USER.user_id'), nullable=False)
assigned_to = db.Column(db.Integer, db.ForeignKey('USER.user_id'), nullable=True)

name = db.Column(db.String, nullable=False)
quantity = db.Column(db.String)

wg = db.relationship('Wg', back_populates='shopping_items')

added_by_user = db.relationship(
    'User',
    foreign_keys=[added_by],
    back_populates='added_items'
)

assigned_to_user = db.relationship(
    'User',
    foreign_keys=[assigned_to],
    back_populates='assigned_items'
)

```

Und in User so:

python

 Code kopieren

```

added_items = db.relationship('ShoppingItem', foreign_keys='ShoppingItem.added_by')
assigned_items = db.relationship('ShoppingItem', foreign_keys='ShoppingItem.assigned_to')

```



➡ Damit verschwinden auch deine SAWarnings, und `assigned_to_user.username` ist zuverlässig, wenn `assigned_to` gesetzt ist.

2) Warum ist `assigned_to_user` bei dir `None` ?

Weil `assigned_to` in der DB bei diesen Items `NULL` ist. Check das kurz:

python

 Code kopieren

```

print([(i.item_id, i.assigned_to) for i in ShoppingItem.query.order_by(ShoppingItem.item_id)])

```



Wenn da `None` steht → logisch, dass `assigned_to_user` auch `None` ist.

Schnell-Fix (einmalig): alle alten Items zufällig zuweisen

Wenn du willst, dass **alle bestehenden** Items einen zuständigen User bekommen:

python

 Code kopieren

```
from sqlalchemy.sql import func

wg_id = 1
users = User.query.filter_by(wg_id=wg_id).all()

if users:
    for item in ShoppingItem.query.filter_by(wg_id=wg_id).all():
        if item.assigned_to is None:
            u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
            item.assigned_to = u.user_id
    db.session.commit()
```

Bonus: In deinem Template richtig anzeigen

So ist es korrekt:

jinja2

 Code kopieren

```
{% if item.assigned_to_user %}
    <small class="assigned_to">{{ item.assigned_to_user.username }}</small>
{% else %}
    <small class="assigned_to">-</small>
{% endif %}
```

Wenn du willst, kann ich dir auch kurz zeigen, wie du beim Speichern **garantierst**, dass `assigned_to` nie `NULL` ist (z.B. `nullable=False` + Logik), ohne dass dir die App crasht, wenn gerade keine WG-User existieren.



```

(venv)
C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project>flask run
Traceback (most recent call last):
  File "<frozen runpy>", line 198, in _run_module_as_main
  File "<frozen runpy>", line 88, in _run_code
  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-
Project\venv\Scripts\flask.exe\__main__.py", line 6, in
<module>
    sys.exit(main())
          ~~~~^^

  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\flask\cli.py", line 1131, in main
    cli.main()
    ~~~~~~^^

  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\click\core.py", line 1406, in main
    rv = self.invoke(ctx)
  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\click\core.py", line 1873, in invoke
    return
_process_result(sub_ctx.command.invoke(sub_ctx))

~~~~~^
  File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-
packages\click\core.py", line 1269, in invoke
    return ctx.invoke(self.callback, **ctx.params)

~~~~~^
^
  File

```

```
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\click\core.py", line 824, in invoke
    return callback(*args, **kwargs)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\click\decorators.py", line 93, in new_func
    return ctx.invoke(f, obj, *args, **kwargs)

~~~~~^

File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\click\core.py", line 824, in invoke
    return callback(*args, **kwargs)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\cli.py", line 979, in run_command
    raise e from None
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\cli.py", line 963, in run_command
    app: WSGIApplication = info.load_app() # pyright:
ignore
    ~~~~~^

File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\cli.py", line 353, in load_app
    app = locate_app(import_name, None,
raise_if_not_found=False)
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\venv\Lib\site-packages\flask\cli.py", line 245, in locate_app
    __import__(module_name)
    ~~~~~^

File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 11, in
```



```

<module>
    from db import Activity, CleaningTask,
CleaningTemplate, Idea, Idea_Comment, Idea_Like,
ShoppingItem,db, User
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\db.py", line 164, in
<module>
    print([(i.item_id, i.assigned_to) for i in
ShoppingItem.query.order_by(ShoppingItem.item_id).all())
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask_sqlalchemy\model.py", line 23, in __get__
    cls, session=cls.__fsa__.session() # type: ignore[arg-type]
            ~~~~~^
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\orm\scoping.py", line 218, in __call__
    sess = self.registry()
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\sqlalchemy\util_collections.py", line 634, in
__call__
    key = self.scopefunc()
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask_sqlalchemy\session.py", line 111, in
_app_ctx_id
    return id(app_ctx._get_current_object()) # type:
ignore[attr-defined]
            ~~~~~^
    File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\werkzeug\local.py", line 519, in
_get_current_object
    raise RuntimeError(unbound_message) from None
RuntimeError: Working outside of application context.

```

This typically means that you attempted to use functionality that needed the current application. To solve this, set up an application context with `app.app_context()`. See the documentation for more information.

Der Fehler kommt **nicht** von deinen Models – sondern weil du diese Zeile in **db.py** stehen hast:

```
python
```

```
print([(i.item_id, i.assigned_to) for i in ShoppingItem.query.order_by(ShoppingIt
```



 Code kopieren

➔ In `db.py` läuft das **beim Import** (also bevor Flask überhaupt eine App / DB-Session Context hat). Deshalb: **“Working outside of application context.”**

✅ Fix (sofort)

Option A (empfohlen): Print aus `db.py` rauswerfen

Lösche die Query/Print-Zeile in `db.py`.

Und wenn du debuggen willst, mach das in einer Route (z.B. `einkaufsplan`) oder im Flask shell.

Option B: Debug-Print in eine Route packen

Zum Testen z.B. in `einkaufsplan()`:

```
python
```

```
shopping_items = ShoppingItem.query.order_by(ShoppingItem.item_id).all()
print([(i.item_id, i.assigned_to) for i in shopping_items])
```

 Code kopieren

Option C: Flask shell benutzen (ohne Code im Projekt)

Im Terminal:

```
bash
```

```
flask shell
```

 Code kopieren

Dann:

```
python
```

```
from db import ShoppingItem
[(i.item_id, i.assigned_to) for i in ShoppingItem.query.order_by(ShoppingItem.ite
```

<

 Code kopieren ,

Warum das so ist (kurz)

`ShoppingItem.query...` braucht eine aktive Flask App + Request/App Context. Beim Import von `db.py` existiert der noch nicht.

Wenn du mir sagst, ob du `app = Flask(__name__)` in `app.py` hast oder `create_app()` nutzt, kann ich dir auch die saubere Variante zeigen, wie man sowas als **Flask CLI Command** macht (z.B. `flask debug-shopping`).

    ...

ich will einfach einen test user ohne login

Alles klar — dann machen wir's **hart verdrahtet**: Wenn niemand eingeloggt ist, nimm einfach **immer** den Test-User (und setz ihn notfalls automatisch an).

1) Test-User automatisch erstellen (einmalig)

Pack das in `app.py` (nachdem `db.init_app(app)` gemacht wurde, aber **vor** `flask run` startet). Wichtig: **nicht** in `db.py`.

```
python
```

 Code kopieren

```

from werkzeug.security import generate_password_hash

@app.before_first_request
def create_test_user():
    test = User.query.filter_by(username="testuser").first()
    if not test:
        test = User(
            username="testuser",
            password_hash=generate_password_hash("test123"), # egal, du loggst e
            email="testuser@example.com",
            role="user",
            wg_id=1
        )
        db.session.add(test)
        db.session.commit()

```

Falls du `before_first_request` in deiner Flask-Version nicht hast/benutzt: sag kurz Bescheid, dann mache ich dir die Variante mit `with app.app_context(): ...` direkt beim Start.

2) In deinen Routes immer „current_user“ fallbacken (ohne Login)

Beispiel für `einkaufsplan()`:

python

 Code kopieren

```

from sqlalchemy.sql import func
from sqlalchemy.orm import joinedload

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    # Immer Testuser nehmen, wenn keiner in session ist
    user_id = session.get("user_id")
    current_user = User.query.get(user_id) if user_id else None
    if not current_user:
        current_user = User.query.filter_by(username="testuser").first()

    wg_id = current_user.wg_id or 1

```

```

if form.validate_on_submit():
    # random assigned_to aus derselben WG (notfalls testuser)
    u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
    assigned_to = u.user_id if u else current_user.user_id

    new_item = ShoppingItem(
        wg_id=wg_id,
        added_by=current_user.user_id,
        name=form.item.data,
        quantity=form.quantity.data,
        assigned_to=assigned_to
    )
    db.session.add(new_item)
    db.session.commit()
    flash("Artikel erfolgreich hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))

shopping_items = (ShoppingItem.query
    .filter_by(wg_id=wg_id)
    .options(joinedload(ShoppingItem.assigned_to_user),
             joinedload(ShoppingItem.added_by_user))
    .order_by(ShoppingItem.item_id.desc())
    .all()
)

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

3) Wichtig: WG-Zuordnung

Wenn du randomisieren willst, muss mindestens **ein User** `wg_id=1` haben. Der Testuser oben bekommt `wg_id=1`, damit funktioniert das sofort.

Wenn du willst, passe ich dir das gleiche „Testuser-Fallback“ auch für `activity_board`, `innovation_board`, `join_activity/leave_activity` etc. in genau deinem Stil an.



kannst du auch bitte request-method beibehalten

Ja — hier ist die **gleiche Route**, aber mit deinem Stil `if request.method == "POST":` beibehalten (und trotzdem „Testuser ohne Login“):

python

 Code kopieren

```
from sqlalchemy.sql import func
from sqlalchemy.orm import joinedload

@app.route("/einkaufsplan/", methods=["GET", "POST"])
def einkaufsplan():
    form = EinkaufsplanForm()

    # --- Fake current_user: Session -> sonst testuser ---
    user_id = session.get("user_id")
    current_user = User.query.get(user_id) if user_id else None
    if not current_user:
        current_user = User.query.filter_by(username="testuser").first()

    # Falls es den testuser noch nicht gibt -> abbrechen/Fehlermeldung
    if not current_user:
        flash("Testuser fehlt. Bitte einmal anlegen (username='testuser', wg_id=1)", "error")
        return render_template("einkaufsplan.html", form=form, shopping_items=[])

    wg_id = current_user.wg_id or 1

    # --- POST: Item speichern ---
    if request.method == "POST":
        print("Einkaufsplan POST angekommen")
        print("form.validate_on_submit():", form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            # random zuständig aus der WG (fallback: current_user)
            u = User.query.filter_by(wg_id=wg_id).order_by(func.random()).first()
            assigned_to = u.user_id if u else current_user.user_id

            new_item = ShoppingItem(
```

```

        wg_id=wg_id,
        added_by=current_user.user_id,
        name=form.item.data,
        quantity=form.quantity.data,
        assigned_to=assigned_to
    )
    db.session.add(new_item)
    db.session.commit()

    flash("Artikel erfolgreich hinzugefügt!", "success")
    return redirect(url_for("einkaufsplan"))
else:
    flash("Fehler beim Hinzufügen. Bitte Eingaben prüfen.", "error")

# --- GET (oder POST mit Fehlern): Items anzeigen ---
shopping_items = (ShoppingItem.query
    .filter_by(wg_id=wg_id)
    .options(
        joinedload(ShoppingItem.assigned_to_user),
        joinedload(ShoppingItem.added_by_user)
    )
    .order_by(ShoppingItem.item_id.desc())
    .all()
)

return render_template("einkaufsplan.html", form=form, shopping_items=shoppin

```

Wichtig (dein letzter Crash): Entferne alle `print(ShoppingItem.query...)` aus `db.py` — Queries dürfen nicht beim Import laufen („Working outside of application context“).

Wenn du willst, gebe ich dir noch den mini-Block, um den `testuser` *automatisch* beim Start anzulegen (auch ohne Login), aber so, dass es **nicht** crasht.



wie würde ich einen farbe picker zur card machen bei innovationboard? Also wenn der user beim erstellen der card die farbe auswählen möchte, wie müsste ich rangehen?

So würde ich es sauber angehen: **Farbe als Feld in der DB speichern**, im **Formular per Color-Picker auswählen**, und beim Rendern die **Card damit stylen**.

1) Datenbank: Farbspalte zur Idee hinzufügen

In deinem `Idea` -Model (db.py) eine Spalte für Hex-Farben (z.B. `#ffcc00`):

python

 Code kopieren

```
class Idea(db.Model):
    __tablename__ = "IDEA"
    idea_id = db.Column(db.Integer, primary_key=True)
    # ...
    color = db.Column(db.String(7), nullable=False, default="#ffffff") # "#RRGGBB"
```

Danach DB aktualisieren:

- Wenn du Flask-Migrate nutzt: Migration erstellen + upgraden
- Wenn ihr noch ohne Migration arbeitet (nur dev): DB neu erstellen oder `ALTER TABLE` (SQLite geht mit Alembic besser).

2) WTForms: Color-Picker ins Formular

WTForms hat nicht immer ein eigenes „ColorField“, aber du kannst ganz einfach ein `StringField` nehmen und im HTML `type="color"` rendern.

python

 Code kopieren

```
from wtforms import StringField, SubmitField
from wtforms.validators import Optional, Regexp

class InnovationForm(FlaskForm):
    title = StringField("Titel", validators=[...])
    description = StringField("Beschreibung", validators=[...])

    color = StringField(
        "Farbe",
        default="#ffffff",
        validators=[
            Optional(),
```



```

        Regexp(r"^[0-9A-Fa-f]{6}$", message="Ungültige Farbe.")
    ],
    render_kw={"type": "color"} # macht daraus einen Color Picker
)

submit = SubmitField("Speichern")

```

3) Route: Farbe speichern

In deiner `innovation_board()` Route beim Erstellen der Idee:

python

 Code kopieren

```

new_idea = Idea(
    wg_id=1,
    created_by=user.user_id,
    title=form.title.data,
    description=form.description.data,
    color=form.color.data or "#ffffff",
    created_at=db.func.now()
)
db.session.add(new_idea)
db.session.commit()

```

4) HTML: Feld im Modal anzeigen

In `innovationboard.html` im Modal:

html

 Code kopieren

```

<div style="margin-bottom: 15px;">
    <label>Farbe:</label>
    {{ form.color(style="display:block; width:60px; height:40px; padding:0; border:
</div>

```

5) Card: Hintergrundfarbe verwenden

Beim Rendern jeder Card:

html

 Code kopieren

```
<div class="idea-card" style="background-color: {{ idea.color or '#ffffff' }};">
  <h3>{{ idea.title }}</h3>
  <p class="idea-desc">{{ idea.description }}</p>
</div>
```



Optional: Nur „schöne“ Farben erlauben (Palette statt freier Picker)

Wenn du nicht willst, dass jemand Neon-Grün wählt, nimm lieber `SelectField`:

python

 Code kopieren

```
from wtforms import SelectField

color = SelectField(
    "Farbe",
    choices=[
        ("#ffffff", "Weiß"),
        ("#f8d7da", "Rosa"),
        ("#d1e7dd", "Grün"),
        ("#cfff4c", "Blau"),
        ("#fff3cd", "Gelb"),
    ],
    default="#ffffff"
)
```

Und im HTML einfach `{{ form.color(...) }}` rendern.

Wenn du mir kurz deine aktuelle `Idea`-Klasse + `InnovationForm` zeigst (nur die relevanten Felder), passe ich dir das 1:1 so an, dass es ohne Überraschungen läuft (inkl. Default-Farbe und Styling).



kann man das auch ohne db migrate machen?

Nachgedacht für 17s >

Ja — **aber**: irgendwas muss die DB-Struktur trotzdem ändern. Ohne `db migrate` hast du im Prinzip 2 praktikable Wege:

Option A (am einfachsten im Uni/Dev): DB neu erstellen (löscht Daten)

Wenn du SQLite nutzt (`.db` Datei) und es egal ist, dass alle Daten weg sind:

1. App stoppen
2. SQLite-Datei löschen (z.B. `instance/app.db` oder wo sie bei dir liegt)
3. Danach DB neu erstellen (je nach Setup):
 - entweder per `db.create_all()` (z.B. in Flask Shell)
 - oder dein init-script, falls du eins hast

Wichtig: `db.create_all()` **ändert keine bestehenden Tabellen**, es erstellt nur neue.

Darum muss die alte DB weg.

Option B (ohne Migration, Daten bleiben): Spalte per SQL hinzufügen (ALTER TABLE)

Das ist der typische "quick & dirty"-Weg ohne Alembic/Migrate.

1) Model erweitern (wie vorher)

```
python
```

 Code kopieren

```
color = db.Column(db.String(7), nullable=False, default="#ffffff")
```

2) Einmalig: SQL ausführen, um die Spalte hinzuzufügen

Mach das **nicht in `db.py`**, sondern einmal im **Flask Shell** (damit du App-Context hast):

```
bash
```

 Code kopieren

```
flask shell
```

Dann:

```
python
```

 Code kopieren

```

from sqlalchemy import text
from app import db # oder: from yourapp import db

db.session.execute(text(
    "ALTER TABLE IDEA ADD COLUMN color VARCHAR(7) NOT NULL DEFAULT '#ffffff';"
))
db.session.commit()

```

✅ Danach hat deine bestehende Tabelle die neue Spalte, ohne dass du migrieren musst.

Falls deine Tabelle anders heißt

Du nutzt bei dir oft `__tablename__ = 'IDEA' / uppercase`. Wenn dein Tabellename wirklich "IDEA" ist, passt das.

Häufige Stolpersteine

- Du darfst keine DB-Queries in `db.py` beim Import machen → sonst kommt "Working outside of application context".
- Wenn du den ALTER TABLE Befehl 2x ausführst, bekommst du "duplicate column name". Dann einfach weglassen.

Wenn du mir kurz sagst, **welche DB du nutzt (SQLite?)** und wie deine Tabelle heißt (IDEA oder Idea), schreibe ich dir den exakten ALTER TABLE -Befehl passend zu deinem Setup.



```

.idea-card{
    background-color: "{{ idea.color }}" or #ffffff;
    border:1px solid #000;
    border-radius:8px;
    padding:15px;
    box-sizing:border-box;
    box-shadow:2px 2px 5px rgba(0,0,0,0.1);
    flex: 0 1 520px; /* Card ist "bis zu" 520px breit,
schrumpft aber */
}

```

```

max-width: 100%; /* nie breiter als Bildschirm */
min-width: 0; /* wichtig in Flexbox, damit Text
wirklich umbrechen darf */
} kann man das so machen?

```

Fast — aber so wie du's geschrieben hast, funktioniert es nicht.

Warum nicht?

- CSS kann **kein Jinja** in einer `.css`-Datei / in einem globalen `<style>` für *jede* Card unterschiedlich auswerten.
- Und `background-color: "{{ idea.color }}" or #ffffff`; ist **kein gültiges CSS** (die Anführungszeichen + `or` machen's kaputt).

✓ Richtig (empfohlen): pro Card inline `style=...`

So bekommt *jede einzelne* Card ihre Farbe aus `idea.color`:

html

```

<div class="idea-card" style="background-color: {{ idea.color or '#ffffff' }};">
  ...
</div>

```



Code kopieren

Und dein CSS bleibt clean:

css

```

.idea-card{
  border:1px solid #000;
  border-radius:8px;
  padding:15px;
  box-sizing:border-box;
  box-shadow:2px 2px 5px rgba(0,0,0,0.1);
  flex: 0 1 520px;
  max-width: 100%;
  min-width: 0;
}

```

✓ Alternative (auch gut): CSS-Variable pro Card

Noch sauberer, weil du "nur" eine Variable setzt:

html

```
<div class="idea-card" style="--card-bg: {{ idea.color or '#ffffff' }};">
  ...
</div>
```

 Code kopieren

CSS

```
.idea-card{
  background-color: var(--card-bg, #ffffff);
  border:1px solid #000;
  border-radius:8px;
  padding:15px;
  box-sizing:border-box;
  box-shadow:2px 2px 5px rgba(0,0,0,0.1);
  flex: 0 1 520px;
  max-width: 100%;
  min-width: 0;
}
```

 Code kopieren

Mini-Fix: wenn du unbedingt alles im CSS lassen willst

Geht nur, wenn du eine Klasse pro Farbe machst (z.B. `.bg-red` , `.bg-blue`), aber dafür müsstest du Farbe auf Klassen mappen → unpraktisch.

Wenn du willst, zeig ich dir auch direkt, wie du im **Formular** einen Farbwähler (`<input type="color">`) einbindest und das beim Speichern in `idea.color` landet.

    ...

```
<table>
  <thead>
    <tr>
```

```

<th>Aufgabe</th>
<th>Zeitraum</th>
<th>Zuständig</th>
</tr>
</thead>

<tbody>
{% for eintrag in putzplan %}

<tr>
<td>
{% set task = eintrag.tasks[0] if eintrag.tasks else None
%}

{% if task %}
<form method="POST"
    action="{{ url_for('toggle_cleaning_task',
task_id=task.task_id) }}"
    class="m-0">
<label class="d-flex align-items-center gap-2 m-0">
<input class="form-check-input m-0"
    type="checkbox"
    name="done"
    onchange="this.form.submit()"
    {% if task.status == 'completed' %}checked{%
endif %}>
<span>{{ eintrag.name }}</span>
</label>
</form>
{% else %}
{{ eintrag.name }}
{% endif %}
</td>

<td>{{ eintrag.description }}</td>

<td>
{% if eintrag.tasks and eintrag.tasks[0].assigned_user
%}
{{ eintrag.tasks[0].assigned_user.username }}
{% else %}
-
{% endif %}

```

```

        </td>

    </tr>
{% endfor %}
</tbody>
</table>

```

ich bin jetzt wieder im putzplan. kannst du das so umschreiben, wie ich es bei den anderen seiten habe?

Klar — du kannst die Tabelle komplett durch **Cards** wie beim Innovation-/ActivityBoard ersetzen. Ersetze einfach deinen ganzen `<table>...</table>` -Block durch diesen **Card-Block**:

html

```

<div class="putzplan-list">
    {% for eintrag in putzplan %}
        {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
        {% set assigned_name = (eintrag.tasks[0].assigned_user.username if eintrag.ta

    <div class="putzplan-card">

        <!-- Top Row: Checkbox + Aufgabe -->
        <div class="putzplan-top">
            <div class="putzplan-title">
                {% if task %}
                    <form method="POST"
                        action="{% url_for('toggle_cleaning_task', task_id=task.task_id
                        class="m-0">
                        <label class="putzplan-check">
                            <input type="checkbox"
                                name="done"
                                onchange="this.form.submit()"
                                {% if task.status == 'completed' %}checked{% endif %}>
                            <span class="putzplan-name">{{ eintrag.name }}</span>
                        </label>
                    </form>
                {% else %}
                    <span class="putzplan-name">{{ eintrag.name }}</span>
                {% endif %}
            </div>

```



```

        <!-- Zuständig (Badge rechts) -->
        <div class="putzplan-assigned">
            {{ assigned_name }}
        </div>
    </div>

    <!-- Zeitraum -->
    <div class="putzplan-meta">
        <span class="putzplan-label">Zeitraum:</span>
        <span>{{ eintrag.description }}</span>
    </div>

</div>
{% else %}
    <p>Noch keine Einträge im Putzplan.</p>
{% endfor %}
</div>

<style>
.putzplan-list{
    display: flex;
    flex-direction: column;
    gap: 14px;
    margin-top: 18px;
    align-items: center;        /* Cards zentrieren */
}

.putzplan-card{
    width: 100%;
    max-width: 700px;
    border: 1px solid #000;
    border-radius: 12px;
    padding: 14px 16px;
    background: #f9f9f9;
    box-shadow: 2px 2px 5px rgba(0,0,0,0.08);
    box-sizing: border-box;
}

.putzplan-top{
    display: flex;
    justify-content: space-between;
    gap: 12px;

```

```
    align-items: flex-start;
}

.putzplan-title{
  min-width: 0;
  flex: 1;
}

.putzplan-check{
  display: flex;
  align-items: center;
  gap: 10px;
  margin: 0;
  cursor: pointer;
}

.putzplan-check input{
  width: 18px;
  height: 18px;
  cursor: pointer;
}

.putzplan-name{
  font-size: 1.05rem;
  font-weight: 700;
  overflow-wrap: anywhere;
  word-break: break-word;
}

.putzplan-assigned{
  flex: 0 0 auto;
  padding: 6px 10px;
  border-radius: 10px;
  background: #e0e0e0;
  border: 1px solid #000;
  font-style: italic;
  white-space: nowrap;
}

.putzplan-meta{
  margin-top: 10px;
  color: #444;
  overflow-wrap: anywhere;
```

```

}

.putzplan-label{
  font-weight: 700;
  margin-right: 6px;
}
</style>

```

Wenn du willst, kann ich dir als nächsten Schritt auch noch:

- den Status **"completed"** visuell machen (z.B. Aufgabe durchgestrichen + leicht ausgegraut),
- oder die Cards **nebeneinander** im Grid (wie bei deinen Idea-Cards) machen.



```

<h1>Putzplan</h1>
<link rel="stylesheet" href="putzplan.css">

{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
        {{ message }}

      </div>
    {% endfor %}
  {% endif %}
{% endwith %}

<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')

```

```
}}">InnovationBoard</a>  
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>  
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>  
</nav>
```

```
{% set remaining = 100 - progress %}
```

```
<div class="mt-3 mb-4">  
  <div class="d-flex justify-content-between align-items-  
center mb-2">  
    <strong>Fortschritt</strong>  
    <span class="text-muted">{{ completed_tasks }}/{{  
total_tasks }} erledigt ({{ progress }}%)</span>  
  </div>
```

```
<div class="progress-stacked" style="height: 18px;">  
  <div class="progress" role="progressbar"  
    aria-label="Erledigt"  
    aria-valuenow="{{ progress|int }}" aria-valuemin="0"  
aria-valuemax="100"  
    style="width: {{ progress|int }}%;">  
    <div class="progress-bar bg-success"></div>  
  </div>
```

```
<div class="progress" role="progressbar"  
  aria-label="Offen"  
  aria-valuenow="{{ remaining|int }}" aria-valuemin="0"  
aria-valuemax="100"  
  style="width: {{ remaining|int }}%;">  
  <div class="progress-bar bg-secondary"></div>  
</div>  
</div>  
</div>
```

```
<input type="checkbox" id="modalToggle" style="display:  
none;">
```

```
<label for="modalToggle" style="cursor: pointer; padding:  
10px 20px; background-color: #007bff; color: white;  
border-radius: 5px; display: inline-block;">
```

```
+ Neuen Eintrag erstellen
</label>
```

```
<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>
```

```
<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
```

```
  <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;"> × </label>
```

```
  <h2>Neuen Eintrag erstellen</h2>
```

```
  <h3>Bestimme, wer für was zuständig ist</h3>
```

```
  <form method="POST">
```

```
    {{ form.hidden_tag() }}
```

```
  <div style="margin-bottom: 15px;">
```

```
    <label for="woche">Kalenderwoche:</label>
```

```
    {{ form.woche(style="display: block; margin-top:
5px; padding: 8px; width: 100%;" ) }}
```

```
  </div>
```

```
  <div style="margin-bottom: 15px;">
```

```
    <label for="von_datum">Von:</label>
```

```
    {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;" ) }}
```

```
  </div>
```

```
  <div style="margin-bottom: 15px;">
```

```
    <label for="bis_datum">Bis:</label>
```

```
    {{ form.bis_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;" ) }}
```

```
  </div>
```

```
  <div style="margin-bottom: 15px;">
```

```
    <label for="aufgabe">Aufgabe:</label>
```

```
    {{ form.aufgabe(style="display: block; margin-top:
5px; padding: 8px; width: 100%;" ) }}
```

```
  </div>
```

```
  <div style="margin-bottom: 15px;">
```

```

<label for="zustaendig">Zuständig:</label>

{{ form.zustaendig(
    list="users",
    id="zustaendig",
    style="display:block; margin-top:5px; padding:8px;
width:100%;";
    placeholder="z.B. Max"
) }}

<datalist id="users">
    {% for user in all_users %}
    <option value="{{ user.username }}"></option>
    {% endfor %}
</datalist>

{% if form.zustaendig.errors %}
    <span style="color:red;">{{
form.zustaendig.errors[0] }}</span>
{% endif %}
</div>

{{ form.submit() }}
</form>
</div>

<style>
    #modalToggle:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggle:checked ~ #modalContent {
        display: block !important;
    }
</style>

<div class="putzplan-list">
    {%for eintrag in eintraege%}
    {%set task = eintrag.tasks[0] if eintrag.tasks else None %}
    {%set assigned_name =
(eintrag.tasks[0].assigned_user.username if eintrag.tasks
and eintrag.tasks[0].assigned_user else "-")%}
    <div class="putzplan-card">

```

```

<div class =putzplan-top>
  <div class ="putzplan-title">
    {%if task%}
      <form method ="POST" action="
{{url_for('toggle_cleaning_task',task_id=task.task_id)}}" class
= "m-0">
        <label class="putzplan-checkbox">
          <input type="checkbox" name ="done"
onchange="this.form.submit()"
          {%if task.status == "completed" %}checked{%
endif %}>
          <span class ="putzplan-name">{{eintrag.name}}
</span>
        </label>
      </form>
    {%else%}
      <span class="putzplan-name">{{eintrag.name}}
</span>
    {%endif%}
  </div>
  <div class = "putzplan-assigned">{{assigned_name}}
</div>
</div>
<div class="putzplan-metadata">
  <span class ="putzplan-label">Zeitraum:</span>
  <span>{{eintrag.description}}</span>
</div>
</div>
{%else%}
  <p>Noch keine Einträge im Putzplan. Sei die/der Erste!
</p>
{%endfor%}
</div>

<p>Einträge geladen: {{ putzplan|length }}</p>

<style>

.putzplan-list{
  display:flex;
  flex-direction: table-column;

```

```
gap: 14px;  
margin-top: 18px;  
align-items: flex-start;  
}
```

```
.putzplan-card{  
width: 100%;  
max-width: 700px;  
border: 1px solid #000  
border-radius 12px;  
padding: 14px 16px;  
background: #f9f9f9;  
box-shadow: 2px 2px 2px;  
box-sizing: border-box;  
}
```

```
.putzplan-top{  
display: flex;  
justify-content: space-between;  
gap: 12px;  
align-items: flex-start;  
}
```

```
.putzplan-title{  
min-width: 0;  
flex: 1;  
}
```

```
.putzplan-checkbox{  
  
display: flex;  
align-items: center;  
gap: 10px;  
margin: 0;  
cursor: pointer;  
}
```

```
.putzplan-name{  
font-size: 1.05rem;  
font-weight: 700;  
overflow-wrap: anywhere;  
word-break: break-word;  
}
```



```
.putzplan-assigned{
  flex: 0 0 auto;
  padding: 6px 10px;
  border-radius: 10px;
  background: #e0e0e0;
  border: 1px solid #000;
  font-style: oblique;
  white-space: nowrap;
}

.putzplan-metadata{
  margin-top: 10px;
  color: #444;
  overflow-wrap: anywhere;
}

.putzplan-label{
  font-weight: 700px;
  margin-right: 6px;
}

.progress-stacked {
  height: 18px !important;
  border-radius: 10px !important;
  overflow: hidden !important;
}

.progress {
  height: 18px !important;
  background-color: #0080ff !important;
}

.progress-bar {
  transition: width .6s ease !important;
}

</style>
warum zeigt er die einträge nicht an?
```

Der Grund ist einfach ein Variablen-Mismatch + Jinja-Else falsch platziert:

1. Du renderst die Cards mit

jinja2

```
{% for eintrag in eintraege %}
```

 Code kopieren

aber unten debugst du mit

```
jinja2
```

```
{{ putzplan|length }}
```

 Code kopieren

→ Wenn du im `render_template(...)` **putzplan** übergibst (wie vorher), dann ist `eintraege` **leer/undefiniert** ⇒ es wird nichts gerendert.

2. Dein `{% else %}` im Loop ist ok *prinzipiell*, aber du hast zusätzlich noch ein paar Syntax-Fehler im CSS (die stören zwar das Rendering nicht unbedingt, aber sind trotzdem falsch). Der Hauptfehler bleibt Punkt 1.

Fix (minimal): Nimm überall dieselbe Variable

Ändere diesen Block:

```
jinja2
```

```
<div class="putzplan-list">
  {%for eintrag in eintraege%}
```

 Code kopieren

zu:

```
jinja2
```

```
<div class="putzplan-list">
  {% for eintrag in putzplan %}
```

 Code kopieren

Und dann am Ende debug auch passend:

```
jinja2
```

```
<p>Einträge geladen: {{ putzplan|length }}</p>
```

 Code kopieren

Fertig.

Bonus: deine Schleife sauber (inkl. else)

So sollte der ganze Block aussehen:

jinja2

```
<div class="putzplan-list">
  {% for eintrag in putzplan %}
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
    {% set assigned_name = (eintrag.tasks[0].assigned_user.username if eintrag.ta

<div class="putzplan-card">
  <div class="putzplan-top">
    <div class="putzplan-title">
      {% if task %}
        <form method="POST"
          action="{{ url_for('toggle_cleaning_task', task_id=task.task_id
            class="m-0">
          <label class="putzplan-checkbox">
            <input type="checkbox"
              name="done"
              onchange="this.form.submit()"
              {% if task.status == "completed" %}checked{% endif %}>
            <span class="putzplan-name">{{ eintrag.name }}</span>
          </label>
        </form>
      {% else %}
        <span class="putzplan-name">{{ eintrag.name }}</span>
      {% endif %}
    </div>

    <div class="putzplan-assigned">{{ assigned_name }}</div>
  </div>

  <div class="putzplan-metadata">
    <span class="putzplan-label">Zeitraum:</span>
    <span>{{ eintrag.description }}</span>
  </div>
</div>

{% else %}
  <p>Noch keine Einträge im Putzplan. Sei die/der Erste!</p>
{% endfor %}
</div>

<p>Einträge geladen: {{ putzplan|length }}</p>
```

Nebenbei: du hast CSS-Fehler (solltest du fixen)

In `.putzplan-card` fehlen Semikolons/Colon:

CSS

```
.putzplan-card{  
  border: 1px solid #000; /* ; fehlt bei dir */  
  border-radius: 12px; /* : und ; fehlen bei dir */  
}
```

Und in `.putzplan-label`:

 Code kopieren

CSS

```
.putzplan-label{  
  font-weight: 700; /* nicht 700px */  
}
```

 Code kopieren

Wenn du mir kurz sagst, wie du in Flask renderst (z.B. `render_template("putzplan.html", putzplan=..., ...)`), kann ich dir exakt sagen, welche Variable du wirklich bekommst — aber mit hoher Wahrscheinlichkeit ist es einfach **putzplan** statt **eintraege**.



```
<form method = "POST" action="{  
  url_for('delete_cleaning_task', task_id=task.task_id) }"  
  class="delete-form">  
  <button type="submit" class="delete-btn"  
  
    onclick="return confirm('Aufgabe wirklich  
löschen?');">   
  </button>  
</form>
```

```
<div class="putzplan-list">  
  {%for eintrag in putzplan%}
```

```

{%set task = eintrag.tasks[0] if eintrag.tasks else None %}
{%set assigned_name =
(eintrag.tasks[0].assigned_user.username if eintrag.tasks
and eintrag.tasks[0].assigned_user else "-")%}
<div class="putzplan-card">
  <div class="putzplan-top">
    <div class="putzplan-title">
      {%if task%}

        <form method="POST" action="
{{url_for('toggle_cleaning_task',task_id=task.task_id)}}" class
="m-0">
          <label class="putzplan-checkbox">
            <input type="checkbox" name="done"
onchange="this.form.submit()"
            {%if task.status=="completed"%}checked{%
endif %}>
            <span class="putzplan-name">{{eintrag.name}}
          </span>
        </label>
      </form>
      {%else%}
        <span class="putzplan-name">{{eintrag.name}}
      </span>
      {%endif%}
    </div>
    <div class="putzplan-assigned">{{assigned_name}}
  </div>
</div>
<div class="putzplan-metadata">
  <span class="putzplan-label">Zeitraum:</span>
  <span>{{eintrag.description}}</span>
</div>
</div>
{%else%}
  <p>Noch keine Einträge im Putzplan. Sei die/der Erste!
</p>
{%endif%}
</div>

```

wo gehört der löschen button rein?

Der Löschen-Button muss in die Card rein, und zwar **pro Eintrag/Task**, also **innerhalb** der `{% for eintrag in putzplan %}`-Schleife – am besten **oben rechts** in der `.putzplan-card` (genau wie bei Activity/Innovation).

1) So platzierst du ihn richtig (oben rechts in der Card)

Wichtig: Der Delete-Form muss **nachdem** `task` **gesetzt wurde** kommen, also **innerhalb** der Card und nur wenn `task` existiert.

jinja2

```
<div class="putzplan-list">
  {% for eintrag in putzplan %}
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
    {% set assigned_name = (eintrag.tasks[0].assigned_user.username if eintrag.ta

  <div class="putzplan-card">

    {% if task %}
      <form method="POST"
        action="{{ url_for('delete_cleaning_task', task_id=task.task_id) }}"
        class="delete-form">
        <button type="submit" class="delete-btn"
          onclick="return confirm('Aufgabe wirklich löschen?');">  </butt
        </form>
    {% endif %}

  <div class="putzplan-top">
    <div class="putzplan-title">
      {% if task %}
        <form method="POST"
          action="{{ url_for('toggle_cleaning_task', task_id=task.task_id) }}"
          class="m-0">
          <label class="putzplan-checkbox">
            <input type="checkbox" name="done" onchange="this.form.submit()"
              {% if task.status == "completed" %}checked{% endif %}>
            <span class="putzplan-name">{{ eintrag.name }}</span>
          </label>
        </form>
      {% else %}
        <span class="putzplan-name">{{ eintrag.name }}</span>
      {% endif %}
    </div>
```

```

        <div class="putzplan-assigned">{{ assigned_name }}</div>
    </div>

    <div class="putzplan-metadata">
        <span class="putzplan-label">Zeitraum:</span>
        <span>{{ eintrag.description }}</span>
    </div>
</div>

{% else %}
    <p>Noch keine Einträge im Putzplan. Sei die/der Erste!</p>
{% endfor %}
</div>

```

2) CSS, damit er wirklich oben rechts sitzt

CSS

```

.putzplan-card{
    position: relative; /* wichtig! */
}

.delete-form{
    margin: 0;
}

.delete-btn{
    position: absolute;
    top: 12px;
    right: 12px;
    border: none;
    background: transparent;
    cursor: pointer;
    font-size: 18px;
    line-height: 1;
}

```

Kurz gesagt

- ✓ Der Button gehört in jede `.putzplan-card` , innerhalb der for-loop, nicht `Code kopieren` gesetzt wurde.
- ✓ Mit `position: relative` auf der Card und `position: absolute` auf dem Button landet er oben rechts.

Wenn du willst, kann ich dir auch zeigen, wie du ihn **nur anzeigen lässt**, wenn der aktuelle User der Ersteller ist / zuständig ist.



die aufgabe wird nicht aus der seite gelöscht, sondern ein strich ist im zuständig und es gibt keine checkbox

Nachgedacht für 42s >

Das Verhalten passt ziemlich genau zu „du löschst nur den `CleaningTask` , aber dein UI rendert die Liste aus `CleaningTemplate` “:

- Du klickst  → `CleaningTask` wird gelöscht
- Der **Template-Eintrag bleibt** (weil `CleaningTemplate` weiter existiert)
- Dadurch hat `eintrag.tasks` danach **keinen Task mehr** → **keine Checkbox** (weil `task = None`)
- Und bei „Zuständig“ siehst du dann nur noch „-“ (dein „Strich“)

Heißt: Dein Delete-Button löscht aktuell das falsche Objekt (Task statt Eintrag/Template) – oder du musst die Liste anders aufbauen.

Option A (empfohlen): UI zeigt `CleaningTask` -Einträge, Delete löscht wirklich den Eintrag

1) Route: Liste aus `CleaningTask` holen

Statt Templates zu laden, lädst du Tasks:

```
python
```

```
putzplan = (CleaningTask.query
            .filter_by(wg_id=1)    # später user.wg_id
```



```

        .options(joinedload(CleaningTask.template),
                  joinedload(CleaningTask.assigned_user))
        .order_by(CleaningTask.task_id.desc())
        .all()

```

```

return render_template("putzplan.html", putzplan=putzplan, form=form, all_users=a

```

2) Template: rendere pro task

 Code kopieren

⏪ Dann brauchst du nicht mehr eintrag.tasks[0] : ⏩

```

jinja2

```

```

<div class="putzplan-list">
    {% for task in putzplan %}
        <div class="putzplan-card">

            <!-- Delete (löscht Task wirklich) -->
            <form method="POST"
                action="{% url_for('delete_cleaning_task', task_id=task.task_id) %}"
                class="delete-form">
                <button type="submit" class="delete-btn"
                    onclick="return confirm('Aufgabe wirklich löschen?');">🗑️</button>
            </form>

            <!-- Checkbox -->
            <form method="POST"
                action="{% url_for('toggle_cleaning_task', task_id=task.task_id) %}"
                class="m-0">
                <label class="putzplan-checkbox">
                    <input type="checkbox" name="done" onchange="this.form.submit()"
                        {% if task.status == 'completed' %}checked{% endif %}>
                    <span class="putzplan-name">{% task.template.name %}</span>
                </label>
            </form>

            <div class="putzplan-assigned">
                {% task.assigned_user.username if task.assigned_user else '-' %}
            </div>

            <div class="putzplan-metadata">
                <span class="putzplan-label">Zeitraum:</span>
                <span>{% task.template.description %}</span>

```

```

        </div>

    </div>
{% else %}
    <p>Noch keine Einträge im Putzplan.</p>
{% endfor %}
</div>

```

➔ Ergebnis: Delete löscht die Zeile wirklich, Checkbox bleibt logisch.

Option B: Du willst Templates anzeigen, dann muss Delete auch das Template löschen

Dann brauchst du eine `delete_cleaning_template(template_id)` Route und im HTML:

 Code kopieren

jinja2

```

<form method="POST"
    action="{{ url_for('delete_cleaning_template', template_id=eintrag.template_id) }}"
    class="delete-form">
    <button type="submit" class="delete-btn"
        onclick="return confirm('Eintrag wirklich löschen?');">🗑️</button>
</form>

```

◀

▶

 Code kopieren

Und in der Route:

python

```

@app.post("/putzplan/template/<int:template_id>/delete")
def delete_cleaning_template(template_id):
    template = CleaningTemplate.query.get_or_404(template_id)
    db.session.delete(template) # falls Tasks dranhängen -> cascade nötig
    db.session.commit()
    flash("Eintrag gelöscht.", "success")
    return redirect(url_for("putzplan"))

```

Wichtig: Damit beim Template-Löschen auch Tasks mit weggehen, brauchst du eine `cascade=True` Beziehung z.B.:

 Code kopieren ▶

Relationship z.B.:

python

```
tasks = db.relationship("CleaningTask", back_populates="template"
```

 Code kopieren


Was sollst du nehmen?

Wenn du willst: „Diese Aufgabe (für diese Woche/Periode) löschen“ → Option A.

Wenn du willst: „Die Aufgabe-Art komplett entfernen (Template)“ → Option B.

Wenn du mir kurz sagst, ob dein `putzplan` im Backend aktuell **Templates** oder **Tasks** enthält, sag ich dir exakt, welche Option bei dir am wenigsten Umbau braucht.



ich will einfach den eintrag löschen können und ihn von der seite weghaben

Nachgedacht für 45s >

Alles klar — bei dir bleibt der „Eintrag“ sichtbar, weil du **nur den** `CleaningTask` **löscht**, aber du **rendert die Seite aus** `CleaningTemplate`. Dann hat das Template danach keinen Task mehr → Checkbox weg, „Zuständig“ wird -, aber **die Card bleibt**.

Du hast 2 saubere Wege. Wenn du einfach willst:  klicken → **Card ist weg**, nimm Option 1 (schnell). Wenn du willst: **Eintrag komplett aus DB entfernen**, nimm Option 2 (richtig "löschen").

✅ Option 1 (schnell): Card nur anzeigen, wenn es einen Task gibt

Dann ist nach dem Löschen des Tasks die Card sofort weg (ohne DB-Änderungen).

In deinem `putzplan.html`:

```
jinja2
```

```

<div class="putzplan-list">
  {% for eintrag in putzplan %}
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
    {% if task %}

      <div class="putzplan-card">

        <!-- DELETE (löscht den Task, und weil kein task mehr existiert, verschwi
        <form method="POST"
          action="{% url_for('delete_cleaning_task', task_id=task.task_id) %}"
          class="delete-form">
            <button type="submit" class="delete-btn"
              onclick="return confirm('Aufgabe wirklich löschen?');">  </butt
        </form>

        <div class="putzplan-top">
          <div class="putzplan-title">
            <form method="POST"
              action="{% url_for('toggle_cleaning_task', task_id=task.task_id) %}"
              class="m-0">
              <label class="putzplan-checkbox">
                <input type="checkbox" name="done" onchange="this.form.submit()"
                  {% if task.status == "completed" %}checked{% endif %}>
                <span class="putzplan-name">{{ eintrag.name }}</span>
              </label>
            </form>
          </div>

          <div class="putzplan-assigned">
            {{ task.assigned_user.username if task.assigned_user else "-" }}
          </div>
        </div>

        <div class="putzplan-metadata">
          <span class="putzplan-label">Zeitraum:</span>
          <span>{{ eintrag.description }}</span>
        </div>

      </div>

    {% endif %}
  {% else %}
    <p>Noch keine Einträge im Putzplan.</p>

```

```
{% endfor %}
</div>
```

👉 Ergebnis: Task löschen = Card weg (weil du nur Cards mit Task renderst).

✅ Option 2 (richtig löschen): Template + alle zugehörigen Tasks löschen

Wenn „Eintrag löschen“ bedeutet: **diese Aufgabe existiert gar nicht mehr**, dann musst du `CleaningTemplate` löschen (und seine Tasks mit).

 Code kopieren

1) In `CleaningTemplate` Relationship Cascade setzen

In `db.py` beim Template:

```
python

tasks = db.relationship(
    "CleaningTask",
    back_populates="template",
    cascade="all, delete-orphan"
)
```

 Code kopieren

2) Route zum Löschen des Templates

```
python

@app.route("/putzplan/template/<int:template_id>/delete", methods=["POST"])
def delete_cleaning_template(template_id):
    template = CleaningTemplate.query.get_or_404(template_id)

    db.session.delete(template)
    db.session.commit()

    flash("Eintrag gelöscht.", "success")
    return redirect(url_for("putzplan"))
```

3) Button im HTML (pro Card)

 Code kopieren

```
jinja2
```

```
<form method="POST"
  action="{{ url_for('delete_cleaning_template', template_id=eintrag.template_id,
  class="delete-form">
  <button type="submit" class="delete-btn"
    onclick="return confirm('Eintrag wirklich löschen?');">🗑️</button>
</form>
```

 Code kopieren

⚠️ eintrag.template_id musst du ggf. anpassen, falls dein PK anders heißt (z.B. cleaning_id o.ä.).

Wo gehört der Delete-Button hin?

Am besten oben rechts in die Card, also direkt innerhalb von `<div class="putzplan-card">` als erstes Element und dann per CSS absolut positionieren:

CSS

```
.putzplan-card { position: relative; }

.delete-form { position: absolute; top: 10px; right: 10px; margin: 0; }
.delete-btn { background: transparent; border: none; cursor: pointer; font-size:
```

 Code kopieren

Meine Empfehlung für dich

Wenn du ohne viel Umbau willst, dass es einfach „weg ist“: **Option 1.**

Wenn du wirklich „Eintrag löschen“ im Sinne von **komplett entfernen** meinst: **Option 2.**

Wenn du mir sagst, wie der Primary Key bei `CleaningTemplate` heißt (z.B. `template_id` ?), schreibe ich dir die Delete-Route + Button 1:1 passend zu deinen Klassen.



File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgramm\Web-Development-Project\venv\Lib\site-

```
packages\werkzeug\routing\map.py", line 924, in build
    raise BuildError(endpoint, values, method, self)
werkzeug.routing.exceptions.BuildError: Could not build url
for endpoint 'delete_cleaning_task' with values
['template_id']. Did you forget to specify values ['task_id']?
```

```
@app.route("/putzplan/task/<int:task_id>/delete",
methods=["POST"])
def delete_cleaning_task(template_id):
    #current_user_id = session.get("user_id")
    #if idea.creator.user_id != current_user_id:
    #    flash("Sie können nur Ihre eigenen Tasks löschen.",
    "error")
    #abort(403)
    template =
CleaningTemplate.query.get_or_404(template_id)
    db.session.delete(template)
    db.session.commit()
    flash("Aufgabe erfolgreich gelöscht.", "success")
    return redirect(url_for("putzplan"))
```

Nachgedacht für 15s >

Der Fehler ist 100% ein **Mismatch** zwischen Route / Funktionsparameter / url_for():

- Deine Route heißt: `"/putzplan/task/<int:task_id>/delete"` → erwartet **task_id**
- Du rufst aber `url_for('delete_cleaning_task', template_id=...)` auf → gibst **template_id**
- Und deine Funktion ist `def delete_cleaning_task(template_id):` → nimmt **template_id**, obwohl die Route **task_id** liefert

Darum: `BuildError ... Did you forget to specify values ['task_id']?`

✅ Lösung A (Eintrag komplett löschen = Template löschen)

Du willst den **Eintrag** (CleaningTemplate) löschen, nicht nur den Task → dann mach es sauber so:

1) Route + Funktion richtig

python

```

@app.route("/putzplan/template/<int:template_id>/delete", methods=["POST"])
def delete_cleaning_template(template_id):
    template = CleaningTemplate.query.get_or_404(template_id)
    db.session.delete(template)
    db.session.commit()
    flash("Eintrag erfolgreich gelöscht.", "success")
    return redirect(url_for("putzplan"))

```

✓ Wichtig: Endpoint heißt jetzt **delete_cleaning_template** und Parameter **template_id**.  Code kopieren

2) Button im HTML richtig

Diesen Button packst du **IN die Card** (ganz oben in `.putzplan-card`), und nutzt `eintrag.template_id` (oder wie dein PK heißt):

jinja2

```

<form method="POST"
    action="{{ url_for('delete_cleaning_template', template_id=eintrag.template_id) }}"
    class="delete-form">
    <button type="submit" class="delete-btn"
        onclick="return confirm('Eintrag wirklich löschen?');"> ✖ </button>
</form>

```

Falls dein Primary Key anders heißt (z.B. `eintrag.id` oder `eintrag.cleaning_id`),
musst du das dort anpassen.  Code kopieren

3) (Optional aber empfohlen) Tasks automatisch mitlöschen (Cascade)

In `CleaningTemplate`:

python

```

tasks = db.relationship(
    "CleaningTask",
    back_populates="template",
    cascade="all, delete-orphan"
)

```

 Code kopieren

✓ Lösung B (nur Task löschen = Checkbox verschwindet, Eintrag bleibt)

Wenn du wirklich nur den Task löschen willst, dann muss alles auf `task_id` laufen:

python

```
@app.route("/putzplan/task/<int:task_id>/delete", methods=["POST"])
def delete_cleaning_task(task_id):
    task = CleaningTask.query.get_or_404(task_id)
    db.session.delete(task)
    db.session.commit()
    flash("Aufgabe erfolgreich gelöscht.", "success")
    return redirect(url_for("putzplan"))
```

HTML:

 Code kopieren

jinja2

```
<form method="POST"
      action="{{ url_for('delete_cleaning_task', task_id=task.task_id) }}"
      class="delete-form">
  <button type="submit" class="delete-btn"
          onclick="return confirm('Aufgabe wirklich löschen?');">🗑️</button>
</form>
```

 Code kopieren

Was du gerade gemacht hast

Du hast eine **Task-Route** gebaut, aber darin ein **Template** gelöscht und im HTML `template_id` übergeben → das passt nicht zusammen.

Wenn du mir kurz sagst, wie der Primary Key bei `CleaningTemplate` heißt (z.B. `template_id` oder `cleaning_id` ?), passe ich dir den `url_for(...)` genau korrekt an.

    ...

okay, wie könnte ich jetzt nachdem die checkbox

angeklickt wurde, den task ausgrauen bzw. wegstreichen?

Du kannst das komplett über **CSS + eine Klasse** lösen. Idee: Wenn `task.status == "completed"` → gib der Zeile/Card eine Klasse wie `is-done`. Dann macht CSS durchgestrichen + ausgegraut.

1) Template anpassen (Jinja)

Unten bei deiner Checkbox/Task-Ausgabe:

jinja2

```
{% set task = eintrag.tasks[0] if eintrag.tasks else None %}
```

```
<div class="putzplan-card {% if task and task.status == 'completed' %}is-done{% e
  <div class="putzplan-top">
    <div class="putzplan-title">
      {% if task %}
        <form method="POST"
          action="{% url_for('toggle_cleaning_task', task_id=task.task_id) %}"
          class="m-0">
          <label class="putzplan-checkbox">
            <input type="checkbox"
              name="done"
              onchange="this.form.submit()"
              {% if task.status == 'completed' %}checked{% endif %}>
            <span class="putzplan-name">{{ eintrag.name }}</span>
          </label>
        </form>
      {% else %}
        <span class="putzplan-name">{{ eintrag.name }}</span>
      {% endif %}
    </div>

    <div class="putzplan-assigned">{{ assigned_name }}</div>
  </div>

  <div class="putzplan-metadata">
    <span class="putzplan-label">Zeitraum:</span>
    <span>{{ eintrag.description }}</span>
```

```
</div>  
</div>
```

2) CSS hinzufügen

Pack das in deinen `<style>` Bereich oder in `putzplan.css` :

CSS

```
/* wenn erledigt */  
.putzplan-card.is-done {  
  opacity: 0.6; /* ausgrauen */  
}  
  
.putzplan-card.is-done .putzplan-name,  
.putzplan-card.is-done .putzplan-metadata {  
  text-decoration: line-through; /* durchstreichen */  
}  
  
/* optional: checkbox etwas "ruhiger" */  
.putzplan-card.is-done input[type="checkbox"] {  
  filter: grayscale(100%);  
}
```

✅ Ergebnis: Sobald der Task in der DB `status="completed"` ist (nach dem POST), wird beim nächsten Rendern die Card automatisch grau + durchgestrichen angezeigt.

 Code kopieren

Optional: nur Text streichen, nicht alles ausgrauen

Dann entferne `opacity: 0.6` und nutze stattdessen z.B.:

CSS

```
.putzplan-card.is-done {  
  background: #f0f0f0;  
}
```

 Code kopieren

Wenn du willst, kann ich dir das auch so machen, dass **nur der Name** durchgestrichen wird, aber Zeitraum/Zuständig normal bleiben.

    ...

<https://bootswatch.com/zephyr/> kannst du mir eine base.html aus diesem bootstrap theme erstellen

Nachgedacht für 34s >

Klar — hier ist eine base.html für Flask/Jinja mit dem Bootswatch "Zephyr" Theme (Bootstrap 5). Du bekommst:

- Zephyr CSS per CDN
- Bootstrap JS Bundle (für Dropdowns, Alerts usw.)
- Navigation + Container Layout
- Flash-Messages als Bootstrap Alerts
- Jinja-Blöcke (title , content , extra_head , scripts)

Zephyr CDN: [bootswatch@5.3.8/dist/zephyr/bootstrap.min.css](https://cdn.jsdelivr.net/npm/bootswatch@5.3.8/dist/zephyr/bootstrap.min.css) jsDelivr

html

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

  <title>{% block title %}WG Planner{% endblock %}</title>

  <!-- Bootswatch Zephyr (Bootstrap 5) -->
  <link rel="stylesheet"
        href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.8/dist/zephyr/bootstrap

  {% block extra_head %}{% endblock %}
</head>

<body>
  <!-- Top Navbar -->
  <nav class="navbar navbar-expand-lg navbar-dark bg-primary mb-4">
    <div class="container">
      <a class="navbar-brand" href="{{ url_for('dashboard') }}">WG Planner</a>
```

```

<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data
  <span class="navbar-toggler-icon"></span>
</button>

<div class="collapse navbar-collapse" id="navMain">
  <ul class="navbar-nav ms-auto gap-1">
    <li class="nav-item">
      <a class="nav-link {% if request.endpoint == 'dashboard' %}active{% e
        href="{{ url_for('dashboard') }}">Start</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {% if request.endpoint == 'putzplan' %}active{% en
        href="{{ url_for('putzplan') }}">Putzplan</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {% if request.endpoint == 'innovation_board' %}act
        href="{{ url_for('innovation_board') }}">InnovationBoard</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {% if request.endpoint == 'activity_board' %}activ
        href="{{ url_for('activity_board') }}">ActivityBoard</a>
    </li>
    <li class="nav-item">
      <a class="nav-link {% if request.endpoint == 'einkaufsplan' %}active{%
        href="{{ url_for('einkaufsplan') }}">Einkauf</a>
    </li>
  </ul>
</div>
</nav>

```

```

<main class="container mb-5">

```

```

<!-- Flash messages -->

```

```

{% with messages = get_flashed_messages(with_categories=true) %}

```

```

{% if messages %}

```

```

  <div class="mb-3">

```

```

    {% for category, message in messages %}

```

```

      {% set bs = 'info' %}

```

```

      {% if category in ['success'] %}{% set bs = 'success' %}{% endif %}

```

```

      {% if category in ['error', 'danger'] %}{% set bs = 'danger' %}{% end

```

```

      {% if category in ['warning'] %}{% set bs = 'warning' %}{% endif %}

```

```

        <div class="alert alert-{{ bs }} alert-dismissible fade show" role="a
            {{ message }}
            <button type="button" class="btn-close" data-bs-dismiss="alert" ari
        </div>
    {% endfor %}
</div>
{% endif %}
{% endwith %}

{% block content %}{% endblock %}
</main>

<!-- Bootstrap JS (Bundle incl. Popper) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bun

{% block scripts %}{% endblock %}
</body>
</html>

```

So benutzt du sie in deinen Seiten

Beispiel putzplan.html :

```

html

{% extends "base.html" %}

{% block title %}Putzplan{% endblock %}

{% block content %}
    <h1 class="mb-3">Putzplan</h1>
    <!-- dein Inhalt -->
{% endblock %}

```

Wenn du willst, passe ich dir als nächstes deinen Putzplan (Cards + Modal + Progressbar) **komplett auf Bootstrap/Zephyr** an, sodass du kaum noch Inline-Styles brauchst.

 Code kopieren

    ...  Quellen

 Code kopieren

Error: While importing 'Web-Development-Project.app', an ImportError was raised:

Traceback (most recent call last):

```
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\venv\Lib\site-
packages\flask\cli.py", line 245, in locate_app
  __import__(module_name)
  ~~~~~^
File
"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\app.py", line 13, in
<module>
    from db import Activity, CleaningTask,
CleaningTemplate, Idea, Idea_Comment, Idea_Like,
ShoppingItem, db, User, Wg
ModuleNotFoundError: No module named 'db'
```

nach git merge

Der Fehler heißt: **Python findet dein Modul db nicht mehr**. Nach dem Merge ist sehr wahrscheinlich **die Projektstruktur/Import-Form** kaputt gegangen (z.B. Datei verschoben, Ordner umbenannt, oder ihr habt jetzt ein Package).

Hier sind die 3 häufigsten Fälle + Fix (mach nur den passenden):

Fall A: db.py liegt nicht mehr neben app.py

Soll-Zustand (einfachstes Setup):

```
cpp
```

```
Web-Development-Project/
```

```
  app.py
```

```
  db.py
```

```
templates/
static/
```

 Code kopieren

✓ Fix:

- Prüfe im Explorer: **gibt es db.py wirklich im gleichen Ordner wie app.py ?**
- Wenn db.py z.B. in docs/ oder src/ gelandet ist, dann importiere korrekt, z.B.:

python

```
from docs.db import Activity, CleaningTask, ...
```

 Code kopieren

Fall B: Ihr habt jetzt ein Package (Ordner) und db.py liegt *drin*

Beispielstruktur nach Merge:

markdown

```
Web-Development-Project/
  app.py
  docs/
    __init__.py
    db.py
```

✓ Fix im app.py :

 Code kopieren

python

```
from docs.db import Activity, CleaningTask, CleaningTemplate, Idea, Idea_Comment,
```

◀

 Code kopieren ▶

Wichtig: docs/ muss eine Datei `__init__.py` haben, sonst ist es kein Package.

Fall C: Ihr startet Flask aus dem falschen Ordner

Wenn du `flask run` **nicht** im Projektroot ausführst, findet Python manchmal lokale Module nicht.

✓ Fix:

- Öffne Terminal in **dem Ordner, wo app.py liegt**
- Dann:


```
bash
```

```
flask --app app run
```

 Code kopieren

Oder Windows (PowerShell):

```
powershell
```

```
set FLASK_APP=app
```

```
flask run
```

 Code kopieren

Schnelltest (so findest du sofort raus, was los ist)

1. Öffne Terminal im Projektordner (wo `app.py` ist)
2. Starte Python:

```
bash
```

```
python
```

 Code kopieren

3. Teste:

```
python
```

```
import os
print(os.getcwd())
import db
print(db.__file__)
```

- Wenn `import db` hier schon knallt → Datei/Struktur stimmt nicht (Fall C)  Code kopieren
- Wenn es hier geht, aber `flask run` nicht → meist Fall C (Start aus falschem Ordner / falsches `FLASK_APP`).

Empfehlung für Git-Merge (damit das nie wieder passiert)

Wenn ihr mehrere Module habt, macht daraus ein klares Package:

```
cpp
```

```
Web-Development-Project/
```

```
  wgp/
```

```
    __init__.py
```

```
app.py
db.py
templates/
static/
```

Dann importiert ihr immer stabil:

[Code kopieren](#)

```
python
```

```
from wgp.db import db, User, ...
```

[Code kopieren](#)

und startet:

```
bash
```

```
flask --app wgp.app run
```

[Code kopieren](#)

Wenn du mir kurz sagst, wo `db.py` **jetzt liegt** (Screenshot vom Projektbaum oder nur Pfade), sag ich dir exakt den richtigen Import (1 Zeile) für deinen Fall.



<https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js>

Ja — das ist das **Bootstrap JS Bundle** (inkl. Popper). Du bindest es in deiner `base.html` ganz unten vor `</body>` ein.

So sieht's korrekt aus (mit Zephyr/Bootswatch CSS + Bootstrap JS):

```
html
```

```
<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```

<!-- Bootswatch Zephyr (Bootstrap CSS) -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.2/dist

<title>{% block title %}WG Planner{% endblock %}</title>
</head>

<body>

{% block content %}{% endblock %}

<!-- Bootstrap JS Bundle (dein Link) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"
        integrity="sha384-C6RzsynM9WDrMNeT87bh95OGNyZPhcTNXj1NW7RuBCsyN/o0jlpcV
        crossorigin="anonymous"></script>

{% block scripts %}{% endblock %}
</body>
</html>

```



Wenn du **keine integrity** willst (geht auch), dann einfach:

html

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js"></script>
```



 Code kopieren 

Wichtig:

- **JS gehört ans Ende** (damit Modals/Dropdowns sicher funktionieren).
- Wenn du in Flask **static** nutzt, kommt dein eigenes JS **nach** Bootstrap oder in `{% block scripts %}`.



```
{% extends "base.html" %}
```

```
{% block title %}Putzplan{% endblock %}
```

```
{% block content %}
```

```
<h1>Putzplan</h1>
```

```
<link rel="stylesheet" href="putzplan.css">
```

```
{% with messages =
get_flashed_messages(with_categories=true) %}
  {% if messages %}
    {% for category, message in messages %}
      <div class="alert-message {% if category == 'success'
%}alert-success{% else %}alert-error{% endif %}"
style="padding: 15px; margin-bottom: 20px; border-
radius: 5px;">
        {{ message }}
      </div>
    {% endfor %}
  {% endif %}
{% endwith %}
```

```
<nav>
  <a href="{{ url_for('dashboard') }}">Start</a>
  <a href="{{ url_for('putzplan') }}">Putzplan</a>
  <a href="{{ url_for('innovation_board')
}}">InnovationBoard</a>
  <a href="{{ url_for('activity_board') }}">ActivityBoard</a>
  <a href="{{ url_for('einkaufsplan') }}">Einkauf</a>
</nav>
```

```
{% set remaining = 100 - progress %}
```

```
<div class="mt-3 mb-4">
  <div class="d-flex justify-content-between align-items-
center mb-2">
    <strong>Fortschritt</strong>
    <span class="text-muted">{{ completed_tasks }}/{{
total_tasks }} erledigt ({{ progress }}%)</span>
  </div>
```

```
<div class="progress-stacked" style="height: 18px;">
  <div class="progress" role="progressbar"
    aria-label="Erledigt"
    aria-valuenow="{{ progress|int }}" aria-valuemin="0"
    aria-valuemax="100"
```

```

        style="width: {{ progress|int }}%;">
    <div class="progress-bar bg-success"> </div>
</div>

<div class="progress" role="progressbar"
    aria-label="Offen"
    aria-valuenow="{{ remaining|int }}" aria-valuemin="0"
    aria-valuemax="100"
    style="width: {{ remaining|int }}%;">
    <div class="progress-bar bg-secondary"> </div>
</div>
</div>
</div>

<input type="checkbox" id="modalToggle" style="display:
none;">

<label for="modalToggle" style="cursor: pointer; padding:
10px 20px; background-color: #007bff; color: white;
border-radius: 5px; display: inline-block;">
    + Neuen Eintrag erstellen
</label>

<div style="position: fixed; top: 0; left: 0; width: 100%;
height: 100%; background-color: rgba(0, 0, 0, 0.5); display:
none; z-index: 1;" id="modalBackdrop"> </div>

<div style="position: fixed; top: 50%; left: 50%; transform:
translate(-50%, -50%); background-color: white; padding:
30px; border-radius: 8px; width: 400px; z-index: 2; display:
none;" id="modalContent">
    <label for="modalToggle" style="float: right; cursor:
pointer; font-size: 24px; font-weight: bold;"> × </label>
    <h2>Neuen Eintrag erstellen</h2>
    <h3>Bestimme, wer für was zuständig ist</h3>
    <form method="POST">
        {{ form.hidden_tag() }}

        <div style="margin-bottom: 15px;">
            <label for="woche">Kalenderwoche:</label>
            {{ form.woche(style="display: block; margin-top:

```

```
5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="von_datum">Von:</label>
  {{ form.von_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="bis_datum">Bis:</label>
  {{ form.bis_datum(style="display: block; margin-
top: 5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="aufgabe">Aufgabe:</label>
  {{ form.aufgabe(style="display: block; margin-top:
5px; padding: 8px; width: 100%;") }}
</div>

<div style="margin-bottom: 15px;">
  <label for="zustaendig">Zuständig:</label>

  {{ form.zustaendig(
    list="users",
    id="zustaendig",
    style="display:block; margin-top:5px; padding:8px;
width:100%;",
    placeholder="z.B. Max"
  ) }}

  <datalist id="users">
    {% for user in all_users %}
    <option value="{{ user.username }}"> </option>
    {% endfor %}
  </datalist>

  {% if form.zustaendig.errors %}
    <span style="color:red;">{{
form.zustaendig.errors[0] }}</span>
  {% endif %}
</div>
```

```

        {{ form.submit() }}
    </form>
</div>

<style>
    #modalToggle:checked ~ #modalBackdrop {
        display: block !important;
    }

    #modalToggle:checked ~ #modalContent {
        display: block !important;
    }
</style>

<div class="putzplan-list">
    {%for eintrag in putzplan%}
        {%set task = eintrag.tasks[0] if eintrag.tasks else None %}
        {%set assigned_name =
(eintrag.tasks[0].assigned_user.username if eintrag.tasks
and eintrag.tasks[0].assigned_user else "-")%}

        <div class="putzplan-card">
            {%if task%}
                <form method = "POST" action="{{
url_for('delete_cleaning_task',
template_id=eintrag.template_id) }}" class="delete-form">
                    <button type="submit" class="delete-btn"
onclick="return confirm('Aufgabe wirklich löschen?');"> 
                </button>
            </form>
            {%endif%}
            <div class =putzplan-top>
                <div class ="putzplan-title">
                    {%if task%}

                        <form method ="POST" action="
{{url_for('toggle_cleaning_task',task_id=task.task_id)}}" class
= "m-0">
                            <label class="putzplan-checkbox">
                                <input type="checkbox" name ="done"
onchange="this.form.submit()"
                                {%if task.status == "completed" %}checked{%

```

```
endif %}>
    <span class ="putzplan-name">{{eintrag.name}}
</span>
    </label>
</form>
{%else%}
    <span class="putzplan-name">{{eintrag.name}}
</span>
    {%endif%}
</div>
    <div class = "putzplan-assigned">{{assigned_name}}
</div>
</div>
<div class="putzplan-metadata">
    <span class ="putzplan-label">Zeitraum:</span>
    <span>{{eintrag.description}}</span>
</div>
</div>
{%else%}
    <p>Noch keine Einträge im Putzplan. Sei die/der Erste!
</p>
    {%endfor%}
</div>

<p>Einträge geladen: {{ putzplan|length }}</p>
```

```
<style>

.putzplan-list{
    display:flex;
    flex-direction:column;
    gap: 14px;
    margin-top: 18px;
    align-items:flex-start;
}

.putzplan-card{
    position: relative;
    width: 100%;
    max-width: 700px;
    border: 1px solid #000
```



```
border-radius 12px;
padding: 14px 16px;
background: #f9f9f9;
box-shadow: 2px 2px 2px;
box-sizing: border-box;
border-radius: 10px;
}

.putzplan-top{
display:flex;
justify-content: space-between;
gap:12px;
align-items: flex-start;
}

.putzplan-title{
min-width: 0;
flex:1;
}

.putzplan-checkbox{

display:flex;
align-items: center;
gap:10px;
margin:0;
cursor:pointer;
}

.putzplan-name{
font-size: 1.05rem;
font-weight:700;
overflow-wrap: anywhere;
word-break: break-word;
}

.putzplan-assigned{
flex: 0 0 auto;
padding: 6px 10px;
border-radius: 10px;
background: #e0e0e0;
border: 1px solid #000;
font-style:oblique;
```

```
white-space: nowrap;
}

.putzplan-metadata{
  margin-top:10px;
  color:#444;
  overflow-wrap: anywhere;
}

.putzplan-label{
  font-weight:bold;
  margin-right:6px;
}

.delete-btn{
  position:absolute;
  bottom: 10px;
  right: 10px;
  width: 34px;
  height: 34px;
  border-radius: 10px;
  border: 1px solid #fecaca;
  background: #fff;
  cursor: pointer;
  display:flex;
  align-items:center;
  justify-content:center;
  transition: 0.15s ease;
}

.delete-btn:hover{
  background: #fef2f2;
  border-color: #fca5a5;
  transform: translateY(-1px);
}

@media (max-width: 700px){
  .item-forms{
    grid-template-columns: 1fr;
  }
}

/* wenn erledigt */
.putzplan-card.is-done {
```

```

opacity: 0.6;          /* ausgrauen */
}

.putzplan-card.is-done .putzplan-name,
.putzplan-card.is-done .putzplan-metadata {
  text-decoration: line-through; /* durchstreichen */
}

.progress-stacked {
  height: 18px !important;
  border-radius: 10px !important;
  overflow: hidden !important;
}
.progress {
  height: 18px !important;
  background-color: #0080ff !important;
}
.progress-bar {
  transition: width .6s ease !important;
}
</style>
{%endblock %}

```

kannst du das auf die cdn anpassen, die ich dir gerade gegeben habe?

Klar — du musst dafür **nichts in deinem putzplan.html ändern**, außer: **base.html muss die Bootstrap-CDNs (CSS + dein JS-Link) laden** und du solltest das lokale CSS korrekt über `url_for('static', ...)` einbinden.

1) base.html anpassen (Zephyr + dein JS-Link)

Pack das in deine `base.html` :

html

```

<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">

```

```

<!-- Bootswatch Zephyr (Bootstrap CSS) -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.2/dist

<title>{% block title %}{% endblock %}</title>

{% block head %}{% endblock %}
</head>
<body>

<div class="container py-4">
  {% block content %}{% endblock %}
</div>

<!-- Bootstrap JS Bundle (genau dein Link) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle

{% block scripts %}{% endblock %}
</body>
</html>

```

2) In putzplan.html CSS-Link richtig machen

Du hast aktuell:

```

html

<link rel="stylesheet" href="putzplan.css">

```

 Code kopieren

Das funktioniert in Flask meistens **nicht**, außer die Datei liegt genau im richtigen Pfad.

Korrekt wäre (wenn putzplan.css in static/putzplan.css liegt):

```

html

{% block head %}
<link rel="stylesheet" href="{{ url_for('static', filename='putzplan.css') }}">
{% endblock %}

```

 Code kopieren

 Also **oben** in deinem putzplan.html ergänzen/ersetzen:

```

jinja2

```

```
{% extends "base.html" %}

{% block title %}Putzplan{% endblock %}

{% block head %}
<link rel="stylesheet" href="{% url_for('static', filename='putzplan.css') %}">
{% endblock %}

{% block content %}
...
{% endblock %}
```

 Code kopieren

3) Bootstrap-Klassen nutzen (optional)

Deine Seite nutzt schon `d-flex`, `mt-3`, `mb-4`, `text-muted` etc. — das funktioniert dann automatisch, sobald `base.html` die CDNs lädt.

Wenn du willst, kann ich dir auch kurz zeigen, wie du dein Modal **von “Checkbox-Trick” auf Bootstrap-Modal** umstellst (dann brauchst du kein `#modalToggle` mehr) — das wäre dann genau der Sinn vom Bootstrap JS Bundle.



ohne zephyr

Alles klar — dann nimm **nur Bootstrap 5.3.2** (ohne Bootswatch/Zephyr).

base.html (nur Bootstrap CDN)

```
html

<!doctype html>
<html lang="de">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
```

```

<!-- Bootstrap CSS (ohne Zephyr) -->
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/

<title>{% block title %}{% endblock %}</title>

{% block head %}{% endblock %}
</head>
<body>

<div class="container py-4">
  {% block content %}{% endblock %}
</div>

<!-- Bootstrap JS Bundle (dein Link) -->
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bun

{% block scripts %}{% endblock %}
</body>
</html>

```

putzplan.html: CSS-Link korrekt einbinden

Ersetze dein

html

```
<link rel="stylesheet" href="putzplan.css">
```

 Code kopieren

durch:

jinja2

```

{% block head %}
<link rel="stylesheet" href="{% url_for('static', filename='putzplan.css') %}">
{% endblock %}

```

 Code kopieren

Und stelle sicher, dass `putzplan.css` in `/static/putzplan.css` liegt.

Damit läuft alles mit Bootstrap, aber ohne Zephyr.



wie passe ich mein css an das design an?

Wenn du jetzt Bootstrap nutzt, solltest du dein CSS so schreiben, dass es **Bootstrap ergänzt** (Spacing, Cards, Buttons) statt alles komplett "inline" zu machen.

Hier ist ein sauberes Vorgehen + fertige CSS-Beispiele, die zu Bootstrap 5.3 passen (modern, "Figma-like").

1) Nutze Bootstrap-Klassen im HTML (weniger eigenes CSS)

Beispiel: Wrapper + Liste

Statt eigener Container/Abstände:

html

```
<div class="container py-4">
  <div class="d-flex align-items-center justify-content-between mb-3">
    <h1 class="m-0">Putzplan</h1>
    <label for="modalToggle" class="btn btn-primary">+ Neuen Eintrag</label>
  </div>

  <!-- Progress -->
  <div class="mb-4">
    ...
  </div>

  <!-- Cards -->
  <div class="putzplan-list">
    ...
  </div>
</div>
```

Dadurch "passt" es automatisch zu Bootstrap (Abstände, Typografie).

 Code kopieren

2) Dein CSS sollte Bootstrap-Variablen nutzen (damit es "wie Bootstrap" aussieht)

Lege in `static/putzplan.css` sowas an:

CSS

```
/* Basis: alles fühlt sich wie Bootstrap an */
.putzplan-list{
  display: flex;
  flex-direction: column;
  gap: 12px;
}

/* Card-Look wie Bootstrap "card", nur etwas moderner */
.putzplan-card{
  position: relative;
  border: 1px solid var(--bs-border-color);
  border-radius: 12px;
  background: var(--bs-body-bg);
  padding: 14px 16px;
  box-shadow: 0 .25rem .75rem rgba(0,0,0,.06);
}

/* Header-Zeile */
.putzplan-top{
  display: flex;
  justify-content: space-between;
  gap: 12px;
  align-items: flex-start;
}

.putzplan-title{ flex: 1; min-width: 0; }

.putzplan-checkbox{
  display: flex;
  gap: .6rem;
  align-items: center;
  margin: 0;
  cursor: pointer;
}

.putzplan-name{
  font-weight: 700;
  font-size: 1.05rem;
  overflow-wrap: anywhere;
}
```



```
}

/* "Badge" für Zuständig */
.putzplan-assigned{
  white-space: nowrap;
  padding: .35rem .6rem;
  border-radius: 999px;
  border: 1px solid var(--bs-border-color);
  background: var(--bs-tertiary-bg);
  color: var(--bs-body-color);
  font-size: .9rem;
}

/* Zeitraum */
.putzplan-metadata{
  margin-top: .5rem;
  color: var(--bs-secondary-color);
  overflow-wrap: anywhere;
}

.putzplan-label{
  font-weight: 600;
  color: var(--bs-body-color);
  margin-right: .35rem;
}

/* Delete Button oben rechts */
.delete-btn{
  position: absolute;
  top: 10px;
  right: 10px;

  width: 34px;
  height: 34px;
  border-radius: 10px;

  border: 1px solid var(--bs-border-color);
  background: var(--bs-body-bg);
  display: flex;
  align-items: center;
  justify-content: center;

  cursor: pointer;
}
```

```

    transition: transform .12s ease, background .12s ease, border-color .12s ease;
  }
  .delete-btn:hover{
    background: var(--bs-danger-bg-subtle);
    border-color: var(--bs-danger-border-subtle);
    transform: translateY(-1px);
  }

  /* Completed Style */
  .putzplan-card.is-done{
    opacity: .65;
  }
  .putzplan-card.is-done .putzplan-name,
  .putzplan-card.is-done .putzplan-metadata{
    text-decoration: line-through;
  }

  /* Progress-Bar hübscher */
  .progress-stacked{
    height: 18px;
    border-radius: 999px;
    overflow: hidden;
  }

```

Das nutzt **Bootstrap CSS Variablen** (--bs-...) → dein Design wirkt automatisch "aus einem Guss".

3) Wichtig: Entferne Inline-Styles, sonst "gewinnen" die gegen dein CSS

Wenn du sowas hast:

```

html

<label style="background-color:#007bff; ...">

```

 Code kopieren

dann wird dein eigenes CSS oft ignoriert. Besser:

```

html

<label class="btn btn-primary" for="modalToggle">+ Neuen Eintrag</label>

```

 Code kopieren

4) "Wie passe ich das an das Design an?" — Checkliste

- **Buttons:** `btn btn-primary` / `btn-outline-danger`
- **Spacing:** `mb-3 mt-2 py-4` statt `style="margin-bottom:20px"`
- **Cards:** `border` , `box-shadow` , `border-radius: 12px`
- **Textfarben:** `var(--bs-secondary-color)` statt harte `#444`
- **Badges/Pills:** `border-radius:999px` + `--bs-tertiary-bg`

 Code kopieren

Wenn du willst, schick mir einmal **deinen aktuellen putzplan.html Teil (nur Cards + Progress + Button)** und ich schreibe ihn dir 1:1 um, komplett Bootstrap-konform (ohne Inline-Styles), damit alles gleich aussieht.



kann ich die defaults von zephyr benutzen? muss man dafür eigenen css code schreiben? beispielsweise will ich die checkbox von zephyr in putzplan, wie mache ich das?

Ja – du kannst die **Defaults von Zephyr** benutzen, und dafür musst du **nicht** zwingend eigenes CSS schreiben.

Wichtig ist nur:

1. Du musst **Zephyr-CSS laden** (statt dem normalen Bootstrap-CSS).
 2. Du musst im HTML die **Bootstrap-Form-Klassen** benutzen (z. B. `form-check` , `form-check-input` , `form-check-label`).
- Dann bekommt deine Checkbox automatisch das Zephyr-Design.

1) Zephyr einbinden (ohne eigenes CSS)

In deiner `base.html` im `<head>` :

```
html
```

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css" integrity="sha384-T3C6C93Rf+kXlT6G6J9Li8Jf74StEAfY6y8W5aMbQpmb:104857670116" crossorigin="anonymous">
```

 Code kopieren

Und am Ende vor `</body>` (dein JS-Link ist ok):

```
html
```

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js" integrity="sha384-YvpcrYfjT4pbEPz5GpxAFwYkZlRBytkv02/76jT4pPz6UkQowpN776M40SbBY" crossorigin="anonymous">
```

 Code kopieren

Wenn du Zephyr nutzt, brauchst du **kein zusätzliches Bootstrap-CSS** mehr. Zephyr ist Bootstrap + Theme.

2) Zephyr-Checkbox im Putzplan verwenden

Dein aktueller Checkbox-Block ist fast richtig, aber du brauchst die Bootstrap-Struktur:

```
html
```

```
<form method="POST" action="{% url_for('toggle_cleaning_task', task_id=task.task_id) %}">
  <div class="form-check">
    <input class="form-check-input"
      type="checkbox"
      id="task{{ task.task_id }}"
      name="done"
      onchange="this.form.submit()"
      {% if task.status == 'completed' %}checked{% endif %}>
    <label class="form-check-label" for="task{{ task.task_id }}">
      {{ eintrag.name }}
    </label>
  </div>
</form>
```

✅ Damit wird die Checkbox automatisch im Zephyr-Stil gerendert.

 Code kopieren

3) Wenn du "Toggle/Switch" willst (falls Zephyr das hübscher macht)

Bootstrap hat Switches direkt eingebaut:

```
html
```

```
<div class="form-check form-switch">
  <input class="form-check-input" type="checkbox" id="task{{ task.task_id }}"
    onchange="this.form.submit()"
    {% if task.status == 'completed' %}checked{% endif %}>
  <label class="form-check-label" for="task{{ task.task_id }}">{{ eintrag.name }}
</div>
```



Code kopieren

4) Muss man eigenes CSS schreiben?

- Für die Zephyr-Checkbox: nein.
- Eigenes CSS brauchst du nur, wenn du **zusätzliche Effekte** willst (z. B. "done = ausgegraut/durchgestrichen").

Wenn du das "done" ohne JS machen willst, kannst du es serverseitig per Klasse lösen:

html

```
<div class="putzplan-card {% if task and task.status == 'completed' %}opacity-50{
```



Code kopieren

Und den Text durchstreichen:

html

```
<span class="{% if task and task.status == 'completed' %}text-decoration-line-thr
  {{ eintrag.name }}">
</span>
```



Code kopieren

Alles davon sind Bootstrap-Klassen → kein eigenes CSS nötig.

Wenn du mir sagst: **Willst du normale Checkbox oder Switch?** (und ob "done" auch durchgestrichen werden soll), schreibe ich dir den Putzplan-Card-Block komplett in sauberem Zephyr/Bootstrap-Stil um.



was ist mit meinen cards? Kannst du die cards von zephyr nehmen?

Ja – du kannst deine Putzplan-“Cards” komplett über **Zephyr/Bootstrap Cards** bauen, ohne dein eigenes `.putzplan-card` CSS.

Hier ist eine **Drop-in** Variante für deinen Listenteil (inkl. Checkbox im Zephyr-Stil, Assigned-Badge, Zeitraum, Delete oben rechts):

Ersetze deinen kompletten `<div class="putzplan-list"> ... </div>` Block durch das hier:

jinja2

```
<div class="d-flex flex-column gap-3 mt-3">

{% for eintrag in putzplan %}
    {% set task = eintrag.tasks[0] if eintrag.tasks else None %}
    {% set assigned_name = (eintrag.tasks[0].assigned_user.username if eintrag.ta
    {% set is_done = (task and task.status == 'completed') %}

    <div class="card shadow-sm {% if is_done %}opacity-75{% endif %}" style="max-w: 100%; margin-bottom: 10px;">
        <div class="card-body">

            <!-- Top row: checkbox/title + assigned + delete -->
            <div class="d-flex justify-content-between align-items-start gap-3">

                <div class="flex-grow-1">
                    {% if task %}
                        <form method="POST" action="{% url_for('toggle_cleaning_task', task_id=task.task_id) %}">
                            <div class="form-check">
                                <input class="form-check-input"
                                    type="checkbox"
                                    id="task{% task.task_id %}"
                                    name="done"
                                    onchange="this.form.submit()"
                                    {% if is_done %}checked{% endif %}>
                                <label class="form-check-label fw-semibold {% if is_done %}text-success{% endif %}"
                                    for="task{% task.task_id %}">
                                    {{ task.task_name }}
```

```

        {{ eintrag.name }}
    </label>
</div>
</form>
{% else %}
    <div class="fw-semibold">{{ eintrag.name }}</div>
{% endif %}

<div class="mt-2 small text-body-secondary {% if is_done %}text-decoration: underline;">
    <span class="fw-semibold">Zeitraum:</span> {{ eintrag.description }}
</div>
</div>

<div class="d-flex flex-column align-items-end gap-2">
    <span class="badge text-bg-secondary">{{ assigned_name }}</span>

    {% if task %}
        <form method="POST" action="{{ url_for('delete_cleaning_task', temp_id=task.id) }}">
            <button type="submit"
                class="btn btn-sm btn-outline-danger"
                onclick="return confirm('Aufgabe wirklich löschen?');">
                
            </button>
        </form>
    {% endif %}
</div>

</div>
</div>
</div>

{% else %}
    <div class="alert alert-info" style="max-width: 720px;">
        Noch keine Einträge im Putzplan. Sei die/der Erste!
    </div>
{% endfor %}

</div>

```

Was du dafür brauchst (in `base.html`)

Damit Zephyr/Bootstrap Card-Design greift:

html

```
<link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/bootswatch@5.3.2/dist/z
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundl
```



Code kopieren

Vorteil

- **Cards, Badges, Buttons, Checkbox** kommen direkt aus Zephyr
- Du kannst fast dein ganzes Custom-CSS weglassen (nur `max-width` habe ich inline gelassen)

Code kopieren

Wenn du willst, kann ich dir auch:

- den **“+ Neuen Eintrag erstellen”** Button als Bootstrap Button (`btn btn-primary`) machen
- das Modal von deinem Checkbox-Hack auf ein **echtes Bootstrap Modal** umstellen (dann sieht's nochmal sauberer aus)



```
<div class="progress-bar-container d-flex">
  <div class="progress" role="progressbar"
    aria-label="Erledigt"
    <div class="progress-bar progress-bar-striped
progress-bar-animated"
    role="progressbar"

    aria-valuemin="0" aria-valuemax="100"
    style="width: 75%;">
  </div>
  <div class="progress-bar bg-success"> </div>
</div>
```

Dein HTML ist kaputt, weil du ein `<div>` in ein **geöffnetes Tag** geschrieben hast:

html

```
<div class="progress" ... aria-label="Erledigt" <div class="progress-bar" ...>
```

Code kopieren

Das ist ungültig. Du brauchst bei Bootstrap **nur**:

- außen: `.progress`
- innen: `.progress-bar`

So ist es korrekt (75% Beispiel):

html

```
<div class="progress" role="progressbar" aria-label="Erledigt"
    aria-valuenow="75" aria-valuemin="0" aria-valuemax="100">
  <div class="progress-bar bg-success progress-bar-striped progress-bar-animated"
    style="width: 75%;">
    75%
  </div>
</div>
```



 Code kopieren

Wenn du **“Erledigt + Offen”** (stacked) willst

Dann so (mit 75% erledigt, 25% offen):

html

```
<div class="progress-stacked" style="height: 18px;">
  <div class="progress" role="progressbar" aria-label="Erledigt"
    aria-valuenow="75" aria-valuemin="0" aria-valuemax="100"
    style="width: 75%;">
    <div class="progress-bar bg-success progress-bar-striped progress-bar-animate
      75%
    </div>
  </div>

  <div class="progress" role="progressbar" aria-label="Offen"
    aria-valuenow="25" aria-valuemin="0" aria-valuemax="100"
    style="width: 25%;">
    <div class="progress-bar bg-secondary"></div>
  </div>
</div>
```



Wenn du mir sagst, ob du **einfach** oder **stacked** willst, passe ich es dir 1:1 in deinen Jinja-Code mit `{{ progress }}` / `{{ remaining }}` ein.

 Code kopieren



```
{% set remaining = 100 - progress %}

<div class="progress-bar-container d-flex">
  <div class="progress" role="progressbar" aria-
label="Erledigt"
    <div class="progress-bar progress-bar-striped
progress-bar-animated"
      role="progressbar" aria-valuemin="0" aria-
valuemax="100" style="width: 75%;"> </div>
    <div class="progress-bar bg-success"> </div>
  </div>

  <div class="progress" role="progressbar" aria-
label="Offen"
    aria-valuenow="{{ remaining }}" aria-valuemin="0"
aria-valuemax="100"
    style="width: {{ remaining }}%;">
    <div class="progress-bar bg-secondary"> </div>
  </div>
</div> kannst du es mir richtig schreiben
```

Klar — hier ist es **korrektes Bootstrap-HTML** (stacked), direkt mit deinem `progress` und `remaining`.

html

```
{% set remaining = 100 - progress %}

<div class="progress-stacked" style="height: 18px;">

  <!-- ERLEDIGT -->
  <div class="progress"
    role="progressbar"
    aria-label="Erledigt"
    aria-valuenow="{{ progress|int }}"
    aria-valuemin="0"
    aria-valuemax="100"
    style="width: {{ progress|int }}%;">
```

```

    <div class="progress-bar bg-success progress-bar-striped progress-bar-animate
      {{ progress|int }}%
    </div>
  </div>

  <!-- OFFEN -->
  <div class="progress"
    role="progressbar"
    aria-label="Offen"
    aria-valuenow="{{ remaining|int }}"
    aria-valuemin="0"
    aria-valuemax="100"
    style="width: {{ remaining|int }}%;">
    <div class="progress-bar bg-secondary">
      {{ remaining|int }}%
    </div>
  </div>

</div>

```

Wenn du **nur** beim "Erledigt"-Teil den Text sehen willst (und bei "Offen" nicht), sag kurz Bescheid, dann kürze ich es.



```

<div class="ideas-list">
  {% for idea in ideas %}
    <div class="" style="background-color: {{ idea.color or
      '#ffffff' }};">

      <div class="d-flex justify-content-between align-
items-center mb-2">
        <!-- Like -->
        <form method="POST" action="{{
url_for('toggle_like', idea_id=idea.idea_id) }}" class="m-0">

          <button type="submit" class="btn btn-sm btn-
outline-primary"
            style="cursor:pointer; padding:10px 20px;
background-color:#007bff; color:white; border-radius:5px;

```

```

display:inline-block;">
    👍 {{ idea.likes|length }}
  </button>
</form>
</div>
<!-- Delete -->
<div class="d-flex justify-content-end align-items-
center mb-2">
  <form method="POST" action="{{
url_for('delete_idea', idea_id=idea.idea_id) }}" class="m-0">

    <button type="submit" class="delete-btn"
      style="cursor:pointer; padding:10px 20px;
background-color:#dc3545; color:white; border-radius:5px;
display:inline-block;"
      onclick="return confirm('Idee wirklich
löschen?');">
      🗑️
    </button>
  </form>
</div>

<!-- Card Content -->
<h3 class="mb-1">{{ idea.title }}</h3>
<p class="idea-desc mb-2">{{ idea.description }}</p>
<small class="text-muted">
  Erstellt von {{ idea.creator.username }} am {{
idea.created_at.strftime('%d.%m.%Y') }}
</small>

<details class="mt-2">
  <summary style="cursor:pointer; color:#28a745; font-
weight:600;">
    💬 Kommentare ({{ idea.comments|length if
idea.comments else 0 }})
  </summary>

  <!-- bestehende Kommentare -->
  <div style="margin-top:10px;">
    {% for c in idea.comments %}
      <div style="border-top:1px solid #ddd; padding-
top:6px; margin-top:6px;">

```

```

        <small>
            <strong>{{ c.user.username }}</strong>
            · {{ c.created_at.strftime('%d.%m.%Y %H:%M') }}
        </small>
        <div class ="comment-text">{{ c.content }}</div>
    </div>
    {% else %}
        <small class="text-muted">Noch keine Kommentare.
    </small>
    {% endfor %}
</div>

<!-- Kommentar schreiben -->
<form method="POST"
    action="{{ url_for('post_comment',
idea_id=idea.idea_id) }}"
    style="margin-top:10px;">
    {{ comment_form.hidden_tag() }}

    <div style="display:flex; gap:10px; align-
items:center;">
        {{ comment_form.content(
            id="content-" ~ idea.idea_id,
            class="form-control",
            rows="3",
            style ="resize:vertical; flex-grow:1;",
            placeholder="Kommentar hinzufügen..."
        ) }}
        <button type="submit" class="btn btn-sm btn-
outline-secondary">
            Kommentieren
        </button>
    </div>
</form>

</details>

</div>

```

kannst du es mir auch für das innovationboard anpassen

Ja — ich passe dir das **InnovationBoard** genauso an (Card mit `idea.color` , Like links, Delete oben rechts, Content, Kommentare + Formular), aber in **sauberem Bootstrap-Layout**, damit nichts "auseinanderfällt".

✓ Innovationboard.html (Cards)

Wichtig: Dein Card- `div` braucht eine **Card-Klasse** + **padding**, sonst sehen Like/Delete "lose" aus und Kommentare rutschen komisch.

html

```
<div class="ideas-list d-flex flex-wrap justify-content-center gap-3">

  {% for idea in ideas %}
    <div class="card shadow-sm"
      style="width: 520px; max-width: 100%; background-color: {{ idea.color or

    <div class="card-body position-relative">

      <!-- Top row: Like links, Delete rechts -->
      <div class="d-flex justify-content-between align-items-center mb-3">

        <!-- Like -->
        <form method="POST"
          action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}"
          class="m-0">
          <button type="submit" class="btn btn-sm btn-primary">
            👍 {{ idea.likes|length }}
          </button>
        </form>

        <!-- Delete (oben rechts) -->
        <form method="POST"
          action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}"
          class="m-0">
          <button type="submit"
            class="btn btn-sm btn-danger"
            onclick="return confirm('Idee wirklich löschen?');">
            🗑️
          </button>
        </form>
      </div>
    </div>
```

```

<!-- Content -->
<h5 class="card-title mb-2">{{ idea.title }}</h5>
<p class="card-text mb-2" style="white-space: pre-wrap; overflow-wrap: any
  {{ idea.description }}
</p>

<small class="text-muted d-block mb-2">
  Erstellt von {{ idea.creator.username }} am {{ idea.created_at.strftime
</small>

<!-- Kommentare -->
<details class="mt-2">
  <summary class="fw-semibold" style="cursor:pointer; color:#198754;">
    💬 Kommentare ({{ idea.comments|length if idea.comments else 0 }})
  </summary>

  <div class="mt-2">
    {% for c in idea.comments %}
      <div class="border-top pt-2 mt-2">
        <small class="d-block">
          <strong>{{ c.user.username }}</strong>
          • {{ c.created_at.strftime('%d.%m.%Y %H:%M') }}
        </small>
        <div style="white-space: pre-wrap; overflow-wrap: anywhere;">
          {{ c.content }}
        </div>
      </div>
    {% else %}
      <small class="text-muted">Noch keine Kommentare.</small>
    {% endfor %}
  </div>

  <!-- Kommentar schreiben -->
  <form method="POST"
    action="{{ url_for('post_comment', idea_id=idea.idea_id) }}"
    class="mt-3">
    {{ comment_form.hidden_tag() }}

    <div class="d-flex gap-2 align-items-start">
      {{ comment_form.content(
        id="content-" ~ idea.idea_id,
        class="form-control",
        rows="3",

```

```

        style="resize:vertical; flex:1;",
        placeholder="Kommentar hinzufügen..."
    ) }}
    <button type="submit" class="btn btn-outline-secondary btn-sm">
        Kommentieren
    </button>
</div>
</form>

</details>

</div>
</div>
{% else %}
    <p>Noch keine Ideen eingereicht. Sei der Erste!</p>
{% endfor %}

</div>

```

✓ Minimal-CSS (optional, falls du `.ideas-list` bisher custom hattest)

Wenn du schon Bootstrap nutzt, brauchst du fast nichts mehr. Nur falls du willst:

```

html

<style>
    .ideas-list { margin-top: 20px; }
</style>

```

 Code kopieren

Wichtig (damit es bei dir wirklich funktioniert)

- Du nutzt `{{ comment_form.hidden_tag() }}` → dann muss `comment_form` im `render_template()` auch wirklich übergeben werden.
- Beispiel in Flask:

```

python

return render_template("innovationboard.html",
                        ideas=ideas,

```



```
form=form,  
comment_form=CommentForm())
```

 Code kopieren

Wenn du mir kurz sagst, wie dein `innovation_board()` aktuell aussieht (nur der return-Teil reicht), passe ich es dir 1:1 passend an.



```
<h5 class="text-primary-emphasis" style="background-  
color: grey; border: black; flex-direction: row; block-size:  
auto;" >{{ idea.title }}</h5>
```

```
<p class="card-text mb-2" style="white-space: pre-  
wrap; overflow-wrap: anywhere;">
```

```
    {{ idea.description }}
```

```
</p>
```

wie mache ich hier, dass der title in einer schönen grauen card liegt und abgerundet ist

Mach's am einfachsten mit **Bootstrap** statt viel Inline-CSS: pack den Titel in ein kleines "Badge/Card"-Element.

Variante A (empfohlen): Bootstrap "pill"-Box (schön, rund, grau)

html

```
<h5 class="mb-2">
```

```
  <span class="d-inline-block px-3 py-2 rounded-3 bg-light border">
```

```
    {{ idea.title }}
```

```
  </span>
```

```
</h5>
```

```
<p class="card-text mb-2" style="white-space: pre-wrap; overflow-wrap: anywhere;"
```

```
  {{ idea.description }}
```

```
</p>
```



Variante B: ganze Titelzeile als grauer Block über volle Breite

 Code kopieren

html

```
<h5 class="mb-2 px-3 py-2 rounded-3 bg-light border">
  {{ idea.title }}
</h5>
```

```
<p class="card-text mb-2" style="white-space: pre-wrap; overflow-wrap: anywhere;"
  {{ idea.description }}
</p>
```



Code kopieren

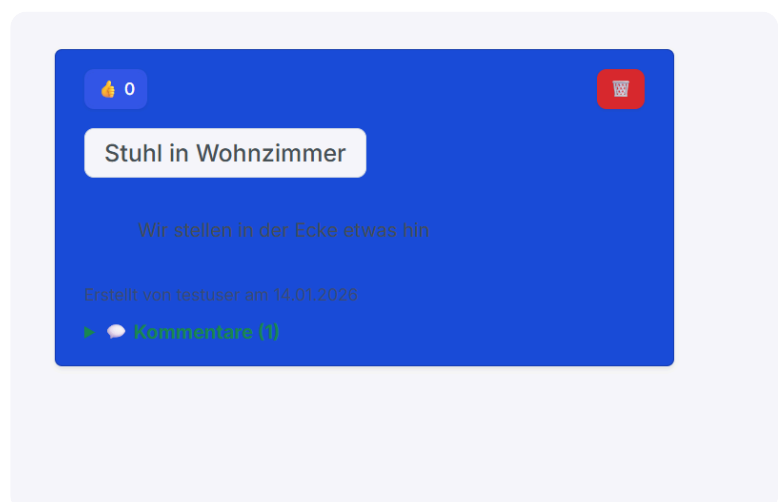
Variante C: dunkleres Grau (wenn du's wirklich "grey" willst)

html

```
<h5 class="mb-2 px-3 py-2 rounded-3 border text-white"
  style="background-color: #6c757d;">
  {{ idea.title }}
</h5>
```

Hinweis: `border: black;` allein bringt nix (Bootstrap braucht `border` + `border-dark`), also lieber `class="border border-dark"` oder wie oben `border`.

Code kopieren



das ist die card, wie könnte man mit bootstrap das design verbessern?

Du kannst die Card deutlich "Bootstrap-mäßiger" machen, indem du:

- echte card nutzt (Padding/Shadow/Roundings sauber)
- die Buttons in eine **Top-Bar** packst (rechts/links)
- Titel/Description in **Card-Body** (bessere Typo)
- auf farbigem Hintergrund **Textfarben** korrekt setzt (sonst schlechter Kontrast)
- Kommentare als **Collapse/Accordion** statt `<details>` (optional)

Hier ist ein gutes Bootstrap-Layout (ohne eigenes CSS nötig), das deine Like/Delete/Comments beibehält:

html

```
<div class="card shadow-sm border-0"
  style="background-color: {{ idea.color or '#0d6efd' }};">
  <div class="card-body text-white">

    <!-- Top row: Like left, Delete right -->
    <div class="d-flex justify-content-between align-items-start mb-3">

      <form method="POST" action="{{ url_for('toggle_like', idea_id=idea.idea_id) }}">
        <button type="submit" class="btn btn-light btn-sm rounded-pill">
          👍 <span class="ms-1">{{ idea.likes|length }}</span>
        </button>
      </form>

      <form method="POST" action="{{ url_for('delete_idea', idea_id=idea.idea_id) }}">
        <button type="submit"
          class="btn btn-danger btn-sm rounded-circle"
          style="width: 38px; height: 38px;"
          onclick="return confirm('Idee wirklich löschen?');">
          🗑️
        </button>
      </form>

    </div>

    <!-- Title -->
    <h5 class="mb-2">
      <span class="d-inline-block px-3 py-2 rounded-3 bg-white text-dark shadow-s"
        {{ idea.title }}
      </span>
    </h5>

    <!-- Description -->
```

```

<p class="mb-3 text-white-50" style="white-space: pre-wrap; overflow-wrap: any
  {{ idea.description }}
</p>

<!-- Meta -->
<small class="d-block mb-3 text-white-50">
  Erstellt von {{ idea.creator.username }} am {{ idea.created_at.strftime('%d

</small>

<!-- Comments toggle (Bootstrap collapse) -->
<a class="link-light fw-semibold text-decoration-none"
  data-bs-toggle="collapse"
  href="#comments-{{ idea.idea_id }}"
  role="button"
  aria-expanded="false"
  aria-controls="comments-{{ idea.idea_id }}">
  💬 Kommentare ({{ idea.comments|length if idea.comments else 0 }})
</a>

<div class="collapse mt-2" id="comments-{{ idea.idea_id }}">
  <div class="bg-white text-dark rounded-3 p-3">

    {% for c in idea.comments %}
      <div class="border-top pt-2 mt-2">
        <small class="text-muted">
          <strong>{{ c.user.username }}</strong> · {{ c.created_at.strftime('%
        </small>
        <div style="overflow-wrap: anywhere;">{{ c.content }}</div>
      </div>
    {% else %}
      <small class="text-muted">Noch keine Kommentare.</small>
    {% endfor %}

    <form method="POST" action="{{ url_for('post_comment', idea_id=idea.idea_
      {{ comment_form.hidden_tag() }}
      <div class="input-group">
        {{ comment_form.content(class="form-control", placeholder="Kommentar
        <button class="btn btn-outline-secondary" type="submit">Kommentieren<
      </div>
    </form>

  </div>
</div>

```

```
</div>
</div>
```

Warum das besser aussieht:

- Like/Delete sind sauber oben ausgerichtet.
- Titel sitzt in einer "White chip" Box → super lesbar.
- Description nutzt `text-white-50` → angenehme Hierarchie.
- Kommentare sind in einem hellen Container → perfekt lesbar, egal welche Card-Farbe.

Wenn du willst, kann ich dir auch eine Version machen, die **nicht** inline

`style="background-color: ..."` nutzt, sondern über eine CSS-Variable/klasse (sauberer).



ich habe ja es ja so dass man in der card die farbe auswählen kann. aber die farben sind nicht so schön, das macht ja wtforms. kann man irgendwie die farben schöner machen mit bootstrap?

Ja – du kannst den **WTForms-Colorpicker** lassen (der liefert nur den HEX), aber du kannst ihn mit Bootstrap so "schöner" machen, indem du:

1. den `<input type="color">` in Bootstrap-Layout packst (Input-Group + Preview)
2. optional **vordefinierte schöne Farben** als Buttons/Swatches anbietest (die setzen den Wert im Color-Input)
3. die Card-Farbe beim Anzeigen leicht "**soften**" (z.B. Hintergrund halbtransparent), damit es nicht knallt

Wichtig: **Nur mit Bootstrap (ohne eigenes CSS/JS)** kannst du das Aussehen von `input[type=color]` nur begrenzt ändern (Browser rendert das native). Aber du kannst es *drumherum* richtig gut aussehen lassen.

Variante A (ohne JS): schöne Preset-Farben + normaler Colorpicker

Du gibst dem Nutzer **schöne Presets** (Bootstrap Buttons) + darunter "eigene Farbe" via Colorpicker.

WTForms (Form)

python

```
from wtforms import StringField
from wtforms.validators import Optional

color = StringField("Farbe", validators=[Optional()], default="#0d6efd")
```

 Code kopieren

HTML im Modal/Form

html

```
<div class="mb-3">
  <label class="form-label">Farbe wählen</label>

  <!-- Presets -->
  <div class="d-flex flex-wrap gap-2 mb-2">
    {% for c in ["#0d6efd", "#6610f2", "#6f42c1", "#d63384", "#dc3545", "#fd7e14", "#ff
    <label class="btn btn-outline-secondary btn-sm rounded-pill">
      <input type="radio" class="btn-check" name="color" value="{{ c }}"
        {% if form.color.data == c %}checked{% endif %}>
      <span class="me-1" style="display:inline-block;width:14px;height:14px;bor
        {{ c }}
      </span>
    </label>
    {% endfor %}
  </div>

  <!-- Fallback: eigener Colorpicker -->
  <div class="input-group">
    <span class="input-group-text">Eigene</span>
    <input type="color" class="form-control form-control-color" name="color"
      value="{{ form.color.data or '#0d6efd' }}" title="Wähle eine Farbe">
    <input type="text" class="form-control" name="color_text"
      value="{{ form.color.data or '#0d6efd' }}" placeholder="#RRGGBB">
  </div>

  <div class="form-text">Tipp: Presets sehen meist besser aus als "random" Farben
</div>
```

Backend: Priorität setzen

- Wenn Radio gewählt → `request.form["color"]` ist gesetzt
- Wenn eigener Picker → auch `color` gesetzt
- `color_text` optional: kannst du ignorieren oder als Fallback nehmen

python

```
chosen_color = request.form.get("color") or request.form.get("color_text") or "#000000"
new_idea.color = chosen_color
```



Code kopieren

Variante B (mit wenig JS, sieht am besten aus): Swatches klicken setzt Colorpicker + Preview

Wenn du minimal JS erlaubst, kannst du die UX wie in Figma machen: Klick auf Swatch → setzt Input und zeigt Preview sofort.

Wenn du **kein JS** willst, nimm Variante A.

Bonus: Card-Farben “weicher” machen (damit es nicht knallt)

Wenn du z.B. knalliges Blau hast, ist Text manchmal schlecht lesbar. Mach die Card stattdessen so:

html

```
<div class="card shadow-sm border-0"
    style="background-color: color-mix(in srgb, {{ idea.color or '#0d6efd' }} 25%, white);">
```



Code kopieren

Falls `color-mix()` in deinem Browser nicht geht, nutze simpler:

- Card bleibt weiß
- Farbe nur als “Accent” (Header/Border)

Zum Beispiel:

html

```
<div class="card shadow-sm border-0">
  <div class="card-header"
    style="background-color: {{ idea.color or '#0d6efd' }};">
```

```

    <span class="badge bg-light text-dark">Idee</span>
  </div>
  <div class="card-body">
    ...
  </div>
</div>

```

Das sieht fast immer "clean" aus.

 Code kopieren

Empfehlung für dich

Für "schöne Farben" ohne Stress:

- **Preset-Farben** anbieten (Variante A)
- Card selber **weiß lassen** und Farbe nur im Header/Badge/Border nutzen
→ wirkt sofort professioneller als vollflächig knallige Hintergründe.

Wenn du willst, paste mir kurz deinen aktuellen **Modal-Teil für "Neue Idee"** (Form fields), dann bau ich dir das direkt passend in deinen Code ein.



wie mache ich statt der color codes meine namen hin

Du kannst statt `#0d6efd` usw. einfach ein **Label (Name)** anzeigen – und trotzdem den **HEX-Wert** als `value` behalten. Der Trick: Du speicherst weiterhin den HEX in der DB, aber du zeigst dem User "Blau", "Mint", "Koralle" etc.

1) Preset-Liste mit Namen + Farbe (Jinja)

Ersetze deinen Loop durch eine Liste von Tupeln `(name, hex)` :

html

```

<div class="d-flex flex-wrap gap-2 mb-2">
  {% set presets = [
    ("Blau", "#0d6efd"),
    ("Violett", "#6610f2"),
    ("Lila", "#6f42c1"),
  ] %}
  {% for preset in presets %}
    <div class="card">
      <div class="card-header">
        <span class="badge bg-{{ preset.hex }} text-dark">{{ preset.name }}</span>
      </div>
      <div class="card-body">
        ...
      </div>
    </div>
  {% endfor %}
</div>

```



```

        ("Pink", "#d63384"),
        ("Rot", "#dc3545"),
        ("Orange", "#fd7e14"),
        ("Gelb", "#ffc107"),
        ("Grün", "#198754"),
        ("Mint", "#20c997"),
        ("Cyan", "#0dcaf0"),
        ("Grau", "#6c757d"),
        ("Schwarz", "#212529")
    ] %}

{% for name, c in presets %}
    <label class="btn btn-outline-secondary btn-sm rounded-pill">
        <input type="radio" class="btn-check" name="color" value="{{ c }}"
            {% if (form.color.data or '#0d6efd') == c %}checked{% endif %}>
        <span class="me-2" style="display:inline-block;width:14px;height:14px;border
            {{ name }}>
    </label>
{% endfor %}
</div>

```

👉 Ergebnis: User sieht "Blau / Grün / Mint ...", aber gesendet wird weiterhin der HEX.

2) Optional: Den gewählten Namen später in der Card anzeigen

 Code kopieren

Wenn du nur HEX speicherst, musst du beim Rendern wieder "zurückübersetzen". ➤

Oben in deinem Template:

```

html

{% set color_names = {
    "#0d6efd": "Blau",
    "#6610f2": "Violett",
    "#6f42c1": "Lila",
    "#d63384": "Pink",
    "#dc3545": "Rot",
    "#fd7e14": "Orange",
    "#ffc107": "Gelb",
    "#198754": "Grün",
    "#20c997": "Mint",
    "#0dcaf0": "Cyan",

```

```

"#6c757d": "Grau",
"#212529": "Schwarz"
} %}

```

In der Card:

html

```

<small class="text-muted">
  Farbe: {{ color_names.get(idea.color, "Eigene Farbe") }}
</small>

```

 Code kopieren

3) Backend bleibt gleich

python

```

chosen_color = request.form.get("color") or "#0d6efd"
new_idea.color = chosen_color

```

 Code kopieren

Wenn du willst, kann ich dir auch eine Variante machen, wo du **zusätzlich den Namen in der DB speicherst** (z.B. `color_name`), dann brauchst du diese Mapping-Tabelle nicht mehr.



```

<a class="link-light fw-semibold text-decoration-none
text-black-50"

```

```

  data-bs-toggle="collapse"

```

```

  href="#comments-{{ idea.idea_id }}"

```

```

  role="button"

```

```

  aria-expanded="false"

```

```

  aria-controls="comments-{{ idea.idea_id }}">

```

```

  🗨 Kommentare ({{ idea.comments|length if
idea.comments else 0 }})

```

 wie mache ich, wenn ich kommentare collapse,
dass die anderen cards nicht betroffen sind?

Damit **nur die Kommentare dieser einen Card** auf/zugehen, brauchst du pro Card eine **eindeutige ID** (machst du schon mit `comments-{{ idea.idea_id }}`) – und du musst sicherstellen, dass du **nicht** aus Versehen `data-bs-parent` nutzt (das würde ein "Accordion" machen und andere Collapses schließen/öffnen).

So ist es korrekt (jede Card unabhängig):

html

 Code kopieren

```
<a class="link-light fw-semibold text-decoration-none text-black-50"
  data-bs-toggle="collapse"
  href="#comments-{{ idea.idea_id }}"
  role="button"
  aria-expanded="false"
  aria-controls="comments-{{ idea.idea_id }}">
  💬 Kommentare ({{ idea.comments|length if idea.comments else 0 }})
</a>

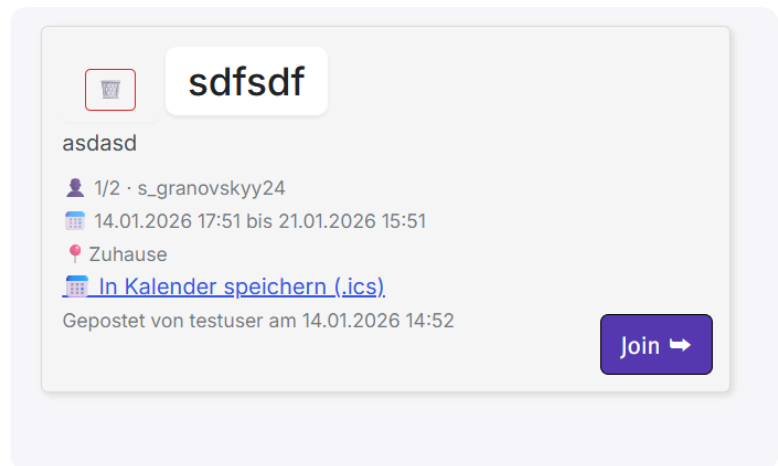
<div class="collapse mt-2" id="comments-{{ idea.idea_id }}">
  <!-- Kommentarbereich nur für diese Idee -->
  <div class="card card-body">
    ...
  </div>
</div>
```

Wichtig:

-  `id="comments-{{ idea.idea_id }}"` muss **einzigartig** sein (ist es).
-  Verwende **kein** `data-bs-parent="#irgendwas"` bei `.collapse`, sonst verhalten sich die Cards wie ein Accordion und beeinflussen sich gegenseitig.
-  Achte auch darauf, dass du nicht mehrfach dieselbe ID irgendwo kopiert hast.

Wenn bei dir trotzdem andere Cards "mitbewegen": Das ist oft nicht Bootstrap, sondern CSS (z.B. wenn du absolute Positioning oder kaputte Div-Struktur hast). Dann schick mir kurz den HTML-Block der Card (von `<div class="idea-card">` bis `</div>`) und ich sag dir genau wo der Fehler sitzt.

    ...



das ist jetzt mein activityboard. wie mache ich die card so, dass eine durchgängige linie die teilnehmer von zeit, ort, und gepostet von trennt?

```
<div class="activities-list">
  {% for activity in activities %}
    <div class="activity-card">
      <form method="POST" action="{%
url_for('delete_activity', activity_id=activity.activity_id) %}"
class="btn ">
        <button type="submit" class="btn btn-sm btn-
outline-danger"

          onclick="return confirm('Aktivität wirklich
löschen?');">  </button>
      </form>

      <h3 class="d-inline-block px-3 py-2 rounded-3 bg-
white text-dark shadow-sm">{{ activity.title }}</h3>
      <br>
      <p class="activity-desc mb-2">{{ activity.description }}
</p>
      <small class="text-muted">
        {% if activity.max_participants %}
           {{ activity.participants|length }}/{{
activity.max_participants }}
        {% else %}
           {{ activity.participants|length }}
        {% endif %}

        {% if activity.participants|length > 0 %}
```

```

        {% for u in activity.participants %}
            {{ u.username }}{% if not loop.last %}, {% endif %}
        {% endfor %}
    {% endif %}
</small>

<br>

    <small class="text-muted">📅 {{
activity.date.strftime('%d.%m.%Y %H:%M') }} bis {{
activity.updated_at.strftime('%d.%m.%Y %H:%M') }}
</small>
    {% if activity.location %}
        <br>
        <small class="text-muted">📍 {{ activity.location }}
</small>
    {% endif %}
    <br>

    <a href="{{ url_for('activity_ics',
activity_id=activity.activity_id) }}">📅 In Kalender
speichern (.ics)</a>
    <br>
    <small class="text-muted" style="margin-top: 20px;
margin-bottom: 5px;">Gepostet von {{
activity.creator.username }} am {{
activity.created_at.strftime('%d.%m.%Y %H:%M') }}
</small>
    <br>
    {% set is_joined = current_user_id and
(activity.participants
| selectattr('user_id', 'equalto', current_user_id)
| list | length > 0) %}

    <form method="POST"
        action="{% if is_joined %}{{ url_for('leave_activity',
activity_id=activity.activity_id) }}{% else %}{{
url_for('join_activity', activity_id=activity.activity_id) }}{%
endif %}"
        class="m-0 action-form">

        <button type="submit"

```

```

        class="action-btn {% if is_joined %}action-
leave{% else %}action-join{% endif %}"
        onclick="return confirm('{% if is_joined %}Von
der Aktivität wirklich abmelden?{% else %}Aktivität wirklich
beitreten?{% endif %}');">
        {% if is_joined %}Leave ✕{% else %}Join ➡{% endif
%}
    </button>
</form>

</div>

{% else %}
    <p>Noch keine Aktivitäten geteilt. Sei der/die Erste!
</p>
{% endfor %}

</div>

```

Du willst optisch einen Trenner (durchgängige Linie) zwischen dem Teilnehmer-Block und dem Rest (Zeit/Ort/Gepostet von). Das geht am saubersten mit einem Bootstrap-Divider (`<hr>`) oder einer `border-top` -Zeile.

Variante A (am einfachsten): `<hr>` direkt nach den Teilnehmern

Füge direkt nach dem Teilnehmer-`<small>` folgendes ein:

```
html
```

 Code kopieren

```

</small>

<hr class="my-3">

```

`my-3` gibt dir oben/unten angenehmen Abstand. Wenn du die Linie dezent willst:

```
html
```

 Code kopieren

```
<hr class="my-3 opacity-50">
```

Variante B (mehr "Layout-Figma"): Teilnehmer als eigener Block + Border

Wickel den Teilnehmerteil in einen Block und gib dem Block eine Bottom-Border:

html

 Code kopieren

```
<div class="pb-3 mb-3 border-bottom">
  <small class="text-muted">
    {% if activity.max_participants %}
      👤 {{ activity.participants|length }}/{{ activity.max_participants }}
    {% else %}
      👥 {{ activity.participants|length }}
    {% endif %}

    {% if activity.participants|length > 0 %}
      .
      {% for u in activity.participants %}
        {{ u.username }}{% if not loop.last %}, {% endif %}
      {% endfor %}
    {% endif %}
  </small>
</div>
```

Danach kommt wie gehabt Zeit/Ort/Gepostet.

Optional: Linie "durchgehend" über volle Card-Breite (ohne Padding)

Wenn deine Card innen `padding` hat und du willst die Linie wirklich **bis zum Rand**, dann:

html

 Code kopieren

```
<hr class="my-3 mx-n3">
```

`mx-n3` zieht die Linie über den horizontalen Padding hinaus (funktioniert, wenn dein Card-Padding ungefähr `p-3` ist). Wenn du anderes Padding nutzt: `mx-n2`, `mx-n4` etc.

Quick-Fix in deinem Code (minimaler Eingriff)

Direkt hier:

html

 Code kopieren

```

<small class="text-muted">
    ...
</small>

<hr class="my-3 opacity-50">

<small class="text-muted">📅 ...</small>

```

Das macht genau das, was du beschrieben hast.



```

@app.route("/activityboard/", methods=['GET', 'POST'])
def activity_board():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    form = ActivityForm()

    all_users = User.query.all()

    if request.method == "POST":
        print("Aktivität hinzufügen POST angekommen")
        print("form.validate_on_submit():",
              form.validate_on_submit())
        print("form.errors:", form.errors)

        if form.validate_on_submit():
            flash("Aktivität erfolgreich hinzugefügt!", "success")

            user =
            User.query.filter_by(username="testuser").first() # To be
            replaced with current_user() or session user
            new_activity = Activity(
                wg_id=1, # wg_id = user.wg_id
                created_by=user.user_id,
                title=form.title.data,
                description=form.description.data,
                date=form.date.data,
                updated_at=form.updated_at.data,

```



```

        location=form.location.data,

        max_participants=form.max_participants.data,
        created_at=db.func.now()
    )
    db.session.add(new_activity)
    db.session.commit()

    return redirect(url_for("activity_board"))
else:
    flash("Fehler beim Hinzufügen der Aktivität. Bitte
    überprüfen Sie die Eingaben.", "error")

activities = (Activity.query
               .filter_by(wg_id=1) # wg_id = user.wg_id
               .options(joinedload(Activity.creator),
                        joinedload(Activity.participants))
               .order_by(Activity.created_at.desc())
               .all())

    return render_template("activityboard.html", form=form,
                           activities=activities, all_users=all_users, current_user_id=1)
#To be replaced with current_user().user_id

@app.route("/activity/<int:activity_id>/join_activity",
           methods=["POST"])
def join_activity(activity_id):
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user_id = session.get("user_id") #Temporary set to 1 for
testing
    #if not user_id:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    user = User.query.get(user_id) #To be replaced with
user_id=user_id
    activity = Activity.query.get_or_404(activity_id)
    #existing_participant =
    Activity_Participant.query.filter_by(activity_id=activity_id,
user_id=user_id).first()

    if user in activity.participants:

```

```

flash("Du nimmst bereits teil.", "error")
return redirect(url_for("activity_board"))

if activity.max_participants and len(activity.participants)
>= activity.max_participants:
    flash("Die Aktivität ist bereits voll.", "error")
    return redirect(url_for("activity_board"))

activity.participants.append(user)
db.session.commit()
flash("Du bist beigetreten!", "success")
#if not existing_participant:

#db.session.add(Activity_Participant(activity_id=activity_id,
user_id=user_id))
    #db.session.commit()
    #flash("Erfolgreich teilgenommen!", "success")

return redirect(url_for("activity_board"))

@app.route("/activity/<int:activity_id>/leave_activity",
methods=["POST"])
def leave_activity(activity_id):
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user_id = session.get("user_id") #Temporary set to 1 for
testing
    #if not user_id:
        #flash("Bitte zuerst einloggen.", "error")
        #return redirect(url_for("login"))
    user = User.query.get(user_id) #To be replaced with
user_id=user_id
    activity = Activity.query.get_or_404(activity_id)

    if user not in activity.participants:
        flash("Du nimmst nicht teil.", "error")
        return redirect(url_for("activity_board"))

    activity.participants.remove(user)
    db.session.commit()
    flash("Du hast die Aktivität verlassen.", "success")
    #existing_participant =

```

```
Activity_Participant.query.filter_by(activity_id=activity_id,
user_id=user_id).first()
```

```
#if existing_participant:
    #db.session.delete(existing_participant)
    #db.session.commit()
    #flash("Erfolgreich ausgetreten!", "success")
```

```
return redirect(url_for("activity_board"))
```

```
<br>
```

```
{% set is_joined = current_user_id and
(activity.participants
| selectattr('user_id', 'equalto', current_user_id)
| list | length > 0) %}
```

```
<form method="POST"
    action="{% if is_joined %}{{ url_for('leave_activity',
activity_id=activity.activity_id) }}{% else %}{{
url_for('join_activity', activity_id=activity.activity_id) }}{%
endif %}"
```

```
    class="m-0 action-form">
```

```
    <button type="submit"
        class="action-btn {% if is_joined %}action-
leave{% else %}action-join{% endif %}"
        onclick="return confirm('{% if is_joined %}Von
der Aktivität wirklich abmelden?{% else %}Aktivität wirklich
beitreten?{% endif %}');">
        {% if is_joined %}Leave ✕{% else %}Join ➡{% endif
    %}
```

```
</button>
```

```
</form>
```

```
</div>
```

warum wechselt der join button nicht MEHR auf leave?

Der Button wechselt nicht, weil du im Template `current_user_id=1` **fest verdrahtet** übergibst – aber beim Join/Leave benutzt du `session["user_id"]`. Sobald der

eingeloggte User **nicht** die ID 1 hat (oder `current_user_id` nicht zur Session passt), bleibt `is_joined` im Template "falsch" und der Button bleibt auf **Join**.

Außerdem: Wenn `current_user_id` als **String** in der Session liegt (z.B. "1"), dann matcht `equalto` gegen `user_id` (int) nicht → ebenfalls immer "nicht beigetreten".

Fix (richtig): `current_user_id` aus der Session nehmen und als int übergeben

In `activity_board()`:

python

```
@app.route("/activityboard/", methods=["GET", "POST"])
def activity_board():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    form = ActivityForm()

    current_user_id = int(session["user_id"]) # <-- WICHTIG

    all_users = User.query.all()

    if request.method == "POST":
        if form.validate_on_submit():
            user = User.query.get(current_user_id) # statt testuser
            new_activity = Activity(
                wg_id=1, # später user.wg_id
                created_by=user.user_id,
                title=form.title.data,
                description=form.description.data,
                date=form.date.data,
                updated_at=form.updated_at.data,
                location=form.location.data,
                max_participants=form.max_participants.data,
                created_at=db.func.now()
            )
            db.session.add(new_activity)
            db.session.commit()
            flash("Aktivität erfolgreich hinzugefügt!", "success")
            return redirect(url_for("activity_board"))
        else:
```

```

flash("Fehler beim Hinzufügen der Aktivität.", "error")

activities = (Activity.query
              .filter_by(wg_id=1)
              .options(joinedload(Activity.creator), joinedload(Activity.part
              .order_by(Activity.created_at.desc())
              .all())

return render_template(
    "activityboard.html",
    form=form,
    activities=activities,
    all_users=all_users,
    current_user_id=current_user_id
)

```

In `join_activity()` / `leave_activity()` (auch int casten):

python

```

user_id = int(session["user_id"])
user = User.query.get_or_404(user_id)

```

 Code kopieren

 Code kopieren

Template bleibt fast gleich, nur robust machen (optional)

Dein `is_joined` ist okay. Ich würde es nur so schreiben (kürzer/sauberer):

jinja2

```

{% set is_joined = activity.participants
  | selectattr('user_id', 'equalto', current_user_id)
  | list | length > 0 %}

```

 Code kopieren

(Kein `current_user_id` and ... nötig, wenn du immer eine int übergibst.)

Mini-Debug (wenn du prüfen willst ob es daran liegt)

Temporär in die Card:

jinja2

```
<small>DEBUG current_user_id={{ current_user_id }}</small>
```

 Code kopieren

Wenn da nicht die echte Session-ID steht, ist genau das der Grund.

Wenn du willst, kann ich dir auch direkt zeigen, wie du **Join/Leave Button nur anzeigen lässt wenn noch Plätze frei sind** und bei "voll" disabled machst (Bootstrap `disabled`).



```
<form method = "POST" action="{{url_for('activity_ics',
activity_id=activity.activity_id)}}" style="position: absolute;
top: 12px; right: 50px; margin-right: 10px;">
    <button type="submit" class="btn btn-sm btn-
outline-secondary">
```



```
</button>
</form>
```

wenn der user über diesen button hovert, will ich, dass angezeigt wird, wofür der button steht in einer kleinen message bei der maus

Mach das einfach mit dem HTML-Attribut `title` (kein JS nötig). Das zeigt beim Hover so einen kleinen Tooltip neben der Maus.

html

```
<form method="POST"
    action="{{ url_for('activity_ics', activity_id=activity.activity_id) }}"
    style="position:absolute; top:12px; right:50px; margin-right:10px;">
    <button type="submit"
        class="btn btn-sm btn-outline-secondary"
        title="In Kalender speichern (.ics)"
        aria-label="In Kalender speichern (.ics)">
```



```
</button>
</form>
```

- `title` = Hover-Text
- `aria-label` = für Screenreader (und falls nur Emoji angezeigt wird)

 Code kopieren


File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 458, in activity_board

`user = User.query.filter_by(current_user_id) # To be replaced with current_user() or session user`

TypeError: Query.filter_by() takes 1 positional argument but 2 were given

`filter_by()` nimmt **nur Keyword-Argumente** (also `spalte=value`). Du hast ihm aber nur `current_user_id` als positional übergeben – daher der Fehler.

So fixst du's:

Variante A (einfach & sauber): `get()`

Wenn `current_user_id` die `user_id` ist:

```
python

current_user_id = session.get("user_id")
user = User.query.get(current_user_id)
```

 Code kopieren

Variante B: `filter_by(user_id=...)`

```
python

current_user_id = session.get("user_id")
user = User.query.filter_by(user_id=current_user_id).first()
```

 Code kopieren

Variante C: `filter(User.user_id == ...)`

```
python
```

```
current_user_id = session.get("user_id")
user = User.query.filter(User.user_id == current_user_id).first()
```

 Code kopieren

Wichtig in deinem activity_board

Du solltest **nicht** sowas machen:

```
python
```

```
user = User.query.filter_by(current_user_id)
```

 Code kopieren

sondern z.B.:

```
python
```

```
current_user_id = session.get("user_id")
user = User.query.get_or_404(current_user_id)
```

 Code kopieren

Wenn du willst, paste kurz deinen `activity_board()` Block um Zeile ~458, dann sag ich dir exakt, welche Variante bei dir am besten passt.



```
<form method = "POST" action="{{url_for('activity_ics',
activity_id=activity.activity_id)}}" style="position: absolute;
top: 12px; right: 50px; margin-right: 10px;">
```

```
    <button type="submit" class="btn btn-sm btn-
outline-primary" title="In Kalender speichern (.ics)" aria-
label="In Kalender speichern (.ics)"
        onclick="return href="">>
```



```
</button>
```

```
</form>
```

method not allowed

Method Not Allowed kommt hier fast sicher daher, dass deine Route `activity_ics` **nur GET** erlaubt (Standard), du aber im HTML ein **POST-Formular** verwendest.

Du hast außerdem ein kaputtes `onclick="return href=">` drin.

Fix (empfohlen): mach es zu einem Link (GET)

html

```
<a href="{{ url_for('activity_ics', activity_id=activity.activity_id) }}"
  class="btn btn-sm btn-outline-primary"
  style="position:absolute; top:12px; right:50px; margin-right:10px;"
  title="In Kalender speichern (.ics)"
  aria-label="In Kalender speichern (.ics)">

</a>
```

Damit wird die Route per **GET** aufgerufen → kein "method not allowed".

 Code kopieren

Alternative: Wenn du unbedingt POST willst

Dann muss deine Flask-Route POST erlauben:

python

```
@app.route("/activity/<int:activity_id>/ics", methods=["POST"])
def activity_ics(activity_id):
    ...
```

 Code kopieren

und im Template:

html

```
<form method="POST" action="{{ url_for('activity_ics', activity_id=activity.activity_id) }}"
  style="position:absolute; top:12px; right:50px; margin-right:10px;">
  <button type="submit"
    class="btn btn-sm btn-outline-primary"
    title="In Kalender speichern (.ics)"
    aria-label="In Kalender speichern (.ics)">

  </button>
</form>
```

Kurz: warum's gerade kaputt ist

- Route erwartet GET, du sendest POST ⇒ **405 Method Not Allowed**
- `onclick="return href=">` ist syntaktisch falsch ⇒ entfernen.

Wenn du mir kurz deine Flask-Route `activity_ics` zeigst (Decorator + methods), sag ich dir direkt, welche Variante bei dir passt.



```
<div class="progress-stacked" style="height: 18px;">
```

```
<!-- ERLEDIGT -->
<div class="progress"
  role="progressbar"
  aria-label="Erledigt"
  aria-valuenow="{{ progress|int }}"
  aria-valuemin="0"
  aria-valuemax="100"
  style="width: {{ progress|int }}%;">
  <div class="progress-bar bg-success progress-bar-
    striped progress-bar-animated">
    {{ progress|int }}%
  </div>
</div>
```

```
<!-- OFFEN -->
<div class="progress"
  role="progressbar"
  aria-label="Offen"
  aria-valuenow="{{ remaining|int }}"
  aria-valuemin="0"
  aria-valuemax="100"
  style="width: {{ remaining|int }}%;">
```

```
<div class="progress-bar bg-secondary">
  {{ remaining|int }}%
</div>
</div>
```

```
{% if progress == 100 %}
  <div class="position-absolute top-5 start-50 translate-
middle-x text-success fw-bold" style="padding-top: 2px;
">
    Alle Aufgaben erledigt! 🎉
  </div>
{% endif %}
```

wie mache ich, dass dieser text nicht mehr in der leiste drinne ist, sondern auf der leiste oder ein wenig drüber

Du bist schon nah dran — der Text liegt "in" der Leiste, weil der Container nicht als **Positionierungs-Referenz** gesetzt ist und weil du ihn mit `top-5` eher *in* die Leiste schiebst.

Mach den äußeren Wrapper `position: relative` und setz den Text **über** die Leiste (z.B. `top: -10px`).

So klappt's sauber:

html

```
<div class="position-relative" style="margin-top: 14px;">
  <div class="progress-stacked" style="height: 18px;">
    <!-- ERLEDIGT -->
    <div class="progress"
      role="progressbar"
      aria-label="Erledigt"
      aria-valuenow="{{ progress|int }}"
      aria-valuemin="0"
      aria-valuemax="100"
      style="width: {{ progress|int }}%;">
      <div class="progress-bar bg-success progress-bar-striped progress-bar-anima
        {{ progress|int }}%
      </div>
    </div>
  </div>

  <!-- OFFEN -->
```

```

<div class="progress"
    role="progressbar"
    aria-label="Offen"
    aria-valuenow="{{ remaining|int }}"
    aria-valuemin="0"
    aria-valuemax="100"
    style="width: {{ remaining|int }}%;">
    <div class="progress-bar bg-secondary">
        {{ remaining|int }}%
    </div>
</div>
</div>

{% if progress == 100 %}
    <div class="position-absolute start-50 translate-middle-x fw-bold text-success"
        style="top: -14px; white-space: nowrap;">
        Alle Aufgaben erledigt! 🎉
    </div>
{% endif %}
</div>

```

Optional: "auf der Leiste" statt "drüber"

Wenn du ihn *auf* der Leiste haben willst (mittig auf der Bar), nimm:

```

html

style="top: 50%; transform: translate(-50%, -50%);"

```

 Code kopieren

statt `top: -14px;` (und `translate-middle-x` dann weglassen oder anpassen).

Wenn du sagst: "**drüber**" oder "**mittig drauf**", sag ich dir die exakten Werte für deinen Screenshot (18px Höhe).

    ...

```

<form method = "POST" action="{{ url_for('einkaufsplan')
}}" >
    {{ form.hidden_tag() }}
    <div class="d-inline-block px-3 py-2 rounded-3 bg-

```

```

white text-dark shadow-sm">
    <div class="form-group mb-2 rounded-3">
        {{ form.item(size=32) }}
        {% for error in form.item.errors %}
            <span style="color: red;">{{ error }}</span>
        {% endfor %}

        {{ form.quantity() }}<br>
        {% for error in form.quantity.errors %}
            <span style="color: red;">{{ error }}</span>
        {% endfor %}
    </div>
    <div class="btn-group mt-2">
        {{ form.submit() }}
    </div>
</div>
</form> kannst du diese bitte anpassen im einkaufsplan

```

Klar — ich würde das im Einkaufsplan „bootstrap-sauber“ als **Row mit 2 Inputs + Button** machen (nebeneinander, gleiche Höhe, responsive). So kannst du deinen Block ersetzen:

html

```

<form method="POST" action="{{ url_for('einkaufsplan') }}" class="mb-4">
    {{ form.hidden_tag() }}

    <div class="card shadow-sm border-0">
        <div class="card-body">
            <div class="row g-2 align-items-start">

                <!-- Item -->
                <div class="col-12 col-md-6">
                    {{ form.item(class="form-control", placeholder="Artikel...") }}
                    {% for error in form.item.errors %}
                        <div class="text-danger small mt-1">{{ error }}</div>
                    {% endfor %}
                </div>

                <!-- Quantity -->
                <div class="col-12 col-md-3">
                    {{ form.quantity(class="form-control", placeholder="Menge...") }}

```

```

    {% for error in form.quantity.errors %}
      <div class="text-danger small mt-1">{{ error }}</div>
    {% endfor %}
  </div>

  <!-- Submit -->
  <div class="col-12 col-md-3 d-grid">
    {{ form.submit(class="btn btn-primary") }}
  </div>

</div>
</div>
</div>
</form>

```

Was das fixt/verbessert:

- Inputs & Button sind nebeneinander (ab md), auf Mobile untereinander.
- Keine `<br>` mehr nötig.
- Fehlertexte sind Bootstrappiger (small + rot).
- Der Button füllt in seiner Spalte die Breite (d-grid).

 Code kopieren

Wenn du willst, kann ich dir auch zeigen, wie du Labels hübsch drüber machst (oder komplett label-frei mit Placeholdern wie oben).



```

<div class="shopping-list">
  {% for item in shopping_items %}
    <div class="shopping-item-card">
      <form method="POST" action="{{
url_for('delete_shopping_item', item_id=item.item_id) }}"
class="delete-form">
        <button type="submit" class="delete-btn"

        onclick="return confirm('Einkaufsartikel wirklich
löschen?');"> 
      </button>
    </form>

    <h3 class="mb-1">{{ item.name }}</h3>

```

```
<p class="item-quantity mb-2">Menge: {{
item.quantity }}</p>
```

```
<small class="text-muted">
Hinzugefügt von:
{% if item.added_by_user %}
  {{ item.added_by_user.username }}
{% else %}
  {{ item.added_by }}
{% endif %}
</small>
<br>
```

```
{% if item.assigned_to_user %}
  <small class = "assigned-to">{{
item.assigned_to_user.username }}</small>
```

```
{% endif %}
```

```
</div>
{% else %}
  <p>Noch keine Einkäufe eingetragen.</p>
```

```
{% endfor %}
</div>
```

So kannst du das **Bootstrap-mäßig** (Zephyr/Bootstrap 5) als „echte Cards“ bauen – inkl. Delete oben rechts und „assigned-to“ als Badge:

html

```
<div class="row g-3">
  {% for item in shopping_items %}
    <div class="col-12 col-md-6 col-lg-4">
      <div class="card shadow-sm h-100 position-relative">

        <!-- Delete (oben rechts) -->
        <form method="POST"
          action="{{ url_for('delete_shopping_item', item_id=item.item_id) }}"
          class="position-absolute top-0 end-0 p-2">
          <button type="submit"
```

```

        class="btn btn-sm btn-outline-danger"
        onclick="return confirm('Einkaufsartikel wirklich löschen?');"
        title="Löschen">

    </button>
</form>

<div class="card-body">
    <div class="d-flex align-items-start justify-content-between gap-2">
        <h5 class="card-title mb-1">{{ item.name }}</h5>

        {% if item.assigned_to_user %}
            <span class="badge text-bg-secondary">
                {{ item.assigned_to_user.username }}
            </span>
        {% endif %}
    </div>

    <p class="card-text mb-2">
        <span class="fw-semibold">Menge:</span> {{ item.quantity }}
    </p>

    <small class="text-muted">
        Hinzugefügt von:
        {% if item.added_by_user %}
            {{ item.added_by_user.username }}
        {% else %}
            {{ item.added_by }}
        {% endif %}
    </small>
</div>

</div>
</div>
{% else %}
    <div class="col-12">
        <p class="text-muted mb-0">Noch keine Einkäufe eingetragen.</p>
    </div>
{% endfor %}
</div>

```

Wenn du willst, kann ich dir auch noch:

- eine „Assigned-to“ Badge farblich nach User (oder zufällig) gestalten,
- oder unten in der Card eine „Gekauft“-Optik (durchstreichen) reinbauen.



```
<div class="card-body">
  <div class="d-flex align-items-start justify-content-between gap-2">
    <h5 class="card-title mb-1">{{ item.name }}</h5>

    {% if item.assigned_to_user %}
      <span class="badge text-bg-secondary">
        {{ item.assigned_to_user.username }}
      </span>
    {% endif %}
  </div> wie mache ich das unten rechts in die card
rein?
```

Du willst das Badge **unten rechts** in der Card, nicht oben neben dem Titel. Nimm dafür `position-relative` am Card-Container und setz das Badge `position-absolute` unten rechts.

So:

html

 Code kopieren

```
<div class="card shadow-sm h-100 position-relative">
  <div class="card-body">
    <h5 class="card-title mb-1">{{ item.name }}</h5>
    <p class="card-text mb-2">
      <span class="fw-semibold">Menge:</span> {{ item.quantity }}
    </p>

    <small class="text-muted">
      Hinzugefügt von:
      {% if item.added_by_user %}{{ item.added_by_user.username }}{% else %}{{ it
    </small>
```

```

{% if item.assigned_to_user %}
    <span class="badge text-bg-secondary position-absolute bottom-0 end-0 m-2">
        {{ item.assigned_to_user.username }}
    </span>
{% endif %}
</div>
</div>

```

Optional (damit das Badge nicht über Text „drüber liegt“): gib der Card etwas mehr Platz unten:

html

 Code kopieren

```
<div class="card-body pb-5">
```

(oder `style="padding-bottom:3rem"`).



```

<div class="row g-3">
    {% for item in shopping_items %}
        <div class="col-12 col-md-6 col-lg-4">
            <div class="card shadow-sm h-100 position-relative">

                <!-- Delete (oben rechts) -->
                <form method="POST"
                    action="{{ url_for('delete_shopping_item',
item_id=item.item_id) }}"
                    class="position-absolute top-0 end-0 p-2">
                    <button type="submit"
                        class="btn btn-sm btn-outline-danger"
                        onclick="return confirm('Einkaufsartikel wirklich
löschen?');"
                        title="Löschen">
                        🗑️
                    </button>
                </form>

```

```
<div class="card-body">
```

```

<h5 class="card-title mb-1">{{ item.name }}</h5>
<p class="card-text mb-2">
  <span class="fw-semibold">Menge:</span> {{
item.quantity }}
</p>

<small class="text-muted">
  Hinzugefügt von:
  {% if item.added_by_user %}{{
item.added_by_user.username }}{% else %}{{
item.added_by }}{% endif %}
</small>

  {% if item.assigned_to_user %}
    <span class="badge text-bg-secondary position-
absolute bottom-0 end-0 m-2">
      {{ item.assigned_to_user.username }}
    </span>
  {% endif %}
</div>
</div>

<p class="card-text mb-2">
  <span class="fw-semibold">Menge:</span> {{
item.quantity }}
</p>

<small class="text-muted">
  Hinzugefügt von:
  {% if item.added_by_user %}
    {{ item.added_by_user.username }}
  {% else %}
    {{ item.added_by }}
  {% endif %}
</small>
</div>

</div>
</div>
{% else %}
<div class="col-12">
  <p class="text-muted mb-0">Noch keine Einkäufe

```

```

eingetragen.</p>
</div>
{% endfor %}
</div>

```

kannst du mir die einrückung korrekt machen etc

Klar — bei dir ist unten **doppelt** Menge/Hinzugefügt von drin und ein paar `</div>` sind zu viel/verschoben. Hier ist die **sauber eingerückte, korrigierte Version** (ohne Duplikate, Tags passen):

html

 Code kopieren

```

<div class="row g-3">
  {% for item in shopping_items %}
    <div class="col-12 col-md-6 col-lg-4">
      <div class="card shadow-sm h-100 position-relative">

        <!-- Delete (oben rechts) -->
        <form method="POST"
          action="{{ url_for('delete_shopping_item', item_id=item.item_id) }}"
          class="position-absolute top-0 end-0 p-2">
          <button type="submit"
            class="btn btn-sm btn-outline-danger"
            onclick="return confirm('Einkaufsartikel wirklich löschen?');"
            title="Löschen">
            
          </button>
        </form>

        <div class="card-body">
          <h5 class="card-title mb-1">{{ item.name }}</h5>

          <p class="card-text mb-2">
            <span class="fw-semibold">Menge:</span> {{ item.quantity }}
          </p>

          <small class="text-muted d-block">
            Hinzugefügt von:
            {% if item.added_by_user %}
              {{ item.added_by_user.username }}
            {% else %}
              {{ item.added_by }}
            {% endif %}
          </small>
        </div>
      </div>
    </div>
  {% endfor %}
</div>

```

```

        {% endif %}
    </small>

    {% if item.assigned_to_user %}
        <span class="badge text-bg-secondary position-absolute bottom-0 end-0"
            {{ item.assigned_to_user.username }}
        </span>
    {% endif %}
</div>

</div>
</div>
{% else %}
    <div class="col-12">
        <p class="text-muted mb-0">Noch keine Einkäufe eingetragen.</p>
    </div>
{% endfor %}
</div>

```

Wenn du willst, kann ich dir auch noch kurz zeigen, wie du dem Badge unten rechts **Platz gibst**, damit es nie über Text liegt (z.B. `pb-4` / `pb-5`).



wie mache ich diese namen farbig und in einem kreis

Du meinst das `assigned_to_user.username` Badge unten rechts? Mach's wie ein „Avatar“: **rund + farbig**.

Variante A (einfach): Kreis mit Initialen + feste Bootstrap-Farbe

Ersetze deinen Badge durch das hier:

```

html

{% if item.assigned_to_user %}
    <span class="position-absolute bottom-0 end-0 m-2
        rounded-circle d-inline-flex align-items-center justify-content-center
        text-white bg-primary fw-semibold"

```

```

        style="width: 36px; height: 36px;"
        title="{{ item.assigned_to_user.username }}">
        {{ item.assigned_to_user.username[:2]|upper }}
    </span>
{% endif %}

```



- zeigt 2 Buchstaben (z.B. "MA")
- Tooltip zeigt den ganzen Namen beim Hover

Code kopieren



Variante B: Kreis + Name daneben (schön & lesbar)

html

```

{% if item.assigned_to_user %}
<div class="position-absolute bottom-0 end-0 m-2 d-flex align-items-center gap-
    <span class="rounded-circle d-inline-flex align-items-center justify-content-
        text-white bg-primary fw-semibold"
        style="width: 32px; height: 32px;"
        {{ item.assigned_to_user.username[:1]|upper }}
    </span>
    <span class="small fw-semibold text-primary">
        {{ item.assigned_to_user.username }}
    </span>
</div>
{% endif %}

```



Code kopieren



Bonus: Farbe automatisch pro User (ohne DB, nur Template)

html

```

{% set colors = ['bg-primary', 'bg-success', 'bg-danger', 'bg-warning', 'bg-info', 'bg-
{% set c = colors[(item.assigned_to_user.user_id or 0) % (colors|length)] %}

<div class="position-absolute bottom-0 end-0 m-2">
    <span class="rounded-circle d-inline-flex align-items-center justify-content-ce
        text-white fw-semibold {{ c }}"
        style="width: 36px; height: 36px;"
        title="{{ item.assigned_to_user.username }}">
        {{ item.assigned_to_user.username[:1]|upper }}

```

```
</span>  
</div>
```

Wenn du mir sagst: **nur Kreis** oder **Kreis + Name**, passe ich dir das direkt  Code kopieren
kompletten Block an.



```
@app.route("/putzplan/", methods=["GET", "POST"])  
def putzplan():  
  
    user_id = int(session["user_id"])  
    if 'user_id' not in session:  
        return redirect(url_for('login'))  
  
    form = PutzplanForm()  
  
    #user_id = session.get("user_id")  
    #if not user_id:  
    #    flash("Bitte zuerst einloggen.", "error")  
    #    return redirect(url_for("login"))  
  
    #user = User.query.get(user_id)  
    #user = current_user()  
    #if not user:  
    #    flash("Bitte zuerst einloggen.", "error")  
    #    return redirect(url_for("login"))  
    zustaendig_name = request.form.get("zustaendig",  
    "").strip()  
    user =  
    User.query.filter_by(username=zustaendig_name).first()  
  
    if request.method == "POST":  
        print("POST angekommen")  
        print("form.validate_on_submit():",  
        form.validate_on_submit())  
        print("form.errors:", form.errors)  
        print("zustaendig raw:",  
        request.form.get("zustaendig"))
```

```
all_users = User.query.all()

if form.validate_on_submit():

    if not user:
        flash("WG-Mitglied existiert nicht", "error")
    else:
        new_template = CleaningTemplate(
            wg_id=user.wg_id,
            name=form.aufgabe.data,
            description=f"KW {form.woche.data}:
{form.von_datum.data} bis {form.bis_datum.data}",
            frequency="weekly",
            is_active=True
        )
        db.session.add(new_template)
        db.session.flush() # template_id ist jetzt da

        new_task = CleaningTask(
            template_id=new_template.template_id,
            assigned_to=user.user_id,
            status="open",
            notes="Zuständig"
        )
        db.session.add(new_task)

        db.session.commit()
        flash("Eintrag erfolgreich erstellt!", "success")
        return redirect(url_for("putzplan"))

putzplan_eintraege = (CleaningTemplate.query
                        .filter_by(wg_id=user.wg_id, is_active=True)

.order_by(CleaningTemplate.template_id.desc())
                        .all())

tasks = (CleaningTask.query
        .join(CleaningTemplate)
        .filter(CleaningTemplate.wg_id == user.wg_id)
        .all())
```



```

        total_tasks = len(tasks)
        completed_tasks = sum(1 for t in tasks if t.status ==
"completed")
        progress = int((completed_tasks / total_tasks) * 100) if
total_tasks else 0

```

```

        return render_template("putzplan.html", form=form,
all_users=all_users, putzplan=putzplan_eintraege,
                        total_tasks=total_tasks,
completed_tasks=completed_tasks, progress=progress)

```

File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgr
am\Web-Development-Project\app.py", line 360, in
putzplan

```

        .filter_by(wg_id=user.wg_id, is_active=True)

```

^^^^^^^^^^

AttributeError: 'NoneType' object has no attribute 'wg_id'

Der Fehler kommt daher, dass `user` **bei dir manchmal** `None` **ist** – und du nutzt später `user.wg_id` (bei GET ist das fast immer `None`, weil `zustaendig` leer ist).

Außerdem hast du die Reihenfolge falsch: du machst `int(session["user_id"])` **bevor du** prüfst, ob `user_id` überhaupt in der Session ist → kann auch knallen.

Fix (sauber + funktioniert)

- Hole `wg_id` immer vom eingeloggten User (`session user`)
- Den `zustaendig_user` nutzt du nur fürs Zuweisen der Aufgabe

python

```

@app.route("/putzplan/", methods=["GET", "POST"])
def putzplan():
    if "user_id" not in session:
        return redirect(url_for("login"))

    current_user = User.query.get(session["user_id"])
    if not current_user:
        return redirect(url_for("login"))

```

```
form = PutzplanForm()
all_users = User.query.filter_by(wg_id=current_user.wg_id).all()

if request.method == "POST":
    print("POST angekommen")
    print("form.validate_on_submit():", form.validate_on_submit())
    print("form.errors:", form.errors)
    print("zustaendig raw:", request.form.get("zustaendig"))

    if form.validate_on_submit():
        zustaendig_name = request.form.get("zustaendig", "").strip()
        assigned_user = User.query.filter_by(
            wg_id=current_user.wg_id,
            username=zustaendig_name
        ).first()

        if not assigned_user:
            flash("WG-Mitglied existiert nicht", "error")
        else:
            new_template = CleaningTemplate(
                wg_id=current_user.wg_id,
                name=form.aufgabe.data,
                description=f"KW {form.woche.data}: {form.von_datum.data} bis
                frequency="weekly",
                is_active=True
            )
            db.session.add(new_template)
            db.session.flush()

            new_task = CleaningTask(
                template_id=new_template.template_id,
                assigned_to=assigned_user.user_id,
                status="open",
                notes="Zuständig"
            )
            db.session.add(new_task)
            db.session.commit()

            flash("Eintrag erfolgreich erstellt!", "success")
            return redirect(url_for("putzplan"))

    putzplan_eintraege = (CleaningTemplate.query
                          .filter_by(wg_id=current_user.wg_id, is_active=True)
```

```

        .order_by(CleaningTemplate.template_id.desc())
        .all()

tasks = (CleaningTask.query
        .join(CleaningTemplate)
        .filter(CleaningTemplate.wg_id == current_user.wg_id)
        .all())

total_tasks = len(tasks)
completed_tasks = sum(1 for t in tasks if t.status == "completed")
progress = int((completed_tasks / total_tasks) * 100) if total_tasks else 0

return render_template(
    "putzplan.html",
    form=form,
    all_users=all_users,
    putzplan=putzplan_eintraege,
    total_tasks=total_tasks,
    completed_tasks=completed_tasks,
    progress=progress
)

```

Warum das funktioniert

- `current_user` ist immer gesetzt (sonst redirect)
- `wg_id` kommt immer vom eingeloggten User
- `assigned_user` wird nur benutzt, wenn das Formular valid ist
- `all_users` zeigt nur Users aus **derselben WG**

Wenn du willst, kann ich dir als nächsten Schritt zeigen, wie du statt

`request.form.get("zustaendig")` sauber `form.zustaendig.data` nutzt (WTForms-Style).



```

@app.route("/dashboard/", methods=['GET'])
def dashboard():
    if 'user_id' not in session:
        return redirect(url_for('login'))

```

```

    user = User.query.get(session['user_id'])

```

```
if not user or not user.wg_id:
    return redirect(url_for('create_or_join_wg'))

wg = Wg.query.get(user.wg_id)

offene_putzaufgaben_count =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status='offen').count()
neue_ideen_count =
Idea.query.filter_by(wg_id=wg.wg_id).count()
kommende_events =
Activity.query.filter(Activity.wg_id==wg.wg_id, Activity.date
>= datetime.now()).count()
einkauf_count =
ShoppingItem.query.filter_by(wg_id=wg.wg_id,
assigned_to=None).count()

counting_boxes = {
    'putzaufgaben': max(offene_putzaufgaben_count, 0),
    'ideen': max(neue_ideen_count, 0),
    'events': max(kommende_events, 0),
    'einkauf': max(einkauf_count, 0)
}

#Hinweis-Box
wichtige_hinweise = []

putz_tasks =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status="offen").all()
if putz_tasks:
    wichtige_hinweise += [f"Denk noch an deine
Putzaufgabe: {t.template.name}" for t in putz_tasks]

offene_einkaufs_items =
ShoppingItem.query.filter_by(wg_id=wg.wg_id,
assigned_to=None).all()
if offene_einkaufs_items:
    wichtige_hinweise += ["Folgende Einkaufsitems
müssen noch besorgt werden:"]
    for item in offene_einkaufs_items:
        wichtige_hinweise.append(f"-{item.name}")
```

```

kommende_events_list =
Activity.query.filter(Activity.wg_id==wg.wg_id, Activity.date
>= datetime.now()).order_by(Activity.date.asc()).limit(5).all()
if kommende_events_list:
    wichtige_hinweise += [f"Kommendes Event: {e.title}
am {e.date.strftime('%d.%m.%Y')}]" for e in
kommende_events_list]

if not wichtige_hinweise:
    wichtige_hinweise = ["Momentan gitbt es keine
offenen Aufgaben oder Hinweise!"]

```

warum zeigt es mir nur die richtige anzahl der einträge in innovationboard? Bei meiner gruppenpartnerin geht es korrekt, aber bei mir nur im innovationboard (idea)

Nachgedacht für 33s >

Weil deine anderen Zähl-Queries auf Werte/Logik filtern, die in deiner DB bei dir vermutlich nicht (mehr) so vorkommen – während

`Idea.query.filter_by(wg_id=wg.wg_id).count()` "einfach nur WG zählen" ist und deshalb korrekt aussieht.

Die zwei häufigsten Gründe in deinem Code:

1) Putzaufgaben: Status-String passt nicht

Du zählst:

```
py
```

```
CleaningTask.query.filter_by(assigned_to=user.user_id, status='offen').count()
```

 Code kopieren

Aber in deinem früheren Code hast du Statuswerte wie "open" / "completed" verwendet. Dann ist bei dir `status='offen' => 0`, obwohl Tasks existieren.

✅ Fix (einheitlich machen, z.B. überall "open"/"completed"):

```
py
```

```

offene_putzaufgaben_count = CleaningTask.query.filter_by(
    assigned_to=user.user_id,
    status="open"
).count()

putz_tasks = CleaningTask.query.filter_by(
    assigned_to=user.user_id,
    status="open"
).all()

```

 Code kopieren

Oder wenn du es kurzfristig robust machen willst:

```

py

from sqlalchemy import or_

offene_putzaufgaben_count = CleaningTask.query.filter(
    CleaningTask.assigned_to == user.user_id,
    or_(CleaningTask.status == "offen", CleaningTask.status == "open")
).count()

```

 Code kopieren

2) Einkauf: du zählst nur "unassigned", aber bei dir ist alles assigned

Du zählst:

```

py

einkauf_count = ShoppingItem.query.filter_by(wg_id=wg.wg_id, assigned_to=None).co

```

 Code kopieren

Wenn du inzwischen **random assigned_to setzt**, ist fast nie `None` → count wird 0, obwohl Items da sind.

✅ Was willst du zählen?

A) Alle Einkaufsitems in der WG:

```

py

einkauf_count = ShoppingItem.query.filter_by(wg_id=wg.wg_id).count()

```

 Code kopieren

B) Nur Items, die dir zugewiesen sind:

py

```
einkauf_count = ShoppingItem.query.filter_by(wg_id=wg.wg_id, assigned_to=user.use
```

 Code kopieren

C) Nur "offene" Items geht ohne Status eigentlich nicht sauber.

(Ohne Status/boolean kannst du "gekauft" nicht unterscheiden, außer über Workarounds.)

3) Events: besser DB-seitig vergleichen

Manchmal ist `datetime.now()` vs gespeicherte Zeitzone/naive datetime ein Problem.

✓ Stabiler:

py

```
from sqlalchemy import func
kommende_events = Activity.query.filter(
    Activity.wg_id == wg.wg_id,
    Activity.date >= func.now()
).count()
```

 Code kopieren

Mini-Debug, damit du sofort siehst, wo es hakt

Pack das einmal in `dashboard()` direkt vor `counting_boxes`:

py

```
print("WG:", wg.wg_id, "USER:", user.user_id)

print("Putz status sample:",
      [(t.task_id, t.status) for t in CleaningTask.query.limit(5).all()])

print("Shopping assigned_to sample:",
      [(s.item_id, s.assigned_to) for s in ShoppingItem.query.filter_by(wg_id=wg.1
```

Wenn du mir sagst, welche Statuswerte bei dir im Print auftauchen (open,  Code kopieren ob `assigned_to` fast immer gesetzt ist, sag ich dir exakt, welche Variante du fürs Dashboard am sinnvollsten nimmst.



und warum sehe ich die activities nicht?

Sehr wahrscheinlich weil du in deiner **ActivityBoard-Query** hart `wg_id=1` filterst, während du im Dashboard korrekt mit `wg.wg_id` arbeitest. Wenn deine WG **nicht** die ID 1 hat (oder deine Activities in einer anderen WG gespeichert wurden), dann kommt bei dir **0 Activities** an – und du "siehst keine".

Du hast ja oben sowas (oder hattest es):

py

Code kopieren

```
activities = (Activity.query
    .filter_by(wg_id=1)
    ...
    .all())
```

✓ Fix: überall die **aktuelle WG** benutzen:

py

Code kopieren

```
@app.route("/activityboard/", methods=["GET", "POST"])
def activity_board():
    if "user_id" not in session:
        return redirect(url_for("login"))

    user = User.query.get(session["user_id"])
    if not user or not user.wg_id:
        return redirect(url_for("create_or_join_wg"))

    wg_id = user.wg_id
    form = ActivityForm()

    if request.method == "POST":
        if form.validate_on_submit():
            new_activity = Activity(
                wg_id=wg_id,
                created_by=user.user_id,
```



```

        title=form.title.data,
        description=form.description.data,
        date=form.date.data,
        updated_at=form.updated_at.data,
        location=form.location.data,
        max_participants=form.max_participants.data,
        created_at=db.func.now()
    )
    db.session.add(new_activity)
    db.session.commit()
    return redirect(url_for("activity_board"))

activities = (Activity.query
    .filter_by(wg_id=wg_id)
    .options(joinedload(Activity.creator), joinedload(Activity.participants))
    .order_by(Activity.created_at.desc())
    .all())

return render_template(
    "activityboard.html",
    form=form,
    activities=activities,
    current_user_id=user.user_id
)

```

Und wichtig: deine Dashboard-Zählung zeigt nur "kommende"

Du zählst:

```
py
```

 Code kopieren

```

kommende_events = Activity.query.filter(
    Activity.wg_id==wg.wg_id,
    Activity.date >= datetime.now()
).count()

```

Wenn all deine Activities **in der Vergangenheit** sind oder `date` falsch gesetzt wurde, ist das auch 0.

☒ Teste kurz ohne Datum-Filter:

```
py
```

 Code kopieren

```
print("All activities in WG:", Activity.query.filter_by(wg_id=wg.wg_id).count())
print("Upcoming activities in WG:", Activity.query.filter(Activity.wg_id==wg.wg_i
```



Wenn "All activities" > 0 aber "Upcoming" = 0, dann ist es nur der Datum-Filter.

Wenn du willst, schick mir kurz:

- deine Activity -Model-Spalte date (Typ/nullable)
- und die aktuelle Query aus deiner /activityboard/ route
dann sag ich dir sofort, ob's wg_id oder date-Filter ist (oder beides).



beide sind 1, was korrekt ist

Nachgedacht für 36s >

Wenn **beide Counts = 1** sind (also "alle Activities in WG" **und** "kommende Activities"), dann ist die DB/Query grundsätzlich ok – dann liegt's fast sicher an **Template/HTML/CSS** (wird nicht gerendert oder ist "da aber unsichtbar").

Hier sind die schnellsten Checks, ohne viel Rumraten:

1) Prüfe im Route-Code, ob wirklich eine Activity im activities -Array steckt

Direkt nach deiner Query:

py

Code kopieren

```
activities = (Activity.query
    .filter_by(wg_id=wg_id)
    .order_by(Activity.created_at.desc())
    .all()
)
```

```
print("activities len:", len(activities))
print("first title:", activities[0].title if activities else None)
```

Wenn das `len: 1` und ein Title kommt → Backend passt.

2) Prüfe im Template, ob activities wirklich ankommt

Ganz oben in `activityboard.html` (irgendwo sichtbar):

jinja2

 Code kopieren

```
<p class="text-muted">DEBUG activities={{ activities|length }}</p>
```

Wenn da `1` steht, aber du siehst trotzdem keine Card → **CSS/HTML**.

3) 90%-Fehler: HTML ist "kaputt" (ein falsches `</div>` oder ein stray `<`)

So ein einzelnes falsches Tag kann dazu führen, dass **alles danach komisch "rausfliegt"**.

Mach folgenden Mini-Test im Template **direkt im Loop**, ganz simpel:

jinja2

 Code kopieren

```
<div class="activities-list">
  {% for activity in activities %}
    <div style="border:2px solid red; padding:10px; margin:10px 0;">
      TEST: {{ activity.title }}
    </div>
  {% endfor %}
</div>
```

Wenn du das **siehst**, dann ist dein ursprüngliches Card-Markup oder CSS der Übeltäter.

4) Wenn "TEST" sichtbar ist → dann versteckt dein CSS die Cards

Öffne DevTools (F12) → Inspect auf `.activity-card` und check:

- `display: none;`
- `visibility: hidden;`

- `opacity: 0;`
- oder die Card liegt **hinter einem Overlay** (z.B. Modal-Backdrop).

Schneller Fix-Test: setze (temporär) in CSS:

CSS

 Code kopieren

```
.activities-list, .activity-card {
  display: block !important;
  visibility: visible !important;
  opacity: 1 !important;
}
```

Wenn es dann auftaucht → du hast irgendwo `display:none` oder Overlay.

5) Typischer Overlay-Fall: Modal-Backdrop ist dauerhaft aktiv

Wenn dein Modal-Backdrop (`#modalBackdrop`) fälschlich `display:block` hat oder per CSS getriggert wird, kann er alles "überdecken".

Check: ist ein halbtransparentes Overlay über der Seite? Dann ist das der Grund.

Sag mir kurz das Ergebnis von Check 2:

Was zeigt `DEBUG activities={ { activities|length } }` im Browser an – 0 oder 1?

Und wenn 1: siehst du den roten "TEST"-Block aus Check 3?



```
<style>
  .activities-list {
    display: flex;
    gap: 15px;
    align-items: flex-start;
    justify-content: center;
    margin-top: 20px;
  }
```

```
.activity-card {  
  
    position: relative;  
    border: 1px solid #ddd;  
    border-radius: 5px;  
    padding: 15px;  
    padding-bottom: 60px;  
    background-color: #f9f9f9;  
    box-shadow: 2px 2px 5px rgba(0,0,0,0.1);  
    width: 100%;  
    max-width: 500px;  
}
```

```
.activity-card .action-form{  
    position: absolute;  
    right: 12px;  
    bottom: 12px;  
    margin: 0;  
}
```

```
.activity-card .action-btn{  
    display: inline-block;  
    width: auto;  
    height: auto;  
    padding: 8px 14px;  
    border-radius: 6px;  
    border: 1px solid black;  
    color: white;  
    font-size: 18px;  
    cursor: pointer;  
    white-space: nowrap;  
}
```

```
.activity-card .action-join{  
    background-color: rgb(89, 55, 175);  
}
```

```
.activity-card .action-leave{  
    background-color: rgb(218, 84, 84);  
}
```

```
.activity-card .action-btn:hover{
```

```

    opacity: 0.85;
  }

  .activity-card h3 {
    padding-right: 60px; /* Platz für Delete schaffen */
  }

  .activity-card {
    padding-bottom: 40px; /* Platz für Join schaffen */
  }

  .activity-desc {
    white-space: pre-wrap; /* Zeilenumbrüche behalten
  */
    overflow-wrap: anywhere; /* Lange Wörter
    umbrechen */
    word-break: break-word; /* Fallback */
    max-width: 100%;
  }

```

mein css

Dein CSS ist grundsätzlich ok – aber **der Grund, warum du “Activities nicht siehst” oder nur 1 Card komisch dargestellt wird, ist sehr wahrscheinlich:**

✅ **.activities-list ist display:flex ohne flex-wrap**

Heißt: wenn mehrere Cards kommen, bleiben sie **in einer Zeile** und laufen rechts aus dem Bildschirm raus → du “siehst” dann nur die erste.

Fix: flex-wrap: wrap; hinzufügen (und optional zentrieren)

CSS

```

.activities-list {
  display: flex;
  flex-wrap: wrap;          /* <<< WICHTIG */
  gap: 15px;
  align-items: flex-start;
  justify-content: center;

```

```
margin-top: 20px;
}
```

 Code kopieren

Card-Breite: "max 500px", aber auf großen Screens soll sie nicht 100% nehmen

Aktuell hast du:

CSS

```
width: 100%;
max-width: 500px;
```

 Code kopieren

Das ist ok – aber schöner/sauberer ist:

CSS

```
.activity-card {
  flex: 0 1 500px;      /* Card ist bis zu 500px breit, schrumpft */
  max-width: 100%;
}
```

 Code kopieren

Kleiner Bug in deinem CSS: .activity-card ist doppelt definiert

Du hast weiter unten nochmal:

CSS

```
.activity-card {
  padding-bottom: 40px;
}
```

 Code kopieren

Damit überschreibst du dein padding-bottom: 60px; oben → kann Buttons "quetschen".

Mach es einmal sauber zusammen:

CSS

```
.activity-card {
  position: relative;
  border: 1px solid #ddd;
```

```
border-radius: 5px;
padding: 15px;
padding-bottom: 60px; /* bleibt 60 */
background-color: #f9f9f9;
box-shadow: 2px 2px 5px rgba(0,0,0,0.1);
flex: 0 1 500px;
max-width: 100%;
}
```

 Code kopieren

Bonus: Wenn du wirklich “untereinander” willst (wie vorher)

Dann statt Flex einfach:

CSS

```
.activities-list {
display: flex;
flex-direction: column;
gap: 15px;
align-items: center;
}
```

 Code kopieren

Wenn du mir sagst, ob du die Activities **nebeneinander (Grid)** oder **untereinander (Liste)** willst, geb ich dir die beste fertige CSS-Variante für genau dein Layout.



ich sehe die den activity count nicht im dashboard

Wenn **dein** `kommende_events` in Python wirklich 1 ist, aber im Dashboard nicht angezeigt wird, dann ist es fast immer **kein Query-Problem**, sondern eins von diesen 3 Dingen:

1) Du übergibst `counting_boxes` (**oder** `kommende_events`) **nicht ans Template**

In deinem Snippet sehe ich **kein** `return render_template(...)` . Stell sicher, dass du `counting_boxes` wirklich mitgibst:

python

```
return render_template(
    "dashboard.html",
    wg=wg,
    user=user,
    counting_boxes=counting_boxes,
    wichtige_hinweise=wichtige_hinweise
)
```

Wenn das fehlt, ist im Template `counting_boxes` einfach leer/undefined –  Code kopieren
nix.

2) Key-Mismatch im Template (events vs. activities etc.)

Du speicherst so:

python

```
counting_boxes = {
    'putzaufgaben': ...,
    'ideen': ...,
    'events': max(kommende_events, 0),
    'einkauf': ...
}
```

Dann muss im Template auch **genau** `events` verwendet werden:

 Code kopieren

✅ richtig:

jinja2

```
{{ counting_boxes['events'] }}
```

 Code kopieren

oder (geht auch):

jinja2

```
{{ counting_boxes.events }}
```

 Code kopieren

✗ falsch wäre z.B.:

```
jinja2
```

```
{{ counting_boxes.activity }}  
{{ counting_boxes.activities }}  
{{ counting_boxes.kommende_events }}
```

 Code kopieren

Tip: Nimm die sichere Variante mit `['events']` .

3) Du überschreibst irgendwo `counting_boxes` oder renderst ein anderes Template

Manchmal passiert das nach Merge/Refactor: Route rendert z.B. `dashboard2.html` oder du baust `counting_boxes` neu, bevor du renderst.

Mini-Debug (kurz vor `render_template`):

```
python
```

```
print("kommende_events:", kommende_events)  
print("counting_boxes:", counting_boxes)
```

 Code kopieren

Quick-Fix im Template (damit du sofort siehst ob es da ist)

Pack das einmal irgendwo ins `dashboard.html` :

```
jinja2
```

```
<p>DEBUG events = {{ counting_boxes['events'] }}</p>
```

 Code kopieren

Wenn das 1 zeigt, ist es **nur ein Layout/CSS/Box-Problem**. Wenn es leer/Fehler ist, ist es **Übergabe/Key**.

Wenn du mir den relevanten Ausschnitt aus deinem `dashboard.html` (die Box, wo Events stehen sollen) schickst, sag ich dir in 10 Sekunden, welcher Punkt es ist.

    ...

events=0

Nachgedacht für 28s >

Wenn bei dir `events=0` rauskommt, heißt das: dein Filter

```
python
```

```
Activity.wg_id == wg.wg_id
Activity.date >= datetime.now()
```

 Code kopieren

liefert **gerade keine Treffer**. Das passiert fast immer aus **einem** dieser Gründe:

1) Deine Activities haben ein Datum, das (aus Sicht von `datetime.now()`) schon "in der Vergangenheit" ist

Auch wenn du "morgen" meinst: Wenn z.B. die Zeit/Zeitzone oder das gespeicherte Datum nicht so ist wie erwartet, fällt es raus.

✅ Debug (direkt im `dashboard()` vor dem count):

```
python
```

```
from datetime import datetime
```

```
now = datetime.now()
```

```
acts = (Activity.query
        .filter(Activity.wg_id == wg.wg_id)
        .order_by(Activity.date.asc())
        .all())
```

```
print("NOW:", now)
```

```
print("WG:", wg.wg_id)
```

```
print("ACTS:", [(a.activity_id, a.title, a.date) for a in acts])
```

```
kommende_events = (Activity.query
```

```
    .filter(Activity.wg_id == wg.wg_id, Activity.date >= now)
```

```

        .count())
print("kommende_events:", kommende_events)

```

Wenn du da siehst, dass `a.date` **kleiner** als `now` ist → deshalb `0`.

 Code kopieren

2) WG-ID passt doch nicht (oder Activity hat `wg_id = NULL`/anderer Wert)

Du hast zwar gesagt "beide sind 1", aber prüf's mit dem Debug oben: Dort siehst du pro Activity das `wg_id` indirekt (über Filter) – wenn ACTS leer ist, passt `wg_id` nicht.

3) Du speicherst das Datum anders als du denkst (WTForms/DateTimeLocalField)

Wenn du ein `DateTimeLocalField` nutzt, kommt **naive local time** rein. Das ist okay – aber nur, wenn du auch mit `datetime.now()` vergleichst (nicht `utcnow()` gemischt).

Wichtig: Wenn du irgendwo `datetime.utcnow()` nutzt und woanders `datetime.now()`, kann's schiefgehen.

Schnell-Fix (sauber & stabil)

Wenn deine gespeicherten Dates "lokal" sind (typisch bei `datetime-local` im Browser), dann nutz überall `datetime.now()`:

```

python

now = datetime.now()
kommende_events = Activity.query.filter(
    Activity.wg_id == wg.wg_id,
    Activity.date >= now
).count()

```

 Code kopieren

Wenn du mir 2 Zeilen Debug-Ausgabe schickst...

- NOW: ...
- ACTS: [...]

...kann ich dir sofort sagen, ob es **Datum/Zeit** oder **wg_id** ist und was du genau ändern musst.



NOW: 2026-01-15 17:14:56.812374

WG: 2

ACTS: [(3, 'fgohfgh', datetime.datetime(2026, 1, 15, 16, 38)),
(4, 'opij', datetime.datetime(2026, 1, 15, 16, 55))]

Perfekt – damit ist der Grund glasklar:

- **NOW:** 17:14
- Deine Activities sind um **16:38** und **16:55**
➔ Beide liegen **in der Vergangenheit**, deshalb liefert dein Filter `Activity.date >= NOW` **0**.

Option A (Dashboard soll "heute" zählen, auch wenn Uhrzeit schon vorbei ist)

Dann filtere ab **Tagesanfang** statt ab "jetzt":

```
python
```

```
from datetime import datetime, time
```

```
today_start = datetime.combine(datetime.now().date(), time.min)
```

```
kommende_events = Activity.query.filter(  
    Activity.wg_id == wg.wg_id,  
    Activity.date >= today_start  
)>.count()
```

Damit zählen auch Events von heute früh/mittags.

Code kopieren

Option B (Dashboard soll wirklich nur zukünftige Events zählen)

Dann ist dein Code korrekt – du musst nur **künftige Zeiten** speichern.

Wenn du “nur Datum” eingibst oder die Uhrzeit automatisch zu früh gesetzt wird, setz beim Speichern z.B. defaultmäßig eine spätere Uhrzeit oder lass den User die Uhrzeit wählen.

Bonus: “Nächste 24 Stunden” statt “ab jetzt”

python

```
from datetime import datetime, timedelta

now = datetime.now()
next_24h = now + timedelta(hours=24)

kommende_events = Activity.query.filter(
    Activity.wg_id == wg.wg_id,
    Activity.date.between(now, next_24h)
).count()
```

 Code kopieren

Wenn du willst, sag mir kurz: Soll im Dashboard **heute** mitgezählt werden (Option A) oder wirklich nur **ab jetzt** (Option B)?



```
letzte_aktivitaeten.append({"zeitpunkt": t.completed_at,
    "text": f"<strong>{t.user.username}</strong> hat {t.template.name} geputzt", "typ": "putz"})
hier nach username ein leerzeichen packen
```

Klar – füge einfach ein Leerzeichen **nach dem Username** in den String ein:

python

 Code kopieren

```
letzte_aktivitaeten.append({
    "zeitpunkt": t.completed_at,
```

```

    "text": f"<strong>{t.user.username}</strong> hat {t.template.name} geputzt",
    "typ": "putz"
})

```

Wenn du ein bisschen mehr Abstand willst (HTML), nimm z.B. ein geschütztes Leerzeichen:

python

 Code kopieren

```

"text": f"<strong>{t.user.username}</strong>&nbsp;hat {t.template.name} geputzt",

```



```

erledigte_putzaufgaben =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status="completed").order_by(CleaningTask.completed_at.
desc()).limit(10).all()
    for t in erledigte_putzaufgaben:
        letzte_aktivitaeten.append({"zeitpunkt":
t.completed_at, "text": f"<strong>
{t.assigned_user.username}</strong> hat
{t.template.name} geputzt", "typ": "putz"})

```

warum zeigt er mir completed_at nicht an?

Das passiert fast immer aus einem von diesen Gründen:

1. completed_at wird beim "completed" setzen nie befüllt

Du setzt zwar status="completed" , aber nirgendwo steht t.completed_at = datetime.now() (oder db.func.now()).

2. completed_at ist None (NULL) in der DB

Dann ist t.completed_at leer → in deiner Ausgabe "steht nichts" und auch das order_by(...desc()) wirkt komisch (NULLs).

3. Du hast gar keine completed_at -Spalte in der Tabelle (wenn du ohne Migration/DB-Update gearbeitet hast)

Dann würdest du meist schon früher einen DB-Fehler bekommen – aber je nach Codepfad merkst du es erst später.

Fix (empfohlen): completed_at beim Toggle setzen

In deiner Route/Funktion, wo du die Checkbox togglest (`toggle_cleaning_task`), mach so:

```
python

from datetime import datetime

@app.route("/putzplan/task/<int:task_id>/toggle", methods=["POST"])
def toggle_cleaning_task(task_id):
    task = CleaningTask.query.get_or_404(task_id)

    if task.status == "completed":
        task.status = "offen"    # oder "open" - wichtig: konsistent!
        task.completed_at = None
    else:
        task.status = "completed"
        task.completed_at = datetime.now()

    db.session.commit()
    return redirect(url_for("putzplan"))
```

Wichtig: Bei dir taucht manchmal `status='offen'` auf. Wenn du an anderer Stelle `"open"` nutzt, bekommst du falsche Counts/Filter. Entscheide dich für **einen** Wert (z.B. `"open"` oder `"offen"`) und nutze ihn überall gleich.

 Code kopieren

Quick-Debug: ist completed_at wirklich leer?

Kurz vorm append:

```
python

print("DEBUG completed:", [(t.task_id, t.status, t.completed_at) for t in erledigt])
```

◀

 Code kopieren ▶

Wenn da `None` steht → Problem #1 (nie gesetzt).

Falls completed_at als Spalte fehlt

In deinem `CleaningTask` Model muss sowas drin sein:

python

```
completed_at = db.Column(db.DateTime, nullable=True)
```

 Code kopieren

Und dann musst du die DB-Struktur aktualisieren (Migration oder Tabelle neu erstellen), sonst kann es nicht gespeichert werden.

Wenn du mir deine `toggle_cleaning_task` -Route + `CleaningTask` -Model zeigst, sag ich dir exakt, wo du es einbauen musst.



```
if not assigned_user:
    flash("WG-Mitglied existiert nicht", "error")
else:
    new_template = CleaningTemplate(
        wg_id=user.wg_id,
        name=form.aufgabe.data,
        description=f"KW {form.woche.data}:
{form.von_datum.data} bis {form.bis_datum.data}",
        frequency="weekly",
        is_active=True
    )
    db.session.add(new_template)
    db.session.flush()

    new_task = CleaningTask(
        template_id=new_template.template_id,
        assigned_to=assigned_user.user_id,
        status="open",
        notes="Zuständig"
    )
    db.session.add(new_task)
    db.session.commit()

    flash("Eintrag erfolgreich erstellt!", "success")
    return redirect(url_for("putzplan"))

wo packe ich es hier rein?
```

In **diesen Block** musst du es **nicht** reinpacken – hier erstellst du den Task ja nur als **offen/open**. `completed_at` setzt du erst dann, wenn der User die Checkbox anklickt (also in deiner **toggle**-Route).

Trotzdem kannst du beim Erstellen sauber initialisieren (optional):

python

```
new_task = CleaningTask(
    template_id=new_template.template_id,
    assigned_to=assigned_user.user_id,
    status="open",
    notes="Zuständig",
    completed_at=None # optional, aber sauber
)
```

✅ Der richtige Ort für `completed_at = ...` ist die Toggle-Funktion, z.B.  Code kopieren

python

```
from datetime import datetime

@app.route("/putzplan/task/<int:task_id>/toggle", methods=["POST"])
def toggle_cleaning_task(task_id):
    task = CleaningTask.query.get_or_404(task_id)

    if task.status == "completed":
        task.status = "open"
        task.completed_at = None
    else:
        task.status = "completed"
        task.completed_at = datetime.now()

    db.session.commit()
    return redirect(url_for("putzplan"))
```

Wichtig: Achte darauf, dass du **überall denselben Status-Wert** nutzt ("open" vs "offen"). Wenn du beim Erstellen "open" setzt, dann auch beim Filtern/Counten "open" verwenden.  Code kopieren



```
@app.route("/dashboard/", methods=['GET'])
def dashboard():
    if 'user_id' not in session:
        return redirect(url_for('login'))

    user = User.query.get(session['user_id'])
    if not user or not user.wg_id:
        return redirect(url_for('create_or_join_wg'))
    if not user:
        session.clear()
        return redirect(url_for("login"))

    wg = Wg.query.get(user.wg_id)

    offene_putzaufgaben_count =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status='open').count()
    neue_ideen_count =
Idea.query.filter_by(wg_id=wg.wg_id).count()

    kommende_events =
Activity.query.filter(Activity.wg_id==wg.wg_id, Activity.date
>= datetime.now()).count()

    einkauf_count =
ShoppingItem.query.filter_by(wg_id=wg.wg_id,
assigned_to=user.user_id).count()

    counting_boxes = {
        'putzaufgaben': max(offene_putzaufgaben_count, 0),
        'ideen': max(neue_ideen_count, 0),
        'events': max(kommende_events, 0),
        'einkauf': max(einkauf_count, 0)
    }
    #Hinweis-Box
    wichtige_hinweise = {
        "putz": [],
        "einkauf": [],
        "events": []
```

```

    }

    putz_tasks =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status="open").all()
    for t in putz_tasks:
        wichtige_hinweise ["putz"].append(t.template.name)

    offene_einkaufs_items =
ShoppingItem.query.filter(ShoppingItem.wg_id ==
wg.wg_id).all()
    for item in offene_einkaufs_items:
        wichtige_hinweise["einkauf"].append(item.name)

    kommende_events_list =
Activity.query.filter(Activity.wg_id==wg.wg_id, Activity.date
>= datetime.now()).order_by(Activity.date.asc()).limit(5).all()
    for e in kommende_events_list:
        wichtige_hinweise["events"].append({
            "title": e.title,
            "date": e.date
        })

    if not wichtige_hinweise:
        wichtige_hinweise = ["Momentan gitbt es keine
offenen Aufgaben oder Hinweise!"]

#Activity-Box
letzte_aktivitaeten = []

    erledigte_putzaufgaben =
CleaningTask.query.filter_by(assigned_to=user.user_id,
status="completed").order_by(CleaningTask.completed_at.
desc()).limit(10).all()
    for t in erledigte_putzaufgaben:
        letzte_aktivitaeten.append({"zeitpunkt":
t.completed_at, "text": f"<strong>
{t.assigned_user.username}</strong> hat
{t.template.name} geputzt", "typ": "putz"})

    neue_einkaufs_items =
ShoppingItem.query.filter(ShoppingItem.wg_id ==

```

```
wg.wg_id).order_by(ShoppingItem.created_at.desc()).limit(10).all()

    for ein in neue_einkaufs_items:
        letzte_aktivitaeten.append({"zeitpunkt":
ein.created_at, "text": f"<strong>
{ein.added_by_user.username}</strong> hat '{ein.name}'
zur Einkaufsliste hinzugefügt", "typ": "einkauf"})

    letzte_ideen =
Idea.query.filter_by(wg_id=wg.wg_id).order_by(Idea.create
d_at.desc()).limit(10).all()
    for i in letzte_ideen:
        letzte_aktivitaeten.append({"zeitpunkt": i.created_at,
"text": f"<strong>{i.creator.username}</strong> hat eine
Idee hinzugefügt: '{i.title}'", "typ": "idee"})

    kommende_events_list =
Activity.query.filter(Activity.wg_id == wg.wg_id,
Activity.date >=
datetime.now()).order_by(Activity.date.asc()).limit(5).all()
    for e in kommende_events_list:
        letzte_aktivitaeten.append({"zeitpunkt": e.created_at
or e.date, "text": f"<strong>{e.creator.username}</strong>
hat ein Event hinzugefügt: " f"'{e.title}' am
{e.date.strftime('%d.%m.%Y')}", "typ": "event"})

    if not letzte_aktivitaeten:
        letzte_aktivitaeten.append({"zeitpunkt": None, "text":
"Momentan gibt es keine Aktivitäten"})

    letzte_aktivitaeten.sort(key=lambda x: x.get("zeitpunkt")
or datetime.min, reverse=True)
    letzte_aktivitaeten = letzte_aktivitaeten[:10]

    wg.mitglieder =
User.query.filter_by(wg_id=wg.wg_id).all()

    heute = datetime.now().strftime("%A, %d.%m.%Y")

    return render_template("dashboard.html",
active_page="dashboard", username=user.username,
wg_name=wg.name, heute=heute,
counting_boxes=counting_boxes,
```

```
wichtige_hinweise=wichtige_hinweise,
letzte_aktivitaeten=letzte_aktivitaeten,
wg_mitglieder=wg_mitglieder)sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) no such table: USER
[SQL: SELECT "USER".user_id AS "USER_user_id",
"USER".username AS "USER_username",
"USER".password_hash AS "USER_password_hash",
"USER".email AS "USER_email", "USER".role AS "USER_role",
"USER".wg_id AS "USER_wg_id"
```

warum geht das nicht mehr, obwohl das vor kurzem noch funktioniert hat und ich nixhts geändert habe?

Nachgedacht für 10s >

Der Fehler bedeutet ganz simpel: **Du läufst gerade gegen eine SQLite-Datei, in der die Tabelle USER nicht existiert.** Das passiert fast immer, wenn du **nicht mehr dieselbe Datenbank-Datei** verwendest wie "vor kurzem", auch wenn du am Python-Code nichts geändert hast.

Typische Ursachen (in der Praxis die Top 3)

1) Du hast jetzt aus einem anderen Ordner gestartet → SQLite legt eine *neue* DB an

Wenn deine URI so aussieht wie z.B. `sqlite:///app.db` (relativer Pfad), dann hängt die Datei vom **aktuellen Working Directory** ab. Startest du `flask run` einmal aus einem anderen Ordner, entsteht eine neue leere DB → keine Tabellen.

✅ Check:

- Suche auf deinem Rechner nach mehreren `app.db` / `database.db` / `*.sqlite`
- Schau in deinem Projekt: `instance/` , `Projektroot`, etc.

Fix (empfohlen): benutze einen absoluten Pfad:

```
python
```

```
import os
```

```
basedir = os.path.abspath(os.path.dirname(__file__))
```

```
app.config["SQLALCHEMY_DATABASE_URI"] = "sqlite://" + os.path.join(basedir, "ins
```

2) Deine `SQLALCHEMY_DATABASE_URI` wurde geändert (z.B. nach Merge)

Nach Git-Merge passiert das oft über:

- `config.py` überschrieben
- `.env` geändert
- anderes `instance/` Setup

✓ Quick-Debug: ganz oben beim App-Start einmal ausgeben:

python

```
print("DB URI =", app.config.get("SQLALCHEMY_DATABASE_URI"))
```

 Code kopieren

Wenn da ein anderer Pfad steht als vorher → du bist in einer anderen DB.

3) DB-Datei wurde gelöscht/überschrieben oder `instance/` ist weg

Wenn die Datei neu ist, gibt es natürlich keine Tabellen.

✓ Check: Gibt es deine DB-Datei noch? Hat sie "normale" Größe oder ist sie 0KB?

So findest du in 30 Sekunden heraus, welche DB gerade benutzt wird und ob Tabellen existieren

Füge temporär in `app.py` (nach `app/db init`) ein:

python

```
from sqlalchemy import text
```

```
@app.route("/__dbcheck")
```

```
def __dbcheck():
```

```
    uri = current_app.config["SQLALCHEMY_DATABASE_URI"]
```

```
    tables = db.session.execute(text("SELECT name FROM sqlite_master WHERE type=''
```

```
    return {"uri": uri, "tables": [t[0] for t in tables]}
```

<

>

Dann rufe im Browser auf: `http://127.0.0.1:5000/__dbcheck`

 Code kopieren

Wenn `USER` nicht in der Liste ist → falsche/neu erzeugte DB.

Fix: Tabellen neu erstellen (ohne Migrate)

Wenn du kein Flask-Migrate nutzt und es dir gerade nur ums Laufen geht:

```
python
```

```
from app import app
from db import db

with app.app_context():
    db.create_all()
```

Oder schnell in flask shell :

 Code kopieren

```
bash
```

```
flask shell
>>> from app import app
>>> from db import db
>>> with app.app_context():
...     db.create_all()
```

⚠️ Wenn du Daten behalten willst: **nicht** einfach neu erstellen, sondern zu  Code kopieren sicherstellen, dass du die richtige DB-Datei nutzt.

Kleiner Hinweis zu deinem Code

Du hast das hier:

```
python
```

```
if not user or not user.wg_id:
    return redirect(url_for('create_or_join_wg'))
if not user:
    session.clear()
    return redirect(url_for("login"))
```

Die zweite `if not user:` ist redundant (die erste fängt das schon ab). Nic  Code kopieren r den DB-Fehler, aber kannst du sauberer machen.

Wenn du mir 1 Zeile gibst, kann ich dir sofort sagen welche Ursache es ist:

Was printet DB URI = ... **bei dir?** (oder Screenshot von __dbcheck Output)



File

"C:\Users\ximma\Desktop\Uni\HWR\4Semester\WebProgram\Web-Development-Project\app.py", line 469, in delete_idea

```
if idea.creator.user_id != user_id:
```

```
    ^^^^
```

UnboundLocalError: cannot access local variable 'idea' where it is not associated with a value

#post route, da das Formular data ändert (löschen)

```
@app.route("/innovation_board/idea/<int:idea_id>/delete", methods=["POST"])
```

```
def delete_idea(idea_id):
```

```
    #Guard Klausel: Sicherstellen, dass der User eingeloggt ist (sollte nicht passieren.)
```

```
    if 'user_id' not in session:
```

```
        return redirect(url_for('login'))
```

```
    #aktuellen User laden und konkrete Idee laden, prüfen ob der User der Ersteller der Idee ist, wenn nicht -> 403 und Flashmeldung
```

```
    #Bezug auf creator: Beziehung in Idea Model
```

```
    user_id = int(session["user_id"])
```

```
    if idea.creator.user_id != user_id:
```

```
        flash("Sie können nur Ihre eigenen Ideen löschen.", "error")
```

```
        abort(403)
```

```
    #Idee löschen
```

```
    idea = Idea.query.get_or_404(idea_id)
```

```
    db.session.delete(idea)
```

```
    db.session.commit()
```

```
    flash("Idee erfolgreich gelöscht.", "success")
```

```
    return redirect(url_for("innovation_board"))
```

Der Fehler kommt, weil du `idea` **benutzt** bevor du sie lädst:

python

```
if idea.creator.user_id != user_id:
    ...
# Idee löschen
idea = Idea.query.get_or_404(idea_id)
```

`idea` existiert da noch nicht → `UnboundLocalError`.

 Code kopieren

So ist es korrekt (erst laden, dann prüfen, dann löschen):

python

```
from flask import abort, flash, redirect, session, url_for
from db import db, Idea # ggf. anpassen

@app.route("/innovation_board/idea/<int:idea_id>/delete", methods=["POST"])
def delete_idea(idea_id):
    if "user_id" not in session:
        return redirect(url_for("login"))

    user_id = int(session["user_id"])

    # 1) Idee laden
    idea = Idea.query.get_or_404(idea_id)

    # 2) Berechtigung prüfen (creator_id oder creator.user_id)
    if idea.created_by != user_id: # falls du created_by Spalte hast (empfohlen
        flash("Sie können nur Ihre eigenen Ideen löschen.", "error")
        abort(403)

    # Alternative, wenn du nur die Beziehung nutzt:
    # if not idea.creator or idea.creator.user_id != user_id:
    #     flash("Sie können nur Ihre eigenen Ideen löschen.", "error")
    #     abort(403)

    # 3) löschen
    db.session.delete(idea)
    db.session.commit()
```

```
flash("Idee erfolgreich gelöscht.", "success")
return redirect(url_for("innovation_board"))
```

Warum `idea.created_by` **besser ist als** `idea.creator.user_id`:

- ist robuster (kein `None`, kein zusätzlicher Join nötig)
- schneller und sauberer für Permission-Checks

 Code kopieren

Wenn du mir kurz sagst, ob dein `Idea`-Model eine Spalte wie `created_by` / `creator_id` hat, passe ich dir den Check exakt auf deine Model-Namen an.



```
if activity.creator.user_id != current_user_id:
    flash("Sie können nur Ihre eigenen Aktivitäten
    löschen.", "error")
    abort()
wie mache ich, dass der user nur seine eigenen aktivitäten
löschen kann?
```

So wie bei der Idee: **erst Activity laden**, dann **Owner prüfen**, dann löschen. Am saubersten über die FK-Spalte (`created_by`), nicht über `activity.creator.user_id`.

python

```
from flask import abort, flash, redirect, session, url_for
from db import db, Activity

@app.route("/activity/<int:activity_id>/delete", methods=["POST"])
def delete_activity(activity_id):
    if "user_id" not in session:
        return redirect(url_for("login"))

    current_user_id = int(session["user_id"])

    activity = Activity.query.get_or_404(activity_id)

    #  nur Ersteller darf löschen
    if activity.created_by != current_user_id:
```

```
flash("Sie können nur Ihre eigenen Aktivitäten löschen.", "error")
abort(403)
```

```
db.session.delete(activity)
db.session.commit()
flash("Aktivität erfolgreich gelöscht.", "success")
return redirect(url_for("activity_board"))
```

Template (Button):

html

```
<form method="POST" action="{{ url_for('delete_activity', activity_id=activity.ac
  <button type="submit" class="btn btn-sm btn-outline-danger"
    onclick="return confirm('Aktivität wirklich löschen?');">
```



```
</button>
</form>
```



Optional (besser UX): Button nur anzeigen, wenn es seine Aktivität ist:

 Code kopieren

jinja2

```
{% if current_user_id == activity.created_by %}
  ...delete form...
{% endif %}
```

 Code kopieren

Wichtig: Im `abort()` immer `abort(403)` geben, sonst ist es ein 500er.



abort geht nicht, da steht, dass es kein argument gibt

Dann importierst du gerade sehr wahrscheinlich **das falsche** `abort` (oder gar keins). In Flask muss es **so** importiert werden:

python

 Code kopieren

```
from flask import abort
```

Danach funktioniert `abort(403)` ganz normal.

Kompletter Fix:

python

 Code kopieren

```
from flask import abort, flash, redirect, session, url_for

@app.route("/activity/<int:activity_id>/delete", methods=["POST"])
def delete_activity(activity_id):
    if "user_id" not in session:
        return redirect(url_for("login"))

    current_user_id = int(session["user_id"])
    activity = Activity.query.get_or_404(activity_id)

    if activity.created_by != current_user_id:
        flash("Sie können nur Ihre eigenen Aktivitäten löschen.", "error")
        abort(403) # <- braucht den richtigen Import

    db.session.delete(activity)
    db.session.commit()
    flash("Aktivität erfolgreich gelöscht.", "success")
    return redirect(url_for("activity_board"))
```

Wenn du **ohne abort** arbeiten willst (auch okay), dann so:

python

 Code kopieren

```
if activity.created_by != current_user_id:
    flash("Sie können nur Ihre eigenen Aktivitäten löschen.", "error")
    return redirect(url_for("activity_board"))
```

Wenn du magst: schick mir kurz deine aktuellen Imports oben in `app.py` (die ersten ~20 Zeilen), dann sag ich dir exakt, welches `abort` bei dir gerade "im Weg" ist.



```

@login_required
@app.route("/welcome/create_wg/", methods=['GET',
'POST'])
def create_wg():

    if request.method == 'POST':
        wg_name = request.form.get("wg_name").strip()

        if not wg_name:
            flash("Bitte einen WG-Namen eingeben", "danger")
            return redirect(url_for("create_wg"))

        invite_code = generate_unique_code()

        new_wg = Wg(name=wg_name,
invite_code=invite_code)

        db.session.add(new_wg)
        db.session.commit()

        flash(f"WG '{wg_name}' erstellt! Dein Einladungscode:
{invite_code}", "success")
        return redirect(url_for("join_wg"))

    return render_template("create_wg.html")
@login_required
^^^^^^^^^^^^^^^^

```

```

TypeError: login_required() takes 0 positional arguments
but 1 was given

```

Nachgedacht für 17s >

Der Fehler heißt ganz simpel: **Dein** `login_required` **ist nicht als Decorator definiert** (oder du importierst nicht das richtige). Ein Decorator muss **die Funktion als Argument bekommen** – bei dir nimmt `login_required()` offenbar **0 Argumente**, aber Python übergibt ihm **die zu dekorierende Funktion** ⇒ deshalb:

```

TypeError: login_required() takes 0 positional arguments but 1 was given

```

✓ Lösung 1: Eigenen login_required richtig bauen (Session-basiert)

Oben in deiner app.py (oder in auth.py und dann importieren):

python

 Code kopieren

```
from functools import wraps
from flask import session, redirect, url_for, flash, request

def login_required(f):
    @wraps(f)
    def decorated(*args, **kwargs):
        if "user_id" not in session:
            flash("Bitte zuerst einloggen.", "danger")
            return redirect(url_for("login"))
        return f(*args, **kwargs)
    return decorated
```

Und dann wichtig: Decorator-Reihenfolge in Flask:

python

 Code kopieren

```
@app.route("/welcome/create_wg/", methods=["GET", "POST"])
@login_required
def create_wg():
    if request.method == "POST":
        wg_name = (request.form.get("wg_name") or "").strip()

        if not wg_name:
            flash("Bitte einen WG-Namen eingeben", "danger")
            return redirect(url_for("create_wg"))

        invite_code = generate_unique_code()
        new_wg = Wg(name=wg_name, invite_code=invite_code)

        db.session.add(new_wg)
        db.session.commit()

        flash(f"WG '{wg_name}' erstellt! Dein Einladungscode: {invite_code}", "su
        return redirect(url_for("join_wg"))

    return render_template("create_wg.html")
```

Warum diese Reihenfolge?

`@app.route` muss **die bereits geschützte** Funktion registrieren. Deshalb: `@app.route` **oben**, `@login_required` **direkt darüber** der Funktion.

✅ Lösung 2: Wenn du Flask-Login nutzt (nur falls ihr das habt)

Dann muss es heißen:

python

 Code kopieren

```
from flask_login import login_required
```

Und auch hier gilt die Reihenfolge:

python

 Code kopieren

```
@app.route("/welcome/create_wg/", methods=["GET", "POST"])
@login_required
def create_wg():
    ...
```

Wenn du mir kurz zeigst, wie dein `login_required` **aktuell aussieht** (oder deine Imports oben in `app.py`), sag ich dir exakt, was du gerade "falsch" importierst/definierst.



```
@app.route("/putzplan/task/<int:template_id>/delete",
methods=["POST"])
def delete_cleaning_task(template_id):
    user_id = session.get("user_id")
    template =
CleaningTemplate.query.get_or_404(template_id)
    if template.creator.user_id != user_id:
        flash("Sie können nur Ihre eigenen Aufgaben
löschen.", "error")
        return redirect(url_for("putzplan"))
```



```
db.session.delete(template)
db.session.commit()
flash("Aufgabe erfolgreich gelöscht.", "success")
return redirect(url_for("putzplan"))
```

'CleaningTemplate' object has no attribute 'creator'

```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True,
index=True)
    wg_id = db.Column(db.Integer,
db.ForeignKey('WG.wg_id'), nullable=False)
    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)
    #Beziehungen
    wg = db.relationship('Wg',
back_populates='cleaning_templates')
    tasks = db.relationship('CleaningTask',
back_populates='template', cascade='all, delete-orphan')
```

```
class CleaningTask(db.Model):
    __tablename__ = 'CLEANING_TASK'
    task_id = db.Column(db.Integer, primary_key=True,
index=True)
    template_id = db.Column(db.Integer,
db.ForeignKey('CLEANING_TEMPLATE.template_id'),
nullable=False)
    assigned_to = db.Column(db.Integer,
db.ForeignKey('USER.user_id'), nullable=False)
    status = db.Column(db.String, nullable=False)
    completed_at = db.Column(db.DateTime)
    notes = db.Column(db.String)
    #Beziehungen
    template = db.relationship('CleaningTemplate',
back_populates='tasks')
    assigned_user = db.relationship('User',
back_populates='cleaning_tasks')
```

Der Fehler ist korrekt: `CleaningTemplate` hat kein Feld/keine Relationship `creator`, deshalb gibt's `template.creator` nicht.

Du hast aktuell nur:

- `CleaningTemplate` (hat `wg_id`, `name`, ...)
- `CleaningTask` (hat `assigned_to` und `assigned_user`)

Damit gibt es 3 sinnvolle Wege, "nur eigene löschen" umzusetzen:

Option A (empfohlen): Creator-Feld im Template hinzufügen (sauberste Lösung)

Wenn "eigene Aufgabe" bedeutet: nur der Ersteller des Eintrags darf löschen, dann brauchst du im Template sowas wie `created_by`.

Model erweitern

python

```
class CleaningTemplate(db.Model):
    __tablename__ = 'CLEANING_TEMPLATE'
    template_id = db.Column(db.Integer, primary_key=True, index=True)
    wg_id = db.Column(db.Integer, db.ForeignKey('WG.wg_id'), nullable=False)

    created_by = db.Column(db.Integer, db.ForeignKey('USER.user_id'), nullable=False)

    name = db.Column(db.String, nullable=False)
    description = db.Column(db.String)
    frequency = db.Column(db.String, nullable=False)
    is_active = db.Column(db.Boolean, default=True)
    position = db.Column(db.Integer)

    wg = db.relationship('Wg', back_populates='cleaning_templates')
    tasks = db.relationship('CleaningTask', back_populates='template', cascade='all, delete')
    creator = db.relationship('User', foreign_keys=[created_by])
```

Beim Erstellen:

python

```

new_template = CleaningTemplate(
    wg_id=user.wg_id,
    created_by=session["user_id"],
    name=form.aufgabe.data,
    description=...,
    frequency="weekly",
    is_active=True
)

```

 Code kopieren

Delete-Route:

python

```

@app.route("/putzplan/task/<int:template_id>/delete", methods=["POST"])
def delete_cleaning_task(template_id):
    user_id = session.get("user_id")
    template = CleaningTemplate.query.get_or_404(template_id)

    if template.created_by != user_id:
        flash("Sie können nur Ihre eigenen Aufgaben löschen.", "error")
        return redirect(url_for("putzplan"))

    db.session.delete(template)
    db.session.commit()
    flash("Aufgabe erfolgreich gelöscht.", "success")
    return redirect(url_for("putzplan"))

```

➔ Das braucht dann aber DB-Migration / DB neu erstellen, weil du eine neue Spalte brauchst.

 Code kopieren

Option B (ohne DB-Änderung): "Eigene Aufgabe" = nur der Zuständige darf löschen

Dann prüfst du nicht am Template, sondern am Task (assigned_to).

Du hast pro Template eh `tasks[0]` benutzt → also:

python

```

@app.route("/putzplan/task/<int:template_id>/delete", methods=["POST"])
def delete_cleaning_task(template_id):
    user_id = session.get("user_id")

```

```

template = CleaningTemplate.query.get_or_404(template_id)

task = template.tasks[0] if template.tasks else None
if not task:
    flash("Kein Task gefunden.", "error")
    return redirect(url_for("putzplan"))

if task.assigned_to != user_id:
    flash("Nur die zuständige Person kann diese Aufgabe löschen.", "error")
    return redirect(url_for("putzplan"))

db.session.delete(template) # löscht wegen cascade auch den task
db.session.commit()
flash("Aufgabe erfolgreich gelöscht.", "success")
return redirect(url_for("putzplan"))

```

✅ Vorteil: kein migrate nötig

⚠️ Nachteil: "eigene" bedeutet dann "mir zugewiesen".

 Code kopieren

Option C: Jeder in der WG darf löschen (einfacher WG-Check)

Wenn Löschen für alle WG-Mitglieder ok ist:

python

```

@app.route("/putzplan/task/<int:template_id>/delete", methods=["POST"])
def delete_cleaning_task(template_id):
    user_id = session.get("user_id")
    current_user = User.query.get(user_id)
    template = CleaningTemplate.query.get_or_404(template_id)

    if not current_user or template.wg_id != current_user.wg_id:
        flash("Nicht erlaubt.", "error")
        return redirect(url_for("putzplan"))

    db.session.delete(template)
    db.session.commit()
    flash("Aufgabe erfolgreich gelöscht.", "success")
    return redirect(url_for("putzplan"))

```

 Code kopieren

Welche willst du?

- Wenn "nur der Ersteller darf löschen" → **Option A**
- Wenn "nur der Zuständige darf löschen" → **Option B** (ohne DB migrate)
- Wenn "jeder in der WG darf löschen" → **Option C**

Wenn du sagst, welche Regel du meinst, passe ich dir Template + Button + Route exakt passend an (auch mit dem korrekten `url_for` und IDs).



```
<button type="submit" class="action-btn {% if is_joined
%}action-leave{% else %}action-join{% endif %}"
onclick="return confirm('{% if is_joined %}Von der Aktivität
wirklich abmelden?{% else %}Aktivität wirklich beitreten?
{% endif %}');"> {% if is_joined %}Leave X{% else %}Join
➡{% endif %} </button>
```

wie würde man das ohne javascript machen? Da müssen ja dann verschiedene neue get bzw. post Routes angelegt werden? Wie könnte man diese in modalen machen statt auf seiten?

Du kannst das **ohne JavaScript** machen – aber dann gibt es **keinen "Confirm"-Dialog**, weil `confirm()` eben JS ist. Ohne JS hast du im Prinzip 3 saubere Alternativen:

1) Ohne JS: Bestätigung als eigene Seite (POST → Confirm-Page → POST)

Du lässt den Button **nicht direkt joinen/leaven**, sondern schickst erst zu einer "Bestätigen"-Route, die eine kleine Seite rendert.

Routes (Beispiel)

- POST `/activity/<id>/join/request` → zeigt Confirm-Seite ("Willst du beitreten?")
- POST `/activity/<id>/join/confirm` → führt Join aus
- genauso für leave

Das ist die klassische "Server-side confirm" Variante.

Vorteil: 100% ohne JS, sehr zuverlässig

Nachteil: es ist eine extra Seite

2) Ohne JS: Bestätigung "inline" auf derselben Seite (per Query-Param)

Du klickst auf "Join", landest wieder auf `/activityboard/?confirm_join=3`, und die Card zeigt dann **statt dem Button** eine Bestätigungsleiste mit "Ja / Abbrechen".

Beispiel im Template

jinja2

```
{% set confirm_join_id = request.args.get('confirm_join', type=int) %}

{% if confirm_join_id == activity.activity_id %}
    <form method="POST" action="{{ url_for('join_activity', activity_id=activity.ac
        <button class="btn btn-sm btn-primary" type="submit">Ja, beitreten</button>
    </form>
    <a class="btn btn-sm btn-secondary" href="{{ url_for('activity_board') }}">Abbr
{% else %}
    <a class="btn btn-sm btn-outline-primary"
        href="{{ url_for('activity_board', confirm_join=activity.activity_id) }}">
        Join ➡
    </a>
{% endif %}
```

⏪ ⏩

Für Leave genauso mit `confirm_leave`.

Vorteil: keine extra Seite, kein JS

 Code kopieren

Nachteil: "Bestätigungs-UI" ist nicht modal, sondern inline

3) "Modal" ohne JS geht nicht wirklich (nur CSS-Trick)

Ein echtes Bootstrap-Modal braucht JS. Ohne JS kannst du höchstens einen **CSS-Modal Hack** machen (Checkbox/ `:target`), aber:

- du kannst **nicht dynamisch** pro Activity-ID sauber modalisieren ohne Verrenkungen
- und es fühlt sich oft "hacky" an

Wenn du wirklich "Modal wie Bootstrap" willst, dann brauchst du Bootstrap JS (oder eigenes JS). (Du nutzt ja sowieso Bootstrap auf euren Seiten – dann ist das auch ok.)

Wenn Modal gewünscht ist (mit Bootstrap, minimal JS, sauber)

Du brauchst **keine neuen GET/POST-Routen** nur für Confirm. Du behältst deine jetzigen Join/Leave POST-Routen – das Modal macht nur die Bestätigung.

Button öffnet Modal (kein Submit)

html

```
<button type="button"
      class="action-btn {% if is_joined %}action-leave{% else %}action-join{% e
      data-bs-toggle="modal"
      data-bs-target="#confirmModal-{{ activity.activity_id }}">
  {% if is_joined %}Leave X{% else %}Join ➡{% endif %}
</button>
```



Code kopieren

Modal pro Activity (mit echtem POST-Submit im Modal)

html

```
<div class="modal fade" id="confirmModal-{{ activity.activity_id }}" tabindex="-1"
  <div class="modal-dialog modal-dialog-centered">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title">
          {% if is_joined %}Aktivität verlassen?{% else %}Aktivität beitreten?{%
        </h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-labe
      </div>

      <div class="modal-body">
        <p class="mb-0">
          {% if is_joined %}
            Möchtest du dich wirklich von <strong>{{ activity.title }}</strong> a
          {% else %}
            Möchtest du wirklich <strong>{{ activity.title }}</strong> beitreten?
          {% endif %}
        </p>
      </div>
    </div>
  </div>
```

```

    </p>
</div>

<div class="modal-footer">
    <button type="button" class="btn btn-secondary" data-bs-dismiss="modal">A

    <form method="POST"
        action="{% if is_joined %}{% url_for('leave_activity', activity_id=
        class="m-0">
        <button type="submit" class="btn {% if is_joined %}btn-danger{% else %}
        {% if is_joined %}Ja, verlassen{% else %}Ja, beitreten{% endif %}
        </button>
    </form>
</div>
</div>
</div>
</div>

```

➡ Damit bleibt dein Backend exakt gleich: nur `join_activity` / `leave_activity` als POST.

Wichtig: Bootstrap JS muss eingebunden sein (du hattest ja schon `bootstrap.bundle.min.js`).

Empfehlung

 Code kopieren

- **Ohne JS:** nimm Variante (2) **Inline Confirm** (beste UX ohne Seitenwechsel)
- **Mit Bootstrap:** nimm das **Modal** oben (sauberste UX, keine neuen Routen nötig)

Wenn du willst, schreibe ich dir die Inline-Confirm-Version einmal komplett für Join + Leave (inkl. Query-Param Logik und sauberem Layout in deiner Card).

    ...

```

<button type="submit"
    class="action-btn {% if is_joined %}action-
    leave{% else %}action-join{% endif %}"
    {% set confirm_join_id =
    request.args.get('confirm_join', type=int) %}

```



```

{% if confirm_join_id == activity.activity_id %}
    <form method="POST" action="{{
url_for('join_activity', activity_id=activity.activity_id) }}"
class="d-inline">
        <button class="btn btn-sm btn-primary"
type="submit">Ja, beitreten</button>
    </form>
    <a class="btn btn-sm btn-secondary" href="{{
url_for('activity_board') }}">Abbrechen</a>
{% else %}
    <a class="btn btn-sm btn-outline-primary"
href="{{ url_for('activity_board',
confirm_join=activity.activity_id) }}">
        Join ➡
    </a>
{% endif %}

</button>

```

wie würde man das hier jetzt nach Variante 2 unschreiben?

Du darfst **nicht** `<form>` / `<a>` **in einen** `<button>` **packen** (ungültiges HTML) und du brauchst bei Variante 2 **zwei Zustände**:

1. Normal: Link setzt `?confirm_join=<id>` (oder `confirm_leave=<id>`)
2. Confirm-State: zeigt **Ja (POST)** + **Abbrechen (Link)**

So sieht das korrekt aus (1:1 in deine Card, an die Stelle wo aktuell dein Join/Leave Button ist):

jinja2

 Code kopieren

```

{% set confirm_join_id = request.args.get('confirm_join', type=int) %}
{% set confirm_leave_id = request.args.get('confirm_leave', type=int) %}

{# Ist der User schon drin? #}
{% set is_joined = current_user_id and (
    activity.participants
    | selectattr('user_id', 'equalto', current_user_id)
    | list | length > 0
) %}

```

```

<div class="action-area">
  {# ===== FALL A: User ist bereits beigetreten -> LEAVE flow ===== #}
  {% if is_joined %}

    {% if confirm_leave_id == activity.activity_id %}
      <div class="d-flex gap-2 justify-content-end">
        <form method="POST" action="{{ url_for('leave_activity', activity_id=activity_id) }}">
          <button type="submit" class="btn btn-sm btn-danger">Ja, verlassen</button>
        </form>
        <a class="btn btn-sm btn-secondary" href="{{ url_for('activity_board') }}">
      </div>
    {% else %}
      <a class="btn btn-sm action-btn action-leave"
        href="{{ url_for('activity_board', confirm_leave=activity.activity_id) }}"
        Leave X
      </a>
    {% endif %}

  {# ===== FALL B: User ist NICHT beigetreten -> JOIN flow ===== #}
  {% else %}

    {% if confirm_join_id == activity.activity_id %}
      <div class="d-flex gap-2 justify-content-end">
        <form method="POST" action="{{ url_for('join_activity', activity_id=activity_id) }}">
          <button type="submit" class="btn btn-sm btn-primary">Ja, beitreten</button>
        </form>
        <a class="btn btn-sm btn-secondary" href="{{ url_for('activity_board') }}">
      </div>
    {% else %}
      <a class="btn btn-sm action-btn action-join"
        href="{{ url_for('activity_board', confirm_join=activity.activity_id) }}"
        Join ➡
      </a>
    {% endif %}

  {% endif %}
</div>

```

CSS (damit es wie vorher unten rechts sitzt)

Wenn du bisher `.action-form` absolut positioniert hast, mach stattdessen das hier:

CSS

 Code kopieren

```
.activity-card { position: relative; padding-bottom: 70px; }

.activity-card .action-area{
  position: absolute;
  right: 12px;
  bottom: 12px;
}
```

Wichtig:

- Die "Join/Leave"-Anzeige ist jetzt ein `<a>` (GET → setzt confirm param)
- Der echte Join/Leave passiert erst beim "Ja" via `<form method="POST">`
- Kein JavaScript nötig.

Wenn du willst, kann ich dir auch zeigen, wie du beim Confirm die Card leicht "highlightest" (z.B. gelber Rahmen), wenn `confirm_join_id == activity.activity_id`.



```
{% set confirm_join_id = request.args.get('confirm_join',
type=int) %}
{% set confirm_leave_id = request.args.get('confirm_leave',
type=int) %}
```

es sagt int "is undefined"

Das kommt daher, dass deine Jinja/Flask-Jinja Umgebung **keinen** `int` -Typ im Template-Kontext hat. In Jinja funktioniert `type=int` nur, wenn `int` verfügbar ist – bei dir ist es das nicht.

Nimm einfach **ohne type** und caste selbst mit `|int`:

jinja2

Code kopieren

```
{% set confirm_join_id = (request.args.get('confirm_join') | int) %}
{% set confirm_leave_id = (request.args.get('confirm_leave') | int) %}
```

Dann vergleichst du wie gehabt:

jinja2

 Code kopieren

```
{% if confirm_join_id == activity.activity_id %}  
...  
{% endif %}
```

✅ Vorteil: Kein `int` is undefined , und wenn der Parameter fehlt, wird `None` | `int` zu `0` (also matcht es nicht).

Wenn du 0 vermeiden willst (optional), kannst du zusätzlich prüfen:

jinja2

 Code kopieren

```
{% set confirm_join_raw = request.args.get('confirm_join') %}  
{% set confirm_join_id = confirm_join_raw|int if confirm_join_raw else None %}
```

